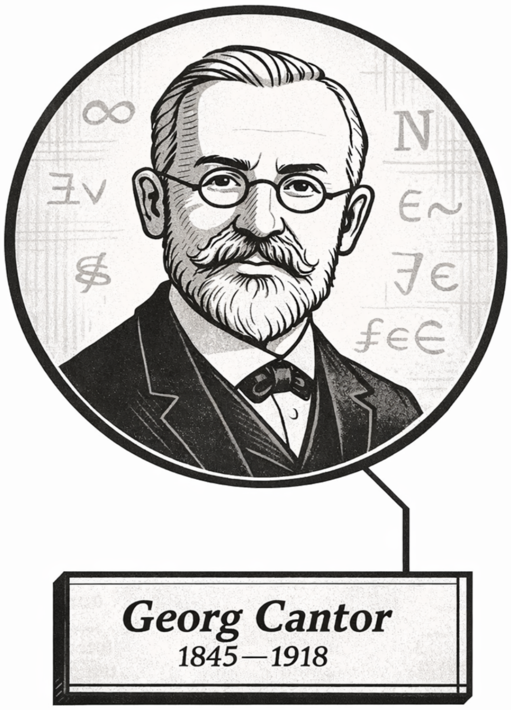
Contents

Part I

Logic, Sets, and Proof

Volume I

Logic, Sets, and Proof



To infinity and beyond!

Georg Cantor (probably)

Volume I

Logic, Sets, and Proof

Self-Study Notes

William Sollers

June 8, 2026

Chapter 1

Propositional Logic

propositional-logic

Proof Techniques → **Propositional Logic** → Predicate Logic

Status: Planned

Roadmap

This chapter develops the formal syntax, semantics, algebraic laws, and normal forms for propositional logic.

Chapter structure

The routed files below collect the notes, exercises, capstone problem, and proofs for the chapter.

1.1 Notation and Conventions

Toolkit

This notation ledger records the propositional symbols, metavariables, semantic notation, and normal-form abbreviations used in the chapter.

Remark (Exposition). The notation section fixes the symbolic vocabulary used throughout the chapter. It identifies which symbols are global conventions, which are local to propositional logic, and where each item is introduced by the formal development.

Symbol	Name	Meaning	Introduced by
$\mathbb{B}$	truth-value set	the set of truth values (global, canonical)	Truth Value

Prop	propositional variable set	the set of propositional variables (global, canonical)	Propositional Language
$\neg$	negation	unary propositional connective (global, canonical)	Propositional Language
$\wedge$	conjunction	binary propositional connective (global, canonical)	Propositional Language
$\vee$	disjunction	binary propositional connective (global, canonical)	Propositional Language
$\to$	conditional	binary propositional connective (global, canonical)	Propositional Language
$\leftrightarrow$	biconditional	binary propositional connective (global, canonical)	Propositional Language
P, Q, R	propositional variables	schematic propositional variables (chapter, canonical)	Propositional Variable
$P_1, \ldots, P_n$	finite list of propositional variables	a finite schematic list of propositional variables (chapter, local)	Propositional Variable
$\circ$	generic binary connective	an arbitrary member of $\{\wedge, \vee, \to, \leftrightarrow\}$ (global, canonical)	Logical Connective
WFF	well-formed formula set	the set of propositional well-formed formulas (chapter, canonical)	Well-Formed Formula
$\varphi, \psi, \chi, \theta$	formula metavariables	metavariables ranging over propositional well-formed formulas (global, canonical)	Well-Formed Formula
α, β	additional formula metavariables	formula metavariables used in comparison statements (chapter, local)	Constructor Injectivity for Propositional Formulas
$\mathrm{Tree}(\varphi)$	syntax tree	parse tree or formation tree of a formula (chapter, canonical)	Parse Tree
$\mathrm{Var}(\varphi)$	variable set of a formula	the set of propositional variables occurring in a formula (chapter, canonical)	Variable Set of a Formula
$\mathrm{Sub}(\varphi)$	subformula set	the set of subformulas of a formula (chapter, canonical)	Subformula
$\mathrm{depth}(\varphi)$	formula depth	height, rank, or complexity of the formula formation tree (chapter, canonical)	Formula Depth
$\mathrm{conn}(\varphi)$	connective count	the number of connective occurrences in a formula (chapter, canonical)	Connective Count
v	truth assignment	a truth assignment on propositional variables (chapter, canonical)	Truth Assignment

w	second truth assignment	truth assignment on propositional variables (global, canonical)	Truth Assignment
$\widehat{v}$	extended truth assignment	unique extension of a truth assignment to formulas (chapter, canonical)	Unique Extension of a Truth Assignment
$\neg, \wedge, \vee, \rightarrow, \leftrightarrow$	standard connectives	standard propositional connectives (global, canonical)	Unique Extension of a Truth Assignment
$v \models \varphi$	satisfaction of a formula	truth assignment satisfies a formula (chapter, canonical)	Satisfaction of a Formula
$\models \varphi$	tautology notation	formula valid under every truth assignment (global, canonical)	Satisfaction of a Formula
$v \models \Gamma$	satisfaction of a set	truth assignment satisfies each formula in a set (chapter, canonical)	Satisfaction of a Set of Formulas
Γ	set of premise formulas	a set of propositional formulas (chapter, canonical)	Satisfaction of a Set of Formulas
$\Gamma \models \varphi$	semantic consequence	semantic consequence from premises to conclusion (chapter, canonical)	Logical Consequence
$\varphi \equiv \psi$	logical equivalence	same truth value under every truth assignment (chapter, canonical)	Logical Equivalence
$f : \mathbb{B}^n \to \mathbb{B}$	n-ary Boolean function	function from truth-value tuples to a truth value (chapter, canonical)	Boolean Function
f_φ	truth function induced by a formula	Boolean function represented by a formula (chapter, canonical)	Truth Function Induced by a Formula
$\equiv$	logical equivalence symbol	logical equivalence relation between formulas (global, canonical)	Propositional Equivalence Law
$\top$	tautological formula	a formula chosen as a true constant (chapter, canonical)	Identity and Domination Laws
$\bot$	contradictory formula	a formula chosen as a false constant (chapter, canonical)	Identity and Domination Laws
$C[-]$	formula context	a one-hole propositional formula context (chapter, canonical)	Formula Context

$C[\varphi]$	context substitution	formula obtained by filling a context with a formula (chapter, canonical)	Formula Context

1.2 Syntax

Toolkit

This section uses truth values, propositional variables, connectives, well-formed formulas, formation trees, and subformulas to build syntax.

Remark (Exposition). The syntax section builds the formal language before assigning it meaning. It isolates the recursive formation rules for formulas and the structural notions, such as subformulas and depth, that later support semantic induction and proof arguments.

Definition (Truth Value)

Definition 1.2.1 (Truth Value). The set of truth values is

$$\mathbb{B} := \{\text{False}, \text{True}\}.$$

Some sources identify this set with $\{0, 1\}$, with 0 read as false and 1 read as true.

Remark (Standard quantified statement).

$$\mathbb{B} = \{\text{False}, \text{True}\}.$$

Remark (Interpretation). Truth values are the semantic targets of propositional formulas. Syntax determines which strings are formulas; semantics later assigns elements of $\mathbb{B}$ to formulas.

Remark (Dependencies).

Definition (Propositional Language)

Definition 1.2.2 (Propositional Language). A propositional language consists of:

1. a set Prop of propositional variables;
2. logical connectives;
3. punctuation symbols such as parentheses;
4. recursive formation rules specifying the well-formed formulas.

Remark (Standard quantified statement).

$$\mathcal{L}_{\text{prop}} = (\mathsf{Prop}, \{\neg, \wedge, \vee, \to, \leftrightarrow\}, \texttt{punctuation}, \texttt{formation rules}).$$

Remark (Interpretation). The language supplies symbols and grammar. It does not yet assign truth values, truth conditions, or logical consequence.

Remark (Dependencies).

- Truth Value

Definition (Propositional Variable)

Definition 1.2.3 (Propositional Variable). A propositional variable is an atomic symbol intended to stand for a proposition. A standard supply of variables is

$$\mathsf{Prop} = \{P_1, P_2, P_3, \ldots\}.$$

Informally, variables are often written $P, Q, R, S, \ldots$.

Remark (Standard quantified statement).

$$\text{PropositionalVariable}(P) \iff P \in \mathsf{Prop}.$$

Remark (Definition predicate reading). $\text{PropositionalVariable}(P)$ means $P \in \mathsf{Prop}$.

Remark (Interpretation). A propositional variable has no internal syntactic structure in propositional logic. It is a basic symbol from which more complex formulas are formed.

Remark (Examples). P, Q, R, P_1, P_2 are propositional variables when they belong to Prop.

Remark (Non-examples). $\neg P$, $P \wedge Q$, and $(P \to Q)$ are not propositional variables; they are compound formulas.

Remark (Dependencies).

- Propositional Language

Definition (Logical Connective)

Definition 1.2.4 (Logical Connective). The standard primitive propositional connectives are

$$\neg, \quad \wedge, \quad \vee, \quad \to, \quad \leftrightarrow .$$

The connective $\neg$ is unary. The connectives $\wedge, \vee, \to, \leftrightarrow$ are binary.

Symbol	Name	Arity	Formation pattern	Reading
$\neg$	negation	unary	$\neg\varphi$	not φ
$\wedge$	conjunction	binary	$(\varphi \wedge \psi)$	φ and ψ
$\vee$	disjunction	binary	$(\varphi \vee \psi)$	φ or ψ
$\to$	conditional	binary	$(\varphi \to \psi)$	if φ, then ψ
$\leftrightarrow$	biconditional	binary	$(\varphi \leftrightarrow \psi)$	φ iff ψ

Remark (Standard quantified statement).

$$\text{UnaryConnective}(\neg) \quad \text{and} \quad \text{BinaryConnective}(\wedge) \wedge \text{BinaryConnective}(\vee) \wedge \text{BinaryConnective}(\to) \wedge \text{Bin}$$

Remark (Interpretation). Arity records how many formulas a connective uses to form a new formula. The truth behavior of these connectives belongs to semantics, not to the syntactic definition.

Remark (Dependencies).

- Propositional Language

Definition (Well-Formed Formula)

Definition 1.2.5 (Well-Formed Formula). The set WFF of well-formed formulas is the smallest set of strings satisfying the following formation rules:

1. Every propositional variable $P \in \mathsf{Prop}$ belongs to WFF.
2. If $\varphi \in \mathsf{WFF}$, then $\neg\varphi \in \mathsf{WFF}$.
3. If $\varphi, \psi \in \mathsf{WFF}$ and $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$, then $(\varphi \circ \psi) \in \mathsf{WFF}$.
4. No string is a well-formed formula unless it is obtained by finitely many applications of the previous clauses.

Remark (Standard quantified statement).

$$\mathsf{WFF} = \bigcap \big\{S : \mathsf{Prop} \subseteq S,\ (\forall \varphi \in S)(\neg\varphi \in S),\ (\forall \varphi, \psi \in S)(\forall \circ \in \{\wedge, \vee, \to, \leftrightarrow\})\big((\varphi \circ \psi) \in S\big)\big\}.$$

Remark (Definition predicate reading). $\mathrm{WellFormedFormula}(\varphi)$ means $\varphi \in \mathsf{WFF}$, read as φ is a well-formed formula.

Remark (Negated quantified statement).

$$\sigma \notin \mathsf{WFF} \iff \exists S\, [\mathsf{Prop} \subseteq S \wedge (\forall \varphi \in S)(\neg\varphi \in S) \wedge (\forall \varphi, \psi \in S)(\forall \circ \in \{\wedge, \vee, \to, \leftrightarrow\})((\varphi \circ \psi) \in S) \wedge \sigma \notin$$

Remark (Negation predicate reading). $\neg\,\mathrm{WellFormedFormula}(\sigma)$ means the string σ is excluded by at least one formation-closed set containing Prop; equivalently, it is not generated by finitely many applications of the formation rules.

Remark (Failure modes). A string fails to be well formed when it violates the formation grammar: an operand may be missing, a connective may be missing, parentheses may be unmatched, or the string may not be generated by finitely many formation steps.

Remark (Failure mode decomposition).

$$\underbrace{P\wedge}_{\texttt{missing right operand}} \qquad \underbrace{(P\ \ Q)}_{\texttt{missing binary connective}} \qquad \underbrace{\neg \vee P}_{\texttt{negation applied to a non-formula}}.$$

Remark (Interpretation). Well-formed formulas are exactly the strings to which the semantic and proof-theoretic machinery of propositional logic applies.

Remark (Examples). P, $\neg P$, $(P \to Q)$, and $(P \to (Q \vee R))$ are well-formed formulas.

Remark (Non-examples). $P\wedge$, (PQ), and $\neg \vee P$ are not well-formed formulas.

Remark (Dependencies).

- Propositional Variable
- Logical Connective

Lemma (Closure Under Formation)

Lemma 1.2.6 (Closure Under Formation). *The set* WFF *is closed under the propositional formation rules.*
That is, if $\varphi \in \mathsf{WFF}$*, then*

$$\neg\varphi \in \mathsf{WFF}.$$

If $\varphi, \psi \in \mathsf{WFF}$ *and*

$$\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\},$$

then

$$(\varphi \circ \psi) \in \mathsf{WFF}.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall\varphi \in \mathsf{WFF})(\neg\varphi \in \mathsf{WFF})$$

and

$$(\forall\varphi, \psi \in \mathsf{WFF})(\forall\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((\varphi \circ \psi) \in \mathsf{WFF}).$$

Remark (Interpretation). Closure says that every legal use of a formation rule produces another well-formed formula. It is the forward direction of the recursive grammar.

Remark (Dependencies).

- Well-Formed Formula

Theorem (Minimality of Well-Formed Formulas)

Theorem 1.2.7 (Minimality of Well-Formed Formulas). *Let* S *be a set of strings over the propositional alphabet. Suppose:*

1. $\mathsf{Prop} \subseteq S$*;*
2. *if* $\varphi \in S$*, then* $\neg\varphi \in S$*;*
3. *if* $\varphi, \psi \in S$ *and* $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$*, then* $(\varphi \circ \psi) \in S$*.*

Then

$$\mathsf{WFF} \subseteq S.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall S)\Big[\mathsf{Prop} \subseteq S \wedge (\forall\varphi \in S)(\neg\varphi \in S) \\ \wedge (\forall\varphi, \psi \in S)(\forall\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((\varphi \circ \psi) \in S)\Big] \rightarrow \mathsf{WFF} \subseteq S.$$

Remark (Interpretation). Minimality is the formal content of the clause "nothing else is a wff." It says that WFF is contained in every set of strings that contains the propositional variables and is closed under the formation rules.

Remark (Dependencies).

- Well-Formed Formula

Definition (Atomic Formula)

Definition 1.2.8 (Atomic Formula). An atomic formula is a well-formed formula that is a propositional variable.

Remark (Standard quantified statement).

$$\mathrm{AtomicFormula}(\varphi) \iff \varphi \in \mathsf{WFF} \wedge \varphi \in \mathsf{Prop}.$$

Remark (Definition predicate reading). $\mathrm{AtomicFormula}(\varphi)$ means that φ is a propositional variable viewed as a well-formed formula.

Remark (Interpretation). Atomic formulas are the base cases of the recursive grammar. They contain no connective structure.

Remark (Dependencies).

- Well-Formed Formula
- Propositional Variable

Definition (Molecular Formula)

Definition 1.2.9 (Molecular Formula). A molecular formula is a well-formed formula that is not atomic. Equivalently, it is a well-formed formula formed using at least one logical connective.

Remark (Standard quantified statement).

$$\mathrm{MolecularFormula}(\varphi) \iff \varphi \in \mathsf{WFF} \wedge \neg\,\mathrm{AtomicFormula}(\varphi).$$

Remark (Definition predicate reading). $\mathrm{MolecularFormula}(\varphi)$ means that φ has nontrivial connective structure.

Remark (Interpretation). Molecular formulas are exactly the formulas obtained by applying negation or a binary connective at least once.

Remark (Dependencies).

- Well-Formed Formula
- Atomic Formula
- Logical Connective

Definition (Main Connective)

Definition 1.2.10 (Main Connective). The main connective of a molecular formula is the connective introduced at the final formation step:

1. for $\neg\varphi$, the main connective is $\neg$;
2. for $(\varphi \circ \psi)$, the main connective is $\circ$.

Atomic formulas have no main connective.

Remark (Standard quantified statement).

$$\mathrm{MainConnective}(\neg\varphi, \neg) \quad \text{and} \quad \mathrm{MainConnective}((\varphi \circ \psi), \circ).$$

Remark (Definition predicate reading). $\mathrm{MainConnective}(\varphi, c)$ means that c is the outermost connective introduced in the final formation step of φ.

Remark (Interpretation). The main connective identifies the outermost syntactic operation. It is the first connective used when analyzing a formula from the outside inward.

Remark (Dependencies).

- Well-Formed Formula
- Molecular Formula
- Logical Connective

Theorem (Structural Induction for Propositional Formulas)

Theorem 1.2.11 (Structural Induction for Propositional Formulas). *Let $A(\varphi)$ be a property of well-formed formulas. Suppose:*

1. *$A(P)$ holds for every $P \in \mathsf{Prop}$;*
2. *if $A(\varphi)$ holds, then $A(\neg\varphi)$ holds;*
3. *if $A(\varphi)$ and $A(\psi)$ hold, then $A((\varphi \circ \psi))$ holds for every $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$.*

Then $A(\theta)$ holds for every $\theta \in \mathsf{WFF}$.
Go to proof.

Remark (Standard quantified statement).

$$\big[(\forall P \in \mathsf{Prop})A(P) \wedge (\forall\varphi \in \mathsf{WFF})(A(\varphi) \rightarrow A(\neg\varphi)) \wedge (\forall\varphi, \psi \in \mathsf{WFF})(\forall\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})\big((A(\varphi) \wedge A(\psi)$$

Remark (Interpretation). To prove a statement for all formulas, it suffices to prove it for propositional variables and show it is preserved by each formula-forming operation.

Remark (Dependencies).

- Well-Formed Formula

Theorem (Structural Recursion for Propositional Formulas)

Theorem 1.2.12 (Structural Recursion for Propositional Formulas). *To define a function F on* WFF*, it suffices to specify:*

1. *$F(P)$ for every $P \in \mathsf{Prop}$;*
2. *$F(\neg\varphi)$ in terms of $F(\varphi)$;*
3. *$F((\varphi \circ \psi))$ in terms of $F(\varphi)$, $F(\psi)$, and $\circ$.*

When these clauses respect the formation rules, they determine a unique function on WFF*.*
Go to proof.

Remark (Standard quantified statement).

recursive data on the formation clauses of $\mathsf{WFF} \Longrightarrow \exists! F : \mathsf{WFF} \to X$ satisfying those cla

Remark (Interpretation). Structural recursion justifies recursive definitions such as variables of a formula, subformulas, depth, connective count, syntax tree, and truth evaluation.

Remark (Dependencies).

- Well-Formed Formula
- Structural Induction for Propositional Formulas

Lemma (Constructor Disjointness for Propositional Formulas)

Lemma 1.2.13 (Constructor Disjointness for Propositional Formulas). *The three outer formation cases for propositional formulas are disjoint:*

1. *no propositional variable is a negation formula;*
2. *no propositional variable is a binary formula;*
3. *no negation formula is a binary formula.*

Go to proof.

Remark (Standard quantified statement).

$$P \in \mathsf{Prop} \Rightarrow [(\forall \varphi \in \mathsf{WFF})(P \neq \neg\varphi) \wedge (\forall \varphi, \psi \in \mathsf{WFF})(\forall \circ)(P \neq (\varphi \circ \psi))]$$

and

$$(\forall \varphi, \psi, \chi \in \mathsf{WFF})(\forall \circ)\big(\neg\varphi \neq (\psi \circ \chi)\big).$$

Remark (Predicate reading).

$$\underbrace{P}_{\texttt{atomic}} \qquad \underbrace{\neg\varphi}_{\texttt{negation case}} \qquad \underbrace{(\varphi \circ \psi)}_{\texttt{binary case}}$$

belong to three distinct outer-form classes.

Remark (Interpretation). A formula cannot be parsed as two different outer formation types.

Remark (Dependencies).

- Well-Formed Formula
- Atomic Formula
- Main Connective

Lemma (Constructor Injectivity for Propositional Formulas)

Lemma 1.2.14 (Constructor Injectivity for Propositional Formulas). *The propositional formula constructors are injective:*

1. *if* $\neg\varphi = \neg\psi$, *then* $\varphi = \psi$;
2. *if* $(\varphi \circ \psi) = (\alpha \star \beta)$, *then* $\varphi = \alpha$, $\psi = \beta$, *and* $\circ = \star$.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})(\neg\varphi = \neg\psi \to \varphi = \psi)$$

and

$$(\forall \varphi, \psi, \alpha, \beta \in \mathsf{WFF})(\forall \circ, \star \in \{\wedge, \vee, \to, \leftrightarrow\})\left[(\varphi \circ \psi) = (\alpha \star \beta) \to (\varphi = \alpha \wedge \psi = \beta \wedge \circ = \star)\right].$$

Remark (Interpretation). If two formulas are built by the same outer constructor and are equal as strings, then their immediate constituents are equal.

Remark (Dependencies).

- Well-Formed Formula
- Logical Connective

Theorem (Unique Decomposition for Propositional Formulas)

Theorem 1.2.15 (Unique Decomposition for Propositional Formulas). *Every* $\varphi \in \mathsf{WFF}$ *has exactly one outermost form:*

1. $\varphi = P$ *for a unique* $P \in \mathsf{Prop}$;
2. $\varphi = \neg\psi$ *for a unique* $\psi \in \mathsf{WFF}$;
3. $\varphi = (\psi \circ \chi)$ *for unique* $\psi, \chi \in \mathsf{WFF}$ *and a unique binary connective* $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$.

Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi \in \mathsf{WFF}, \quad \text{exactly one of the following holds:}$$

$$\varphi \in \mathsf{Prop}, \qquad \exists! \psi \in \mathsf{WFF}\,(\varphi = \neg\psi), \qquad \exists! \psi, \chi \in \mathsf{WFF}\ \exists! \circ \in \{\wedge, \vee, \to, \leftrightarrow\}\,(\varphi = (\psi \circ \chi)).$$

Remark (Predicate reading).

$$\varphi \in \underbrace{\mathsf{Prop}}_{\texttt{atomic case}} \quad \text{or} \quad \varphi = \underbrace{\neg\psi}_{\texttt{negation case}} \quad \text{or} \quad \varphi = \underbrace{(\psi \circ \chi)}_{\texttt{binary case}},$$

and exactly one of these alternatives holds.

Remark (Interpretation). Unique decomposition makes recursive definitions and induction on formulas unambiguous.

Remark (Dependencies).

- Well-Formed Formula
- Constructor Disjointness for Propositional Formulas
- Constructor Injectivity for Propositional Formulas

Definition (Parse Tree)

Definition 1.2.16 (Parse Tree). The parse tree, or syntax tree, of a well-formed formula is the tree representation of its recursive construction:

1. a propositional variable is represented by a single leaf;
2. a formula $\neg\varphi$ is represented by a root labeled $\neg$ with one child, the parse tree of φ;
3. a formula $(\varphi \circ \psi)$ is represented by a root labeled $\circ$ with two children, the parse trees of φ and ψ.

Remark (Standard quantified statement).

$$\mathrm{Tree} : \mathsf{WFF} \to \{\texttt{finite rooted labeled trees}\}$$

is defined recursively by the three formation clauses for propositional formulas.

Remark (Interpretation). The parse tree records the same structure described by unique decomposition. It turns the recursive construction of a formula into a visible tree.

Remark (Dependencies).

- Well-Formed Formula
- Unique Decomposition for Propositional Formulas

Theorem (Unique Syntax Tree for Propositional Formulas)

Theorem 1.2.17 (Unique Syntax Tree for Propositional Formulas). *Every propositional well-formed formula has a unique parse tree.*
Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi \in \mathsf{WFF}, \quad \exists! T \ (T = \mathrm{Tree}(\varphi)).$$

Remark (Interpretation). A formula has exactly one tree representation of its recursive construction. This is the tree form of unique decomposition.

Remark (Dependencies).

- Parse Tree
- Unique Decomposition for Propositional Formulas

Definition (Variable Set of a Formula)

Definition 1.2.18 (Variable Set of a Formula). The variable set $\mathrm{Var}(\varphi)$ of a formula φ is defined recursively:

1. if $\varphi = P \in \mathsf{Prop}$, then $\mathrm{Var}(\varphi) = \{P\}$;
2. if $\varphi = \neg\psi$, then $\mathrm{Var}(\varphi) = \mathrm{Var}(\psi)$;
3. if $\varphi = (\psi \circ \chi)$, then

$$\mathrm{Var}(\varphi) = \mathrm{Var}(\psi) \cup \mathrm{Var}(\chi).$$

Remark (Standard quantified statement).

$$\mathrm{Var}(P) = \{P\}, \qquad \mathrm{Var}(\neg\psi) = \mathrm{Var}(\psi), \qquad \mathrm{Var}((\psi \circ \chi)) = \mathrm{Var}(\psi) \cup \mathrm{Var}(\chi).$$

Remark (Interpretation). The variable set records exactly which propositional variables occur in a formula.

Remark (Dependencies).

- Well-Formed Formula
- Structural Recursion for Propositional Formulas

Definition (Subformula)

Definition 1.2.19 (Subformula). The set $\mathrm{Sub}(\varphi)$ of subformulas of a formula φ is defined recursively:

1. if φ is atomic, then $\mathrm{Sub}(\varphi) = \{\varphi\}$;
2. if $\varphi = \neg\psi$, then
$$\mathrm{Sub}(\varphi) = \{\varphi\} \cup \mathrm{Sub}(\psi);$$
3. if $\varphi = (\psi \circ \chi)$, then

$$\mathrm{Sub}(\varphi) = \{\varphi\} \cup \mathrm{Sub}(\psi) \cup \mathrm{Sub}(\chi).$$

Remark (Standard quantified statement).

$$\mathrm{Sub}(P) = \{P\}, \qquad \mathrm{Sub}(\neg\psi) = \{\neg\psi\} \cup \mathrm{Sub}(\psi),$$

$$\mathrm{Sub}((\psi \circ \chi)) = \{(\psi \circ \chi)\} \cup \mathrm{Sub}(\psi) \cup \mathrm{Sub}(\chi).$$

Remark (Interpretation). Subformulas are the formulas that occur at nodes of the parse tree.

Remark (Dependencies).

- Well-Formed Formula
- Unique Decomposition for Propositional Formulas

Definition (Proper Subformula)

Definition 1.2.20 (Proper Subformula). Let $\varphi \in \mathsf{WFF}$. A *proper subformula* of φ is a formula ψ such that

$$\psi \in \mathrm{Sub}(\varphi) \quad \text{and} \quad \psi \neq \varphi.$$

Remark (Standard quantified statement).

$$\mathrm{ProperSubformula}(\psi, \varphi) \iff \psi \in \mathrm{Sub}(\varphi) \land \psi \neq \varphi.$$

Remark (Interpretation). Proper subformulas are the strict syntactic parts of a formula. They exclude the whole formula itself.

Remark (Dependencies).

- Subformula
- Well-Formed Formula

Definition (Formula Depth)

Definition 1.2.21 (Formula Depth). The depth of a formula φ, denoted $\mathrm{depth}(\varphi)$, is defined recursively:

1. if φ is atomic, then $\mathrm{depth}(\varphi) = 0$;
2. if $\varphi = \neg\psi$, then $\mathrm{depth}(\varphi) = \mathrm{depth}(\psi) + 1$;
3. if $\varphi = (\psi \circ \chi)$, then

$$\mathrm{depth}(\varphi) = \max\{\mathrm{depth}(\psi), \mathrm{depth}(\chi)\} + 1.$$

Remark (Standard quantified statement).

$$\mathrm{depth}(P) = 0, \qquad \mathrm{depth}(\neg\psi) = \mathrm{depth}(\psi) + 1,$$

$$\mathrm{depth}((\psi \circ \chi)) = \max\{\mathrm{depth}(\psi), \mathrm{depth}(\chi)\} + 1.$$

Remark (Interpretation). Formula depth measures the height of the syntax tree. Some authors call this the rank, complexity, or formation depth of the formula.

Remark (Dependencies).

- Well-Formed Formula
- Parse Tree

Definition (Connective Count)

Definition 1.2.22 (Connective Count). The connective count of a formula φ, denoted $\mathrm{conn}(\varphi)$, is defined recursively:

1. if φ is atomic, then $\mathrm{conn}(\varphi) = 0$;
2. if $\varphi = \neg\psi$, then $\mathrm{conn}(\varphi) = \mathrm{conn}(\psi) + 1$;
3. if $\varphi = (\psi \circ \chi)$, then

$$\mathrm{conn}(\varphi) = \mathrm{conn}(\psi) + \mathrm{conn}(\chi) + 1.$$

Remark (Standard quantified statement).

$$\mathrm{conn}(P) = 0, \qquad \mathrm{conn}(\neg\psi) = \mathrm{conn}(\psi) + 1,$$

$$\mathrm{conn}((\psi \circ \chi)) = \mathrm{conn}(\psi) + \mathrm{conn}(\chi) + 1.$$

Remark (Interpretation). Formula depth measures the height of the syntax tree. Connective count measures the number of connective nodes in the syntax tree.

Remark (Dependencies).

- Well-Formed Formula
- Parse Tree

Lemma (Proper Subformula Depth)

Lemma 1.2.23 (Proper Subformula Depth). *If ψ is a proper subformula of φ, then*

$$\mathrm{depth}(\psi) < \mathrm{depth}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\left[\psi \in \mathrm{Sub}(\varphi) \land \psi \neq \varphi \to \mathrm{depth}(\psi) < \mathrm{depth}(\varphi)\right].$$

Remark (Interpretation). Every descent into a proper subformula lowers formula depth. This justifies induction on formula depth and recursive proofs over subformulas.

Remark (Dependencies).

- Subformula
- Formula Depth

Lemma (Finiteness of Variables in a Formula)

Lemma 1.2.24 (Finiteness of Variables in a Formula). *For every $\varphi \in \mathsf{WFF}$, the set $\mathrm{Var}(\varphi)$ is finite.*
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF}) \quad \mathrm{Var}(\varphi) \text{ is finite.}$$

Remark (Interpretation). Every formula depends on only finitely many propositional variables, even though the language may contain infinitely many variables.

Remark (Dependencies).

- Variable Set of a Formula
- Structural Induction for Propositional Formulas

Lemma (Finiteness of Subformulas)

Lemma 1.2.25 (Finiteness of Subformulas). *For every $\varphi \in \mathsf{WFF}$, the set $\mathrm{Sub}(\varphi)$ is finite.*
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF}) \quad \mathrm{Sub}(\varphi) \text{ is finite.}$$

Remark (Interpretation). Every formula has only finitely many syntactic parts.

Remark (Dependencies).

- Subformula
- Structural Induction for Propositional Formulas

1.3 Semantics

Toolkit

This section uses truth assignments, recursive extensions, satisfaction, truth tables, and logical equivalence to assign meanings to formulas.

Remark (Exposition). The semantics section assigns truth conditions to the syntactic formulas introduced earlier. Truth assignments, recursive extensions, satisfaction, and truth tables provide the machinery for validity, consequence, equivalence, and semantic classification.

Truth Assignments

Definition (Truth Assignment)

Definition 1.3.1 (Truth Assignment). A *truth assignment* is a function

$$v : \mathsf{Prop} \to \mathbb{B}.$$

For each propositional variable $P \in \mathsf{Prop}$, the value $v(P)$ is the truth value assigned to P.

Remark (Standard quantified statement).

$$\mathrm{TruthAssignment}(v) \iff v : \mathsf{Prop} \to \mathbb{B}.$$

Remark (Definition predicate reading).

$$\mathrm{TruthAssignment}(v)$$

means that v assigns exactly one truth value to each propositional variable.

Remark (Interpretation). A truth assignment gives semantic values to the atomic formulas. The recursive truth conditions for the connectives then determine the truth value of every well-formed formula.

Remark (Examples). If $\mathsf{Prop} = \{P_1, P_2, P_3, \ldots\}$, then a truth assignment may satisfy

$$v(P_1) = \mathsf{True}, \qquad v(P_2) = \mathsf{False}, \qquad v(P_3) = \mathsf{True},$$

with values assigned to every remaining propositional variable.

Remark (Non-Examples). A partial assignment that gives values only to P_1 and P_2 is not a truth assignment on Prop. It may be useful for local computations, but it is not a function $\mathsf{Prop} \to \mathbb{B}$.

Remark (Dependencies).

- Truth Value
- Propositional Variable
- Well-Formed Formula

Definition (Domain of a Truth Assignment)

Definition 1.3.2 (Domain of a Truth Assignment). The *domain* of a truth assignment

$$v : \mathsf{Prop} \to \mathbb{B}$$

is the set Prop of propositional variables.

Remark (Standard quantified statement).

$$\mathrm{TruthAssignment}(v) \implies \mathrm{dom}(v) = \mathsf{Prop}.$$

Remark (Interpretation). A truth assignment is global: it assigns truth values to all propositional variables in the language, even when a particular formula uses only finitely many of them.

Remark (Dependencies).

- Truth Assignment

Extension of a Truth Assignment

Definition (Extension of a Truth Assignment to Formulas)

Definition 1.3.3 (Extension of a Truth Assignment to Formulas). Let

$$v : \mathsf{Prop} \to \mathbb{B}$$

be a truth assignment. The *extension* of v to well-formed formulas is the function

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

defined recursively as follows:

$$\widehat{v}(P) = v(P) \quad \text{for every } P \in \mathsf{Prop},$$

$$\widehat{v}(\neg\varphi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{False},$$

$$\widehat{v}(\varphi \wedge \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ and } \widehat{v}(\psi) = \mathsf{True},$$

$$\widehat{v}(\varphi \vee \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ or } \widehat{v}(\psi) = \mathsf{True},$$

$$\widehat{v}(\varphi \to \psi) = \mathsf{False} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ and } \widehat{v}(\psi) = \mathsf{False},$$

and

$$\widehat{v}(\varphi \leftrightarrow \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \widehat{v}(\psi).$$

Remark (Standard quantified statement). For every truth assignment $v : \mathsf{Prop} \to \mathbb{B}$, its extension is a function

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

satisfying the recursive truth clauses for propositional variables, negation, conjunction, disjunction, material implication, and biconditional.

Remark (Definition predicate reading).

$$\mathrm{ExtendedTruthAssignment}(\widehat{v}, v)$$

means that $\widehat{v}$ is the recursively defined truth-value function on WFF induced by the atomic truth assignment v.

Remark (Interpretation). The original assignment v gives truth values only to propositional variables. The extension $\widehat{v}$ evaluates every formula by following the formula's syntactic construction.

Remark (Examples).

φ	$\neg\varphi$
True	False
False	True

φ	ψ	$\varphi \wedge \psi$
True	True	True
True	False	False
False	True	False
False	False	False

φ	ψ	$\varphi \vee \psi$
True	True	True
True	False	True
False	True	True
False	False	False

φ	ψ	$\varphi \to \psi$
True	True	True
True	False	False
False	True	True
False	False	True

φ	ψ	$\varphi \leftrightarrow \psi$
True	True	True
True	False	False
False	True	False
False	False	True

Remark (Examples). The truth tables are not additional formation rules. They specify how each primitive connective acts on truth values once the formula has already been formed syntactically.

Remark (Dependencies).

- Truth Assignment
- Well-Formed Formula
- Structural Recursion for Propositional Formulas
- Unique Decomposition for Propositional Formulas

Theorem (Unique Extension of a Truth Assignment)

Theorem 1.3.4 (Unique Extension of a Truth Assignment). *Every truth assignment*

$$v : \mathsf{Prop} \to \mathbb{B}$$

has a unique extension

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

satisfying the recursive truth clauses for propositional variables and the primitive connectives.
Go to proof.

Remark (Standard quantified statement).

$$(\forall v : \mathsf{Prop} \to \mathbb{B})(\exists! \widehat{v} : \mathsf{WFF} \to \mathbb{B})$$

such that $\widehat{v}$ agrees with v on Prop and satisfies the recursive truth clauses for

$$\neg, \wedge, \vee, \to, \leftrightarrow .$$

Remark (Predicate reading).

$$\text{TruthAssignment}(v) \implies \exists! \widehat{v}\ \text{ExtendedTruthAssignment}(\widehat{v}, v).$$

Remark (Interpretation). This theorem guarantees that semantic evaluation is well-defined. Once the truth values of the propositional variables are fixed, every well-formed formula has exactly one truth value.

Remark (Dependencies).

- Truth Assignment
- Extension of a Truth Assignment to Formulas

- Structural Recursion for Propositional Formulas
- Unique Decomposition for Propositional Formulas

Definition (Satisfaction of a Formula)

Definition 1.3.5 (Satisfaction of a Formula). Let v be a truth assignment and let $\varphi \in \mathsf{WFF}$. We say that v satisfies φ, written

$$v \models \varphi,$$

if

$$\widehat{v}(\varphi) = \text{True}.$$

Remark (Standard quantified statement).

$$v \models \varphi \iff \text{TruthAssignment}(v) \wedge \varphi \in \mathsf{WFF} \wedge \widehat{v}(\varphi) = \text{True}.$$

Remark (Definition predicate reading).

$$\text{SatisfiesFormula}(v, \varphi)$$

means $v \models \varphi$.

Remark (Negated quantified statement).

$$v \not\models \varphi \iff \widehat{v}(\varphi) = \text{False}.$$

Remark (Interpretation). The notation $v \models \varphi$ means that the formula φ is true under the truth assignment v.

Remark (Dependencies).

- Truth Assignment
- Unique Extension of a Truth Assignment
- Well-Formed Formula

Definition (Satisfaction of a Set of Formulas)

Definition 1.3.6 (Satisfaction of a Set of Formulas). Let $\Gamma \subseteq \mathsf{WFF}$. A truth assignment v satisfies Γ, written

$$v \models \Gamma,$$

if $v \models \gamma$ for every $\gamma \in \Gamma$.

Remark (Standard quantified statement).

$$v \models \Gamma \iff \text{TruthAssignment}(v) \wedge \Gamma \subseteq \mathsf{WFF} \wedge (\forall \gamma \in \Gamma)(v \models \gamma).$$

Remark (Definition predicate reading).

$$\text{SatisfiesSet}(v, \Gamma)$$

means v satisfies every formula in Γ.

Remark (Negated quantified statement).

$$v \not\models \Gamma \iff \exists\gamma \in \Gamma \text{ such that } v \not\models \gamma.$$

Remark (Interpretation). Satisfaction of a set means simultaneous satisfaction of every formula in the set.

Remark (Dependencies).

- Satisfaction of a Formula
- Truth Assignment

Definition (Truth Table)

Definition 1.3.7 (Truth Table). Let $\varphi \in \mathsf{WFF}$ and suppose the variables occurring in φ are among $P_1, \ldots, P_n$. A truth table for φ is a finite table with one row for each assignment

$$a : \{P_1, \ldots, P_n\} \to \mathbb{B}$$

and a final column recording the value of $\widehat{v}(\varphi)$ for any truth assignment v extending a.

Remark (Standard quantified statement).

$$\mathrm{TruthTable}(\varphi) \iff \varphi \in \mathsf{WFF} \wedge \mathrm{Var}(\varphi) = \{P_1, \ldots, P_n\} \wedge \text{the table has } 2^n \text{ assignment rows.}$$

Remark (Interpretation). A truth table is finite because every formula contains only finitely many variables. It records the complete truth behavior of a formula.

Remark (Dependencies).

- Variable Set of a Formula
- Finiteness of Variables in a Formula
- Unique Extension of a Truth Assignment

Definition (Tautology)

Definition 1.3.8 (Tautology). A formula $\varphi \in \mathsf{WFF}$ is a tautology, written

$$\models \varphi,$$

if every truth assignment satisfies φ.

Remark (Standard quantified statement).

$$\models \varphi \iff \varphi \in \mathsf{WFF} \wedge (\forall v : \mathsf{Prop} \to \mathbb{B})(v \models \varphi).$$

Remark (Definition predicate reading).

$$\mathrm{Tautology}(\varphi)$$

means $\models \varphi$.

Remark (Interpretation). A tautology is true under every possible assignment of truth values to propositional variables.

Remark (Dependencies).

- Satisfaction of a Formula
- Truth Assignment

Definition (Contradiction)

Definition 1.3.9 (Contradiction). A formula $\varphi \in \mathsf{WFF}$ is a contradiction if no truth assignment satisfies φ.

Remark (Standard quantified statement).

$$\text{Contradiction}(\varphi) \iff \varphi \in \mathsf{WFF} \land (\forall v : \mathsf{Prop} \to \mathbb{B})(v \not\models \varphi).$$

Remark (Definition predicate reading).

$$\text{Contradiction}(\varphi)$$

means that φ is false under every truth assignment.

Remark (Interpretation). A contradiction has no true row in its truth table.

Remark (Dependencies).

- Satisfaction of a Formula
- Truth Assignment

Definition (Satisfiable Formula)

Definition 1.3.10 (Satisfiable Formula). A formula $\varphi \in \mathsf{WFF}$ is satisfiable if at least one truth assignment satisfies it.

Remark (Standard quantified statement).

$$\text{SatisfiableFormula}(\varphi) \iff \varphi \in \mathsf{WFF} \land \exists v : \mathsf{Prop} \to \mathbb{B} \texttt{ such that } v \models \varphi.$$

Remark (Definition predicate reading).

$$\text{SatisfiableFormula}(\varphi)$$

means that φ is true under at least one truth assignment.

Remark (Interpretation). A satisfiable formula has at least one true row in its truth table.

Remark (Dependencies).

- Satisfaction of a Formula
- Truth Assignment

Definition (Contingency)

Definition 1.3.11 (Contingency)**.** A formula $\varphi \in \mathsf{WFF}$ is a contingency if it is satisfiable but not a tautology.

Remark (Standard quantified statement)*.*

$$\text{Contingency}(\varphi) \iff \text{SatisfiableFormula}(\varphi) \wedge \neg\,\text{Tautology}(\varphi).$$

Remark (Definition predicate reading)*.*

$$\text{Contingency}(\varphi)$$

means that φ is true under some truth assignments and false under others.

Remark (Interpretation)*.* A contingency has at least one true row and at least one false row in its truth table.

Remark (Dependencies).

- Satisfiable Formula
- Tautology

Definition (Logical Consequence)

Definition 1.3.12 (Logical Consequence)**.** Let $\Gamma \subseteq \mathsf{WFF}$ and $\varphi \in \mathsf{WFF}$. The formula φ is a logical consequence of Γ, written

$$\Gamma \models \varphi,$$

if every truth assignment satisfying Γ also satisfies φ.

Remark (Standard quantified statement)*.*

$$\Gamma \models \varphi \iff \Gamma \subseteq \mathsf{WFF} \wedge \varphi \in \mathsf{WFF} \wedge (\forall v : \mathsf{Prop} \to \mathbb{B})\left[v \models \Gamma \to v \models \varphi\right].$$

Remark (Definition predicate reading)*.*

$$\text{LogicalConsequence}(\Gamma, \varphi)$$

means $\Gamma \models \varphi$.

Remark (Negated quantified statement)*.*

$$\Gamma \not\models \varphi \iff \exists v : \mathsf{Prop} \to \mathbb{B}\left[v \models \Gamma \wedge v \not\models \varphi\right].$$

Remark (Interpretation)*.* Logical consequence means truth preservation from premises to conclusion. There is no truth assignment under which all premises are true and the conclusion is false.

Remark (Dependencies).

- Satisfaction of a Set of Formulas
- Satisfaction of a Formula

Definition (Logical Equivalence)

Definition 1.3.13 (Logical Equivalence). Formulas $\varphi, \psi \in \mathsf{WFF}$ are logically equivalent, written

$$\varphi \equiv \psi,$$

if they have the same truth value under every truth assignment.

Remark (Standard quantified statement).

$$\varphi \equiv \psi \iff \varphi, \psi \in \mathsf{WFF} \land (\forall v : \mathsf{Prop} \to \mathbb{B}) \left[\widehat{v}(\varphi) = \widehat{v}(\psi)\right].$$

Remark (Definition predicate reading).

$$\text{LogicalEquivalence}(\varphi, \psi)$$

means $\varphi \equiv \psi$.

Remark (Interpretation). Logically equivalent formulas have identical truth-table columns. They are interchangeable for semantic purposes.

Remark (Dependencies).

- Unique Extension of a Truth Assignment
- Truth Assignment

Lemma (Coincidence Lemma for Propositional Logic)

Lemma 1.3.14 (Coincidence Lemma for Propositional Logic). *Let $v, w : \mathsf{Prop} \to \mathbb{B}$ be truth assignments. If v and w agree on every propositional variable occurring in φ, then they assign the same truth value to φ:*

$$v|_{\mathrm{Var}(\varphi)} = w|_{\mathrm{Var}(\varphi)} \implies \widehat{v}(\varphi) = \widehat{w}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})(\forall v, w : \mathsf{Prop} \to \mathbb{B}) \left[(\forall P \in \mathrm{Var}(\varphi))(v(P) = w(P)) \to \widehat{v}(\varphi) = \widehat{w}(\varphi)\right].$$

Remark (Interpretation). The truth value of a formula depends only on the variables actually occurring in that formula.

Remark (Dependencies).

- Variable Set of a Formula
- Unique Extension of a Truth Assignment
- Structural Induction for Propositional Formulas

Theorem (Finite Truth-Table Theorem)

Theorem 1.3.15 (Finite Truth-Table Theorem). *Every propositional formula has a finite truth table. More precisely, if* $\mathrm{Var}(\varphi)$ *has* n *elements, then* φ *has a truth table with* 2^n *rows.*
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})\,[|\,\mathrm{Var}(\varphi)| = n \to \mathrm{TruthTable}(\varphi)\ \texttt{has}\ 2^n\ \texttt{rows}]\,.$$

Remark (Interpretation). Truth tables are finite because every formula contains only finitely many propositional variables.

Remark (Dependencies).

- Truth Table
- Finiteness of Variables in a Formula

Definition (Boolean Function)

Definition 1.3.16 (Boolean Function). An n-ary Boolean function is a function

$$f : \mathbb{B}^n \to \mathbb{B}.$$

Remark (Standard quantified statement).

$$BooleanFunction_n(f) \iff f : \mathbb{B}^n \to \mathbb{B}.$$

Remark (Definition predicate reading).

$$BooleanFunction_n(f)$$

means that f takes n truth values as input and returns one truth value.

Remark (Interpretation). Boolean functions are the abstract truth-functions that formulas may represent.

Remark (Dependencies).

- Truth Value

Definition (Truth Function Induced by a Formula)

Definition 1.3.17 (Truth Function Induced by a Formula). Let $\varphi \in \mathsf{WFF}$, and suppose $\mathrm{Var}(\varphi) \subseteq \{P_1, \ldots, P_n\}$. The truth function induced by φ is the Boolean function

$$f_\varphi : \mathbb{B}^n \to \mathbb{B}$$

defined by

$$f_\varphi(b_1, \ldots, b_n) = \widehat{v}(\varphi),$$

where $v(P_i) = b_i$ for each i.

Remark (Standard quantified statement).

$$f_\varphi(b_1,\ldots,b_n)=\widehat{v}(\varphi)\quad\text{whenever } v(P_i)=b_i \text{ for } 1\le i\le n.$$

Remark (Definition predicate reading).

$$\mathrm{TruthFunctionOfFormula}(f_\varphi,\varphi)$$

means that f_φ records the truth behavior of φ.

Remark (Interpretation). The induced truth function packages the truth table of a formula as a Boolean function.

Remark (Dependencies).

- Boolean Function
- Variable Set of a Formula
- Unique Extension of a Truth Assignment
- Coincidence Lemma for Propositional Logic

Theorem (Counting Boolean Functions)

Theorem 1.3.18 (Counting Boolean Functions). *There are exactly*

$$2^{2^n}$$

Boolean functions $\mathbb{B}^n\to\mathbb{B}$.
Go to proof.

Remark (Standard quantified statement).

$$|\{f:\mathbb{B}^n\to\mathbb{B}\}|=2^{2^n}.$$

Remark (Interpretation). The domain $\mathbb{B}^n$ has 2^n elements. A Boolean function chooses one of two truth values for each input row.

Remark (Dependencies).

- Boolean Function

1.4 Algebra of Propositions

Toolkit

This section uses logical equivalence, equivalence laws, formula contexts, and replacement of logical equivalents to treat propositions algebraically.

Remark (Exposition). The algebra section treats logically equivalent formulas as interchangeable for calculation. Equivalence laws and replacement principles turn semantic agreement under every truth assignment into a controlled rewrite calculus for propositional formulas.

Definition (Propositional Equivalence Law)

Definition 1.4.1 (Propositional Equivalence Law). A propositional equivalence law is a valid schema of the form

$$\varphi \equiv \psi,$$

with $\varphi, \psi \in \mathsf{WFF}$. Each instance of the schema says that the two formulas have the same truth value under every truth assignment.

Remark (Standard quantified statement).

$$\text{PropositionalEquivalenceLaw}(\varphi, \psi) \iff \varphi, \psi \in \mathsf{WFF} \land \varphi \equiv \psi.$$

Remark (Definition predicate reading).

$$\text{PropositionalEquivalenceLaw}(\varphi, \psi)$$

means that $\varphi \equiv \psi$ is a valid logical-equivalence schema.

Remark (Interpretation). Equivalence laws are the algebraic rewrite rules of propositional logic. They allow formulas to be transformed without changing truth conditions.

Remark (Dependencies).

- Logical Equivalence
- Well-Formed Formula

Theorem (Logical Equivalence is an Equivalence Relation)

Theorem 1.4.2 (Logical Equivalence is an Equivalence Relation). *Logical equivalence is reflexive, symmetric, and transitive on* WFF*:*

$$\varphi \equiv \varphi, \qquad \varphi \equiv \psi \Longrightarrow \psi \equiv \varphi,$$

and

$$(\varphi \equiv \psi \land \psi \equiv \chi) \Longrightarrow \varphi \equiv \chi.$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned}
&(\forall \varphi \in \mathsf{WFF})(\varphi \equiv \varphi),\\
&(\forall \varphi, \psi \in \mathsf{WFF})(\varphi \equiv \psi \to \psi \equiv \varphi),\\
&(\forall \varphi, \psi, \chi \in \mathsf{WFF})((\varphi \equiv \psi \land \psi \equiv \chi) \to \varphi \equiv \chi).
\end{aligned}$$

Remark (Interpretation). Logical equivalence behaves like equality at the semantic level.

Remark (Dependencies).

- Logical Equivalence

Theorem (Basic Boolean Equivalence Laws)

Theorem 1.4.3 (Basic Boolean Equivalence Laws). *For all formulas $\varphi, \psi, \chi \in \mathsf{WFF}$, the following logical equivalences hold.*
Commutativity:

$$\varphi \wedge \psi \equiv \psi \wedge \varphi, \qquad \varphi \vee \psi \equiv \psi \vee \varphi.$$

Associativity:

$$(\varphi \wedge \psi) \wedge \chi \equiv \varphi \wedge (\psi \wedge \chi), \qquad (\varphi \vee \psi) \vee \chi \equiv \varphi \vee (\psi \vee \chi).$$

Idempotence:

$$\varphi \wedge \varphi \equiv \varphi, \qquad \varphi \vee \varphi \equiv \varphi.$$

Double negation:

$$\neg\neg\varphi \equiv \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi \in \mathsf{WFF}) \begin{bmatrix} \varphi \wedge \psi \equiv \psi \wedge \varphi, \quad \varphi \vee \psi \equiv \psi \vee \varphi, \\ (\varphi \wedge \psi) \wedge \chi \equiv \varphi \wedge (\psi \wedge \chi), \quad (\varphi \vee \psi) \vee \chi \equiv \varphi \vee (\psi \vee \chi), \\ \varphi \wedge \varphi \equiv \varphi, \quad \varphi \vee \varphi \equiv \varphi, \quad \neg\neg\varphi \equiv \varphi \end{bmatrix}.$$

Remark (Interpretation). These are the basic algebraic laws for conjunction, disjunction, and negation.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Theorem (Identity and Domination Laws)

Theorem 1.4.4 (Identity and Domination Laws). *Let $\top$ be any tautological formula and let $\bot$ be any contradictory formula. Then for every $\varphi \in \mathsf{WFF}$:*

$$\varphi \wedge \top \equiv \varphi, \qquad \varphi \vee \bot \equiv \varphi,$$

and

$$\varphi \wedge \bot \equiv \bot, \qquad \varphi \vee \top \equiv \top.$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} &(\mathrm{Tautology}(\top) \wedge \mathrm{Contradiction}(\bot)) \to (\forall \varphi \in \mathsf{WFF}) \\ &\quad [(\varphi \wedge \top \equiv \varphi) \wedge (\varphi \vee \bot \equiv \varphi) \wedge (\varphi \wedge \bot \equiv \bot) \wedge (\varphi \vee \top \equiv \top)]. \end{aligned}$$

Remark (Interpretation). Tautologies act like true constants; contradictions act like false constants.

Remark (Dependencies).

- Tautology
- Contradiction
- Logical Equivalence

Theorem (De Morgan Laws for Propositional Logic)

Theorem 1.4.5 (De Morgan Laws for Propositional Logic). *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi,$$

and

$$\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\left[\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi\right],$$
$$(\forall \varphi, \psi \in \mathsf{WFF})\left[\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi\right].$$

Remark (Predicate reading).

$$\underbrace{\neg(\varphi \wedge \psi)}_{\texttt{not both}} \equiv \underbrace{\neg\varphi \vee \neg\psi}_{\texttt{at least one fails}}, \qquad \underbrace{\neg(\varphi \vee \psi)}_{\texttt{neither}} \equiv \underbrace{\neg\varphi \wedge \neg\psi}_{\texttt{both fail}}.$$

Remark (Interpretation). Negation swaps conjunction with disjunction when pushed inward.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Theorem (Distributive Laws for Propositional Logic)

Theorem 1.4.6 (Distributive Laws for Propositional Logic). *For all* $\varphi, \psi, \chi \in \mathsf{WFF}$*,*

$$\varphi \wedge (\psi \vee \chi) \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi),$$

and

$$\varphi \vee (\psi \wedge \chi) \equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi \in \mathsf{WFF})\left[\varphi \wedge (\psi \vee \chi) \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi)\right],$$
$$(\forall \varphi, \psi, \chi \in \mathsf{WFF})\left[\varphi \vee (\psi \wedge \chi) \equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi)\right].$$

Remark (Interpretation). Conjunction distributes over disjunction, and disjunction distributes over conjunction. These laws are the main algebraic engine behind CNF and DNF conversion.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Theorem (Absorption Laws)

Theorem 1.4.7 (Absorption Laws). *For all* $\varphi, \psi \in \mathsf{WFF}$,

$$\varphi \wedge (\varphi \vee \psi) \equiv \varphi,$$

and

$$\varphi \vee (\varphi \wedge \psi) \equiv \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\, [\varphi \wedge (\varphi \vee \psi) \equiv \varphi]\,,$$
$$(\forall \varphi, \psi \in \mathsf{WFF})\, [\varphi \vee (\varphi \wedge \psi) \equiv \varphi]\,.$$

Remark (Interpretation). Once φ is already required, adding $\varphi \vee \psi$ contributes no additional constraint. Dually, once φ is already allowed, adding $\varphi \wedge \psi$ contributes no additional possibility.

Remark (Dependencies).

- Logical Equivalence
- Basic Boolean Equivalence Laws
- Distributive Laws for Propositional Logic

Theorem (Conditional Equivalence Laws)

Theorem 1.4.8 (Conditional Equivalence Laws). *For all* $\varphi, \psi \in \mathsf{WFF}$,

$$\varphi \to \psi \equiv \neg\varphi \vee \psi,$$

and

$$\neg(\varphi \to \psi) \equiv \varphi \wedge \neg\psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\, [\varphi \to \psi \equiv \neg\varphi \vee \psi]\,,$$
$$(\forall \varphi, \psi \in \mathsf{WFF})\, [\neg(\varphi \to \psi) \equiv \varphi \wedge \neg\psi]\,.$$

Remark (Predicate reading).

$$\underbrace{\varphi \to \psi}_{\texttt{no counterexample}} \equiv \underbrace{\neg\varphi \vee \psi}_{\texttt{premise false or conclusion true}}, \quad \underbrace{\neg(\varphi \to \psi)}_{\texttt{counterexample}} \equiv \underbrace{\varphi \wedge \neg\psi}_{\texttt{premise true and conclusion false}}.$$

Remark (Interpretation). A conditional fails exactly when the antecedent is true and the consequent is false.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Theorem (Biconditional Equivalence Laws)

Theorem 1.4.9 (Biconditional Equivalence Laws). *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi),$$

and

$$\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\left[\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi)\right],$$
$$(\forall \varphi, \psi \in \mathsf{WFF})\left[\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi)\right].$$

Remark (Predicate reading).

$$\varphi \leftrightarrow \psi \quad \Longleftrightarrow \quad (\varphi \to \psi) \wedge (\psi \to \varphi)$$

and also

$$\varphi \leftrightarrow \psi \quad \Longleftrightarrow \quad (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

The biconditional says that the two formulas have the same truth value.

Remark (Interpretation). A biconditional says that two formulas have the same truth value: both true or both false.

Remark (Dependencies).

- Logical Equivalence
- Conditional Equivalence Laws

Definition (Formula Context)

Definition 1.4.10 (Formula Context). A formula context $C[-]$ is a well-formed formula with one distinguished placeholder occurrence $[-]$. For each well-formed formula φ, the expression $C[\varphi]$ denotes the formula obtained by replacing the placeholder with φ.

Remark (Standard quantified statement).

$$\mathrm{FormulaContext}(C[-]) \Longrightarrow (\forall \varphi \in \mathsf{WFF})(C[\varphi] \in \mathsf{WFF}).$$

Remark (Definition predicate reading).

$$\text{FormulaContext}(C[-])$$

means that $C[-]$ is a one-hole propositional formula context.

Remark (Interpretation). A formula context is a syntactic environment into which a formula can be inserted. Contexts make precise the idea of replacing a subformula inside a larger formula.

Remark (Examples). If

$$C[-] := ([-] \wedge R),$$

then

$$C[P] = (P \wedge R)$$

and

$$C[Q \vee S] = ((Q \vee S) \wedge R).$$

Remark (Dependencies).

- Well-Formed Formula
- Subformula

Theorem (Replacement of Logical Equivalents)

Theorem 1.4.11 (Replacement of Logical Equivalents). *Replacing logically equivalent formulas inside a formula context preserves logical equivalence:*

$$\varphi \equiv \psi \Longrightarrow C[\varphi] \equiv C[\psi].$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})(\forall C[-])\left[\text{FormulaContext}(C[-]) \wedge \varphi \equiv \psi \to C[\varphi] \equiv C[\psi]\right].$$

Remark (Predicate reading).

$$\varphi \equiv \psi \quad \Longrightarrow \quad C[\varphi] \equiv C[\psi].$$

A logical equivalence may be used inside a larger formula context without changing the truth behavior of the whole formula.

Remark (Interpretation). Logical equivalence is substitutive: equivalent formulas may be replaced inside larger formulas without changing truth behavior.

Remark (Exposition). The equivalence laws in this section may be used as replacement rules inside larger formulas. If a formula contains a subformula matching one side of a listed equivalence law, that subformula may be replaced by the other side.

Common replacement rules include:

$$\neg\neg\varphi \equiv \varphi,$$

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi, \qquad \neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi,$$
$$\varphi \to \psi \equiv \neg\varphi \vee \psi,$$
$$\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi),$$

and

$$\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

Remark (Exposition). The laws in this section allow formulas to be manipulated algebraically. They are not new formation rules; they are semantic equivalences. Later sections use these laws to transform formulas into normal forms and to reduce the set of primitive connectives.

Remark (Exposition). The algebra of propositions is the bridge from semantics to computation: once formulas are equivalent, they can be rewritten without changing truth behavior.

Remark (Dependencies).

- Formula Context
- Logical Equivalence
- Coincidence Lemma for Propositional Logic
- Unique Extension of a Truth Assignment
- Replacement of Logical Equivalents

1.5 Normal Forms

Toolkit

This section uses logical equivalence, truth assignments, literals, clauses, and algebraic rewrite laws to motivate normal forms.

Remark (Exposition). The normal-forms section studies controlled syntactic shapes for formulas. Logical equivalence is the governing invariant: conversion may change a formula's shape, but it must preserve truth under every assignment while exposing regular literal, clause, DNF, CNF, and canonical structure.

Orientation

Remark (Section map). This section uses the preceding syntax, semantics, and algebra of propositions as its starting point. It first isolates literals and clauses, then introduces negation normal form, disjunctive normal form, conjunctive normal form, canonical normal forms, and semantic classification uses. Later arguments will use these forms to organize satisfiability questions, proof search, functional completeness, and resolution-style methods.

Remark (Why normal forms matter). Normal forms trade arbitrary expression for regular structure. In a truth-table argument, regular structure helps classify formulas as tautologies, contradictions, satisfiable formulas, or contingencies. In proof search and satisfiability procedures, regular structure turns a formula into a finite combinatorial object that can be searched. In functional completeness, normal forms show how complicated truth functions can be built from simpler connectives.

Definition (Literal)

Definition 1.5.1 (Literal). A *literal* is either a propositional variable P or the negation $\neg P$ of a propositional variable P.

Remark (Standard quantified statement). For a formula $\ell \in \mathsf{WFF}$,

$$\ell \text{ is a literal} \iff (\exists P \in \mathsf{Prop})(\ell = P \text{ or } \ell = \neg P).$$

Remark (Definition predicate reading).

$$\text{Literal}(\ell)$$

means that ℓ is an atomic propositional variable or the negation of one atomic propositional variable.

Remark (Interpretation). Literals are the indivisible signed atoms used in normal forms. They remember which propositional variable is being tested and whether it appears positively or negatively.

Remark (Examples). The formulas P, Q, $\neg P$, and $\neg Q$ are literals.

Remark (Non-Examples). The formulas $P \wedge Q$, $P \vee Q$, and $\neg(P \wedge Q)$ are not literals. The last formula is negated, but it negates a compound formula rather than a single propositional variable.

Remark (Dependencies).

- Propositional Variable
- Logical Connective

Definition (Positive Literal)

Definition 1.5.2 (Positive Literal). A *positive literal* is a literal of the form P, where P is a propositional variable.

Remark (Standard quantified statement). For a formula $\ell \in \mathsf{WFF}$,

$$\ell \text{ is a positive literal} \iff (\exists P \in \mathsf{Prop})(\ell = P).$$

Remark (Definition predicate reading).

$$\text{PositiveLiteral}(\ell)$$

means that ℓ is a literal whose sign is positive.

Remark (Interpretation). A positive literal asserts the propositional variable directly. It carries no leading negation.

Remark (Examples). The formulas P and Q are positive literals.

Remark (Dependencies).

- Literal

Definition (Negative Literal)

Definition 1.5.3 (Negative Literal). A *negative literal* is a literal of the form $\neg P$, where P is a propositional variable.

Remark (Standard quantified statement). For a formula $\ell \in \mathsf{WFF}$,

$$\ell \text{ is a negative literal} \iff (\exists P \in \mathsf{Prop})(\ell = \neg P).$$

Remark (Definition predicate reading).

$$\text{NegativeLiteral}(\ell)$$

means that ℓ is a literal whose sign is negative.

Remark (Interpretation). A negative literal denies exactly one propositional variable. In normal forms, negation is pushed down until it appears only at this literal level.

Remark (Examples). The formulas $\neg P$ and $\neg Q$ are negative literals.

Remark (Dependencies).

- Literal

Definition (Complementary Literals)

Definition 1.5.4 (Complementary Literals). Two literals are *complementary* if they are P and $\neg P$ for the same propositional variable P.

Remark (Standard quantified statement). For literals ℓ_1, ℓ_2,

$$\ell_1 \text{ and } \ell_2 \text{ are complementary} \iff (\exists P \in \mathsf{Prop})\big((\ell_1 = P \wedge \ell_2 = \neg P) \vee (\ell_1 = \neg P \wedge \ell_2 = P)\big).$$

Remark (Definition predicate reading).

$$\text{ComplementaryLiterals}(\ell_1, \ell_2)$$

means that the two literals name the same propositional variable with opposite signs.

Remark (Interpretation). Complementary literals are the local contradiction pattern inside clauses and terms. The pair $P, \neg P$ is tied to one variable, not merely to two formulas with opposite truth values under a particular assignment.

Remark (Examples). The literals P and $\neg P$ are complementary, as are Q and $\neg Q$.

Remark (Non-Examples). The literals P and $\neg Q$ are not complementary unless $P = Q$.

Remark (Dependencies).

- Literal

Definition (Clause)

Definition 1.5.5 (Clause). A *clause* is a finite disjunction of literals.

Remark (Standard quantified statement). A formula $C \in \mathsf{WFF}$ is a clause if there are literals $\ell_1, \ldots, \ell_n$ with $n \geq 1$ such that

$$C = \ell_1 \vee \cdots \vee \ell_n.$$

The empty clause is treated separately as the empty disjunction.

Remark (Definition predicate reading).

$$\mathrm{Clause}(C)$$

means that C is built by finitely disjoining literals.

Remark (Interpretation). A clause records a finite list of literal-level alternatives. It is satisfied when at least one of its literals is true.

Remark (Examples). The formulas $P \vee Q$, $P \vee \neg Q$, and $\neg P \vee Q \vee R$ are clauses.

Remark (Non-Examples). The formula $P \wedge Q$ is not a clause, because its top-level connective is a conjunction. The formula $\neg(P \vee Q)$ is not a clause, because its negation has not been pushed down to the literal level.

Remark (Dependencies).

- Literal

Definition (Term)

Definition 1.5.6 (Term). A *term*, also called a *conjunction term*, is a finite conjunction of literals.

Remark (Standard quantified statement). A formula $T \in \mathsf{WFF}$ is a term if there are literals $\ell_1, \ldots, \ell_n$ with $n \geq 1$ such that

$$T = \ell_1 \wedge \cdots \wedge \ell_n.$$

The empty conjunction is treated separately.

Remark (Definition predicate reading).

$$\mathrm{Term}(T)$$

means that T is built by finitely conjoining literals.

Remark (Interpretation). A term records a finite list of literal-level requirements. It is satisfied when all of its literals are true.

Remark (Examples). The formulas $P \wedge Q$, $P \wedge \neg Q$, and $\neg P \wedge Q \wedge R$ are terms.

Remark (Non-Examples). The formula $P \vee Q$ is not a term, because its top-level connective is a disjunction. The formula $\neg(P \wedge Q)$ is not a term, because its negation has not been pushed down to the literal level.

Remark (Dependencies).

- Literal

Definition (Empty Clause)

Definition 1.5.7 (Empty Clause). The *empty clause* is the empty disjunction of literals. Semantically, it is read as false.

Remark (Standard quantified statement). The empty clause is the disjunction over no literals and has truth value False under every truth assignment.

Remark (Definition predicate reading).

$$\mathrm{EmptyClause}(C)$$

means that C is the clause with no disjuncts.

Remark (Interpretation). A nonempty clause is true when at least one listed literal is true. With no literals listed, there is no witness to make the disjunction true, so the empty clause is the false boundary case for clauses.

Remark (Dependencies).

- Clause
- Contradiction

Definition (Empty Conjunction)

Definition 1.5.8 (Empty Conjunction). The *empty conjunction* is the conjunction over no literals. Semantically, it is read as true.

Remark (Standard quantified statement). The empty conjunction is the conjunction over no literals and has truth value True under every truth assignment.

Remark (Definition predicate reading).

$$\mathrm{EmptyConjunction}(T)$$

means that T is the term with no conjuncts.

Remark (Interpretation). A nonempty term is true when every listed literal is true. With no literals listed, there is no requirement to violate, so the empty conjunction is the true boundary case for terms.

Remark (Dependencies).

- Term
- Tautology

Definition (Negation Normal Form)

Definition 1.5.9 (Negation Normal Form). A formula is in *negation normal form* if it uses only propositional variables, negated propositional variables, conjunction, and disjunction, and every negation occurs immediately before a propositional variable.

Remark (Standard quantified statement). For a formula $\varphi \in \mathsf{WFF}$,

$$\varphi \text{ is in negation normal form} \iff \varphi \text{ is built from literals using only } \wedge \text{ and } \vee.$$

Remark (Definition predicate reading).

$$\text{NegationNormalForm}(\varphi)$$

means that φ has no conditionals, no biconditionals, and no negation outside the literal level.

Remark (Interpretation). Negation normal form is the first cleanup stage for propositional formulas. It does not force a formula to be a disjunction of terms or a conjunction of clauses; it only restricts the available connectives and pushes negation down to propositional variables.

Remark (Examples). The formulas P, $\neg P$, $P \wedge (\neg Q \vee R)$, and $(P \vee Q) \wedge (\neg R \vee S)$ are in negation normal form.

Remark (Non-Examples). The formulas $P \to Q$, $P \leftrightarrow Q$, and $\neg(P \wedge Q)$ are not in negation normal form. The first two use connectives not allowed in negation normal form, and the last has a negation applied to a compound formula.

Remark (Dependencies).

- Literal
- Well-Formed Formula
- Logical Equivalence

Definition (NNF Conversion Procedure)

Definition 1.5.10 (NNF Conversion Procedure). The *NNF conversion procedure* converts a propositional formula to an equivalent formula in negation normal form by the following steps:

1. eliminate biconditionals using biconditional equivalence laws;
2. eliminate conditionals using conditional equivalence laws;
3. eliminate double negations;
4. push negations inward using De Morgan laws.

Remark (Standard quantified statement). Starting with $\varphi \in \mathsf{WFF}$, repeatedly apply the equivalences

$$\alpha \leftrightarrow \beta \equiv (\alpha \to \beta) \wedge (\beta \to \alpha), \qquad \alpha \to \beta \equiv \neg\alpha \vee \beta,$$

$$\neg\neg\alpha \equiv \alpha, \qquad \neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta, \qquad \neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta,$$

until negations occur only immediately before propositional variables and only $\wedge$ and $\vee$ remain above the literal level.

Remark (Definition predicate reading).

$$\text{NNFConversionProcedure}(\varphi, \psi)$$

means that ψ is obtained from φ by eliminating biconditionals and conditionals, removing double negations, and pushing remaining negations inward.

Remark (Interpretation). The procedure is a syntax-directed rewrite process. Each rewrite step is backed by a logical equivalence law, so the output formula has the same truth conditions as the input formula.

Remark (Dependencies).

- Conditional Equivalence Laws
- Biconditional Equivalence Laws
- De Morgan Laws for Propositional Logic
- Basic Boolean Equivalence Laws

Theorem (Existence of Negation Normal Form)

Theorem 1.5.11 (Existence of Negation Normal Form). *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in negation normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})(\exists \psi \in \mathsf{WFF})\big(\psi \text{ is in negation normal form} \wedge \varphi \equiv \psi\big).$$

Remark (Predicate reading).

$$\text{NegationNormalFormExists}(\varphi)$$

means that φ has a logically equivalent representative in negation normal form.

Remark (Interpretation). Every propositional formula can be rewritten so that negation appears only at the variable level. This makes negation normal form a semantic normalizing step: it changes shape while preserving logical equivalence.

Remark (Dependencies).

- Negation Normal Form
- NNF Conversion Procedure
- Replacement of Logical Equivalents

- Structural Induction for Propositional Formulas

Definition (Elementary Conjunction)

Definition 1.5.12 (Elementary Conjunction). An *elementary conjunction* is a finite conjunction of literals. That is, it is a term of the form

$$L_1 \wedge L_2 \wedge \cdots \wedge L_k,$$

where each L_i is a literal. The one-literal case is permitted.

Remark (Standard quantified statement).

$$L_1 \wedge L_2 \wedge \cdots \wedge L_k, \qquad L_i \text{ is a literal for each } i.$$

Remark (Interpretation). Elementary conjunctions are the conjunction terms used as the building blocks of disjunctive normal form. They record simultaneous literal requirements.

Example (Examples).

$$P, \qquad P \wedge \neg Q, \qquad \neg P \wedge R \wedge S$$

are elementary conjunctions.

Remark (Dependencies).

- Term
- Literal

Definition (Minterm)

Definition 1.5.13 (Minterm). Let $P_1, \ldots, P_n$ be a finite list of propositional variables. A *minterm* relative to $P_1, \ldots, P_n$ is an elementary conjunction that contains exactly one of P_i or $\neg P_i$ for each $i = 1, \ldots, n$.

Remark (Standard quantified statement).

$$M = L_1 \wedge \cdots \wedge L_n,$$

where for each i, exactly one of P_i or $\neg P_i$ appears among the literals $L_1, \ldots, L_n$.

Remark (Interpretation). A minterm relative to $P_1, \ldots, P_n$ specifies one complete truth-value assignment on those variables: choosing P_i requires P_i to be true, and choosing $\neg P_i$ requires P_i to be false.

Example (Examples). Relative to P, Q, R, the formula

$$P \wedge \neg Q \wedge R$$

is a minterm. Relative to P, Q, R, the formula $P \wedge R$ is not a minterm because it omits both Q and $\neg Q$.

Remark (Dependencies).

- Elementary Conjunction
- Variable Set of a Formula

Definition (Disjunctive Normal Form)

Definition 1.5.14 (Disjunctive Normal Form). A formula is in *disjunctive normal form*, abbreviated *DNF*, when it is a finite disjunction of elementary conjunctions. Equivalently, it has the shape

$$C_1 \vee C_2 \vee \cdots \vee C_m,$$

where each C_j is an elementary conjunction.

Remark (Standard quantified statement).

$$\varphi \text{ is in DNF} \iff \varphi = C_1 \vee C_2 \vee \cdots \vee C_m$$

for finitely many elementary conjunctions $C_1, \ldots, C_m$.

Remark (Interpretation). DNF is a syntactic form, but it is used semantically: once a formula is replaced by a logically equivalent DNF formula, questions about satisfiability reduce to questions about individual conjunction terms.

Example (Examples).

$$(P \wedge Q) \vee (\neg P \wedge R) \vee S$$

is in DNF because each disjunct is an elementary conjunction.

Remark (Dependencies).

- Elementary Conjunction
- Logical Equivalence
- Satisfiable Formula

Theorem (Existence of Disjunctive Normal Form)

Theorem 1.5.15 (Existence of Disjunctive Normal Form). *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in disjunctive normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})(\exists \psi \in \mathsf{WFF})\big(\psi \text{ is in DNF} \wedge \varphi \equiv \psi\big).$$

Remark (Interpretation). One first converts φ to negation normal form, then repeatedly applies distributive laws to move conjunctions below disjunctions. Replacement of logical equivalents preserves equivalence at each step.

Remark (Dependencies).

- Disjunctive Normal Form

- Negation Normal Form
- Distributive Laws for Propositional Logic
- Replacement of Logical Equivalents
- Existence of Negation Normal Form

Theorem (DNF Satisfiability Criterion)

Theorem 1.5.16 (DNF Satisfiability Criterion). *Let ψ be a formula in disjunctive normal form. Then ψ is satisfiable if and only if at least one conjunction term in ψ contains no complementary pair of literals.* *Go to proof.*

Remark (Standard quantified statement).

ψ is a satisfiable DNF formula $\iff$ some conjunction term of ψ has no complementary li

Remark (Interpretation). A DNF formula is true exactly when at least one of its conjunction terms is true. A conjunction term can be made true exactly when it does not require both a variable and its negation.

Remark (Dependencies).

- Disjunctive Normal Form
- Complementary Literals
- Satisfiable Formula

Definition (Elementary Disjunction)

Definition 1.5.17 (Elementary Disjunction). An *elementary disjunction* is a finite disjunction of literals. That is, it is a clause of the form

$$L_1 \vee L_2 \vee \cdots \vee L_k,$$

where each L_i is a literal. The one-literal case is permitted.

Remark (Standard quantified statement).

$$L_1 \vee L_2 \vee \cdots \vee L_k, \qquad L_i \text{ is a literal for each } i.$$

Remark (Interpretation). Elementary disjunctions are the clause-level building blocks used in conjunctive normal form. They record finite literal alternatives.

Example (Examples).

$$P, \qquad P \vee \neg Q, \qquad \neg P \vee R \vee S$$

are elementary disjunctions.

Remark (Dependencies).

- Clause

- Literal

Definition (Maxterm)

Definition 1.5.18 (Maxterm). Let $P_1, \ldots, P_n$ be a finite list of propositional variables. A *maxterm* relative to $P_1, \ldots, P_n$ is an elementary disjunction that contains exactly one of P_i or $\neg P_i$ for each $i = 1, \ldots, n$.

Remark (Standard quantified statement).

$$M = L_1 \vee \cdots \vee L_n,$$

where for each i, exactly one of P_i or $\neg P_i$ appears among the literals $L_1, \ldots, L_n$.

Remark (Interpretation). A maxterm relative to $P_1, \ldots, P_n$ specifies one literal-level alternative for every variable in the list. It is false under exactly the assignment that makes each listed literal false.

Example (Examples). Relative to P, Q, R, the formula

$$P \vee \neg Q \vee R$$

is a maxterm. Relative to P, Q, R, the formula $P \vee R$ is not a maxterm because it omits both Q and $\neg Q$.

Remark (Dependencies).

- Elementary Disjunction
- Variable Set of a Formula

Definition (Conjunctive Normal Form)

Definition 1.5.19 (Conjunctive Normal Form). A formula is in *conjunctive normal form*, abbreviated *CNF*, when it is a finite conjunction of clauses. Equivalently, it has the shape

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each C_j is a clause.

Remark (Standard quantified statement).

$$\varphi \text{ is in CNF} \iff \varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

for finitely many clauses $C_1, \ldots, C_m$.

Remark (Interpretation). CNF is a syntactic form used to expose clause-by-clause constraints. Replacing a formula by a logically equivalent CNF formula turns global validity and contradiction questions into questions about the interaction of its clauses.

Example (Examples).

$$(P \vee Q) \wedge (\neg P \vee R) \wedge S$$

is in CNF because each conjunct is a clause.

Remark (Dependencies).

- Clause
- Logical Equivalence
- Contradiction

Theorem (Existence of Conjunctive Normal Form)

Theorem 1.5.20 (Existence of Conjunctive Normal Form). *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in conjunctive normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})(\exists \psi \in \mathsf{WFF})\big(\psi \text{ is in CNF} \wedge \varphi \equiv \psi\big).$$

Remark (Interpretation). One first converts φ to negation normal form, then repeatedly applies distributive laws to move disjunctions below conjunctions. Replacement of logical equivalents preserves equivalence through every local rewrite.

Remark (Dependencies).

- Conjunctive Normal Form
- Negation Normal Form
- Distributive Laws for Propositional Logic
- Replacement of Logical Equivalents
- Existence of Negation Normal Form

Remark (Equivalence-preserving conversion). Normal form conversion is a rewrite process that replaces a formula by a logically equivalent formula with a prescribed syntactic shape. Each local rewrite is justified by an equivalence law, and each replacement inside a larger formula is justified by Replacement of Logical Equivalents.

The goal is not to preserve the exact parse tree of the original formula. The goal is to preserve truth value under every valuation while moving the formula toward negation normal form, disjunctive normal form, or conjunctive normal form.

Remark (Step 1: eliminate biconditionals). Replace every biconditional by an equivalent formula using the Biconditional Equivalence Laws. A standard replacement is

$$\alpha \leftrightarrow \beta \quad \rightsquigarrow \quad (\alpha \to \beta) \wedge (\beta \to \alpha).$$

After this step, no occurrence of $\leftrightarrow$ remains.

Remark (Step 2: eliminate conditionals). Replace every conditional by an equivalent disjunction using the Conditional Equivalence Laws. The standard replacement is

$$\alpha \to \beta \quad \rightsquigarrow \quad \neg\alpha \vee \beta.$$

After this step, the only connectives still present are $\neg$, $\wedge$, and $\vee$.

Remark (Step 3: push negations inward). Use De Morgan Laws for Propositional Logic and double-negation simplification to move every negation down to the literal level:

$$\neg(\alpha \wedge \beta) \rightsquigarrow \neg\alpha \vee \neg\beta, \qquad \neg(\alpha \vee \beta) \rightsquigarrow \neg\alpha \wedge \neg\beta, \qquad \neg\neg\alpha \rightsquigarrow \alpha.$$

When this step terminates, the result is in negation normal form: negations occur only immediately in front of propositional variables.

Remark (Step 4A: distribute for DNF). To obtain DNF from negation normal form, distribute conjunction over disjunction using the Distributive Laws for Propositional Logic:

$$\alpha \wedge (\beta \vee \gamma) \quad \rightsquigarrow \quad (\alpha \wedge \beta) \vee (\alpha \wedge \gamma),$$

and similarly

$$(\beta \vee \gamma) \wedge \alpha \quad \rightsquigarrow \quad (\beta \wedge \alpha) \vee (\gamma \wedge \alpha).$$

Repeatedly applying these rewrites moves disjunctions outward until the formula is a finite disjunction of conjunction terms.

Remark (Step 4B: distribute for CNF). To obtain CNF from negation normal form, distribute disjunction over conjunction:

$$\alpha \vee (\beta \wedge \gamma) \quad \rightsquigarrow \quad (\alpha \vee \beta) \wedge (\alpha \vee \gamma),$$

and similarly

$$(\beta \wedge \gamma) \vee \alpha \quad \rightsquigarrow \quad (\beta \vee \alpha) \wedge (\gamma \vee \alpha).$$

Repeatedly applying these rewrites moves conjunctions outward until the formula is a finite conjunction of clauses.

Remark (DNF conversion workflow). To convert a formula to DNF:

1. eliminate biconditionals;
2. eliminate conditionals;
3. push negations inward to obtain negation normal form;
4. distribute conjunction over disjunction until the result is a finite disjunction of elementary conjunctions.

The resulting formula is logically equivalent to the original formula, but it need not be the shortest or only possible DNF representation.

Remark (CNF conversion workflow). To convert a formula to CNF:

1. eliminate biconditionals;
2. eliminate conditionals;

3. push negations inward to obtain negation normal form;
4. distribute disjunction over conjunction until the result is a finite conjunction of clauses.

The resulting formula is logically equivalent to the original formula, but it need not be the shortest or only possible CNF representation.

Remark (Termination). The first three stages terminate because they remove or strictly simplify specific connective patterns. Eliminating biconditionals reduces the number of $\leftrightarrow$ symbols. Eliminating conditionals reduces the number of $\to$ symbols. Pushing negations inward reduces the distance between each negation and the propositional variables it governs, while double negations shorten the formula.

The distribution stages terminate when guided by a target measure: for DNF, keep applying distributions that move $\vee$ outward past $\wedge$; for CNF, keep applying distributions that move $\wedge$ outward past $\vee$. Each application lowers the number of target-obstructing connective patterns, even though it may duplicate subformulas.

Remark (Correctness). Correctness is by equivalence preservation. Each rewrite instance is one of the listed logical equivalence laws: biconditional laws, conditional laws, De Morgan laws, or distributive laws. Replacement of logical equivalents permits the rewrite to occur inside an arbitrary larger formula. Therefore the final normal form is logically equivalent to the original formula.

Remark (Non-uniqueness). Ordinary DNF and CNF are not unique. Reordering terms or clauses, reordering literals inside a term or clause, duplicating a term, deleting a redundant term, or applying absorption-style simplifications can produce different formulas with the same truth table and the same normal-form type.

Canonical normal forms impose extra conventions to remove this freedom. The ordinary conversion procedures in this topic do not impose those conventions.

Remark (Size blowup). Distribution can duplicate subformulas. As a result, converting a compact formula to DNF or CNF can produce a formula whose size is much larger than the original. This size blowup is a syntactic cost of forcing a formula into a flat disjunction-of-terms or conjunction-of-clauses shape.

Remark (Comparison).

Target form	Final outer connective pattern	Distribution used
NNF	negations only at literals	none
DNF	disjunction of conjunction terms	$\wedge$ over $\vee$
CNF	conjunction of clauses	$\vee$ over $\wedge$

All three workflows use equivalence-preserving rewrites. DNF and CNF share the same first three stages and differ only in the final distribution direction.

Definition (Truth-Table Row Conjunction)

Definition 1.5.21 (Truth-Table Row Conjunction). Fix a finite variable list $P_1, \dots, P_n$ and a row r of the truth table for those variables. The *truth-table row conjunction* for r is the conjunction

$$L_1 \wedge L_2 \wedge \cdots \wedge L_n,$$

where

$$L_i = \begin{cases} P_i, & \texttt{if row } r \texttt{ makes } P_i \texttt{ true,} \\ \neg P_i, & \texttt{if row } r \texttt{ makes } P_i \texttt{ false.} \end{cases}$$

Remark (Standard quantified statement). For a fixed row r on variables $P_1, \dots, P_n$,

$$\bigwedge_{i=1}^{n} L_i, \qquad L_i = \begin{cases} P_i, & r(P_i) = \mathsf{True}, \\ \neg P_i, & r(P_i) = \mathsf{False}. \end{cases}$$

Remark (Interpretation). The row conjunction names exactly one truth-table row. It is true on that row and false on every row that disagrees with it on at least one listed variable.

Remark (Dependencies).

- Truth Table
- Literal
- Variable Set of a Formula

Definition (Truth-Table Row Clause)

Definition 1.5.22 (Truth-Table Row Clause). Fix a finite variable list $P_1, \dots, P_n$ and a row r of the truth table for those variables. The *truth-table row clause* for r is the clause

$$M_1 \vee M_2 \vee \cdots \vee M_n,$$

where

$$M_i = \begin{cases} P_i, & \texttt{if row } r \texttt{ makes } P_i \texttt{ false,} \\ \neg P_i, & \texttt{if row } r \texttt{ makes } P_i \texttt{ true.} \end{cases}$$

Remark (Standard quantified statement). For a fixed row r on variables $P_1, \dots, P_n$,

$$\bigvee_{i=1}^{n} M_i, \qquad M_i = \begin{cases} P_i, & r(P_i) = \mathsf{False}, \\ \neg P_i, & r(P_i) = \mathsf{True}. \end{cases}$$

Remark (Interpretation). The row clause excludes exactly one truth-table row. It is false on row r and true on every row that disagrees with r on at least one listed variable.

Remark (Dependencies).

- Truth Table
- Clause
- Variable Set of a Formula

Definition (Canonical Disjunctive Normal Form)

Definition 1.5.23 (Canonical Disjunctive Normal Form). Let $\varphi \in \mathsf{WFF}$, and fix an ordering $P_1, \dots, P_n$ of the finite variable set of φ. The *canonical disjunctive normal form* of φ relative to this ordering is the disjunction of the truth-table row conjunctions corresponding to the rows on which φ is true.

Remark (Standard quantified statement). For $\mathrm{Var}(\varphi) = \{P_1, \dots, P_n\}$, the canonical disjunctive normal form is

$$\bigvee_{\widehat{v}(\varphi)=\mathsf{True}} (L_1 \wedge \cdots \wedge L_n),$$

where each row conjunction is determined by the truth-table row of v.

Remark (Interpretation). Canonical DNF records exactly the truth-table rows on which the formula is true, using one row conjunction for each true row.

Remark (Examples). If φ is a contradiction, then it has no true rows. Its canonical DNF is the empty disjunction, represented by the empty clause.

Remark (Dependencies).

- Disjunctive Normal Form
- Truth-Table Row Conjunction
- Truth Table
- Empty Clause

Definition (Canonical Conjunctive Normal Form)

Definition 1.5.24 (Canonical Conjunctive Normal Form). Let $\varphi \in \mathsf{WFF}$, and fix an ordering $P_1, \dots, P_n$ of the finite variable set of φ. The *canonical conjunctive normal form* of φ relative to this ordering is the conjunction of the truth-table row clauses corresponding to the rows on which φ is false.

Remark (Standard quantified statement). For $\mathrm{Var}(\varphi) = \{P_1, \dots, P_n\}$, the canonical conjunctive normal form is

$$\bigwedge_{\widehat{v}(\varphi)=\mathsf{False}} (M_1 \vee \cdots \vee M_n),$$

where each row clause is determined by the truth-table row of v.

Remark (Interpretation). Canonical CNF records exactly the truth-table rows on which the formula is false, using one row clause for each false row.

Remark (Examples). If φ is a tautology, then it has no false rows. Its canonical CNF is the empty conjunction, represented by the empty conjunction.

Remark (Dependencies).

- Conjunctive Normal Form
- Truth-Table Row Clause
- Truth Table
- Empty Conjunction

Theorem (Canonical Disjunctive Normal Form)

Theorem 1.5.25 (Canonical Disjunctive Normal Form Theorem). *Every formula $\varphi \in \mathsf{WFF}$ is logically equivalent to its canonical disjunctive normal form relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \mathrm{CDNF}_{P_1,\ldots,P_n}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})\, \varphi \equiv \mathrm{CDNF}_{P_1,\ldots,P_n}(\varphi),$$

where $P_1, \ldots, P_n$ is a fixed ordering of the finitely many variables of φ.

Remark (Interpretation). Canonical DNF rebuilds the formula from exactly the truth-table rows where the formula is true. Each row conjunction recognizes one true row, and the final disjunction accepts any of those rows.

Remark (Dependencies).

- Canonical Disjunctive Normal Form
- Finite Truth-Table Theorem
- Coincidence Lemma for Propositional Logic
- Logical Equivalence

Theorem (Canonical Conjunctive Normal Form)

Theorem 1.5.26 (Canonical Conjunctive Normal Form Theorem). *Every formula $\varphi \in \mathsf{WFF}$ is logically equivalent to its canonical conjunctive normal form relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \mathrm{CCNF}_{P_1,\ldots,P_n}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})\, \varphi \equiv \mathrm{CCNF}_{P_1,\ldots,P_n}(\varphi),$$

where $P_1, \ldots, P_n$ is a fixed ordering of the finitely many variables of φ.

Remark (Interpretation). Canonical CNF rebuilds the formula by excluding exactly the truth-table rows where the formula is false. Each row clause rejects one false row, and the final conjunction requires all false rows to be rejected.

Remark (Dependencies).

- Canonical Conjunctive Normal Form
- Finite Truth-Table Theorem
- Coincidence Lemma for Propositional Logic
- Logical Equivalence

Theorem (Semantic Classification by Normal Forms)

Theorem 1.5.27 (Semantic Classification by Normal Forms). *Normal forms give finite certificates for semantic classification. For any formula $\varphi \in \mathsf{WFF}$:*

1. *φ is satisfiable if and only if some DNF formula equivalent to φ has a satisfiable conjunction term.*
2. *φ is a contradiction if and only if some DNF formula equivalent to φ has no satisfiable conjunction term.*
3. *φ is a tautology if and only if some CNF formula equivalent to φ has no falsifiable clause.*
4. *φ is a contingency if and only if it is satisfiable and not a tautology.*

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned}\varphi \texttt{ is satisfiable} &\iff \texttt{some equivalent DNF term is satisfiable},\\ \varphi \texttt{ is a contradiction} &\iff \texttt{no equivalent DNF term is satisfiable},\\ \varphi \texttt{ is a tautology} &\iff \texttt{no equivalent CNF clause is falsifiable}.\end{aligned}$$

Remark (Interpretation). The truth-table method classifies φ by evaluating every row directly. The normal-form method first rewrites φ into an equivalent DNF or CNF formula and then checks the finite terms or clauses. Canonical normal forms connect the two approaches: canonical DNF is built from true rows, and canonical CNF is built from false rows.

Remark (Exposition). DNF is naturally witness-producing. A satisfiable term gives an explicit way to make the whole disjunction true. CNF is naturally suited for clause checking: each clause is a local obstruction to falsifying the whole conjunction. This clause-centered view is also the starting point for later resolution methods.

Remark (Dependencies).

- Disjunctive Normal Form
- Conjunctive Normal Form
- Satisfiable Formula
- Contradiction
- Tautology
- Contingency
- DNF Satisfiability Criterion
- Canonical Disjunctive Normal Form Theorem
- Canonical Conjunctive Normal Form Theorem

Summary

Remark (Literal vocabulary). The vocabulary begins at the literal level. A literal is a propositional variable or the negation of one propositional variable. Positive literals and negative literals record the sign of that occurrence. Complementary literals are the opposed pair P and $\neg P$. Clauses are finite disjunctions of literals, while terms are finite conjunctions of literals. The empty clause is the false boundary case, and the empty conjunction is the true boundary case.

Remark (Negation normal form). Negation normal form is the first normalization target. It permits only $\neg$, $\wedge$, and $\vee$, with every negation pushed down to the literal level. Its purpose is to remove conditionals and biconditionals and to make negation local. Once a formula is in NNF, the remaining work is a matter of distributing $\wedge$ and $\vee$ in the direction required by the target normal form.

Remark (Disjunctive normal form). Disjunctive normal form writes a formula as a finite disjunction of terms. It is naturally witness-oriented: to make a DNF formula true, it is enough to make one term true, and a satisfiable term directly records compatible literal requirements. This is why DNF is especially useful for satisfiability checks and for building formulas from true rows of a truth table.

Remark (Conjunctive normal form). Conjunctive normal form writes a formula as a finite conjunction of clauses. It is naturally obstruction-oriented: to falsify a CNF formula, one must falsify a clause, and each clause gives a local disjunctive condition. This clause structure makes CNF the preferred shape for later clause checking, resolution-style arguments, and systematic proof search.

Remark (Canonical normal forms). Ordinary DNF and CNF are not unique. Reordering, duplicating, or simplifying terms and clauses can produce different formulas with the same truth table. Canonical normal forms add truth-table discipline. Canonical DNF records the true rows by minterm-style conjunctions, and canonical CNF records the false rows by maxterm-style clauses. These canonical forms show that every finite truth behavior can be represented syntactically.

Remark (Conversion workflow). The conversion workflow has a common beginning. First eliminate biconditionals. Then eliminate conditionals. Then push negations inward until the formula is in negation normal form. From there, distribute conjunction over disjunction to obtain DNF, or distribute disjunction over conjunction to obtain CNF. Each stage is justified by logical equivalence laws and by replacement of logical equivalents inside larger formulas.

Remark (Back to the algebra of propositions). Normal forms are an application of the algebra of propositions. The conversion rules are not new semantic principles; they are uses of earlier equivalence laws such as De Morgan laws, conditional and biconditional equivalences, distributivity, and replacement of equivalents. The normal-form viewpoint therefore shows how algebraic manipulation and semantic preservation work together.

Remark (Forward to functional completeness). Canonical normal forms point forward to functional completeness. If a truth function can be described by its true rows, then it can be represented by a disjunction of conjunctions of literals. If it can be described by its false rows, then it can be represented by a conjunction of disjunctions of literals. This is the bridge from truth tables to the question of which connectives are enough to express every truth function.

Remark (Forward to arguments and proof systems). Normal forms also prepare the transition from single formulas to arguments and proof systems. DNF gives explicit witnesses for satisfiability, while CNF turns formulas into collections of clauses that can be checked and transformed. The same syntactic regularity used here to classify formulas will later support validity tests, derivation systems, resolution methods, and comparisons between semantic consequence and formal proof.

1.6 Functional Completeness

Toolkit

This section uses Boolean functions, truth functions induced by formulas, logical equivalence, canonical normal forms, and connective rewrite laws.

Remark (Exposition). The functional-completeness section studies expressive power. A connective set is complete exactly when formulas built from it can represent every Boolean function, so normal forms and connective reductions become tests for the adequacy of propositional vocabularies.

Orientation

Remark (Section map). This section begins with Boolean functions and formula representation, then introduces connective bases and definability of connectives. It uses normal forms to prove completeness of the standard connectives and then reduces the primitive vocabulary further. The final topics show that NAND and NOR each form a one-connective complete basis, while some familiar connective sets fail to be functionally complete.

Remark (Connection with normal forms). Normal forms explain why functional completeness is possible. Canonical DNF and canonical CNF show that the truth behavior of a formula can be reconstructed from finitely many truth-table rows. Once those canonical constructions are available, proving that a connective set is functionally complete amounts to showing that the connectives needed for those normal forms can be defined from that set.

Boolean Functions and Representation

Definition (Representation of a Boolean Function)

Definition 1.6.1 (Representation of a Boolean Function). Let $f : \mathbb{B}^n \to \mathbb{B}$ be a Boolean function. A formula $\varphi \in \mathsf{WFF}$ with variables among $P_1, \ldots, P_n$ *represents* f if for every truth assignment v,

$$\widehat{v}(\varphi) = f(v(P_1), \ldots, v(P_n)).$$

Remark (Standard quantified statement).

$$\text{Represents}(\varphi, f; P_1, \ldots, P_n) \iff (\forall v : \mathsf{Prop} \to \mathbb{B})\big(\widehat{v}(\varphi) = f(v(P_1), \ldots, v(P_n))\big).$$

Remark (Definition predicate reading). $\text{Represents}(\varphi, f; P_1, \ldots, P_n)$ means that φ has exactly the same truth table as the Boolean function f on the listed variables.

Remark (Interpretation). A formula represents a Boolean function when the formula computes the same truth value as the function for every input row. This turns a semantic table of values into a syntactic object inside the propositional language.

Remark (Dependencies).

- Boolean Function
- Truth Function Induced by a Formula
- Truth Assignment

Theorem (DNF Representation of Boolean Functions)

Theorem 1.6.2 (DNF Representation of Boolean Functions). *Every Boolean function $f : \mathbb{B}^n \to \mathbb{B}$ is represented by a formula in disjunctive normal form using variables $P_1, \ldots, P_n$.*
Go to proof.

Remark (Standard quantified statement).

$$(\forall f : \mathbb{B}^n \to \mathbb{B})(\exists \varphi \in \mathsf{WFF})\big(\varphi \text{ is in DNF} \land \text{Represents}(\varphi, f; P_1, \ldots, P_n)\big).$$

Remark (Predicate reading). Every n-ary Boolean function has a propositional formula representative, and one may choose the representative to be in disjunctive normal form.

Remark (Interpretation). A truth table specifies which input rows make the Boolean function true. For each true row, form the corresponding conjunction of literals; the disjunction of those row conjunctions represents the entire function.

Remark (Dependencies).

- Representation of a Boolean Function
- Canonical Disjunctive Normal Form
- Canonical Disjunctive Normal Form Theorem
- Disjunctive Normal Form

Connective Bases

Definition (Connective Basis)

Definition 1.6.3 (Connective Basis). A *connective basis* is a specified set B of propositional connectives allowed as primitive connectives for building formulas.

Remark (Standard quantified statement).

$$B \text{ is a connective basis} \iff B \subseteq \{\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \ldots\}$$

and formulas over B are built using only connectives from B.

Remark (Definition predicate reading). $\mathrm{ConnectiveBasis}(B)$ means that B is the chosen primitive connective vocabulary for a propositional language or fragment.

Remark (Interpretation). A connective basis controls which connectives are primitive, not which truth functions are expressible. Different bases may have the same expressive power.

Remark (Dependencies).

- Logical Connective
- Propositional Language

Definition (Definable Connective)

Definition 1.6.4 (Definable Connective). Let B be a connective basis. A connective $\circ$ is *definable from* B if there is a formula built using only connectives from B whose truth function agrees with the truth function of $\circ$.

Remark (Standard quantified statement).

$$\circ \text{ is definable from } B \iff \exists\theta \in \mathsf{WFF}_B \left(f_\theta = f_\circ\right),$$

where WFF_B denotes the formulas built using only connectives from B.

Remark (Definition predicate reading). $\mathrm{DefinableFrom}(\circ, B)$ means that $\circ$ can be treated as an abbreviation for a formula pattern using only connectives from B.

Remark (Interpretation). Definability separates notation from expressive power. A connective can be omitted from the primitive syntax without losing its use, provided it is recoverable as a defined connective.

Remark (Dependencies).

- Connective Basis
- Truth Function Induced by a Formula
- Boolean Function

Definition (Functionally Complete Connective Set)

Definition 1.6.5 (Functionally Complete Connective Set). A connective basis B is *functionally complete* if every Boolean function $f : \mathbb{B}^n \to \mathbb{B}$, for every $n \geq 1$, is represented by some formula using only connectives from B.

Remark (Standard quantified statement).

$$\mathrm{FunctionallyComplete}(B) \iff (\forall n \geq 1)(\forall f : \mathbb{B}^n \to \mathbb{B})(\exists \varphi \in \mathsf{WFF}_B)\, \mathrm{Represents}(\varphi, f; P_1, \ldots, P_n).$$

Remark (Definition predicate reading). $\mathrm{FunctionallyComplete}(B)$ means that formulas over B can represent every finite Boolean input-output table.

Remark (Interpretation). Functional completeness is the central expressive-power property for connective sets. It says that the basis can express all possible truth-functional behavior, not merely the connectives originally listed in the language.

Remark (Dependencies).

- Connective Basis
- Representation of a Boolean Function
- Boolean Function

Definition (Expressively Equivalent Connective Sets)

Definition 1.6.6 (Expressively Equivalent Connective Sets). Two connective bases B and C are *expressively equivalent* if every connective in B is definable from C, and every connective in C is definable from B.

Remark (Standard quantified statement).

$$B \equiv_{\mathrm{expr}} C \iff \big((\forall \circ \in B)\, \mathrm{DefinableFrom}(\circ, C)\big) \wedge \big((\forall \star \in C)\, \mathrm{DefinableFrom}(\star, B)\big).$$

Remark (Definition predicate reading). $B \equiv_{\mathrm{expr}} C$ means that the two bases can define exactly the same connective behavior.

Remark (Interpretation). Expressive equivalence lets a proof replace one primitive vocabulary with another. This is why a language can be simplified syntactically without changing which truth functions can be expressed.

Remark (Dependencies).

- Connective Basis
- Definable Connective

Completeness of the Standard Connectives

Theorem (Standard Connectives are Functionally Complete)

Theorem 1.6.7 (Standard Connectives are Functionally Complete). *The connective basis*

$$\{\neg, \wedge, \vee\}$$

is functionally complete.
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \wedge, \vee\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation, conjunction, and disjunction.

Remark (Interpretation). Canonical disjunctive normal form uses only literals, conjunctions, and disjunctions. Since literals use variables and negation, the normal-form theorem shows that $\{\neg, \wedge, \vee\}$ can express every Boolean function.

Remark (Dependencies).

- Functionally Complete Connective Set
- DNF Representation of Boolean Functions
- Disjunctive Normal Form

Theorem (Negation and Conjunction are Functionally Complete)

Theorem 1.6.8 (Negation and Conjunction are Functionally Complete). *The connective basis*

$$\{\neg, \wedge\}$$

is functionally complete.
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \wedge\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation and conjunction.

Remark (Interpretation). Disjunction is definable from negation and conjunction by De Morgan's law:

$$\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi).$$

Thus every formula using $\neg, \wedge, \vee$ can be translated into an equivalent formula using only $\neg$ and $\wedge$.

Remark (Dependencies).

- Standard Connectives are Functionally Complete
- De Morgan Laws for Propositional Logic
- Replacement of Logical Equivalents

Theorem (Negation and Disjunction are Functionally Complete)

Theorem 1.6.9 (Negation and Disjunction are Functionally Complete). *The connective basis*

$$\{\neg, \vee\}$$

is functionally complete.
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \vee\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation and disjunction.

Remark (Interpretation). Conjunction is definable from negation and disjunction by De Morgan's law:

$$\varphi \wedge \psi \equiv \neg(\neg\varphi \vee \neg\psi).$$

Thus every formula using $\neg, \wedge, \vee$ can be translated into an equivalent formula using only $\neg$ and $\vee$.

Remark (Dependencies).

- Standard Connectives are Functionally Complete
- De Morgan Laws for Propositional Logic
- Replacement of Logical Equivalents

Reduction of Connective Sets

Definition (Connective Reduction)

Definition 1.6.10 (Connective Reduction). A *connective reduction* replaces a formula using one connective basis by a logically equivalent formula using a smaller or different connective basis.

Remark (Standard quantified statement).

$$\mathrm{Reduction}_{B\to C}(\varphi,\psi) \iff \varphi \equiv \psi \quad \text{and} \quad \psi \in \mathsf{WFF}_C.$$

Remark (Definition predicate reading). $\mathrm{Reduction}_{B\to C}(\varphi,\psi)$ means that ψ is a translation of φ into the connective basis C without changing truth behavior.

Remark (Interpretation). Reduction shows that some primitive connectives are optional. If a connective is definable from the remaining basis, then formulas using that connective can be translated away.

Remark (Exposition). The standard reductions are:

$$\varphi \to \psi \equiv \neg\varphi \lor \psi,$$

$$\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \land (\psi \to \varphi),$$
$$\varphi \lor \psi \equiv \neg(\neg\varphi \land \neg\psi), \qquad \varphi \land \psi \equiv \neg(\neg\varphi \lor \neg\psi).$$

Each replacement preserves logical equivalence and may be performed inside a larger formula context.

Remark (Exposition). Reducing a connective basis can make formulas longer. Functional completeness is therefore an expressive result, not an efficiency result. A small basis may be powerful enough to express every Boolean function while still being awkward for ordinary human-readable formulas.

Remark (Dependencies).

- Connective Basis
- Definable Connective
- Logical Equivalence

NAND

Definition (NAND Connective)

Definition 1.6.11 (NAND Connective). The *NAND* connective, written $\uparrow$, is the binary connective defined by

$$\varphi \uparrow \psi \equiv \neg(\varphi \land \psi).$$

Remark (Standard quantified statement).

$$\widehat{v}(\varphi \uparrow \psi) = \mathsf{True} \iff \text{not both } \widehat{v}(\varphi) = \mathsf{True} \text{ and } \widehat{v}(\psi) = \mathsf{True}.$$

Remark (Definition predicate reading). $\mathrm{NAND}(\varphi,\psi)$ means the negation of the conjunction of φ and ψ.

Remark (Interpretation). NAND is read as "not both." It is false only when both inputs are true. This single connective can reproduce negation, conjunction, and disjunction.

Remark (Dependencies).

- Logical Connective
- Truth Assignment

Definition (NAND-Only Formula)

Definition 1.6.12 (NAND-Only Formula). A *NAND-only formula* is a well-formed formula whose only connective is $\uparrow$.

Remark (Standard quantified statement).

$$\varphi \text{ is NAND-only} \iff \varphi \in \mathsf{WFF}_{\{\uparrow\}}.$$

Remark (Definition predicate reading). $\mathrm{NANDOnly}(\varphi)$ means that every connective occurrence in φ is an occurrence of $\uparrow$.

Remark (Interpretation). A NAND-only formula is syntactically restricted but expressively powerful. The restriction is severe at the level of primitive symbols, but not at the level of truth functions.

Remark (Dependencies).

- NAND Connective
- Well-Formed Formula

Theorem (NAND is Functionally Complete)

Theorem 1.6.13 (NAND is Functionally Complete). *The one-connective basis $\{\uparrow\}$ is functionally complete.*
Go to proof.

Remark (Standard quantified statement).

$$\mathrm{FunctionallyComplete}(\{\uparrow\}).$$

Remark (Predicate reading). Every Boolean function can be represented by a NAND-only formula.

Remark (Interpretation). NAND can define negation and conjunction:

$$\neg\varphi \equiv \varphi \uparrow \varphi, \qquad \varphi \wedge \psi \equiv (\varphi \uparrow \psi) \uparrow (\varphi \uparrow \psi).$$

Once negation and conjunction are definable, functional completeness follows from the functional completeness of $\{\neg, \wedge\}$.

Remark (Dependencies).

- NAND Connective
- NAND-Only Formula
- Functionally Complete Connective Set
- Negation and Conjunction are Functionally Complete

NOR

Definition (NOR Connective)

Definition 1.6.14 (NOR Connective). The *NOR* connective, written $\downarrow$, is the binary connective defined by

$$\varphi \downarrow \psi \equiv \neg(\varphi \vee \psi).$$

Remark (Standard quantified statement).

$$\widehat{v}(\varphi \downarrow \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{False} \text{ and } \widehat{v}(\psi) = \mathsf{False}.$$

Remark (Definition predicate reading). $\mathrm{NOR}(\varphi, \psi)$ means the negation of the disjunction of φ and ψ.

Remark (Interpretation). NOR is read as "neither." It is true exactly when both inputs are false. Like NAND, this single connective can reproduce the standard functionally complete basis.

Remark (Dependencies).

- Logical Connective
- Truth Assignment

Definition (NOR-Only Formula)

Definition 1.6.15 (NOR-Only Formula). A *NOR-only formula* is a well-formed formula whose only connective is $\downarrow$.

Remark (Standard quantified statement).

$$\varphi \text{ is NOR-only} \iff \varphi \in \mathsf{WFF}_{\{\downarrow\}}.$$

Remark (Definition predicate reading). $\mathrm{NOROnly}(\varphi)$ means that every connective occurrence in φ is an occurrence of $\downarrow$.

Remark (Interpretation). A NOR-only formula uses a single primitive connective. Its expressive power comes from the ability of NOR to define negation and disjunction

Remark (Dependencies).

- NOR Connective
- Well-Formed Formula

Theorem (NOR is Functionally Complete)

Theorem 1.6.16 (NOR is Functionally Complete). *The one-connective basis $\{\downarrow\}$ is functionally complete.*
Go to proof.

Remark (Standard quantified statement).

$$\mathrm{FunctionallyComplete}(\{\downarrow\}).$$

Remark (Predicate reading). Every Boolean function can be represented by a NOR-only formula.

Remark (Interpretation). NOR can define negation and disjunction:

$$\neg\varphi \equiv \varphi \downarrow \varphi, \qquad \varphi \vee \psi \equiv (\varphi \downarrow \psi) \downarrow (\varphi \downarrow \psi).$$

Once negation and disjunction are definable, functional completeness follows from the functional completeness of $\{\neg, \vee\}$.

Remark (Dependencies).

- NOR Connective
- NOR-Only Formula
- Functionally Complete Connective Set
- Negation and Disjunction are Functionally Complete

Incomplete Connective Sets

Definition (Incomplete Connective Set)

Definition 1.6.17 (Incomplete Connective Set). A connective basis is *incomplete* if it is not functionally complete.

Remark (Standard quantified statement).

$$\mathrm{Incomplete}(B) \iff \neg\,\mathrm{FunctionallyComplete}(B).$$

Remark (Definition predicate reading). $\mathrm{Incomplete}(B)$ means that some Boolean function cannot be represented by any formula using only connectives from B.

Remark (Interpretation). Incompleteness is usually proved by finding an invariant preserved by every formula built from the basis. If some Boolean function fails that invariant, the basis cannot express every Boolean function.

Remark (Dependencies).

- Functionally Complete Connective Set
- Boolean Function

Definition (Monotone Boolean Function)

Definition 1.6.18 (Monotone Boolean Function). A Boolean function $f : \mathbb{B}^n \to \mathbb{B}$ is *monotone* if changing input coordinates from False to True never changes the output from True to False.

Remark (Standard quantified statement).

$$\mathrm{Monotone}(f) \iff (\forall a, b \in \mathbb{B}^n)\big(a \le b \to f(a) \le f(b)\big),$$

where $\mathsf{False} \le \mathsf{True}$ coordinatewise.

Remark (Definition predicate reading). $\mathrm{Monotone}(f)$ means that increasing truth inputs cannot make the output decrease from true to false.

Remark (Interpretation). The connectives $\wedge$ and $\vee$ are monotone. Negation is not monotone, because changing P from false to true changes $\neg P$ from true to false.

Remark (Dependencies).

- Boolean Function

Theorem (Conjunction and Disjunction are not Functionally Complete)

Theorem 1.6.19 (Conjunction and Disjunction are not Functionally Complete). *The connective basis $\{\wedge, \vee\}$ is not functionally complete.* *Go to proof.*

Remark (Standard quantified statement).

$$\neg\,\mathrm{FunctionallyComplete}(\{\wedge, \vee\}).$$

Remark (Predicate reading). There is at least one Boolean function that cannot be represented by any formula using only conjunction and disjunction.

Remark (Interpretation). Every formula built using only $\wedge$ and $\vee$ represents a monotone Boolean function. Since negation is not monotone, no formula using only $\wedge$ and $\vee$ can represent negation. Hence the basis is incomplete.

Remark (Dependencies).

- Incomplete Connective Set
- Monotone Boolean Function
- Functionally Complete Connective Set

Minimal Complete Bases

Definition (Minimal Complete Basis)

Definition 1.6.20 (Minimal Complete Basis). A functionally complete connective basis B is *minimal complete* if no proper subset of B is functionally complete.

Remark (Standard quantified statement).

$$\mathrm{MinimalComplete}(B) \iff \mathrm{FunctionallyComplete}(B) \wedge (\forall C \subsetneq B)\neg\,\mathrm{FunctionallyComplete}(C).$$

Remark (Definition predicate reading). $\mathrm{MinimalComplete}(B)$ means that B is complete, but removing any primitive connective from B destroys completeness.

Remark (Interpretation). Minimality is relative to deletion of connectives from the basis. It does not mean that formulas using the basis are short or easy to read.

Remark (Dependencies).

- Functionally Complete Connective Set
- Connective Basis

Theorem (NAND and NOR are Minimal Complete Bases)

Theorem 1.6.21 (NAND and NOR are Minimal Complete Bases). *The one-connective bases $\{\uparrow\}$ and $\{\downarrow\}$ are minimal complete bases.*
Go to proof.

Remark (Standard quantified statement).

$$\mathrm{MinimalComplete}(\{\uparrow\}) \wedge \mathrm{MinimalComplete}(\{\downarrow\}).$$

Remark (Predicate reading). NAND alone is complete, NOR alone is complete, and deleting the sole connective from either basis leaves no functionally complete basis.

Remark (Interpretation). A one-connective complete basis is automatically minimal once the empty basis is not functionally complete. NAND and NOR are therefore minimal complete bases because each is complete by itself.

Remark (Dependencies).

- Minimal Complete Basis
- NAND is Functionally Complete
- NOR is Functionally Complete
- Incomplete Connective Set

Summary and Transition

Remark (Summary). Functional completeness measures the expressive strength of a connective basis. A functionally complete basis can represent every finite Boolean function by a formula using only the connectives in that basis. Normal forms supply the main construction: truth-table rows can be converted into formulas, and those formulas can be assembled into a representative for the desired Boolean function.

Remark (Section map). The standard connectives $\neg$, $\wedge$, and $\vee$ are functionally complete because they can build disjunctive normal forms. De Morgan laws reduce this basis to $\{\neg, \wedge\}$ or $\{\neg, \vee\}$. NAND and NOR go further: each single connective can define a complete two-connective basis and therefore can represent every Boolean function.

Remark (Transition). The next section moves from expressive power to argument forms and informal proof patterns. Functional completeness explains which truth functions can be expressed. Argument forms ask which patterns of inference preserve truth from premises to conclusions.

1.7 Argument Forms and Informal Proof Patterns

Toolkit

This section uses argument forms, inference patterns, replacement rules, informal proof patterns, and semantic validity.

Remark (Exposition). Argument forms organize recurring patterns of propositional reasoning. This section connects valid and invalid forms with replacement principles, informal proof practice, and the semantic tests that justify those patterns.

Orientation

Remark (Working vocabulary). This orientation uses truth assignments, logical consequence, tautology, logical equivalence, formula contexts, and counterexample assignments to justify common argument forms.

Remark (Exposition). Argument forms are reusable patterns of inference. The purpose of this section is to justify the basic inference tools that will be used throughout the rest of the project. The justification is semantic: an argument form is valid when no truth assignment makes every premise true while making the conclusion false.

Remark (Section map). This section introduces argument forms, substitution instances, validity, invalidity, and counterexample assignments. It then proves the validity of the standard inference patterns, isolates common invalid forms, explains rules of replacement, and connects informal proof patterns with semantic consequence. Formal proof systems are previewed but not developed here.

Remark (Boundary). The goal is rigorous senior-undergraduate propositional logic. The section proves that the proof moves used later are truth-preserving, but it does not build Hilbert systems, sequent calculi, cut elimination, or metatheoretic completeness machinery.

Capstone Problem

Problem. Build a complete truth-table analysis for a propositional formula, classify the formula semantically, and convert it to an equivalent normal form.

Remark (Dependency ceiling). The capstone may use the syntax, semantics, algebra, and normal-form results routed earlier in this chapter.

Syntax Proofs

Remark (Return). Return to Lemma (Closure Under Formation).

Lemma (Closure Under Formation). *The set* WFF *is closed under the propositional formation rules.*

Proof. **Professional Standard Proof.** TODO: supply the proof by direct unpacking of the WFF formation rules. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).
Well-Formed Formula

Remark (Return). Return to Theorem (Minimality of Well-Formed Formulas).

Theorem (Minimality of Well-Formed Formulas). *Let S be a set of strings over the propositional alphabet. If S contains* Prop *and is closed under the propositional formation rules, then* $\mathsf{WFF} \subseteq S$.

Proof. **Professional Standard Proof.** TODO: supply the proof by structural induction on formulas or by the intersection characterization of the recursively generated set. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).
Well-Formed Formula

Remark (Return). Return to Theorem (Structural Induction for Propositional Formulas).

Theorem (Structural Induction for Propositional Formulas). *Let $A(\varphi)$ be a property of well-formed formulas. Suppose:*

1. *$A(P)$ holds for every $P \in \mathsf{Prop}$;*
2. *if $A(\varphi)$ holds, then $A(\neg\varphi)$ holds;*
3. *if $A(\varphi)$ and $A(\psi)$ hold, then $A((\varphi \circ \psi))$ holds for every $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$.*

Then $A(\theta)$ holds for every $\theta \in \mathsf{WFF}$.

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Well-Formed Formula

Remark (Return). Return to Theorem (Structural Recursion for Propositional Formulas).

Theorem (Structural Recursion for Propositional Formulas). *To define a function F on* WFF*, it suffices to specify:*

1. *$F(P)$ for every $P \in \mathsf{Prop}$;*
2. *$F(\neg\varphi)$ in terms of $F(\varphi)$;*
3. *$F((\varphi \circ \psi))$ in terms of $F(\varphi)$, $F(\psi)$, and $\circ$.*

When these clauses respect the formation rules, they determine a unique function on WFF.

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Well-Formed Formula Structural Induction for Propositional Formulas

Remark (Return). Return to Lemma (Constructor Disjointness for Propositional Formulas).

Lemma (Constructor Disjointness for Propositional Formulas). *The three outer formation cases for propositional formulas are disjoint:*

1. *no propositional variable is a negation formula;*
2. *no propositional variable is a binary formula;*
3. *no negation formula is a binary formula.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Well-Formed Formula Atomic Formula Main Connective

Remark (Return). Return to Lemma (Constructor Injectivity for Propositional Formulas).

Lemma (Constructor Injectivity for Propositional Formulas). *The propositional formula constructors are injective:*

1. *if* $\neg\varphi = \neg\psi$*, then* $\varphi = \psi$*;*
2. *if* $(\varphi \circ \psi) = (\alpha \star \beta)$*, then* $\varphi = \alpha$*,* $\psi = \beta$*, and* $\circ = \star$*.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Well-Formed Formula Logical Connective

Remark (Return). Return to Theorem (Unique Decomposition for Propositional Formulas).

Theorem (Unique Decomposition for Propositional Formulas). *Every* $\varphi \in \mathsf{WFF}$ *has exactly one outermost form:*

1. $\varphi = P$ *for a unique* $P \in \mathsf{Prop}$;
2. $\varphi = \neg\psi$ *for a unique* $\psi \in \mathsf{WFF}$;
3. $\varphi = (\psi \circ \chi)$ *for unique* $\psi, \chi \in \mathsf{WFF}$ *and a unique binary connective* $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$.

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Well-Formed Formula Constructor Disjointness for Propositional Formulas Constructor Injectivity for Propositional Formulas

Remark (Return). Return to Theorem (Unique Syntax Tree for Propositional Formulas).

Theorem (Unique Syntax Tree for Propositional Formulas). *Every propositional well-formed formula has a unique parse tree.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Parse Tree Unique Decomposition for Propositional Formulas

Remark (Return). Return to Lemma (Proper Subformula Depth).

Lemma (Proper Subformula Depth). *If ψ is a proper subformula of φ, then*

$$\mathrm{depth}(\psi) < \mathrm{depth}(\varphi).$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Subformula Formula Depth

Remark (Return). Return to Lemma (Finiteness of Variables in a Formula).

Lemma (Finiteness of Variables in a Formula). *For every* $\varphi \in \mathsf{WFF}$*, the set* $\mathrm{Var}(\varphi)$ *is finite.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Variable Set of a Formula Structural Induction for Propositional Formulas

Remark (Return). Return to Lemma (Finiteness of Subformulas).

Lemma (Finiteness of Subformulas). *For every $\varphi \in \mathsf{WFF}$, the set $\mathrm{Sub}(\varphi)$ is finite.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). TODO: record the intended proof structure.

Remark (Dependencies).

Subformula Structural Induction for Propositional Formulas

Semantics Proofs

Remark (Return). Return to Theorem (Unique Extension of a Truth Assignment).

Theorem (Unique Extension of a Truth Assignment). *Every truth assignment*

$$v : \mathsf{Prop} \to \mathbb{B}$$

has a unique extension

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

satisfying the recursive truth clauses for propositional variables and the primitive connectives.

Proof. **Professional Standard Proof.** TODO: supply the proof by structural recursion on well-formed formulas, justified by unique decomposition. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed proof by structural recursion on well-formed formulas, justified by unique decomposition. □

Remark (Proof structure). Use structural recursion to define $\widehat{v}$. Use structural induction or uniqueness of recursive definitions to prove uniqueness.

Remark (Dependencies).

- Truth Assignment
- Extension of a Truth Assignment to Formulas
- Structural Recursion for Propositional Formulas
- Unique Decomposition for Propositional Formulas

Remark (Return). Return to Lemma (Coincidence Lemma for Propositional Logic).

Lemma (Coincidence Lemma for Propositional Logic). *Let $v, w : \mathsf{Prop} \to \mathbb{B}$ be truth assignments. If v and w agree on every propositional variable occurring in φ, then they assign the same truth value to φ:*

$$v|_{\mathrm{Var}(\varphi)} = w|_{\mathrm{Var}(\varphi)} \Longrightarrow \widehat{v}(\varphi) = \widehat{w}(\varphi).$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Prove by structural induction on φ. Atomic case follows from agreement on variables. Negation and binary connective cases follow from the induction hypothesis and the recursive truth clauses.

Remark (Dependencies).

- Variable Set of a Formula
- Unique Extension of a Truth Assignment
- Structural Induction for Propositional Formulas

Remark (Return). Return to Theorem (Finite Truth-Table Theorem).

Theorem (Finite Truth-Table Theorem). *Every propositional formula has a finite truth table. More precisely, if* $\mathrm{Var}(\varphi)$ *has* n *elements, then* φ *has a truth table with* 2^n *rows.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use finiteness of $\mathrm{Var}(\varphi)$. If there are n variables, then there are 2^n functions from that finite variable set to $\mathbb{B}$.

Remark (Dependencies).

- Truth Table
- Finiteness of Variables in a Formula

Remark (Return). Return to Theorem (Counting Boolean Functions).

Theorem (Counting Boolean Functions). *There are exactly*

$$2^{2^n}$$

Boolean functions $\mathbb{B}^n \to \mathbb{B}$.

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Count functions from a 2^n-element set to a 2-element set.

Remark (Dependencies).

- Boolean Function

Algebra of Propositions Proofs

Remark (Return). Return to Theorem (Logical Equivalence is an Equivalence Relation).

Theorem (Logical Equivalence is an Equivalence Relation). *Logical equivalence is reflexive, symmetric, and transitive on* WFF*:*

$$\varphi \equiv \varphi, \qquad \varphi \equiv \psi \Longrightarrow \psi \equiv \varphi,$$

and

$$(\varphi \equiv \psi \wedge \psi \equiv \chi) \Longrightarrow \varphi \equiv \chi.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Unfold logical equivalence as equality of truth values under every truth assignment.

Remark (Dependencies).

- Logical Equivalence

Remark (Return). Return to Theorem (Basic Boolean Equivalence Laws).

Theorem (Basic Boolean Equivalence Laws). *For all formulas* $\varphi, \psi, \chi \in \mathsf{WFF}$*, the following logical equivalences hold.*

Commutativity:

$$\varphi \wedge \psi \equiv \psi \wedge \varphi, \qquad \varphi \vee \psi \equiv \psi \vee \varphi.$$

Associativity:

$$(\varphi \wedge \psi) \wedge \chi \equiv \varphi \wedge (\psi \wedge \chi), \qquad (\varphi \vee \psi) \vee \chi \equiv \varphi \vee (\psi \vee \chi).$$

Idempotence:

$$\varphi \wedge \varphi \equiv \varphi, \qquad \varphi \vee \varphi \equiv \varphi.$$

Double negation:

$$\neg\neg\varphi \equiv \varphi.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Prove each equivalence by truth-table calculation or by unfolding the semantic clauses for $\widehat{v}$.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Remark (Return). Return to Theorem (Identity and Domination Laws).

Theorem (Identity and Domination Laws). *Let $\top$ be any tautological formula and let $\bot$ be any contradictory formula. Then for every $\varphi \in \mathsf{WFF}$:*

$$\varphi \wedge \top \equiv \varphi, \qquad \varphi \vee \bot \equiv \varphi,$$

and

$$\varphi \wedge \bot \equiv \bot, \qquad \varphi \vee \top \equiv \top.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Evaluate both sides under an arbitrary truth assignment.

Remark (Dependencies).

- Tautology
- Contradiction
- Logical Equivalence

Remark (Return). Return to Theorem (De Morgan Laws for Propositional Logic).

Theorem (De Morgan Laws for Propositional Logic). *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi,$$

and

$$\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use a two-variable truth table or unfold the semantic clauses for $\neg, \wedge, \vee$.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Remark (Return). Return to Theorem (Distributive Laws for Propositional Logic).

Theorem (Distributive Laws for Propositional Logic). *For all* $\varphi, \psi, \chi \in \mathsf{WFF}$*,*

$$\varphi \wedge (\psi \vee \chi) \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi),$$

and

$$\varphi \vee (\psi \wedge \chi) \equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi).$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use a three-variable truth table or semantic case analysis.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Remark (Return). Return to Theorem (Absorption Laws).

Theorem (Absorption Laws). *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \wedge (\varphi \vee \psi) \equiv \varphi,$$

and

$$\varphi \vee (\varphi \wedge \psi) \equiv \varphi.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Prove by truth table or derive algebraically from distributivity, idempotence, and identity laws.

Remark (Dependencies).

- Logical Equivalence
- Basic Boolean Equivalence Laws
- Distributive Laws for Propositional Logic

Remark (Return). Return to Theorem (Conditional Equivalence Laws).

Theorem (Conditional Equivalence Laws). *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \to \psi \equiv \neg\varphi \vee \psi,$$

and

$$\neg(\varphi \to \psi) \equiv \varphi \wedge \neg\psi.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. ☐

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. ☐

Remark (Proof structure). Use the semantic truth clause for implication.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Remark (Return). Return to Theorem (Biconditional Equivalence Laws).

Theorem (Biconditional Equivalence Laws). *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi),$$

and

$$\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use the truth clause for biconditional, or rewrite using the conditional equivalence laws.

Remark (Dependencies).

- Logical Equivalence
- Conditional Equivalence Laws

Remark (Return). Return to Theorem (Replacement of Logical Equivalents).

Theorem (Replacement of Logical Equivalents). *Replacing logically equivalent formulas inside a formula context preserves logical equivalence:*

$$\varphi \equiv \psi \Longrightarrow C[\varphi] \equiv C[\psi].$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Prove by structural induction on the context $C[-]$. The placeholder case uses $\varphi \equiv \psi$. The connective cases follow from the semantic truth clauses.

Remark (Dependencies).

- Formula Context
- Logical Equivalence
- Coincidence Lemma for Propositional Logic
- Unique Extension of a Truth Assignment

Normal Forms Proofs

Remark (Return). Return to Theorem (Canonical Disjunctive Normal Form).

Theorem (Canonical Disjunctive Normal Form Theorem). *Every formula $\varphi \in \mathsf{WFF}$ is logically equivalent to its canonical disjunctive normal form relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \mathrm{CDNF}_{P_1,\ldots,P_n}(\varphi).$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use the finite truth-table theorem to list all rows over the variables of φ. For each true row, the corresponding row conjunction is true exactly on that row. The disjunction of all such row conjunctions is therefore true exactly on the same rows as φ. The coincidence lemma ensures no variables outside the fixed list affect the comparison.

Remark (Dependencies).

- Canonical Disjunctive Normal Form
- Finite Truth-Table Theorem
- Coincidence Lemma for Propositional Logic
- Logical Equivalence

Remark (Return). Return to Theorem (Canonical Conjunctive Normal Form).

Theorem (Canonical Conjunctive Normal Form Theorem). *Every formula* $\varphi \in \mathsf{WFF}$ *is logically equivalent to its canonical conjunctive normal form relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \mathrm{CCNF}_{P_1,\ldots,P_n}(\varphi).$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use the finite truth-table theorem to list all rows over the variables of φ. For each false row, the corresponding row clause is false exactly on that row and true on every other row. The conjunction of all such row clauses is therefore false exactly on the same rows as φ. The coincidence lemma ensures no variables outside the fixed list affect the comparison.

Remark (Dependencies).

- Canonical Conjunctive Normal Form
- Finite Truth-Table Theorem
- Coincidence Lemma for Propositional Logic
- Logical Equivalence

Remark (Return). Return to Theorem (Existence of Conjunctive Normal Form).

Theorem (Existence of Conjunctive Normal Form). *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in conjunctive normal form and*

$$\varphi \equiv \psi.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. ☐

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. ☐

Remark (Proof structure). First use the existence theorem for negation normal form to replace φ by an equivalent NNF formula. Then prove, by induction on that NNF formula, that repeated use of the distributive laws transforms it into a conjunction of clauses. Replacement of logical equivalents preserves equivalence through each local transformation.

Remark (Dependencies).

- Conjunctive Normal Form
- Negation Normal Form
- Distributive Laws for Propositional Logic
- Replacement of Logical Equivalents
- Existence of Negation Normal Form

Remark (Return). Return to Theorem (Existence of Disjunctive Normal Form).

Theorem (Existence of Disjunctive Normal Form). *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in disjunctive normal form and*

$$\varphi \equiv \psi.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). First use the existence theorem for negation normal form to replace φ by an equivalent NNF formula. Then prove, by induction on that NNF formula, that repeated use of the distributive laws transforms it into a disjunction of elementary conjunctions. Replacement of logical equivalents preserves equivalence through each local transformation.

Remark (Dependencies).

- Disjunctive Normal Form
- Negation Normal Form
- Distributive Laws for Propositional Logic
- Replacement of Logical Equivalents
- Existence of Negation Normal Form

Remark (Return). Return to Theorem (DNF Satisfiability Criterion).

Theorem (DNF Satisfiability Criterion). *Let ψ be a formula in disjunctive normal form. Then ψ is satisfiable if and only if at least one conjunction term in ψ contains no complementary pair of literals.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). For the forward direction, any satisfying valuation makes some DNF disjunct true, and a true conjunction cannot contain complementary literals. For the reverse direction, choose a conjunction term without complementary literals and assign its positive literals true and its negated literals false; extend the assignment arbitrarily to remaining variables.

Remark (Dependencies).

- Disjunctive Normal Form
- Complementary Literals
- Satisfiable Formula

Remark (Return). Return to Theorem (Existence of Negation Normal Form).

Theorem (Existence of Negation Normal Form). *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in negation normal form and*

$$\varphi \equiv \psi.$$

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Prove the result by structural induction on φ. The atomic case is already in negation normal form. The connective cases use the NNF conversion procedure, replacement of logical equivalents, and the equivalence laws that eliminate conditionals, eliminate biconditionals, remove double negations, and push negations inward.

Remark (Dependencies).

- Negation Normal Form
- NNF Conversion Procedure
- Replacement of Logical Equivalents
- Structural Induction for Propositional Formulas

Remark (Return). Return to Theorem (Semantic Classification by Normal Forms).

Theorem (Semantic Classification by Normal Forms). *Normal forms give finite certificates for semantic classification. A formula is satisfiable exactly when some DNF equivalent has a satisfiable term; it is a contradiction exactly when some DNF equivalent has no satisfiable term; it is a tautology exactly when some CNF equivalent has no falsifiable clause; and it is a contingency exactly when it is satisfiable and not a tautology.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use logical equivalence to replace φ by an equivalent normal form. For DNF, apply the DNF satisfiability criterion to detect whether at least one term can be made true; absence of such a term gives contradiction. For CNF, use the dual clause check: if no clause can be falsified, every valuation makes each clause true, so the conjunction is a tautology. Finally, contingency is the semantic case of satisfiable but not tautological.

Remark (Dependencies).

- Disjunctive Normal Form
- Conjunctive Normal Form
- Satisfiable Formula
- Contradiction
- Tautology
- Contingency
- DNF Satisfiability Criterion
- Canonical Disjunctive Normal Form Theorem
- Canonical Conjunctive Normal Form Theorem

Functional Completeness Proofs

Remark (Return). Return to Theorem (DNF Representation of Boolean Functions).

Theorem (DNF Representation of Boolean Functions). *Every Boolean function $f : \mathbb{B}^n \to \mathbb{B}$ is represented by a formula in disjunctive normal form using variables $P_1, \ldots, P_n$.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use the truth table of f. For each row on which f has value True, form the corresponding row conjunction. The disjunction of all such row conjunctions is a DNF formula representing f.

Remark (Dependencies).

- Representation of a Boolean Function
- Canonical Disjunctive Normal Form
- Canonical Disjunctive Normal Form Theorem
- Disjunctive Normal Form

Remark (Return). Return to Theorem (Conjunction and Disjunction are not Functionally Complete).

Theorem (Conjunction and Disjunction are not Functionally Complete). *The connective basis $\{\wedge, \vee\}$ is not functionally complete.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Prove by structural induction that every formula using only $\wedge$ and $\vee$ represents a monotone Boolean function. Since negation is not monotone, $\{\wedge, \vee\}$ cannot represent every Boolean function.

Remark (Dependencies).

- Incomplete Connective Set
- Monotone Boolean Function
- Functionally Complete Connective Set

Remark (Return). Return to Theorem (NAND and NOR are Minimal Complete Bases).

Theorem (NAND and NOR are Minimal Complete Bases). *The one-connective bases* $\{\uparrow\}$ *and* $\{\downarrow\}$ *are minimal complete bases.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use the functional completeness of NAND and NOR. Since each basis contains a single connective, the only proper subset is the empty basis, which is not functionally complete. Therefore each one-connective complete basis is minimal.

Remark (Dependencies).

- Minimal Complete Basis
- NAND is Functionally Complete
- NOR is Functionally Complete
- Incomplete Connective Set

Remark (Return). Return to Theorem (NAND is Functionally Complete).

Theorem (NAND is Functionally Complete). *The one-connective basis $\{\uparrow\}$ is functionally complete.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. ☐

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. ☐

Remark (Proof structure). Show that NAND defines negation and conjunction. Since $\{\neg, \wedge\}$ is functionally complete, any Boolean function can be represented by first using that basis and then replacing the defined connectives by NAND-only patterns.

Remark (Dependencies).

- NAND Connective
- NAND-Only Formula
- Functionally Complete Connective Set
- Negation and Conjunction are Functionally Complete

Remark (Return). Return to Theorem (NOR is Functionally Complete).

Theorem (NOR is Functionally Complete). *The one-connective basis* $\{\downarrow\}$ *is functionally complete.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Show that NOR defines negation and disjunction. Since $\{\neg, \vee\}$ is functionally complete, any Boolean function can be represented by first using that basis and then replacing the defined connectives by NOR-only patterns.

Remark (Dependencies).

- NOR Connective
- NOR-Only Formula
- Functionally Complete Connective Set
- Negation and Disjunction are Functionally Complete

Remark (Return). Return to Theorem (Standard Connectives are Functionally Complete).

Theorem (Standard Connectives are Functionally Complete). *The connective basis* $\{\neg, \wedge, \vee\}$ *is functionally complete.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Apply the DNF representation theorem for Boolean functions. DNF formulas use only literals, conjunctions, and disjunctions, so they use only $\neg, \wedge, \vee$.

Remark (Dependencies).

- Functionally Complete Connective Set
- DNF Representation of Boolean Functions
- Disjunctive Normal Form

Remark (Return). Return to Theorem (Negation and Conjunction are Functionally Complete).

Theorem (Negation and Conjunction are Functionally Complete). *The connective basis* $\{\neg, \wedge\}$ *is functionally complete.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. $\square$

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. $\square$

Remark (Proof structure). Use De Morgan's law to define disjunction from negation and conjunction: $\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi)$. Then translate any formula over $\{\neg, \wedge, \vee\}$ into a formula over $\{\neg, \wedge\}$.

Remark (Dependencies).

- Standard Connectives are Functionally Complete
- De Morgan Laws for Propositional Logic
- Replacement of Logical Equivalents

Remark (Return). Return to Theorem (Negation and Disjunction are Functionally Complete).

Theorem (Negation and Disjunction are Functionally Complete). *The connective basis* $\{\neg, \vee\}$ *is functionally complete.*

Proof. **Professional Standard Proof.** TODO: supply the compact proof. □

Proof. **Detailed Learning Proof.** TODO: supply the detailed learning proof. □

Remark (Proof structure). Use De Morgan's law to define conjunction from negation and disjunction: $\varphi \wedge \psi \equiv \neg(\neg\varphi \vee \neg\psi)$. Then translate any formula over $\{\neg, \wedge, \vee\}$ into a formula over $\{\neg, \vee\}$.

Remark (Dependencies).

- Standard Connectives are Functionally Complete
- De Morgan Laws for Propositional Logic
- Replacement of Logical Equivalents

Argument Forms Proofs

Exercise Proofs

Chapter 2

Predicate Logic

Where You Are in the Journey

Propositional Logic $\rightarrow$ **Predicate Calculus** $\rightarrow$ Sets & Functions $\rightarrow$ Proof Techniques $\rightarrow$ Axiom Systems $\rightarrow$ $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ $\rightarrow$ Real Analysis $\rightarrow$ $\cdots$

How we got here. Propositional logic gave us a language for combining truth values with connectives. But it cannot express "every integer has a successor" or "there exists a prime greater than 100" — statements that quantify over objects. Predicate logic provides the missing machinery.

What this chapter builds. We extend propositional syntax with variables, constants, function symbols, predicate symbols, and quantifiers. The semantics are now structures: a domain of objects plus interpretations of every symbol. The proof theory adds quantifier introduction and elimination rules. Soundness and completeness (Gödel's theorem) are the major metatheorems.

Where this leads. Every subsequent chapter uses predicate logic as its underlying language. The axiomatic construction of $\mathbb{N}$ is a predicate-logic theory. Set theory, ring theory, and topology are all first-order theories.

Structural Roadmap

The same four-layer architecture as propositional logic applies:

$$\textbf{Syntax} \longrightarrow \textbf{Semantics} \longrightarrow \textbf{Proof Theory} \longrightarrow \textbf{Metatheory}$$

The global progression is:

1. Syntax: terms, atomic formulas, well-formed formulas, free and bound variables, substitution
2. Semantics: structures, variable assignments, satisfaction, validity, logical equivalence, substitution lemmas
3. Quantifiers: universal, existential, unique existential, bounded quantifiers, negation, commutation, prenex normal form

4. Proof theory: quantifier rules (UI, UG, EI, EG), equality, soundness and completeness
5. Translation: English to predicate logic, scope, square of opposition
6. Reference: quantifier fallacies, comparison tables

Remark 2.0.1 (Primary source). The primary driver is Bjørndahl's *Logic and Proof*, supplemented by Gerstein's *Introduction to Mathematical Structures and Proofs*.

2.1 Syntax of First-Order Logic: Terms and Formulas

Terms and Formulas Quick Reference

Concept	Meaning	Detail
Variable	Symbol ranging over domain elements	Def
Term	Syntactic object denoting a domain element	Def
Atomic formula	Predicate applied to terms	Def
Well-formed formula	Recursively constructed formula	Def
Molecular formula	Non-atomic wff	Def

Definition (Variable)

A *variable* is a syntactic symbol that ranges over elements of a fixed domain of discourse.
Variables serve as placeholders in formulas and do not refer to specific objects until they are assigned values or bound by quantifiers.

Remark 2.1.1 (English reading). Variables are the unknowns of predicate logic. They pick out no particular object on their own; their value is supplied either by a variable assignment (semantics) or by a quantifier (syntax).

Remark 2.1.2 (Consequence). Because variables have no fixed denotation, the truth of a formula containing free variables depends on what values are assigned to those variables. This dependence is tracked formally by the variable assignment function s.

Definition (Term)

A *term* is a syntactic expression intended to denote an object in the domain of discourse.
The set of terms of a formal language is defined recursively:

1. **Variables.** Every variable is a term.
2. **Constants.** Every constant symbol is a term.
3. **Function application.** If f is an n-ary function symbol and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term.
4. **Closure.** No expression is a term unless it can be obtained by finitely many applications of rules (1)–(3).

Remark 2.1.3 (English reading). Terms are the noun phrases of predicate logic: they name (or describe) objects in the domain. Constants name fixed objects; variables name arbitrary ones; function symbols build complex names from simpler ones.

Remark 2.1.4 (Fully quantified form). Terms are purely syntactic objects. Under an interpretation, each term denotes an element of the domain of discourse, but the term itself is not an object of the domain.

Definition (Atomic Formula)

An *atomic formula* is a well-formed formula obtained by applying an n-ary predicate symbol to n terms.
If P is an n-ary predicate symbol and $t_1, \ldots, t_n$ are terms, then

$$P(t_1, \ldots, t_n)$$

is an atomic formula.
Atomic formulas contain no logical connectives or quantifiers and serve as the base case for the recursive definition of well-formed formulas.

Remark 2.1.5 (English reading). Atomic formulas are the simplest complete statements: they assert that a predicate (property or relation) holds of specific objects. They are the predicate-logic counterpart of propositional variables.

Definition (Well-Formed Formula)

The set of *well-formed formulas* (wffs) of a first-order language is defined recursively:

1. **Atomic formulas.** Every atomic formula is a well-formed formula.
2. **Negation.** If φ is a formula, then $\neg\varphi$ is a formula.
3. **Binary connectives.** If φ and ψ are formulas and $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then $(\varphi \circ \psi)$ is a formula.
4. **Quantification.** If φ is a formula and x is a variable, then $\forall x\, \varphi$ and $\exists x\, \varphi$ are formulas.
5. **Closure.** No expression is a formula unless it can be obtained by finitely many applications of rules (1)–(4).

Remark 2.1.6 (English reading). The wff definition says exactly which strings of symbols count as grammatical sentences of first-order logic. Everything in the language is built bottom-up from atomic formulas by the four formation rules.

Remark 2.1.7 (Distinction from propositional logic). Rule (4) is new: quantifiers bind variables and create formulas from formulas. This extra layer is what makes predicate logic more expressive than propositional logic.

Definition (Molecular Formula)

A *molecular formula* is a well-formed formula that is not atomic.
Equivalently, a formula is molecular if it is formed from one or more atomic formulas by the application of logical connectives or quantifiers.

Remark 2.1.8 (English reading). Atomic formulas express basic properties or relations. Molecular formulas express compound statements built from atomic formulas using the logical apparatus of the language.

2.2 Syntax: Free Variables, Bound Variables, and Substitution

Variables and Substitution Quick Reference

Concept	Meaning	Detail
Scope of a quantifier	Formula governed by the quantifier	Def
Bound/free occurrence	Whether x is captured by a quantifier	Def
Free variable $\mathrm{FV}(\varphi)$	Variables whose values affect truth	Def
Sentence	Formula with $\mathrm{FV}(\varphi) = \varnothing$	Def
Substitution $\varphi[t/x]$	Replace free x by term t	Def
Free for substitution	No variable in t becomes bound	Def
Alpha-equivalence	Renaming bound variables	Def

Definition (Scope of a Quantifier)

Let φ be a formula of a first-order language.
If φ is of the form $\forall x\, \psi$ or $\exists x\, \psi$, then the formula ψ is called the *scope* of the quantifier.
The quantifier is said to *bind* all occurrences of the variable x that appear within its scope.

Remark 2.2.1 (English reading). The scope is the reach of a quantifier the subformula it governs. Scope is syntactically determined by the parenthesisation of the formula, not by proximity to the quantifier symbol.

Example 2.2.2. In $(\forall x\, P(x)) \wedge Q(x)$, the scope of $\forall x$ is $P(x)$ only. The occurrence of x in $Q(x)$ falls outside the scope and is free.

Definition (Bound and Free Occurrences)

An occurrence of a variable x in a formula φ is *bound* if it lies within the scope of a quantifier $\forall x$ or $\exists x$.
An occurrence of x is *free* if it is not bound by any quantifier in φ.

Remark 2.2.3 (English reading). The same variable can have both bound and free occurrences in a single formula. Each occurrence is classified independently by checking whether it falls within the scope of a binding quantifier.

Definition (Free Variables of a Formula)

The set $\mathrm{FV}(\varphi)$ of *free variables* of a formula φ is defined recursively:

1. If $\varphi = P(t_1, \ldots, t_n)$ is atomic, then $\mathrm{FV}(\varphi) = \bigcup_{i=1}^{n} \mathrm{Var}(t_i)$, where $\mathrm{Var}(t_i)$ is the set of variables in term t_i.
2. $\mathrm{FV}(\neg\varphi) = \mathrm{FV}(\varphi)$.
3. $\mathrm{FV}(\varphi \circ \psi) = \mathrm{FV}(\varphi) \cup \mathrm{FV}(\psi)$ for any binary connective $\circ$.
4. $\mathrm{FV}(\forall x\, \varphi) = \mathrm{FV}(\varphi) \setminus \{x\}$.
5. $\mathrm{FV}(\exists x\, \varphi) = \mathrm{FV}(\varphi) \setminus \{x\}$.

Remark 2.2.4 (English reading). $\mathrm{FV}(\varphi)$ collects exactly those variables whose values can influence the truth of φ under a structure. Quantifying over x removes x from the free set because the quantifier takes responsibility for ranging over all (or some) values of x.

Remark 2.2.5 (Common error). Terms themselves contain only free variables. Variables become bound only through quantification in formulas. The interpretation of a formula depends exactly on the values assigned to its free variables.

Definition (Sentence)

A *sentence* (or *closed formula*) is a formula with no free variables.
Formally, φ is a sentence if and only if $\mathrm{FV}(\varphi) = \varnothing$.

Remark 2.2.6 (Consequence). For sentences, truth depends only on the structure, not on the variable assignment. This makes sentences the natural objects to be called true or false in a model, without qualification.

Definition (Substitution Notation)

Let φ be a formula, x a variable, and t a term.
The expression $\varphi[t/x]$ denotes the formula obtained from φ by replacing every *free* occurrence of x with the term t, leaving bound occurrences of x unchanged.

Definition (Free for Substitution)

A term t is *free for* x in φ if no free occurrence of x in φ lies within the scope of a quantifier $\forall y$ or $\exists y$ where y is a variable occurring in t.
Equivalently, t is free for x in φ if the substitution $\varphi[t/x]$ does not result in any variable in t becoming bound.

Remark 2.2.7 (Capture-avoiding substitution). A substitution $\varphi[t/x]$ is admissible only if t is free for x in φ. If this condition is violated, a variable occurring in t may become bound after substitution, silently changing the meaning of the formula. This is called ***variable capture***.

Remark 2.2.8 (Common error). Variable capture is one of the most common syntactic mistakes in predicate logic. Always check that the term being substituted introduces no variables that fall inside a binding quantifier in the target formula.

Definition (Alpha-Equivalence)

Formulas that differ only by the names of bound variables are logically equivalent. This is called *alpha-equivalence* (or *alphabetic variance*):

$$\forall x\, \varphi \;\equiv\; \forall y\, \varphi[y/x] \quad \text{(provided } y \text{ is not free in } \varphi\text{).}$$

Remark 2.2.9 (Consequence). Alpha-equivalence is the formal basis for the bound variable renaming used in prenex normal form conversion and in any proof where variable capture must be avoided. Two alpha-equivalent formulas are interchangeable in all contexts.

2.3 Syntax: Formula Depth

Formula Depth Quick Reference

Concept	Meaning	Detail
Formula depth	Max formation steps from atomic formulas	Def

Definition (Formula Depth)

The *depth* (or *complexity*) of a formula φ, denoted $\text{depth}(\varphi)$, is a natural number defined recursively:

1. **Atomic case.** If φ is atomic, then $\text{depth}(\varphi) = 0$.
2. **Negation.** If $\varphi = \neg\psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$.
3. **Binary connectives.** If $\varphi = (\psi \circ \chi)$ for $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then

$$\text{depth}(\varphi) = \max\{\text{depth}(\psi), \text{depth}(\chi)\} + 1.$$

4. **Quantifiers.** If $\varphi = \forall x\, \psi$ or $\varphi = \exists x\, \psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$.

Remark 2.3.1 (English reading). The depth of a formula measures the maximum number of logical formation steps required to build the formula from atomic formulas. It corresponds to the height of the formula's syntactic parse tree.

Remark 2.3.2 (Proof strategy). Formula depth is the standard measure used in structural induction proofs over first-order formulas. The base case is depth 0 (atomic formulas); the inductive step handles each formation rule and assumes the result holds for sub-formulas of strictly smaller depth.

2.4 Semantics: Structures and Variable Assignments

Structures Quick Reference

Concept	Meaning	Detail
First-order language $\mathcal{L}$	Signature specifying symbols	Def
Structure $\mathcal{M} = \langle D, I \rangle$	Interpretation of $\mathcal{L}$	Def
Variable assignment s	Function assigning domain elements to variables	Def
Modified assignment $s[x \mapsto d]$	Change s at x only	Def
Term interpretation $[\![t]\!]_{\mathcal{M},s}$	Semantic value of a term	Def

Definition (First-Order Language)

A *first-order language* $\mathcal{L}$ consists of:

- a set of constant symbols,
- a set of function symbols, each with a specified arity, and
- a set of predicate symbols, each with a specified arity.

Every first-order language also includes a countable set of variables and the logical symbols $\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \forall, \exists$, and (optionally) $=$.

Remark 2.4.1 (English reading). A first-order language is the alphabet of a formal system. It specifies which non-logical names are available (constants, functions, predicates) but says nothing yet about what those names mean. Meaning is supplied by a structure.

Definition (Structure)

Let $\mathcal{L}$ be a first-order language. A *structure* (or *interpretation*) for $\mathcal{L}$ is a pair

$$\mathcal{M} = \langle D, I \rangle,$$

where D is a nonempty set called the *domain of discourse* (or *universe*), and I is an interpretation function assigning:

- to each constant symbol c: an element $I(c) \in D$,
- to each n-ary function symbol f: a function $I(f) : D^n \to D$,
- to each n-ary predicate symbol P: a relation $I(P) \subseteq D^n$.

Remark 2.4.2 (English reading). A structure gives the language its meaning. The domain specifies the universe of objects being reasoned about, and the interpretation function assigns each non-logical symbol a concrete mathematical object.

Remark 2.4.3 (Nonempty domain convention). Throughout classical first-order logic, the domain D is required to be nonempty. Without this, the inference $\forall x\, \varphi \Rightarrow \exists x\, \varphi$ (subalternation) would fail: a universal statement is vacuously true in an empty domain while the existential statement is false.

Remark 2.4.4 (Source note). Some texts use the term *interpretation* and others use *structure*; the two are synonymous. The notation $\mathcal{M}$, $\mathfrak{A}$, or $\mathbf{A}$ all appear in the literature.

Definition (Variable Assignment)

Let $\mathcal{M} = \langle D, I \rangle$ be a structure for $\mathcal{L}$. A *variable assignment* is a function

$$s : \mathsf{Var} \to D$$

that assigns to each variable an element of the domain.

Remark 2.4.5 (English reading). A structure assigns meaning to non-logical symbols; a variable assignment assigns meaning to variables. Together they determine the truth value of every formula in the language.

Definition (Modified Assignment)

Let s be a variable assignment, x a variable, and $d \in D$. The *modified assignment* $s[x \mapsto d]$ is defined by

$$s[x \mapsto d](y) = \begin{cases} d & \text{if } y = x, \\ s(y) & \text{if } y \neq x. \end{cases}$$

Remark 2.4.6 (Consequence). Modified assignments are the formal device for evaluating quantified formulas. The clause $\mathcal{M}, s \models \forall x\, \varphi$ checks $\mathcal{M}, s[x \mapsto d] \models$

φ for every $d \in D$, testing the formula as x ranges over all domain elements.

Definition (Interpretation of a Term)

The *interpretation* of a term t in $\mathcal{M}$ under s, denoted $[\![t]\!]_{\mathcal{M},s}$, is defined recursively:

1. If t is a variable x: $[\![x]\!]_{\mathcal{M},s} = s(x)$.
2. If t is a constant c: $[\![c]\!]_{\mathcal{M},s} = I(c)$.
3. If $t = f(t_1, \ldots, t_n)$: $[\![f(t_1, \ldots, t_n)]\!]_{\mathcal{M},s} = I(f)\big([\![t_1]\!]_{\mathcal{M},s}, \ldots, [\![t_n]\!]_{\mathcal{M},s}\big)$.

Remark 2.4.7 (English reading). This is the semantic counterpart of the recursive term definition. Each formation rule for terms has a corresponding clause that computes a domain element from the interpretations of its components.

2.5 Semantics: Satisfaction and Truth

Satisfaction and Truth Quick Reference

Concept	Meaning	Detail
Satisfaction $\mathcal{M}, s \models \varphi$	Formula true in $\mathcal{M}$ under s	Def
Truth in a structure $\mathcal{M} \models \varphi$	True for every assignment	Def
Validity / logical truth	True in every structure	Def
Satisfiability	True in some structure	Def

Definition (Satisfaction)

Let $\mathcal{M} = \langle D, I \rangle$ be a structure, s a variable assignment, and φ a formula. The relation $\mathcal{M}, s \models \varphi$ (read "$\mathcal{M}$ satisfies φ under s") is defined recursively:

1. **Atomic.** $\mathcal{M}, s \models P(t_1, \ldots, t_n) \iff ([\![t_1]\!]_{\mathcal{M},s}, \ldots, [\![t_n]\!]_{\mathcal{M},s}) \in I(P)$.
2. **Equality.** $\mathcal{M}, s \models (t_1 = t_2) \iff [\![t_1]\!]_{\mathcal{M},s} = [\![t_2]\!]_{\mathcal{M},s}$.
3. **Negation.** $\mathcal{M}, s \models \neg\varphi \iff \mathcal{M}, s \not\models \varphi$.
4. **Conjunction.** $\mathcal{M}, s \models (\varphi \wedge \psi) \iff \mathcal{M}, s \models \varphi$ and $\mathcal{M}, s \models \psi$.
5. **Disjunction.** $\mathcal{M}, s \models (\varphi \vee \psi) \iff \mathcal{M}, s \models \varphi$ or $\mathcal{M}, s \models \psi$.
6. **Implication.** $\mathcal{M}, s \models (\varphi \to \psi) \iff \mathcal{M}, s \not\models \varphi$ or $\mathcal{M}, s \models \psi$.
7. **Biconditional.** $\mathcal{M}, s \models (\varphi \leftrightarrow \psi) \iff (\mathcal{M}, s \models \varphi \Leftrightarrow \mathcal{M}, s \models \psi)$.
8. **Universal quantification.** $\mathcal{M}, s \models \forall x\, \varphi \iff$ for all $d \in D$, $\mathcal{M}, s[x \mapsto d] \models \varphi$.
9. **Existential quantification.** $\mathcal{M}, s \models \exists x\, \varphi \iff$ there exists $d \in D$ such that $\mathcal{M}, s[x \mapsto d] \models \varphi$.

Remark 2.5.1 (English reading). Satisfaction is the bridge between syntax and semantics. It tells us, for each formula formation rule, what it means for the formula to hold. Clauses (1)-(7) mirror the propositional connectives; clauses (8)-(9) are the new quantifier clauses unique to first-order logic.

Remark 2.5.2 (Fully quantified form for universal quantification). The clause $\mathcal{M}, s \models \forall x\, \varphi$ holds if and only if for every element d of the domain D, the formula φ is satisfied under the assignment that agrees with s everywhere except that it sends x to d. This is what it means for a property to hold universally.

Definition (Truth in a Structure)

A sentence φ is *true* in a structure $\mathcal{M}$, written $\mathcal{M} \models \varphi$, if $\mathcal{M}, s \models \varphi$ for every variable assignment s.
A sentence φ is *false* in $\mathcal{M}$ if $\mathcal{M} \not\models \varphi$.

Remark 2.5.3 (English reading). For sentences (formulas with no free variables), the choice of assignment is irrelevant all assignments agree. So $\mathcal{M} \models \varphi$ is well-defined without reference to any particular s.

Definition (Validity and Satisfiability)

A formula φ is:

- *valid* (or a *logical truth*) if $\mathcal{M}, s \models \varphi$ for every structure $\mathcal{M}$ and every assignment s;
- *satisfiable* if $\mathcal{M}, s \models \varphi$ for some $\mathcal{M}$ and some s;
- *unsatisfiable* (or a *contradiction*) if it is not satisfiable.

Remark 2.5.4 (Consequence). A formula is valid if no structure can make it false. These are the logical truths of predicate logic the formulas provable by logic alone, without any special assumptions about the domain.

2.6 Semantics: Models, Theories, and Logical Consequence

Models and Theories Quick Reference

Concept	Meaning	Detail
Model $\mathcal{M} \models \Gamma$	Structure satisfying all sentences of Γ	Def
Theory	Set of sentences; consistent if it has a model	Def
Logical consequence $\Gamma \models \varphi$	True in every model of Γ	Def
Logical equivalence $\varphi \equiv \psi$	Mutually entailing formulas	Def

Definition (Model)

Let φ be a sentence and $\mathcal{M}$ a structure. We say that $\mathcal{M}$ is a *model* of φ if $\mathcal{M} \models \varphi$. More generally, if Γ is a set of sentences, then $\mathcal{M}$ is a model of Γ if $\mathcal{M} \models \gamma$ for every $\gamma \in \Gamma$. In this case, we write $\mathcal{M} \models \Gamma$.

Definition (Theory)

A *theory* is a set of sentences.
A theory T is *satisfiable* (or *consistent*) if it has at least one model.
A theory T is *complete* if for every sentence φ in the language, either $T \models \varphi$ or $T \models \neg\varphi$.

Remark 2.6.1 (Source note). In some texts, a theory is required to be closed under logical consequence: if $T \models \varphi$ then $\varphi \in T$. In other texts, a theory is simply any set of axioms from which consequences are derived. Both usages appear in the literature.

Definition (Logical Consequence)

Let Γ be a set of formulas and φ a formula. We say that φ is a *logical consequence* of Γ, written $\Gamma \models \varphi$, if for every structure $\mathcal{M}$ and every variable assignment s:

$$\text{if } \mathcal{M}, s \models \gamma \text{ for all } \gamma \in \Gamma, \text{ then } \mathcal{M}, s \models \varphi.$$

Equivalently, every model of Γ is a model of φ.

Remark 2.6.2 (English reading). Logical consequence says: if all the hypotheses in Γ are true in some structure, then φ must be true there too. This is the semantic notion of entailment.

Remark 2.6.3 (Notation). The notation $\models \varphi$ (empty left-hand side) means φ is valid: it is a logical consequence of the empty set of premises.

Definition (Logical Equivalence)

Two formulas φ and ψ are *logically equivalent*, written $\varphi \equiv \psi$, if each is a logical consequence of the other:

$$\varphi \equiv \psi \quad\Longleftrightarrow\quad (\varphi \models \psi \text{ and } \psi \models \varphi).$$

Equivalently, φ and ψ have the same truth value in every structure under every variable assignment.

Remark 2.6.4 (Consequence). Logical equivalence is the semantic counterpart of provable bi-implication. Two logically equivalent formulas are interchangeable in any context without changing truth values.

2.7 Semantics: Predicates, Relations, and Substitution Lemmas

Predicates and Substitution Lemmas Quick Reference

Concept	Meaning	Detail
Predicate	Open formula or truth-valued function on domain	Def
Substitution Lemma (terms)	Assign vs. substitute, same result	Lem
Substitution Lemma (formulas)	Syntactic substitution = semantic update	Lem

Definition (Predicate)

A *predicate* is an expression containing one or more variables that represents a property or relation and becomes a proposition when all its variables are instantiated. Formally, a predicate has two equivalent views:

1. **Syntactic view.** A predicate is an open formula $\varphi(x_1, \dots, x_n)$ that has no truth value until specific objects are substituted for its variables.

2. **Semantic view.** Given a domain D, an n-ary predicate determines a function $P : D^n \to \{\mathsf{T}, \mathsf{F}\}$.

Remark 2.7.1 (Predicates vs. relations). A *predicate* is a syntactic or semantic device: syntactically, an open formula; semantically, a truth-valued function. A *relation* is a purely set-theoretic object: an n-ary relation on D is a subset $R \subseteq D^n$. Under an interpretation, predicates and relations correspond via $P(a_1, \dots, a_n)$ is true $\iff$ $(a_1, \dots, a_n) \in R$. Thus predicates belong to the language of logic, while relations belong to the structures interpreting that language.

Lemma 2.7.2 (Substitution Lemma for Terms). *Let $\mathcal{M}$ be a structure, s a variable assignment, x a variable, and $d \in D$. For any term t,*

$$[\![t]\!]_{\mathcal{M},\, s[x \mapsto d]} = [\![t[d/x]]\!]_{\mathcal{M},\, s}.$$

Remark 2.7.3 (English reading). Evaluating a term after modifying an assignment is the same as substituting the value directly into the term and then evaluating. This shows that syntactic substitution and semantic update commute for terms.

Lemma 2.7.4 (Substitution Lemma for Formulas). *Let φ be a formula, t a term free for x in φ. Then for any structure $\mathcal{M}$ and assignment s,*

$$\mathcal{M}, s \models \varphi[t/x] \quad \iff \quad \mathcal{M},\, s\big[x \mapsto [\![t]\!]_{\mathcal{M},s}\big] \models \varphi.$$

Remark 2.7.5 (English reading). Satisfying φ with x replaced by t is the same as satisfying φ under the assignment that sends x to the value of t. This lemma formally connects syntactic substitution with semantic evaluation and is essential for proving the soundness of Universal Instantiation.

Remark 2.7.6 (Proof strategy). Both lemmas are proved by structural induction on the term (Lemma for terms) or formula (Lemma for formulas). The key cases are variables and quantifiers; the remaining cases follow directly from the inductive hypothesis.

2.8 Quantifiers: Universal, Existential, and Bounded

Basic Quantifiers Quick Reference

Symbol	Reading	Meaning	Detail
$\forall x\, \varphi$	For all x, φ	True for every domain element	Def
$\exists x\, \varphi$	There exists x such that φ	True for some domain element	Def
$\exists! x\, \varphi$	There exists exactly one x such that φ	Existence + uniqueness	Def
$\forall x \in A\, \varphi$	For all x in A, φ	Restricted universal	Def
$\exists x \in A\, \varphi$	There exists x in A such that φ	Restricted existential	Def

Definition (Universal Quantifier)

The *universal quantifier*, denoted $\forall$, is a logical operator that binds a variable and asserts that a formula holds for all elements of the domain of discourse.
If φ is a formula and x is a variable, then $\forall x\, \varphi$ is a formula, read "for all x, φ."

Remark 2.8.1 (English reading). The universal quantifier expresses a global condition over the entire domain. It does not claim the domain is nonempty on its own, but by the nonempty domain convention, there is always at least one element to instantiate.

Definition (Existential Quantifier)

The *existential quantifier*, denoted $\exists$, is a logical operator that binds a variable and asserts that a formula holds for at least one element of the domain of discourse.
If φ is a formula and x is a variable, then $\exists x\, \varphi$ is a formula, read "there exists an x such that φ."

Remark 2.8.2 (English reading). The existential quantifier expresses a local condition. It does not specify which element witnesses the claim; it merely asserts that such a witness exists somewhere in the domain.

Definition (Unique Existential Quantifier)

The *unique existential quantifier* $\exists!$ asserts that there exists exactly one element satisfying a given property. It is an abbreviation:

$$\exists! x\, \varphi \;:=\; \exists x\, \big(\varphi \wedge \forall y\, (\varphi[y/x] \to y = x)\big),$$

where y is a variable not occurring in φ.
Equivalently,

$$\exists! x\, \varphi \;\equiv\; \exists x\, \varphi \;\wedge\; \forall x\, \forall y\, \big((\varphi \wedge \varphi[y/x]) \to x = y\big).$$

Remark 2.8.3 (English reading). $\exists! x\, \varphi$ combines existence (at least one x satisfies φ) and uniqueness (at most one such x exists). It is not a primitive quantifier but a convenient abbreviation.

Example 2.8.4. "There is exactly one even prime number" formalizes as $\exists! x\, (P(x) \wedge E(x))$, where $P(x)$ means "x is prime" and $E(x)$ means "x is even."

Definition (Bounded Quantifiers)

Let A be a set (or a predicate defining a set). The *bounded quantifiers* are abbreviations:

$$\forall x \in A\, \varphi \;:=\; \forall x\, (x \in A \to \varphi),$$
$$\exists x \in A\, \varphi \;:=\; \exists x\, (x \in A \wedge \varphi).$$

Remark 2.8.5 (Asymmetry). Bounded universal uses implication ($\to$), while bounded existential uses conjunction ($\wedge$). This is not arbitrary: if the set A is empty, $\forall x \in A\, \varphi$ is vacuously true (there is no x to violate the implication), while $\exists x \in A\, \varphi$ is false (no witness exists).

Remark 2.8.6 (Common error). Writing $\forall x \in A\, \varphi$ as $\forall x\, (x \in A \wedge \varphi)$ is wrong: this would assert that every element is in A, not just that every element of A satisfies φ.

Theorem 2.8.7 (Negation of Bounded Quantifiers). *Let A be a set and φ a formula. Then:*

$$\neg(\forall x \in A\, \varphi) \;\equiv\; \exists x \in A\, \neg\varphi,$$
$$\neg(\exists x \in A\, \varphi) \;\equiv\; \forall x \in A\, \neg\varphi.$$

Remark (Fully quantified form). $\neg(\forall x\, (x \in A \to \varphi)) \;\equiv\; \exists x\, (x \in A \wedge \neg\varphi)$, and $\neg(\exists x\, (x \in A \wedge \varphi)) \;\equiv\; \forall x\, (x \in A \to \neg\varphi)$.

Remark 2.8.8. The domain restriction $x \in A$ is preserved under negation because it functions as a constraint on which elements are quantified over, not as part of the claim being negated.

2.9 Quantifiers: Quantifier Laws

Quantifier Laws Quick Reference

Law	Equivalence	Condition	Detail
Negation (1)	$\neg\forall x\,\varphi \equiv \exists x\,\neg\varphi$		Thm
Negation (2)	$\neg\exists x\,\varphi \equiv \forall x\,\neg\varphi$		Thm
Same-type commutation	$\forall x\,\forall y\,\varphi \equiv \forall y\,\forall x\,\varphi$	same type	Thm
Distribution $\forall/\wedge$	$\forall x\,(\varphi \wedge \psi) \equiv (\forall x\,\varphi) \wedge (\forall x\,\psi)$		Thm
Distribution $\exists/\vee$	$\exists x\,(\varphi \vee \psi) \equiv (\exists x\,\varphi) \vee (\exists x\,\psi)$		Thm
Vacuous $\forall$	$\forall x\,\varphi \equiv \varphi$	$x \notin \mathrm{FV}(\varphi)$	Thm
Renaming	$\forall x\,\varphi \equiv \forall y\,\varphi[y/x]$	y not free in φ	Thm

Proposition 2.9.1 (Quantifier Negation Laws). *For any formula φ:*

$$\neg\forall x\,\varphi \;\equiv\; \exists x\,\neg\varphi,$$
$$\neg\exists x\,\varphi \;\equiv\; \forall x\,\neg\varphi.$$

Remark 2.9.2 (English reading). To negate a universally quantified statement, swap $\forall$ for $\exists$ and negate the inner formula. To negate an existential, swap $\exists$ for $\forall$ and negate. The negation pushes through the quantifier, flipping it.

Remark 2.9.3 (Procedure for negating nested quantifiers). When negating a statement with multiple quantifiers, push the negation inward past every quantifier, flipping each one, until it reaches the atomic predicate. For example:

$$\neg(\forall x\,\exists y\,\varphi) \equiv \exists x\,\neg(\exists y\,\varphi) \equiv \exists x\,\forall y\,\neg\varphi.$$

Remark 2.9.4 (General schema for negating quantified implications). For $Q_1x_1\cdots Q_nx_n(\varphi$ $\psi)$ where each $Q_i \in \{\forall, \exists\}$:

$$\neg\big(Q_1x_1\cdots Q_nx_n(\varphi \to \psi)\big) \;\equiv\; Q'_1x_1\cdots Q'_nx_n(\varphi \wedge \neg\psi),$$

where $Q'_i = \exists$ when $Q_i = \forall$, and $Q'_i = \forall$ when $Q_i = \exists$.

Remark 2.9.5 (Common error). Negating only the predicate without flipping the quantifier is incorrect: $\neg(\forall x\,P(x)) \not\equiv \forall x\,\neg P(x)$. The right side asserts that *every* element fails P, which is much stronger than merely asserting that not every element satisfies P.

Proposition 2.9.6 (Quantifier Commutation). *Quantifiers of the same type commute:*

$$\forall x\,\forall y\,\varphi \;\equiv\; \forall y\,\forall x\,\varphi,$$
$$\exists x\,\exists y\,\varphi \;\equiv\; \exists y\,\exists x\,\varphi.$$

However, quantifiers of different types do not *commute in general:*

$$\forall x\,\exists y\,\varphi \;\not\equiv\; \exists y\,\forall x\,\varphi.$$

Remark 2.9.7 (Common error). Swapping $\forall$ and $\exists$ changes the meaning. The statement $\exists y\,\forall x\,\varphi$ requires a single uniform witness y that works for all x, whereas $\forall x\,\exists y\,\varphi$ allows a different y for each x.

Proposition 2.9.8 (Quantifier Distribution). *The following distribution equivalences hold:*

$$\begin{aligned}\forall x\,(\varphi \wedge \psi) &\equiv (\forall x\,\varphi) \wedge (\forall x\,\psi),\\ \exists x\,(\varphi \vee \psi) &\equiv (\exists x\,\varphi) \vee (\exists x\,\psi).\end{aligned}$$

The following do not *hold in general:*

$$\begin{aligned}\forall x\,(\varphi \vee \psi) &\not\equiv (\forall x\,\varphi) \vee (\forall x\,\psi),\\ \exists x\,(\varphi \wedge \psi) &\not\equiv (\exists x\,\varphi) \wedge (\exists x\,\psi).\end{aligned}$$

Additionally, when $x \notin \mathrm{FV}(\psi)$:

$$\begin{aligned}\forall x\,(\varphi \wedge \psi) &\equiv (\forall x\,\varphi) \wedge \psi,\\ \exists x\,(\varphi \vee \psi) &\equiv (\exists x\,\varphi) \vee \psi.\end{aligned}$$

Remark 2.9.9 (Mnemonic). Universal distributes over conjunction ($\forall/\wedge$ match because both are "all-or-nothing"); existential distributes over disjunction ($\exists/\vee$ match because both assert "at least one").

Proposition 2.9.10 (Vacuous Quantification). *If x does not occur free in φ, then:*

$$\begin{aligned}\forall x\,\varphi &\equiv \varphi,\\ \exists x\,\varphi &\equiv \varphi.\end{aligned}$$

Remark 2.9.11 (English reading). A quantifier is vacuous if its bound variable does not appear free in the scope. Such quantifiers may be freely added or removed without changing meaning.

Proposition 2.9.12 (Renaming Bound Variables). *If y does not occur in φ, then:*

$$\begin{aligned}\forall x\,\varphi &\equiv \forall y\,\varphi[y/x],\\ \exists x\,\varphi &\equiv \exists y\,\varphi[y/x].\end{aligned}$$

Remark 2.9.13 (Consequence). This is a direct consequence of alpha-equivalence (Definition 2.2). Renaming bound variables is the standard technique for avoiding variable capture when applying substitution rules.

2.10 Quantifiers: Prenex Normal Form

Prenex Normal Form Quick Reference

Concept	Meaning	Detail
Prenex normal form (PNF)	All quantifiers at front, quantifier-free matrix	Def
PNF Theorem	Every formula is equivalent to a PNF formula	Thm
Quantifier-pulling rules	Equivalences for extracting quantifiers	Def

Definition (Prenex Normal Form)

A first-order formula is in *prenex normal form* (PNF) if it has the shape

$$Q_1x_1\,Q_2x_2\cdots Q_nx_n\;\;\psi,$$

where each $Q_i\in\{\forall,\exists\}$ and ψ is quantifier-free. The string $Q_1x_1\cdots Q_nx_n$ is the *quantifier prefix*; ψ is the *matrix*.

Theorem (Prenex Normal Form Theorem)

Every first-order formula is logically equivalent to a formula in prenex normal form.

Remark 2.10.1 (Proof strategy conversion procedure). Given a formula φ, the following four steps produce an equivalent PNF:

1. **Eliminate $\leftrightarrow$ and $\to$.** Rewrite using only $\neg,\wedge,\vee$: $(A\to B)\equiv(\neg A\vee B)$.

2. **Push negations inward (Negation Normal Form).** Use De Morgan's laws and quantifier-negation laws until $\neg$ applies only to atomic formulas.

3. **Standardize bound variables apart.** Rename bound variables so that no variable is quantified twice and no bound variable coincides with any free variable.

4. **Pull quantifiers outward.** Apply the quantifier-pulling equivalences (below) to move all quantifiers to the front.

Remark 2.10.2 (Quantifier-pulling equivalences). The following hold when $x\notin\mathrm{FV}(\psi)$:

Equivalence	Side condition
$(\forall x\,\phi)\wedge\psi\equiv\forall x\,(\phi\wedge\psi)$	$x\notin\mathrm{FV}(\psi)$
$(\exists x\,\phi)\vee\psi\equiv\exists x\,(\phi\vee\psi)$	$x\notin\mathrm{FV}(\psi)$
$(\forall x\,\phi)\vee\psi\equiv\forall x\,(\phi\vee\psi)$	$x\notin\mathrm{FV}(\psi)$
$(\exists x\,\phi)\wedge\psi\equiv\exists x\,(\phi\wedge\psi)$	$x\notin\mathrm{FV}(\psi)$

Example 2.10.3 (PNF conversion). Normalize $\varphi:=\neg\big(\forall x\,P(x)\to\exists y\,Q(y)\big)$.
Step 1 (eliminate $\to$): $\varphi\equiv\neg\big(\neg\forall x\,P(x)\vee\exists y\,Q(y)\big)$.
Step 2 (push $\neg$ inward): $\varphi\equiv(\forall x\,P(x))\wedge(\forall y\,\neg Q(y))$.
Step 3 (standardize apart): Variables are already distinct.
Step 4 (pull quantifiers outward): $\varphi\equiv\forall x\,\forall y\,(P(x)\wedge\neg Q(y))$.

2.11 Quantifiers: Logical Strength and Quantifier Order

Logical Strength Quick Reference

Concept	Meaning	Detail
Entailment $A \models B$	A is true implies B is true	Def
Logical strength	A stronger than B if $A \models B$ but $B \not\models A$	Def
Model-set viewpoint	$A \models B \iff \mathrm{Mod}(A) \subseteq \mathrm{Mod}(B)$	Def
Quantifier alternation	Alternating $\forall/\exists$ in prefix	Def

Definition (Logical Entailment)

Let A and B be sentences. We write $A \models B$ and say that A *entails* B if in every structure in which A is true, B is also true. Equivalently, $A \models B$ means $A \to B$ is valid.

Definition (Logical Strength)

We say that A *is (logically) stronger than* B if $A \models B$ but $B \not\models A$.
A and B are *logically equivalent* if $A \models B$ and $B \models A$.

Remark 2.11.1 (English reading). "Stronger" means "harder to satisfy" (fewer models make it true). If A is stronger than B, then A rules out more possibilities: every world where A holds is a world where B holds, but not conversely.

Definition (Model-Set Viewpoint)

For a statement A, let $\mathrm{Mod}(A)$ denote the class of all structures in which A is true. Then

$$A \models B \quad \iff \quad \mathrm{Mod}(A) \subseteq \mathrm{Mod}(B).$$

Remark 2.11.2 (English reading). Strength is set inclusion of model classes. "A is stronger than B" is literally $\mathrm{Mod}(A) \subsetneq \mathrm{Mod}(B)$: A satisfies fewer structures than B.

Definition (Quantifier Alternation)

A *quantifier alternation* occurs when $Q_i \neq Q_{i+1}$ for some i in the prefix $Q_1x_1 \cdots Q_kx_k$ of a formula. For example, $\forall x\, \exists y\, \Phi(x, y)$ has one alternation, while $\forall x\, \forall y\, \Phi(x, y)$ has none.

Remark 2.11.3 (Why alternation matters for strength). $\forall x\, \exists y\, \Phi(x, y)$ requires a rule assigning a (possibly x-dependent) witness y to each x. In contrast, $\exists y\, \forall x\, \Phi(x, y)$ requires a single uniform witness y that works for all x. Uniformity is strictly stronger than dependence.

Remark 2.11.4 (Strength hierarchy). The following entailments hold:

1. $\forall x\, \Phi(x) \models \exists x\, \Phi(x)$ (universal is stronger).

2. $\exists y\,\forall x\,\Phi(x,y) \models \forall x\,\exists y\,\Phi(x,y)$ (uniform witness is stronger than dependent witness).

Neither converse holds in general.

Example 2.11.5 (Swap failure on $\mathbb{R}$). Let the domain be $\mathbb{R}$ and $\Phi(x,y) = (y > x)$.

$\forall x\,\exists y\,(y > x)$ is true: given any x, take $y = x + 1$.
$\exists y\,\forall x\,(y > x)$ is false: no single real exceeds all reals.
Hence $\exists y\,\forall x$ is strictly stronger than $\forall x\,\exists y$.

Example 2.11.6 (Universal vs. existential). Let the domain be $\mathbb{R}$ and $\Phi(x) = (x^2 \geq 0)$.

$\forall x\,(x^2 \geq 0)$ entails $\exists x\,(x^2 \geq 0)$, but not conversely. Thus the universal statement is strictly stronger.

Remark 2.11.7 (Logical strength increases when you:).

- add conjuncts ($A \wedge B$ is stronger than A),
- replace $\exists$ with $\forall$,
- move existential quantifiers outward (requiring uniform witnesses),
- introduce quantifier alternation.

2.12 Proof Theory: Quantifier Inference Rules

Quantifier Inference Rules Quick Reference

Rule	Schema	Key restriction	Detail
UI	$\forall x\,\varphi \Rightarrow \varphi[t/x]$	t free for x	Def
UG	$\varphi \Rightarrow \forall x\,\varphi$	x not free in assumptions	Def
EI	$\exists x\,\varphi \Rightarrow \varphi[c/x]$	c fresh witness	Def
EG	$\varphi[t/x] \Rightarrow \exists x\,\varphi$	none	Def

Definition (Universal Instantiation UI)

From a universally quantified statement, infer any instance obtained by substituting a term for the bound variable:

$$\forall x\,\varphi \;\Rightarrow\; \varphi[t/x].$$

Remark 2.12.1 (Side condition). The term t must be free for x in φ. If t contains a variable that would become bound after substitution, the inference is invalid.

Definition (Universal Generalization UG)

From a formula, infer a universally quantified statement:

$$\varphi \;\Rightarrow\; \forall x\,\varphi.$$

Remark 2.12.2 (Restriction on UG). The variable x must not occur free in any undischarged assumption on which φ depends. This restriction ensures that x is truly arbitrary not tied to a specific hypothesis about x.

Remark 2.12.3 (Arbitrary element argument). To prove $\forall x\, \varphi(x)$, fix an arbitrary element x (introducing no special assumptions about x), prove $\varphi(x)$, and then apply UG. The variable x must remain unconstrained throughout.

Definition (Existential Instantiation EI)

From an existentially quantified statement, infer an instance with a fresh constant (witness):

$$\exists x\, \varphi \;\Rightarrow\; \varphi[c/x].$$

Remark 2.12.4 (Witness discipline for EI). The constant c must be fresh: it must not appear in φ, in any undischarged assumption, or in the final conclusion of the proof. The constant represents an arbitrary but fixed witness to the existential claim, not a specific known object.

Remark 2.12.5 (Common error). Using a witness constant introduced by EI outside its allowed scope, or choosing c before establishing the existential premise, invalidates the proof.

Definition (Existential Generalization EG)

From a formula containing a term, infer an existential statement:

$$\varphi[t/x] \;\Rightarrow\; \exists x\, \varphi.$$

Remark 2.12.6 (No side conditions). EG has no side conditions: any term t may be replaced by an existentially quantified variable. To prove $\exists x\, \varphi(x)$, explicitly exhibit a term t such that $\varphi(t)$ holds, then apply EG.

Remark 2.12.7 (Counterexample argument). To refute $\forall x\, \varphi(x)$: produce a single term t such that $\neg\varphi(t)$ holds. To refute $\exists x\, \varphi(x)$: show that $\varphi(t)$ fails for every term t in the domain.

2.13 Proof Strategies for Quantified Statements

Quantifier Proof Strategies Quick Reference

Goal shape	Strategy	Key rule
Prove $\forall x\, \varphi(x)$	Fix an arbitrary x; prove $\varphi(x)$ for that x	UG
Prove $\exists x\, \varphi(x)$	Produce a specific witness t; verify $\varphi(t)$	EG
Use $\forall x\, \varphi(x)$	Instantiate at any term t to get $\varphi(t)$	UI
Use $\exists x\, \varphi(x)$	Introduce a fresh name c for the witness	EI
Negate $\forall x\, \varphi$	Becomes $\exists x\, \neg\varphi$	Neg. law
Negate $\exists x\, \varphi$	Becomes $\forall x\, \neg\varphi$	Neg. law
Prove $\forall x\, \exists y\, \varphi(x,y)$	Fix x; then construct y depending on x	UG then EG
Prove $\exists y\, \forall x\, \varphi(x,y)$	Find y first; then verify for all x	EG then UG

The inference rules UI, UG, EI, and EG were defined in the previous section. That section answered the question: what does each rule license? This section answers a different question: when the goal of a proof has a particular quantifier structure, which rule should be used, and what does the proof skeleton look like? The two questions are distinct, and both must be understood clearly before reading analysis proofs.

The single most important organizing principle is this: *the shape of the goal determines the proof structure, and the shape of the hypothesis determines how to use it.* A $\forall$ goal calls for a different move than a $\exists$ goal; a $\forall$ hypothesis is used differently from a $\exists$ hypothesis. The four combinations are the four strategies below.

Strategy 1: Proving a Universal $\forall x\, \varphi(x)$

To prove $\forall x\, \varphi(x)$: introduce a fresh variable x_0 representing an *arbitrary, unspecified* element of the domain. Prove $\varphi(x_0)$ without making any assumption about x_0 beyond its being in the domain. Then apply Universal Generalization (UG) to conclude $\forall x\, \varphi(x)$.

> **Let** x_0 be an arbitrary element of the domain.
> **[prove $\varphi(x_0)$ using only the hypotheses, not any special property of x_0]**
> **Therefore:** $\forall x\, \varphi(x)$. (by UG)

The critical constraint is that x_0 must be *arbitrary*: no hypothesis in the proof may impose a condition on x_0 that is not universally true. Violating this constraint is the UG fallacy ("illicit generalization").

Remark 2.13.1 (Why "let x be arbitrary" works). Fixing an arbitrary element x_0 is the formal counterpart of the English phrase "let x be any element." The word "arbitrary" means: the proof uses no information about x_0 that is specific to x_0 -- only information that applies to every element of the domain. If the argument goes through for such an unspecified x_0, it goes through for every x, so the universal follows.

Remark 2.13.2 (The UG side condition in practice). UG requires that x_0 not appear free in any undischarged assumption. In practice: do not use any hypothesis of the form "x_0 has property P" unless P holds of every domain element. If a hypothesis says "$x_0 > 0$," then x_0 is not arbitrary, and UG to $\forall x$ is invalid.

Strategy 2: Proving an Existential $\exists x\, \varphi(x)$

To prove $\exists x\, \varphi(x)$: exhibit a specific term t and verify that $\varphi(t)$ holds. Then apply Existential Generalization (EG) to conclude $\exists x\, \varphi(x)$.

> **Claim:** $\exists x\, \varphi(x)$.
> **Take:** $t :=$ [specific term].
> **Verify:** $\varphi(t)$ holds. [proof that t works]
> **Therefore:** $\exists x\, \varphi(x)$. (by EG)

The entire burden of an existential proof is finding the right t. Once t is identified and verified, the logic is trivial: one application of EG.

Remark 2.13.3 (Where the difficulty lies). In existential proofs, the logical structure is simple but the mathematical insight is often deep. Finding the witness requires understanding the problem; verifying the witness is usually straightforward. This is why analysis proofs often say "take $\delta = \min(\varepsilon/2, 1)$" without explaining where that choice came from -- the work happened before the formal proof was written.

Strategy 3: Using a Universal $\forall x\, \varphi(x)$

If a hypothesis or established fact has the form $\forall x\, \varphi(x)$, one can instantiate it at *any* specific term t to obtain $\varphi(t)$, by Universal Instantiation (UI). There are no side conditions beyond t being free for x in φ.

Available: $\forall x\, \varphi(x)$.
Choose any term t.
Obtain: $\varphi(t)$. (by UI)

UI can be applied repeatedly to the same universal statement with different choices of t.

Strategy 4: Using an Existential $\exists x\, \varphi(x)$

If a hypothesis has the form $\exists x\, \varphi(x)$, introduce a *fresh constant* c to name the (unknown, unspecified) witness, obtaining $\varphi(c)$ by Existential Instantiation (EI). The constant c must be new: it must not appear anywhere else in the proof.

Available: $\exists x\, \varphi(x)$.
Let c be a witness. (fresh constant, not used elsewhere)
Obtain: $\varphi(c)$. (by EI)

The freshness requirement is not a technicality. Using a constant that already appears in the proof — say, a constant a introduced under a different assumption — would illegally identify the witness with that specific element. The witness c is only known to satisfy $\varphi(c)$; nothing else about it is known, and it cannot be equated with anything specific.

Remark 2.13.4 (The scope of a witness constant). Once c is introduced by EI, it names the witness for the duration of the argument. Conclusions derived using c depend on the existence of such a witness; they are valid (they do not depend on which specific element c actually names). However, c cannot be exported outside the scope in which the existential was instantiated, and no universal generalization may be applied to c.

The Central Asymmetry: Universal vs. Existential Goals

The following comparison is the most important thing to understand about quantifier proof structure. Universal and existential goals require fundamentally different strategies, and confusing them is one of the most common serious errors in mathematical writing.

Universal vs. Existential Goals: The Fundamental Difference

	$\forall x\, \varphi(x)$	$\exists x\, \varphi(x)$
To prove:	Fix *arbitrary* x; prove $\varphi(x)$	*Choose* specific t; verify $\varphi(t)$
Who chooses x?	The adversary (you cannot choose)	You (the prover)
Main rule:	UG	EG
Burden:	Works for every x without assumptions	A single witness suffices
Error:	Assuming something special about x	Failing to exhibit a concrete t

Remark 2.13.5 (Who controls the variable). For a universal goal, the variable x is chosen by the "opponent" of the proof: the proof must work no matter what x is. The prover has no control over x; hence the term "arbitrary." For an existential goal, the prover chooses the witness t; the power is entirely on the prover's side, but so is the obligation to produce something concrete and verifiable.

Nested Quantifiers: The ∀∃ Pattern in Analysis

Every ε-δ proof in analysis is a proof of a statement with the pattern $\forall\varepsilon\, \exists\delta \cdots$. Understanding the proof structure of such statements is the direct payoff of this section.

Proof Skeleton: $\forall x\, \exists y\, \varphi(x, y)$

Step 1. Fix arbitrary x. (Opening move for the outer $\forall$; x is unspecified. This is Strategy 1.)

Step 2. Define y in terms of x. (The witness for $\exists y$ may, and typically does, depend on x. This is Strategy 2 choosing a witness.)

Step 3. Verify $\varphi(x, y)$ using the specific y from Step 2 and the arbitrary x from Step 1.

Step 4. Conclude: $\exists y\, \varphi(x, y)$. (by EG)

Step 5. Conclude: $\forall x\, \exists y\, \varphi(x, y)$. (by UG, since x was arbitrary in Step 1)

The key insight: y is chosen *after* x is fixed, so y is allowed to depend on x.

Proof Skeleton: $\exists y\, \forall x\, \varphi(x, y)$

Step 1. Define y. (The witness for $\exists y$ must be chosen *before* x is introduced. This y must work for *every* x.)

Step 2. Fix arbitrary x. (Now x is introduced.)

Step 3. Verify $\varphi(x, y)$ for the specific y from Step 1 and arbitrary x from Step 2.

Step 4. Conclude: $\forall x\, \varphi(x, y)$. (by UG)

Step 5. Conclude: $\exists y\, \forall x\, \varphi(x, y)$. (by EG)

The key contrast: y is chosen *before* x, so y cannot depend on x. This is why $\exists y\, \forall x$ is a much stronger claim.

Remark 2.13.6 (Why order determines strength). In the $\forall\exists$ skeleton, the witness y is constructed from x, so a different y is allowed for each x. In the $\exists\forall$ skeleton, a single y must work for all x simultaneously. The first pattern appears in continuity (δ may depend on ε and x); the second in uniform continuity (δ may depend on ε but not on x). The difference between continuity and uniform continuity is exactly the difference between these two proof skeletons.

A Complete Worked Example

The following proof traces every UI/UG/EI/EG step explicitly. In practice, proofs are written more concisely, but the logical structure is always this one.

Example 2.13.7 (A worked multi-quantifier proof). **Claim:** If $\forall x\, \exists y\, P(x, y)$ and $\forall x\, \forall y\, (P(x, y) \to Q(x, y))$, then $\forall x\, \exists y\, Q(x, y)$.

Proof:

1. Assume $H_1 := \forall x\, \exists y\, P(x, y)$ and $H_2 := \forall x\, \forall y\, (P(x, y) \to Q(x, y))$. (hypotheses)
2. Let a be an arbitrary element of the domain. (introduce arbitrary variable for $\forall x$ in the goal)
3. From H_1, instantiate at a: $\exists y\, P(a, y)$. (by UI applied to H_1 with $t = a$)
4. Let b be a witness: $P(a, b)$. (by EI applied to Step 3; b is fresh)
5. From H_2, instantiate at a: $\forall y\, (P(a, y) \to Q(a, y))$. (by UI applied to H_2 with $t = a$)
6. From Step 5, instantiate at b: $P(a, b) \to Q(a, b)$. (by UI applied to Step 5 with $t = b$)
7. From Step 4 ($P(a, b)$) and Step 6 ($P(a, b) \to Q(a, b)$): obtain $Q(a, b)$. (by Modus Ponens)
8. From Step 7: $\exists y\, Q(a, y)$. (by EG: b witnesses the existential)

9. Since a was arbitrary: $\forall x\, \exists y\, Q(x, y)$. (by UG: a was not constrained)

Annotation: The proof follows the $\forall\exists$ skeleton exactly. Step 2 fixes the arbitrary x. Steps 3-4 use the first hypothesis to obtain a witness b depending on a. Steps 5-7 apply the second hypothesis to get the desired conclusion about b. Steps 8-9 wrap everything back up with EG and UG.

Remark 2.13.8 (Reading analysis proofs). A proof that begins "Let $\varepsilon > 0$" is executing Step 2 of the $\forall\exists$ skeleton with domain $\mathbb{R}_{>0}$. A sentence like "take $\delta = \varepsilon/2$" is executing Step 2 of the inner existential proof: it is the witness for $\exists\delta$. Everything between "let $\varepsilon > 0$" and "take $\delta = \ldots$" is scaffolding to find the right witness. Everything after "take $\delta = \ldots$" is the verification that the witness works. Once this structure is seen, every ε-δ proof becomes readable by template.

2.14 Proof Theory: Equality Rules

Equality Rules Quick Reference

Rule	Schema	Status	Detail
Reflexivity (EqI)	$t = t$	Primitive rule	Def
Equality Elim (EqE)	$t_1 = t_2,\ \varphi[t_1/x] \Rightarrow \varphi[t_2/x]$	Primitive rule	Def
Symmetry	$t_1 = t_2 \Rightarrow t_2 = t_1$	Derived	Thm
Transitivity	$t_1 = t_2,\ t_2 = t_3 \Rightarrow t_1 = t_3$	Derived	Thm
Term substitution	$f(\ldots, t_1, \ldots) = f(\ldots, t_2, \ldots)$	Derived	Thm
Predicate substitution	$P(\ldots, t_1, \ldots) \Leftrightarrow P(\ldots, t_2, \ldots)$	Derived	Thm

Definition (Equality Introduction Reflexivity)

For any term t, one may infer $t = t$.

Remark 2.14.1 (English reading). Reflexivity requires no premises and holds for all terms under all interpretations. It is the foundational rule that allows equality reasoning to get off the ground.

Definition (Equality Elimination Substitution of Equals)

From $t_1 = t_2$ and $\varphi[t_1/x]$, one may infer $\varphi[t_2/x]$.

Remark 2.14.2 (English reading). Equality elimination permits replacing a term by an equal term in any formula position, provided the substitution is capture-avoiding. This formalizes the intuition that equal things can be substituted for each other.

Theorem 2.14.3 (Symmetry of Equality). *From $t_1 = t_2$, one may infer $t_2 = t_1$.*

Remark (Fully quantified form). $\forall t_1, t_2 :\ t_1 = t_2 \to t_2 = t_1$.

Theorem 2.14.4 (Transitivity of Equality). *From $t_1 = t_2$ and $t_2 = t_3$, one may infer $t_1 = t_3$.*

Remark (Fully quantified form). $\forall t_1, t_2, t_3 : t_1 = t_2 \wedge t_2 = t_3 \to t_1 = t_3$.

Theorem 2.14.5 (Term Substitution under Equality). *If $t_1 = t_2$, then for any function symbol f, $f(\ldots, t_1, \ldots) = f(\ldots, t_2, \ldots)$.*

Remark (Fully quantified form). $\forall t_1, t_2 : t_1 = t_2 \to \forall$ function symbol f: $f(\ldots, t_1, \ldots) = f(\ldots, t_2, \ldots)$.

Theorem 2.14.6 (Predicate Substitution under Equality). *If $t_1 = t_2$ and P is an n-ary predicate symbol, then $P(\ldots, t_1, \ldots) \Leftrightarrow P(\ldots, t_2, \ldots)$.*

Remark (Fully quantified form). $\forall t_1, t_2 : t_1 = t_2 \to \forall$ predicate symbol P: $P(\ldots, t_1, \ldots) \leftrightarrow P(\ldots, t_2, \ldots)$.

Remark 2.14.7 (Summary). These derived rules collectively express that functions and predicates *respect equality*: equal inputs produce equal outputs (for functions) or equivalent propositions (for predicates). Together they amount to the principle of Leibniz substitutivity.

Remark 2.14.8 (Common error). When substituting equals for equals, the substitution must be made consistently. Replacing a term by an equal in one occurrence but not another in the same formula can produce incorrect conclusions.

2.15 Proof Theory: Soundness and Completeness

Soundness and Completeness Quick Reference

Property	Direction	Detail
Soundness	$\Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi$	Def
Completeness	$\Gamma \models \varphi \Rightarrow \Gamma \vdash \varphi$	Def
Soundness Theorem (FOL)	All standard FOL rules are sound	Thm
Gödel's Completeness Theorem	FOL is complete	Thm

Definition (Soundness)

A proof system is *sound* if every provable formula is valid:

$$\Gamma \vdash \varphi \implies \Gamma \models \varphi.$$

Definition (Completeness)

A proof system is *complete* if every valid formula is provable:

$$\Gamma \models \varphi \implies \Gamma \vdash \varphi.$$

Theorem 2.15.1 (Soundness of First-Order Logic). *The standard inference rules for first-order logic (UI, UG, EI, EG, and the propositional rules) are sound: they preserve truth in all structures.*

If $\Gamma \vdash \varphi$, then $\Gamma \models \varphi$.

Remark (Fully quantified form). $\forall \Gamma, \varphi : \; \Gamma \vdash \varphi \; \rightarrow \; \forall \mathcal{M}, \forall s, \; (\forall \gamma \in \Gamma, \; \mathcal{M}, s \models \gamma) \rightarrow \mathcal{M}, s \models \varphi$.

Theorem (Gödel's Completeness Theorem)

First-order logic is complete: every logically valid formula is provable.

$$\Gamma \models \varphi \implies \Gamma \vdash \varphi.$$

Equivalently: if a set of sentences Γ is consistent (has no proof of contradiction), then Γ has a model.

Remark 2.15.2 (English reading). Soundness ensures proofs do not lead us astray: we cannot prove false statements from true premises. Completeness ensures proofs are powerful enough: every statement that is true in all models can be established by a formal proof.

Together, soundness and completeness show that syntactic provability ($\vdash$) and semantic entailment ($\models$) coincide for first-order logic.

Remark 2.15.3 (Completeness vs. incompleteness). Gödel's completeness theorem (1930) should not be confused with his incompleteness theorems (1931). The completeness theorem concerns first-order logic as a proof system. The incompletenes theorems concern the limitations of formal theories capable of expressing arithmetic, and are a separate result.

2.16 Translation: English to Logic and Scope Ambiguity

Translation Quick Reference

English pattern	Logical form	Negation	Detail
Everyone has P	$\forall x\, P(x)$	$\exists x\, \neg P(x)$	Table
Someone has P	$\exists x\, P(x)$	$\forall x\, \neg P(x)$	Table
Everyone likes something	$\forall x\, \exists y\, L(x, y)$	$\exists x\, \forall y\, \neg L(x, y)$	Table
Someone likes everything	$\exists x\, \forall y\, L(x, y)$	$\forall x\, \exists y\, \neg L(x, y)$	Table
Scope ambiguity	Two readings differ by quantifier order		Table

Remark 2.16.1 (Translation discipline). When translating a symbolic formula into natural language, respect the original quantifier nesting of the formula. Do not reorder quantifiers unless explicitly asked to normalize or rewrite the expression.

English	Logical Form	Negation
Everyone has property P	$\forall x\, P(x)$	$\exists x\, \neg P(x)$
Someone has property P	$\exists x\, P(x)$	$\forall x\, \neg P(x)$
Everyone likes something	$\forall x\, \exists y\, L(x, y)$	$\exists x\, \forall y\, \neg L(x, y)$
Someone likes everything	$\exists x\, \forall y\, L(x, y)$	$\forall x\, \exists y\, \neg L(x, y)$
Every student passed every exam	$\forall x\, \forall y\, P(x, y)$	$\exists x\, \exists y\, \neg P(x, y)$
Some student passed every exam	$\exists x\, \forall y\, P(x, y)$	$\forall x\, \exists y\, \neg P(x, y)$

Remark 2.16.2 (Scope determines meaning). Natural language often leaves quantifier scope ambiguous. Formal logic resolves this by fixing a precise quantifier order. Changing the order of quantifiers generally changes the meaning of the statement.

English	Logical Form	Meaning
Everyone loves someone	$\forall x\, \exists y\, L(x, y)$	Each person may love a different person.
	$\exists y\, \forall x\, L(x, y)$	There is one person whom everyone loves.
Every student passed an exam	$\forall x\, \exists y\, P(x, y)$	Each student passed at least one (possibly different) exam.
	$\exists y\, \forall x\, P(x, y)$	There is a single exam that all students passed.
A teacher knows every student	$\exists x\, \forall y\, K(x, y)$	There is one teacher who knows all students.
	$\forall y\, \exists x\, K(x, y)$	Every student is known by at least one teacher.
Every function has a zero	$\forall f\, \exists x\, Z(f, x)$	Each function has at least one (possibly different) zero.
	$\exists x\, \forall f\, Z(f, x)$	There is a single point that is a zero of every function.

2.17 Translation: Square of Opposition

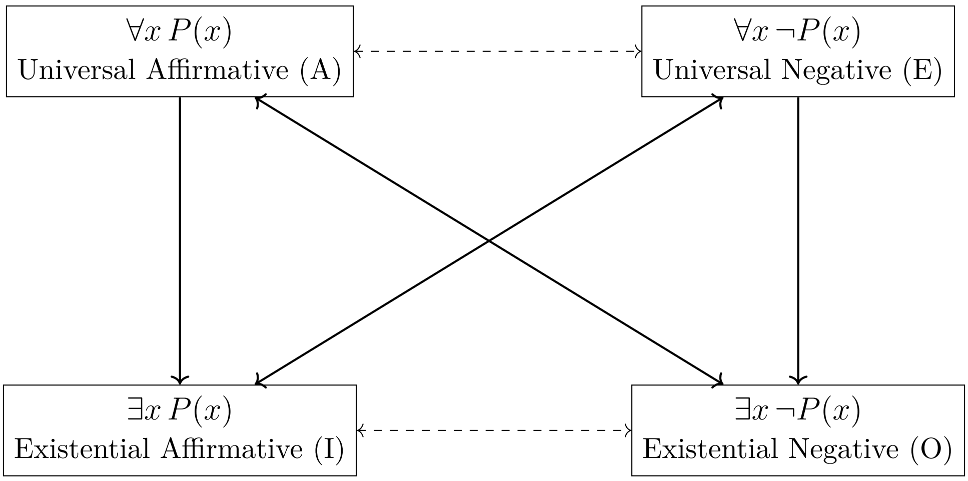
Square of Opposition Quick Reference

Form	Name	Formula	Detail
A	Universal Affirmative	$\forall x\, P(x)$	Def
E	Universal Negative	$\forall x\, \neg P(x)$	Def
I	Existential Affirmative	$\exists x\, P(x)$	Def
O	Existential Negative	$\exists x\, \neg P(x)$	Def

Definition (Quantified Opposition Forms)

Let $P(x)$ be a formula with one free variable. The four standard quantified forms are:

$$\begin{aligned} \textbf{Universal Affirmative (A):}\quad & \forall x\, P(x) \\ \textbf{Universal Negative (E):}\quad & \forall x\, \neg P(x) \\ \textbf{Existential Affirmative (I):}\quad & \exists x\, P(x) \\ \textbf{Existential Negative (O):}\quad & \exists x\, \neg P(x) \end{aligned}$$



Remark 2.17.1 (Logical relations in first-order logic with nonempty domains).

- **Contradictories:** A and O ($\forall x\, P(x)$ vs $\exists x\, \neg P(x)$); E and I ($\forall x\, \neg P(x)$ vs $\exists x\, P(x)$). These pairs cannot both be true and cannot both be false.

- **Subalternation:** A entails I ($\forall x\, P(x) \Rightarrow \exists x\, P(x)$); E entails O ($\forall x\, \neg P(x) \Rightarrow \exists x\, \neg P(x)$). These hold in classical first-order logic with nonempty domains.

- **Contraries and subcontraries** do not generally hold in first-order logic without existential presuppositions.

Remark 2.17.2 (Modern status of the square). In classical first-order logic, only contradiction and subalternation are logically valid relations. The traditional notions of contrariety and subcontrariety rely on existential assumptions and are not preserved in general model-theoretic semantics.

2.18 Reference: Common Errors and Fallacies

Errors and Fallacies Quick Reference

Error category	Where it arises	Detail
Quantifier fallacies	Negation, scope, and swap errors	Table
Common incorrect negations	Forgetting to flip quantifiers	Table
Inference rule errors	Misuse of UI, UG, EI, EG	Rem

Fallacy	Diagnostic Question
Failure to flip quantifier under negation	Was $\forall$ changed to $\exists$ (or vice versa) when negating?
Negating predicate only	Was only the predicate negated while the quantifier was left unchanged?
Partial quantifier negation	When negating nested quantifiers, were *all* quantifiers flipped?
Quantifier order confusion	Was the order of quantifiers changed without justification?
Illicit quantifier swap	Were $\forall$ and $\exists$ commuted when they are not of the same type?
Variable capture	Did substitution introduce a variable that became bound unintentionally?
Illegal existential instantiation	Was a witness chosen before an existential statement was established?
Illicit universal generalization	Was a universally quantified conclusion drawn from a statement depending on a specific object?
Scope ambiguity	Was the scope of a quantifier unclear or implicitly extended beyond its syntactic bounds?
Vacuous quantification misuse	Was a quantifier added or removed even though the variable does not occur free in the formula?

Remark 2.18.1. Quantifier errors almost always stem from ignoring scope or treating quantifiers as informal linguistic modifiers. Every quantifier introduces a binding context that must be respected syntactically before semantic reasoning is applied.

Original	Incorrect Negation	Why It Is Wrong
$\forall x\, P(x)$	$\forall x\, \neg P(x)$	Fails to flip the quantifier; asserts everyone fails P.
$\exists x\, P(x)$	$\exists x\, \neg P(x)$	Negates the predicate but not the existential claim.
$\forall x\, \exists y\, P(x, y)$	$\exists x\, \exists y\, \neg P(x, y)$	Only the outer quantifier was flipped.
$\exists x\, \forall y\, P(x, y)$	$\forall x\, \forall y\, \neg P(x, y)$	Overstrengthened the negation.
$\forall x\, (P(x) \to Q(x))$	$\forall x\, (P(x) \wedge \neg Q(x))$	Correct negation is $\exists x\, (P(x) \wedge \neg Q(x))$.

Remark 2.18.2 (Negation discipline). When negating a quantified statement, push the negation inward across *every* quantifier, flipping each one, until it reaches the atomic predicate. Negating only the predicate or only one quantifier changes the logical claim in an incorrect direction.

Remark 2.18.3 (Common inference rule errors).

- Choosing a witness before establishing an existential premise.
- Using a witness constant introduced by EI outside its allowed scope.
- Generalizing universally over a variable that depends on a special assumption.
- Substituting inside the scope of a quantifier without checking for variable capture.
- Replacing a term by an equal term in one occurrence but not another.

Remark 2.18.4. A useful informal check: if a proof step would still make sense after replacing $\forall$ with "for most" or $\exists$ with "maybe", the step is almost certainly invalid.

2.19 Reference: Summary Tables

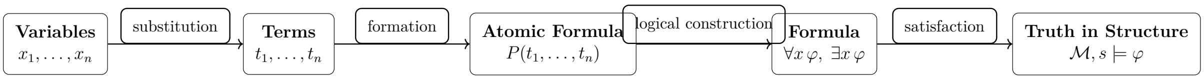
Comparison: Propositional vs. Predicate Logic

Aspect	**Propositional Logic**	**Predicate Logic**
Atomic formulas	Propositional variables (e.g. P, Q)	Predicate symbols applied to terms (e.g. $P(x)$, $R(x, y)$)
Internal structure	No internal structure	Terms, variables, constants, functions
Dependence on variables	None	May contain free variables
Semantic interpretation	Assigned a truth value directly	True or false relative to a structure and assignment
Use of quantifiers	Not available	Essential component
Semantic evaluation	Truth tables	Satisfaction in a structure

Quantifier Rules Summary

Rule Name	**Schema**	**Conditions / Notes**
Universal Instantiation	$\forall x\, \varphi \Rightarrow \varphi[t/x]$	t must be free for x in φ
Existential Generalization	$\varphi[t/x] \Rightarrow \exists x\, \varphi$	t may be any term
Existential Instantiation	$\exists x\, \varphi \Rightarrow \varphi[c/x]$	c is a new constant symbol
Universal Generalization	$\varphi \Rightarrow \forall x\, \varphi$	x not free in any undischarged assumption
Quantifier Negation	$\neg\forall x\, \varphi \equiv \exists x\, \neg\varphi$	
	$\neg\exists x\, \varphi \equiv \forall x\, \neg\varphi$	
Quantifier Commutation	$\forall x\, \forall y\, \varphi \equiv \forall y\, \forall x\, \varphi$	Same quantifier type only
	$\exists x\, \exists y\, \varphi \equiv \exists y\, \exists x\, \varphi$	
Vacuous Quantification	$\forall x\, \varphi \equiv \varphi$	x not free in φ
	$\exists x\, \varphi \equiv \varphi$	
Renaming Bound Variables	$\forall x\, \varphi \equiv \forall y\, \varphi[y/x]$	y not free in φ

Syntax to Semantics Overview



Remark 2.19.1. Variables and terms belong purely to syntax. Predicates form atomic formulas. Logical connectives and quantifiers build complex formulas. A structure and variable assignment then determine whether a formula is satisfied, with quantifiers ranging over the domain of discourse.

Proofs

Quantifier Negation Laws

Remark (Return). ← Back to Proposition (Quantifier Negation Laws) in Notes

Proof. **Claim.** For any formula φ and model $\mathfrak{M}$ with domain D: $\neg\forall x\,\varphi \equiv \exists x\,\neg\varphi$ and $\neg\exists x\,\varphi \equiv \forall x\,\neg\varphi$.
Given. The semantics of the quantifiers: $\mathfrak{M} \models \forall x\,\varphi$ iff φ holds for all $d \in D$; $\mathfrak{M} \models \exists x\,\varphi$ iff φ holds for some $d \in D$. The semantics of negation: $\mathfrak{M} \models \neg\psi$ iff $\mathfrak{M} \not\models \psi$.
Goal. To show that the two equivalences hold in every model.
Strategy. We prove both equivalences by unfolding the semantic definitions and applying classical logic (specifically, the equivalence $\neg\forall \equiv \exists\neg$ and $\neg\exists \equiv \forall\neg$ for sets).

First law: $\neg\forall x\,\varphi \equiv \exists x\,\neg\varphi$.

Fix a model $\mathfrak{M}$ with domain D.

($\Rightarrow$) Suppose $\mathfrak{M} \models \neg\forall x\,\varphi$. Then $\mathfrak{M} \not\models \forall x\,\varphi$, so it is not the case that φ holds for all $d \in D$. Hence there exists some $d \in D$ for which $\varphi[d/x]$ fails, i.e. $\neg\varphi[d/x]$ holds. Therefore $\mathfrak{M} \models \exists x\,\neg\varphi$.

($\Leftarrow$) Suppose $\mathfrak{M} \models \exists x\,\neg\varphi$. Then there exists $d \in D$ with $\mathfrak{M} \models \neg\varphi[d/x]$, so $\varphi[d/x]$ fails. Therefore φ does not hold for all $d \in D$, meaning $\mathfrak{M} \not\models \forall x\,\varphi$, i.e. $\mathfrak{M} \models \neg\forall x\,\varphi$.

Second law: $\neg\exists x\,\varphi \equiv \forall x\,\neg\varphi$.

($\Rightarrow$) Suppose $\mathfrak{M} \models \neg\exists x\,\varphi$. Then $\mathfrak{M} \not\models \exists x\,\varphi$, so no $d \in D$ satisfies $\varphi[d/x]$. Hence for every $d \in D$, $\neg\varphi[d/x]$ holds. Therefore $\mathfrak{M} \models \forall x\,\neg\varphi$.

($\Leftarrow$) Suppose $\mathfrak{M} \models \forall x\,\neg\varphi$. Then for every $d \in D$, $\varphi[d/x]$ fails. So no $d \in D$ satisfies φ, meaning $\mathfrak{M} \not\models \exists x\,\varphi$, i.e. $\mathfrak{M} \models \neg\exists x\,\varphi$. **as required.** □

Remark (Proof shape). Each law is a direct biconditional proof: the two directions are proved by unfolding the semantic definitions of $\neg$, $\forall$, and $\exists$ and applying the classical logical equivalence between "not for all" and "there exists a counterexample." No structural induction is required.

Remark (Classical logic dependence). The first law uses the classical fact that $\neg(\forall d \in D, P(d)) \Leftrightarrow \exists d \in D, \neg P(d)$. This is a standard set-theoretic equivalence that holds for any domain D and property P. In intuitionistic logic, $\neg\forall \Rightarrow \exists\neg$ may fail when D is infinite: one may know the universal quantification fails without being able to produce a witness.

Remark (Dependencies). The proof depends on: the semantics of $\forall$, $\exists$, and $\neg$ in first-order logic, and the classical set-theoretic equivalence between "not all" and "some not."

Vacuous Quantification

Remark (Return). ← Back to Proposition (Vacuous Quantification) in Notes

Proof. **Claim.** If x does not occur free in φ, then $\forall x\,\varphi \equiv \varphi$ and $\exists x\,\varphi \equiv \varphi$.

Given. The semantics of $\forall$ and $\exists$. The fact that if x does not occur free in φ, then the truth value of φ is independent of the value assigned to x: for any model $\mathfrak{M}$, domain D, and assignment σ, $\mathfrak{M}, \sigma \models \varphi$ iff $\mathfrak{M}, \sigma[x \mapsto d] \models \varphi$ for any (hence every) $d \in D$.
Goal. To show both equivalences hold in every nonempty model.

Universal case: $\forall x\, \varphi \equiv \varphi$.
Fix a model $\mathfrak{M}$ with nonempty domain D.
($\Rightarrow$) Suppose $\mathfrak{M} \models \forall x\, \varphi$. Then φ holds for every $d \in D$. Since D is nonempty, pick any $d_0 \in D$. Then $\varphi[d_0/x]$ holds. Since $x \notin \mathrm{FV}(\varphi)$, the truth of φ is independent of x's value, so φ holds.
($\Leftarrow$) Suppose $\mathfrak{M} \models \varphi$. Since $x \notin \mathrm{FV}(\varphi)$, the truth value of φ is unchanged when x is assigned any value $d \in D$. Hence φ holds under any assignment to x, so $\mathfrak{M} \models \forall x\, \varphi$.

Existential case: $\exists x\, \varphi \equiv \varphi$.
($\Rightarrow$) Suppose $\mathfrak{M} \models \exists x\, \varphi$. Then there is some $d \in D$ with $\varphi[d/x]$. Since $x \notin \mathrm{FV}(\varphi)$, this is the same as φ holding, so $\mathfrak{M} \models \varphi$.
($\Leftarrow$) Suppose $\mathfrak{M} \models \varphi$. Since D is nonempty, pick any $d_0 \in D$. Since $x \notin \mathrm{FV}(\varphi)$, φ holds under the assignment $x \mapsto d_0$. Hence $\mathfrak{M} \models \exists x\, \varphi$. **as required.**
□

Remark (Proof shape). Both directions of both equivalences follow from one key semantic fact: if x is not free in φ, the truth of φ is independent of what x is assigned. The universal case uses nonemptiness of D only in the ($\Rightarrow$) direction (to pick a witness). The existential case uses it in the ($\Leftarrow$) direction.

Remark (Why nonemptiness matters). In first-order logic over an empty domain, $\forall x\, \varphi$ is vacuously true but $\exists x\, \varphi$ is false. Standard first-order logic requires nonempty domains, so both equivalences hold. In empty-domain logic, the universal equivalence would fail: $\forall x\, \varphi$ would be true but φ might be false.

Remark (Dependencies). The proof depends on: the semantics of $\forall$ and $\exists$, the definition of free variables ($\mathrm{FV}(\varphi)$), and the assumption that models have nonempty domains.

Renaming Bound Variables

Remark (Return). ← Back to Proposition (Renaming Bound Variables) in Notes

Proof. **Claim.** If y does not occur in φ, then $\forall x\, \varphi \equiv \forall y\, \varphi[y/x]$ and $\exists x\, \varphi \equiv \exists y\, \varphi[y/x]$.
Given. The semantics of $\forall$: $\mathfrak{M} \models \forall x\, \varphi$ iff $\varphi[d/x]$ holds for all $d \in D$. The semantics of substitution: if y does not occur in φ, then $\varphi[y/x][d/y] = \varphi[d/x]$ for any $d \in D$ (substituting y for x and then d for y is the same as substituting d for x directly, since y is fresh and causes no capture).
Goal. To show $\forall x\, \varphi \equiv \forall y\, \varphi[y/x]$ and $\exists x\, \varphi \equiv \exists y\, \varphi[y/x]$.

Universal case. Fix a model $\mathfrak{M}$ with domain D.
$\mathfrak{M} \models \forall x\, \varphi$ iff for all $d \in D$, $\mathfrak{M} \models \varphi[d/x]$ iff for all $d \in D$, $\mathfrak{M} \models (\varphi[y/x])[d/y]$ iff $\mathfrak{M} \models \forall y\, \varphi[y/x]$.

The middle step uses the substitution identity $\varphi[d/x] = (\varphi[y/x])[d/y]$, which holds because y is fresh in φ so no variable capture occurs when replacing x by y.

Existential case. The same chain of equivalences with $\exists$ in place of $\forall$ and "some $d \in D$" in place of "all $d \in D$." **as required.** $\square$

Remark (Proof shape). The proof is a one-step semantic chain in each direction, mediated by the substitution identity $\varphi[d/x] = (\varphi[y/x])[d/y]$. This identity is the core of the argument and holds precisely because y is fresh (does not appear in φ), ensuring no capture occurs.

Remark (The freshness condition is essential). If y already occurs free in φ, the substitution $\varphi[y/x]$ may capture y. For example, $\forall x\,(x < y)$ and $\forall y\,(y < y)$ are not equivalent: the former asserts all elements are less than y (a free variable), while the latter (after careless renaming) asserts all elements are less than themselves, which is false.

Remark (Connection to alpha-equivalence). In type theory and lambda calculus, renaming bound variables is called ***alpha-equivalence***. This proposition is the first-order logic analogue: bound variables are placeholders, and their names carry no semantic content.

Remark (Dependencies). The proof depends on: the semantics of $\forall$ and $\exists$, and the substitution identity $\varphi[d/x] = (\varphi[y/x])[d/y]$ when y is fresh in φ.

Capstone

2.20 Capstone Assessment: Predicate Calculus

Purpose. This capstone assesses mastery of first-order predicate logic, including quantifiers, identity, scope, semantic consequence, and formal proof structure. All arguments must be expressed using first-order logical reasoning. No set-theoretic arguments may be used unless explicitly stated.

Instructions. Each problem requires a complete and rigorous proof. You must explicitly justify quantifier introduction and elimination steps. Appeals to intuition or informal paraphrase are not sufficient.

Problem 1 Quantifier Order Sensitivity

Prove that the following implication is logically valid:

$$\forall x(P(x) \to Q(x)) \;\wedge\; \exists x\, P(x) \;\to\; \exists x\, Q(x).$$

Your proof must explicitly identify where existential instantiation and universal instantiation are applied.

Problem 2 Failure of Converse

Show that the converse of the statement in Problem 1 is not logically valid. That is, show that

$$\exists x\, Q(x) \;\to\; \exists x\, P(x)$$

does not follow from

$$\forall x(P(x) \to Q(x)).$$

Your argument must be semantic (valuation/model-based), not syntactic.

Problem 3 Scope and Negation

Prove that the following two formulas are logically equivalent:

$$\neg\forall x\, P(x) \quad \text{and} \quad \exists x\, \neg P(x).$$

Your proof must make explicit use of the semantics of quantifiers.

Problem 4 Identity Reasoning

Assume identity is part of the language. Prove that the following implication is logically valid:

$$x = y \;\to\; (P(x) \leftrightarrow P(y)).$$

Your proof must not assume substitutivity without justification.

Problem 5 Semantic Consequence

Show that the set of formulas

$$\{\, \forall x(P(x) \to Q(x)),\; \forall x(Q(x) \to R(x)),\; \exists x\, P(x) \,\}$$

logically implies

$$\exists x\, R(x).$$

Your proof must argue that every structure satisfying the premises also satisfies the conclusion.

Completion Criterion. You have mastered predicate calculus if all five proofs:

- use quantifier rules correctly,
- respect variable scope and dependency,
- distinguish syntax from semantics, and
- are written without hidden assumptions.

Successful completion certifies readiness to proceed to set-theoretic foundations.

Chapter 3

Sets, Relations, and Functions

Where You Are in the Journey

Propositional Logic $\rightarrow$ Predicate Calculus $\rightarrow$ **Sets & Functions** $\rightarrow$ Proof Techniques $\rightarrow$ Axiom Systems $\rightarrow$ $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ $\rightarrow$ Real Analysis $\rightarrow$ $\cdots$

How we got here. Logic gave us a precise language. We now need mathematical objects to reason *about*. Sets are the universe of mathematical objects; functions are the structure-preserving maps between them. Together they provide the language of all subsequent mathematics.

What this chapter builds. We develop naïve and axiomatic set theory: membership, subsets, operations, power sets, and families. Relations — including equivalence relations and partial orders — give us structure on sets. Functions capture how sets relate to each other: injections, surjections, bijections, and cardinality.

Where this leads. Every algebraic structure is a set with additional operations. Every proof about $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{R}$ uses the function and relation vocabulary established here. Equivalence relations appear again in the integer and rational constructions; order relations appear throughout analysis.

Structural Roadmap

Each major topic is organised as:

Definitions $\longrightarrow$ Main Theorems $\longrightarrow$ Consequences and Structural Insight

The global progression is:

1. Sets: membership, subsets, operations (union, intersection, difference), power sets, the empty set
2. Relations: binary relations, properties (reflexive, symmetric, transitive), composition
3. Families: indexed families, arbitrary unions and intersections
4. Equivalence relations: equivalence classes, partitions, quotient sets

5. Functions: domain, codomain, image, injection, surjection, bijection, composition, inverse

6. Order: partial orders, total orders, well-orders, Zorn's lemma

Remark 3.0.1 (Primary source). The primary driver is Suppes, *Introduction to Logic*, supplemented by Gerstein, *Introduction to Mathematical Structures and Proofs*.

3.1 Sets, Membership, and ZFC Axioms

Foundations Quick Reference

Concept	Meaning	Detail
Set, membership	Primitive notions; meaning fixed by axioms	Def
Extensionality	Sets equal iff same elements	Ax
Empty Set	Exists a set with no elements	Ax
Pairing	Any two sets can be collected	Ax
Union	Elements of sets in a family	Ax
Power Set	All subsets form a set	Ax
Infinity	An infinite set exists	Ax
Separation	Subsets by definable property	Ax
Replacement	Functional image of a set is a set	Ax
Foundation	No infinite descending $\in$-chains	Ax
Choice	Selection function exists	Ax

Definition (Set and Membership)

In axiomatic set theory, the notions of *set* and *membership* are *primitive*.

- A *set* is an object.
- *Membership* is a binary relation, denoted by $\in$, between objects.

If x is an object and A is a set, the statement $x \in A$ is read as "x is an element of A." No definition of "set" or "$\in$" is given in more basic terms. Their meaning is determined entirely by the axioms governing them.

Remark 3.1.1 (English reading). Primitive means we do not define these in terms of simpler concepts. Rather, we fix their meaning implicitly by stating the axioms they must obey. This is the standard approach in formal mathematics: rules of behaviour replace informal definitions.

Remark 3.1.2 (Consequence). All subsequent notions--subsets, ordered pairs, relations, functions, number systems--are *definitions* introduced within this axiomatic framework. Every proved result is a *theorem* derived logically from the axioms via the rules of inference.

Axiom System (ZFC)

Axiom of Extensionality.
Two sets are equal iff they have the same elements.

$$\forall A \, \forall B \left(A = B \iff \forall x \, (x \in A \leftrightarrow x \in B) \right).$$

Axiom of Empty Set.
There exists a set with no elements.

$$\exists A \, \forall x \, (x \notin A).$$

Axiom of Pairing.
For any two sets, there exists a set containing exactly those two sets.

$$\forall A \, \forall B \, \exists C \, \forall x \, (x \in C \leftrightarrow (x = A \lor x = B)).$$

Axiom of Union.
For any family of sets, there exists a set containing exactly the elements of those sets.

$$\forall A \, \exists U \, \forall x \, \big(x \in U \leftrightarrow \exists B \, (B \in A \land x \in B)\big).$$

Axiom of Power Set.
For any set, there exists a set of all its subsets.

$$\forall A \, \exists P \, \forall x \, (x \in P \leftrightarrow x \subseteq A).$$

Axiom of Infinity.
There exists an infinite set.

$$\exists A \left(\varnothing \in A \land \forall x \, (x \in A \to x \cup \{x\} \in A) \right).$$

Axiom Schema of Separation.
Given a set and a property, there exists a subset containing exactly the elements satisfying that property. For any formula $\varphi(x)$,

$$\forall A \, \exists B \, \forall x \, (x \in B \leftrightarrow (x \in A \land \varphi(x))).$$

Axiom Schema of Replacement.
If $\varphi(x, y)$ defines a functional relation on a set A, then the image of A under φ is a set. For any formula $\varphi(x, y)$,

$$\forall A \left(\big(\forall x \in A \, \exists! y \, \varphi(x, y)\big) \to \exists B \, \forall y \, \big(y \in B \leftrightarrow \exists x \in A \, \varphi(x, y)\big) \right).$$

Axiom of Foundation.
Every nonempty set has an $\in$-minimal element.

$$\forall A \left(A \neq \varnothing \to \exists x \in A \, (x \cap A = \varnothing) \right).$$

Axiom of Choice.
For any family of nonempty sets, there exists a selection function.

Remark 3.1.3 (Role of the axioms). The axioms are not statements to be proved, but rules specifying how $\in$ behaves and which sets are permitted to exist. From this point onward, set theory functions as the underlying language of mathematics: all reasoning about mathematical objects is ultimately grounded in these axioms, even when they are not cited explicitly.

Remark 3.1.4 (Separation vs. Replacement). Separation is a schema: one axiom instance per definable property φ. It carves out subsets of an already-existing set. Replacement is strictly stronger: it can produce sets whose elements are not already contained in any pre-existing set, allowing the construction of large stages of the cumulative hierarchy.

Remark 3.1.5 (Axiom of Choice and right inverses). The Axiom of Choice is equivalent to many statements used later in analysis, including Zorn's Lemma, the well-ordering theorem, and the statement that every surjective function has a right inverse. It is independent of the other ZFC axioms.

3.2 Set Constructions and Operations

Set Operations Quick Reference

Concept	Notation	Membership condition	Detail
Empty set	$\varnothing$	$x \in \varnothing$ never	Def
Subset	$A \subseteq B$	$x \in A \Rightarrow x \in B$	Def
Proper subset	$A \subsetneq B$	$A \subseteq B$ and $A \neq B$	Def
Set equality	$A = B$	same elements both ways	Def
Union	$A \cup B$	in A or in B	Def
Intersection	$A \cap B$	in A and in B	Def
Set difference	$A \setminus B$	in A but not B	Def
Symmetric diff.	$A \triangle B$	in exactly one of A, B	Def
Complement	$A^c = U \setminus A$	in U but not A	Def
Cartesian product	$A \times B$	all ordered pairs (a, b)	Def
Power set	$\mathcal{P}(A)$	all subsets of A	Def
De Morgan	$(A \cup B)^c = A^c \cap B^c$	complement of union	Thm

Definition (Empty Set)

The unique set with no elements is called the *empty set* and is denoted $\varnothing$.

Remark 3.2.1 (Vacuous truth). Statements of the form $\forall x \in \varnothing,\ P(x)$ are *vacuously true*: there is no element $x \in \varnothing$ for which $P(x)$ could fail. This is not a special convention but a consequence of how universal quantification is defined.

Definition (Subset)

Let A and B be sets. We say A is a *subset* of B, written $A \subseteq B$, if every element of A is also an element of B:

$$A \subseteq B \iff \forall x\,(x \in A \to x \in B).$$

Definition (Proper Subset)

A is a *proper subset* of B, written $A \subsetneq B$, if $A \subseteq B$ and $A \neq B$.

Remark 3.2.2 (Notation convention). Some authors write $A \subset B$ for a proper subset; others use it to mean $A \subseteq B$. In these notes, $\subseteq$ always denotes subset (possibly equal) and $\subsetneq$ always denotes proper subset.

Definition (Set Equality)

Two sets A and B are *equal*, written $A = B$, if they have the same elements:

$$A = B \iff \forall x\,(x \in A \leftrightarrow x \in B).$$

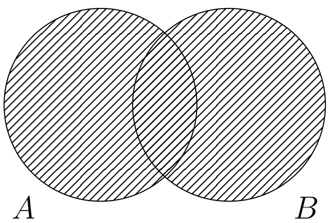
Equivalently, $A = B \iff (A \subseteq B \wedge B \subseteq A)$.

Remark 3.2.3 (Proof strategy). In practice, to prove $A = B$ one shows mutual inclusion: first $A \subseteq B$ (take arbitrary $x \in A$ and deduce $x \in B$), then $B \subseteq A$. This two-step structure appears throughout set-theoretic proofs.

Definition (Union)

The *union* of A and B, denoted $A \cup B$, is the set of all elements belonging to at least one of the two sets:

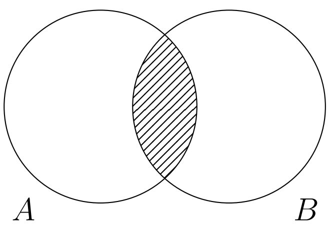
$$A \cup B \;=\; \{\, x \mid x \in A \vee x \in B \,\}.$$



Definition (Intersection)

The *intersection* of A and B, denoted $A \cap B$, is the set of all elements common to both:

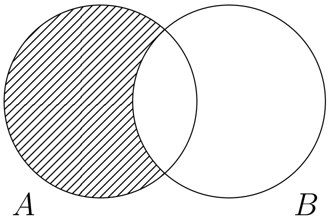
$$A \cap B \;=\; \{\, x \mid x \in A \wedge x \in B \,\}.$$



Definition (Set Difference)

The *set difference* $A \setminus B$ is the set of elements in A but not in B:

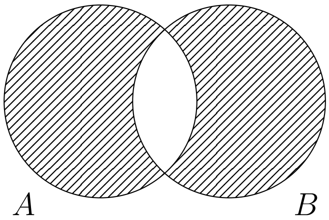
$$A \setminus B \;=\; \{\, x \mid x \in A \wedge x \notin B \,\}.$$



Definition (Symmetric Difference)

The *symmetric difference* $A \triangle B$ is the set of elements belonging to exactly one of A and B:

$$A \triangle B \;=\; (A \setminus B) \cup (B \setminus A) \;=\; \{\, x \mid (x \in A \vee x \in B) \wedge x \notin A \cap B \,\}.$$



Remark 3.2.4 (Algebraic structure of $\triangle$). The symmetric difference is commutative and associative. Under $\triangle$, the power set $\mathcal{P}(U)$ forms an abelian group with identity $\varnothing$ and every element its own inverse ($A \triangle A = \varnothing$).

Definition (Complement)

Let U be a fixed universe and $A \subseteq U$. The *complement* of A relative to U, denoted A^c, is:

$$A^c \;=\; U \setminus A.$$

Definition (Cartesian Product)

The *Cartesian product* of sets A and B is:

$$A \times B = \{\, (a, b) \mid a \in A \text{ and } b \in B \,\}.$$

Remark 3.2.5 (Ordered pairs). The ordered pair (a, b) is formally defined as $\{\{a\}, \{a, b\}\}$. This Kuratowski encoding ensures the key property: $(a, b) = (c, d)$ iff $a = c$ and $b = d$.

Remark 3.2.6 (Non-commutativity). In general, $A \times B \neq B \times A$. The product is however associative up to canonical isomorphism: $(A \times B) \times C \cong A \times (B \times C)$.

Definition (Power Set)

The *power set* of A, denoted $\mathcal{P}(A)$, is the set of all subsets of A:

$$\mathcal{P}(A) \;:=\; \{\, S \mid S \subseteq A \,\}.$$

Remark 3.2.7 (Size). If A has n elements, then $|\mathcal{P}(A)| = 2^n$. For infinite A, Cantor's theorem shows $|\mathcal{P}(A)| > |A|$ strictly.

Theorem (De Morgan's Laws)

Let U be a universe and $A, B \subseteq U$. Then

$$(A \cup B)^c = A^c \cap B^c \quad \text{and} \quad (A \cap B)^c = A^c \cup B^c.$$

Remark (Fully quantified form). $\forall A, B \subseteq U : \forall x \left(x \notin A \cup B \leftrightarrow (x \notin A \land x \notin B)\right)$, and dually $\forall x \left(x \notin A \cap B \leftrightarrow (x \notin A \lor x \notin B)\right)$.

Remark 3.2.8 (Connection to propositional logic). De Morgan's laws mirror the propositional logic identities $\neg(P \lor Q) \equiv \neg P \land \neg Q$ and $\neg(P \land Q) \equiv \neg P \lor \neg Q$ under the correspondence $\cap \leftrightarrow \land$, $\cup \leftrightarrow \lor$, $A^c \leftrightarrow \neg A$.

Definition 3.2.9 (Set Duality). Two set-theoretic expressions over a fixed universe U are ***dual*** if one is obtained from the other by simultaneously replacing $\cup \leftrightarrow \cap$ and $\varnothing \leftrightarrow U$, with complements unchanged.

Remark (Fully quantified form). $\forall$ expressions E, E' over U: E and E' are dual iff E' is obtained from E by the substitution $\cup \mapsto \cap$, $\cap \mapsto \cup$, $\varnothing \mapsto U$, $U \mapsto \varnothing$.

Corollary 3.2.10 (Principle of Set Duality). *Any identity involving $\cup$, $\cap$, $\varnothing$, and U that holds for all subsets of a universe remains valid when each operation and constant is replaced by its dual.*

Remark (Fully quantified form). For any identity $\forall A_1, \ldots, A_n \subseteq U : E(A_i, \cup, \cap, \varnothing, U) = E'(A_i, \cup, \cap, \varnothing, U)$, the dual identity $\forall A_1, \ldots, A_n \subseteq U : E^d = E'^d$ holds, where E^d replaces $\cup \leftrightarrow \cap$ and $\varnothing \leftrightarrow U$.

Remark 3.2.11 (Using duality in proofs). To prove a statement involving unions and intersections, it often suffices to prove one version; the dual statement follows immediately.

Example 3.2.12 (Distributive law via duality). The two distributive laws

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C) \quad \text{and} \quad A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

are duals of each other. Proving either one and applying the Principle of Set Duality yields the other immediately.

Remark 3.2.13 (Transition to relations). The Cartesian product provides a way to encode ordered information. Relations and functions will be defined as special subsets of Cartesian products, so they require no new foundational objects.

3.3 Algebraic Laws of Set Operations

Set Algebra Laws Quick Reference

Law	Union form	Intersection form	Detail
Commutativity	$A \cup B = B \cup A$	$A \cap B = B \cap A$	Thm
Associativity	$(A \cup B) \cup C = A \cup (B \cup C)$	analogous	Thm
Distributivity	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	dual	Thm
Identity	$A \cup \varnothing = A$	$A \cap U = A$	Thm
Absorption	$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$	Thm
Involution	$(A^c)^c = A$	—	Thm

Remark 3.3.1 (Algebraic structure of set operations). The operations $\cup$, $\cap$, and complement satisfy algebraic laws analogous to those of logical connectives. Together they endow $\mathcal{P}(U)$ with the structure of a *Boolean algebra*. These laws justify manipulation of set expressions in later proofs, particularly those involving equivalence classes, partitions, and functions.

Theorem 3.3.2 (Commutativity of Union and Intersection). *Let A, B be sets. Then*

$$A \cup B = B \cup A \quad \textit{and} \quad A \cap B = B \cap A.$$

Remark (Fully quantified form). $\forall A, B : \forall x\,(x \in A \cup B \leftrightarrow x \in B \cup A)$ and $\forall x\,(x \in A \cap B \leftrightarrow x \in B \cap A)$.

Theorem 3.3.3 (Associativity of Union and Intersection). *Let A, B, C be sets. Then*

$$(A \cup B) \cup C = A \cup (B \cup C) \quad \textit{and} \quad (A \cap B) \cap C = A \cap (B \cap C).$$

Remark (Fully quantified form). $\forall A, B, C : \forall x\,\big((x \in (A \cup B) \cup C) \leftrightarrow (x \in A \cup (B \cup C))\big)$, and analogously for $\cap$.

Theorem 3.3.4 (Distributive Laws). *Let A, B, C be sets. Then*

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C),$$

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C).$$

Remark (Fully quantified form). $\forall A, B, C : \forall x\,\big(x \in A \cap (B \cup C) \leftrightarrow x \in (A \cap B) \cup (A \cap C)\big)$, and dually for $\cup$ over $\cap$.

Theorem 3.3.5 (Identity and Absorption Laws). *Let A, B be sets and U a universe with $A \subseteq U$. Then*

$$A \cup \varnothing = A, \qquad A \cap U = A,$$

$$A \cup (A \cap B) = A, \qquad A \cap (A \cup B) = A.$$

Remark (Fully quantified form). $\forall A \subseteq U : \forall x\,(x \in A \cup \varnothing \leftrightarrow x \in A)$ and $\forall x\,(x \in A \cap U \leftrightarrow x \in A)$. Absorption: $\forall A, B : \forall x\,(x \in A \cup (A \cap B) \leftrightarrow x \in A)$, and dually.

Theorem 3.3.6 (Involution of Complement). *For any* $A \subseteq U$*,*

$$(A^c)^c = A.$$

Remark (Fully quantified form). $\forall A \subseteq U : \forall x\,(x \in (A^c)^c \leftrightarrow x \in A)$.

Remark 3.3.7 (Non-commutative and non-associative operations). Set difference $\setminus$ is neither commutative nor associative:

$$A \setminus B \neq B \setminus A, \qquad (A \setminus B) \setminus C \neq A \setminus (B \setminus C) \quad \text{in general}.$$

It interacts with union and intersection via:

$$A \setminus (B \cup C) = (A \setminus B) \cap (A \setminus C), \qquad A \setminus (B \cap C) = (A \setminus B) \cup (A \setminus C).$$

These follow from $A \setminus B = A \cap B^c$ together with De Morgan's laws.

Remark 3.3.8 (Cartesian product). The Cartesian product $\times$ is not commutative, is associative only up to canonical isomorphism, and distributes over union in each coordinate:

$$A \times (B \cup C) = (A \times B) \cup (A \times C).$$

3.4 Ordered Pairs and Relations

Relations Quick Reference

Concept	Meaning	Detail
Ordered pair	$(a, b) := \{\{a\}, \{a, b\}\}$; identity iff both coords equal	Def
Cartesian product	$A \times B$: all ordered pairs from A and B	Def
Relation	Any subset $R \subseteq A \times B$	Def

Definition (Ordered Pair)

Let a and b be sets. The *ordered pair* (a, b) is defined (Kuratowski) as

$$(a, b) \; := \; \{\{a\}, \{a, b\}\}.$$

Theorem 3.4.1 (Uniqueness of Ordered Pairs). *For any sets* a, b, c, d*,*

$$(a, b) = (c, d) \iff (a = c \wedge b = d).$$

Remark (Fully quantified form). $\forall a, b, c, d : \; (a, b) = (c, d) \;\leftrightarrow\; (a = c \wedge b = d)$.

Remark 3.4.2 (Kuratowski encoding). The purpose of the Kuratowski definition is purely to guarantee the uniqueness theorem above. Once $(a, b) = (c, d) \iff a = c \wedge b = d$ is established, we may treat ordered pairs as a primitive with this property and forget the encoding.

Definition (Cartesian Product as foundation for relations)

The *Cartesian product* of sets A and B is:

$$A \times B := \{ (a, b) \mid a \in A \text{ and } b \in B \}.$$

Definition (Relation)

A *relation* from A to B is any subset $R \subseteq A \times B$.
If $(a, b) \in R$ we write $a\,R\,b$ and say "a is related to b." When $A = B$ we say R is a *relation on* A.

Remark 3.4.3 (Relations as structured sets). Relations introduce no new foundational objects: they are simply sets of ordered pairs, constructed using operations already available. Properties of relations can therefore be studied with ordinary set-theoretic reasoning.

3.5 Properties of Relations

Relational Properties Quick Reference

Property	Formal definition	Canonical example	Detail
Reflexive	$\forall a,\ (a, a) \in R$	$=$ on any set	Def
Irreflexive	$\forall a,\ (a, a) \notin R$	$<$ on $\mathbb{N}$	Def
Symmetric	$(a, b) \in R \Rightarrow (b, a) \in R$	"same age as"	Def
Antisymmetric	$(a, b), (b, a) \in R \Rightarrow a = b$	$\leq$ on $\mathbb{N}$	Def
Asymmetric	$(a, b) \in R \Rightarrow (b, a) \notin R$	$<$ on $\mathbb{N}$	Def
Transitive	$(a, b), (b, c) \in R \Rightarrow (a, c) \in R$	$\leq$ on $\mathbb{N}$	Def
Total (Connex)	$(a, b) \in R \vee (b, a) \in R$	$\leq$ on $\mathbb{R}$	Def
Structural classes formed by combining properties:			
Equivalence	Reflexive + Symmetric + Transitive	$=$ on any set	Def
Preorder	Reflexive + Transitive	$\leq$ on $\mathbb{N}$	Def
Partial order	Reflexive + Antisymmetric + Transitive	$\subseteq$ on $\mathcal{P}(A)$	Def
Total order	Partial order + Total	$\leq$ on $\mathbb{R}$	Def

Definition 3.5.1 (Reflexive Relation)**.** A relation R on A is *reflexive* if every element is related to itself:

$$\forall a \in A,\ (a, a) \in R.$$

Definition 3.5.2 (Irreflexive Relation)**.** R is *irreflexive* if no element is related to itself:

$$\forall a \in A,\ (a, a) \notin R.$$

Remark 3.5.3 (Independence of reflexive and irreflexive). A relation may be neither reflexive nor irreflexive (if some but not all elements satisfy $(a, a) \in R$). However, a relation cannot be both reflexive and irreflexive unless $A = \varnothing$.

Definition 3.5.4 (Symmetric Relation). R is *symmetric* if related pairs commute:

$$\forall a, b \in A,\ (a, b) \in R \to (b, a) \in R.$$

Definition 3.5.5 (Antisymmetric Relation). R is *antisymmetric* if mutual relation implies equality:

$$\forall a, b \in A,\ \big((a, b) \in R \wedge (b, a) \in R\big) \to a = b.$$

Definition 3.5.6 (Asymmetric Relation). R is *asymmetric* if related pairs never commute:

$$\forall a, b \in A,\ (a, b) \in R \to (b, a) \notin R.$$

Remark 3.5.7 (Asymmetric vs. antisymmetric). Every asymmetric relation is antisymmetric, but not conversely. $\leq$ on $\mathbb{R}$ is antisymmetric but not asymmetric (since $1 \leq 1$). Strict order $<$ is both asymmetric and irreflexive.

Definition 3.5.8 (Transitive Relation). R is *transitive* if it extends along chains:

$$\forall a, b, c \in A,\ \big((a, b) \in R \wedge (b, c) \in R\big) \to (a, c) \in R.$$

Definition 3.5.9 (Total (Connex) Relation). R is *total* (or *connex*) if every pair is comparable:

$$\forall a, b \in A,\ (a, b) \in R \vee (b, a) \in R.$$

Definition (Equivalence Relation)

R is an *equivalence relation* on A if it is reflexive, symmetric, and transitive.

Remark (Fully quantified form). R is an equivalence relation on A iff:

$$(\forall a \in A,\ (a, a) \in R) \wedge (\forall a, b \in A,\ (a, b) \in R \to (b, a) \in R) \wedge (\forall a, b, c \in A,\ (a, b) \in R \wedge (b, c) \in R \to (a,$$

Remark 3.5.10 (Role in mathematics). Equivalence relations axiomatize the idea of "sameness up to a chosen criterion." They partition sets into equivalence classes and are the basis for quotient constructions throughout algebra, topology, and analysis.

Definition (Preorder)

R is a *preorder* on A if it is reflexive and transitive.

Remark (Fully quantified form). R is a preorder on A iff:

$$(\forall a \in A,\ (a, a) \in R)\ \wedge\ (\forall a, b, c \in A,\ (a, b) \in R \wedge (b, c) \in R \to (a, c) \in R).$$

Definition (Partial Order)

R is a *partial order* on A if it is reflexive, antisymmetric, and transitive.

Remark (Fully quantified form). R is a partial order on A iff:

$$(\forall a \in A,\ (a, a) \in R) \wedge (\forall a, b \in A,\ (a, b) \in R \wedge (b, a) \in R \to a = b) \wedge (\forall a, b, c \in A,\ (a, b) \in R \wedge (b, c) \in R$$

Definition (Total Order)

R is a *total order* on A if it is a partial order and is total.

Remark (Fully quantified form). R is a total order on A iff R is a partial order on A and additionally:

$$\forall a, b \in A,\ (a, b) \in R \vee (b, a) \in R.$$

Remark 3.5.11 (Using structural properties in proofs). When R is asserted to be an equivalence relation or partial order, this is shorthand for the conjunction of its defining properties. In proofs, cite only the specific component needed: "since R is a partial order, antisymmetry implies ..." rather than restating all properties.

Example 3.5.12 (Using a structural property in a proof). Let R be an equivalence relation on A. For any $a, b \in A$, $(a, b) \in R \Rightarrow [a] = [b]$, where $[a] = \{x \in A \mid (a, x) \in R\}$.

Remark 3.5.13 (Properties are logically independent). No basic property implies another in general. A relation may be transitive without being reflexive, or symmetric without being transitive. The structural classes are distinguished precisely by requiring specific combinations.

3.6 Indexed Families of Sets

Indexed Families Quick Reference

Concept	Meaning	Detail
Indexed family	Function $F : I \to \mathcal{P}(U)$; written $\{A_i\}_{i\in I}$	Def
Indexed union	$\bigcup_{i\in I} A_i$: in at least one A_i	Def
Indexed intersection	$\bigcap_{i\in I} A_i$: in every A_i	Def
Pairwise disjoint	$i \neq j \Rightarrow A_i \cap A_j = \varnothing$	Def
Cover	$\bigcup_{C\in\mathcal{C}} C = A$	Def
Arbitrary product	$\prod_{i\in I} A_i$: choice functions $f : I \to \bigcup A_i$	Def

Remark 3.6.1 (Motivation). Many set-theoretic constructions require not just pairs of sets but infinite families. Indexed families provide the formal language for partitions, equivalence classes, unions over countably or uncountably many sets, and products indexed by arbitrary index sets.

Definition (Indexed Family of Sets)

Let I be a set (the *index set*) and U a universe. An *indexed family of sets* is a function

$$F : I \to \mathcal{P}(U),$$

with $F(i)$ typically written A_i. The family is denoted $\{A_i\}_{i\in I}$.

Remark 3.6.2 (Family vs. set of sets). An indexed family is formally a *function*, not a set of sets. Different indices may correspond to the same set: $i \neq j$ does not imply $A_i \neq A_j$. This distinction matters for equivalence class constructions.

Definition (Indexed Union)

The *union* of the indexed family $\{A_i\}_{i\in I}$ is:

$$\bigcup_{i\in I} A_i \;:=\; \{\, x \mid \exists i \in I \text{ such that } x \in A_i \,\}.$$

Definition (Indexed Intersection)

For $I \neq \varnothing$, the *intersection* of $\{A_i\}_{i\in I}$ is:

$$\bigcap_{i\in I} A_i \;:=\; \{\, x \mid \forall i \in I,\; x \in A_i \,\}.$$

Remark 3.6.3 (Why $I \neq \varnothing$ is required). If $I = \varnothing$, then $\forall i \in I, x \in A_i$ holds vacuously for every x, making the intersection the entire universe U. This is normally left undefined or requires specifying a background universe explicitly.

Definition (Pairwise Disjoint Family)

The family $\{A_i\}_{i\in I}$ is *pairwise disjoint* if

$$\forall i, j \in I,\; i \neq j \to A_i \cap A_j = \varnothing.$$

Definition 3.6.4 (Cover). A collection $\mathcal{C} \subseteq \mathcal{P}(A)$ is a *cover* of A if

$$\bigcup_{C\in\mathcal{C}} C = A.$$

Remark (Fully quantified form). $\mathcal{C}$ covers A $\leftrightarrow$ $\forall x \in A,\, \exists C \in \mathcal{C},\; x \in C$.

3.6.1 Arbitrary Cartesian Products

Definition (Arbitrary Cartesian Product)

Let $\{A_i\}_{i\in I}$ be an indexed family of sets. The *Cartesian product* is:

$$\prod_{i\in I} A_i \;:=\; \Big\{ f : I \to \bigcup_{i\in I} A_i \;\Big|\; \forall i \in I,\; f(i) \in A_i \Big\}.$$

Remark 3.6.5 (Elements as choice functions). An element of $\prod_{i\in I} A_i$ is a *choice function*: a function that assigns to each index i an element of the corresponding set A_i. For finite $I = \{1, \ldots, n\}$, this reduces to the familiar n-tuple $(a_1, \ldots, a_n)$.

Remark 3.6.6 (Axiom of Choice connection). For infinite I, the product $\prod_{i\in I} A_i$ is nonempty iff a choice function exists. This existence is not guaranteed by the other ZFC axioms: it is equivalent to the Axiom of Choice. Thus the Axiom of Choice is precisely the assertion that arbitrary products of nonempty sets are nonempty.

3.7 Covers, Subcovers, and the Finite Intersection Property

Covers and FIP Quick Reference

Concept	Meaning	Detail
Cover of A	$\bigcup_{C\in\mathcal{C}} C \supseteq A$	Def
Subcover	Sub-collection of a cover that is still a cover	Def
Finite cover	Cover with finitely many members	Def
Finite subcover	Finite subcollection that is still a cover	Def
Open cover	Cover whose members are all open sets	Def
Finite Intersection Property	Every finite subcollection has nonempty intersection	Def
FIPcover duality	$\mathcal{C}$ covers A $\iff$ closed complements fail FIP	Prop

Compactness is one of the central concepts of analysis, and its definition is stated in terms of covers: a space is compact if every open cover has a finite subcover. This is a purely set-theoretic condition it makes no reference to distance or continuity. The work of understanding compactness therefore begins here, with the vocabulary of covers. Once these definitions are in place, the definition of compactness will be a single sentence, and the difficulty will lie entirely in the geometric and analytic content, not the set-theoretic framework.

Definition (Cover, Subcover, Finite Cover)

Let A be a set and $\mathcal{C}$ a collection of sets.

(i) $\mathcal{C}$ is a *cover* of A if every element of A belongs to at least one member of $\mathcal{C}$:

$$A \subseteq \bigcup_{C\in\mathcal{C}} C.$$

(ii) $\mathcal{C}'$ is a *subcover* of $\mathcal{C}$ if $\mathcal{C}' \subseteq \mathcal{C}$ and $\mathcal{C}'$ is still a cover of A.

(iii) A cover is *finite* if it has finitely many members. A *finite subcover* is a subcover $\mathcal{C}' \subseteq \mathcal{C}$ with $|\mathcal{C}'| < \infty$.

Remark 3.7.1 (The key question). The condition of being a cover is easy to satisfy: take $\mathcal{C} = \mathcal{P}(A)$ and it is trivially a cover. The interesting question is whether large covers can be replaced by smaller ones specifically, finite ones. An infinite cover has infinitely many sets, which is hard to work with. A finite subcover is just as good for covering purposes, but is finite and therefore tractable. When a finite subcover always exists, no matter which cover one starts with, the space is called compact.

Definition 3.7.2 (Open Cover). In a topological space (X, τ), an *open cover* of a subset $A \subseteq X$ is a cover $\mathcal{C}$ of A all of whose members are open sets: $\mathcal{C} \subseteq \tau$.

Remark (Fully quantified form). $\mathcal{C}$ is an open cover of A $\leftrightarrow$ $(\forall C \in \mathcal{C},\ C \in \tau) \wedge (\forall x \in A,\ \exists C \in \mathcal{C},\ x \in C)$.

Remark 3.7.3 (Why open covers). The emphasis on open covers in compactness is because open sets are the sets one controls in a topological space. An open cover $\{U_\alpha\}$ of A means: every point of A has at least one U_α containing it. A finite subcover then gives a list of finitely many open sets whose union already contains all of A. Compactness says this reduction is always possible, which is a very strong structural property.

Definition (Finite Intersection Property)

A collection $\mathcal{F}$ of sets has the *finite intersection property* (FIP) if every finite subcollection has nonempty intersection: for all finite $\mathcal{F}' \subseteq \mathcal{F}$,

$$\bigcap_{F \in \mathcal{F}'} F \neq \varnothing.$$

Remark 3.7.4 (How to read the FIP). The FIP is a consistency condition: no finite list of sets from $\mathcal{F}$ is mutually exclusive. It does *not* say the entire intersection $\bigcap_{F \in \mathcal{F}} F$ is nonempty only that every *finite* sub-intersection is. The difference matters when $\mathcal{F}$ is infinite.

Example: In $\mathbb{R}$, the family $\mathcal{F} = \{(0, 1/n) : n \in \mathbb{N}\}$ has the FIP (any finite subcollection has nonempty intersection, since $(0, 1/n) \supseteq (0, 1/N)$ for the largest N in the subcollection). But $\bigcap_{n=1}^{\infty} (0, 1/n) = \varnothing$. Compact spaces are characterized by the property that the FIP implies nonempty total intersection.

Proposition 3.7.5 (FIP-Cover Duality). *Let X be a set, $A \subseteq X$, and $\{U_\alpha\}_{\alpha \in I}$ a family of subsets of X. Let $F_\alpha = U_\alpha^c = X \setminus U_\alpha$. Then:*

$$\{U_\alpha\} \text{ covers } A \iff \{F_\alpha \cap A\}_{\alpha \in I} \text{ does not have the FIP.}$$

More precisely: the family $\{U_\alpha\}$ does not *cover A if and only if the family of closed complements $\{F_\alpha\}$ has the FIP relative to A.*

Remark (Fully quantified form). $A \subseteq \bigcup_{\alpha \in I} U_\alpha \leftrightarrow \neg(\forall$ finite $I_0 \subseteq I,\ \bigcap_{\alpha \in I_0} F_\alpha \cap A \neq \varnothing)$. Reading pointwise: $\forall x \in A,\ \exists \alpha \in I,\ x \in U_\alpha \leftrightarrow \{F_\alpha \cap A\}$ fails the FIP.

Remark 3.7.6 (Proof by De Morgan). The duality is a direct consequence of De Morgan's law for indexed families (Theorem 3.10.11). The family $\{U_\alpha\}$ fails to cover A means:

$$A \not\subseteq \bigcup_\alpha U_\alpha \iff A \cap \Big(\bigcup_\alpha U_\alpha\Big)^c \neq \varnothing \iff A \cap \bigcap_\alpha U_\alpha^c \neq \varnothing \iff A \cap \bigcap_\alpha F_\alpha \neq \varnothing.$$

Applying this to every finite subcollection gives the equivalence with the FIP.

Remark 3.7.7 (Why this duality matters for compactness). Compactness has two equivalent formulations:

(i) Every open cover has a finite subcover.

(ii) Every family of closed sets with the FIP has nonempty intersection.

These formulations are equivalent by FIP-Cover Duality: take complements and apply De Morgan. Formulation (i) is the standard definition; formulation (ii) is often easier to apply when working with closed sets (e.g., nested compact sets). Both are purely set-theoretic conditions, and the equivalence is entirely a matter of set algebra. When compactness is encountered in topology and metric spaces, the only new content is the geometric meaning the set-theoretic equivalence is established here.

Example 3.7.8 (Nested closed sets and FIP). In $\mathbb{R}$, the family $\{[n,\infty) : n \in \mathbb{N}\}$ has the FIP: any finite subcollection $\{[n_1,\infty),\dots,[n_k,\infty)\}$ has intersection $[\max(n_i),\infty) \neq \varnothing$. But $\bigcap_{n=1}^{\infty}[n,\infty) = \varnothing$. This shows $\mathbb{R}$ is not compact (the family of closed sets has the FIP but empty total intersection). A compact space would not allow this failure.

3.8 Equivalence Classes and Partitions

Equivalence and Partitions Quick Reference

Concept	Meaning	Detail
Equivalence class	$[a]_R = \{x \in A \mid (a,x) \in R\}$	Def
Quotient set	A/R: all equivalence classes	Def
Index of R	Cardinality of A/R	Def
Canonical surjection	$\pi : A \to A/R$, $\pi(a) = [a]$	Def
Partition	Nonempty, disjoint, covering collection of subsets	Def
Rep. independence	$[a] = [b] \iff (a,b) \in R$	Lem
Equiv.–partition correspondence	Bijection between equiv. relations and partitions	Thm

Definition (Equivalence Class)

Let R be an equivalence relation on A. For $a \in A$, the *equivalence class* of a is:

$$[a]_R := \{\, x \in A \mid (a,x) \in R \,\}.$$

When R is clear from context, we write $[a]$.

Remark 3.8.1 (English reading). $[a]_R$ is the set of all elements that R declares "the same as a." Two elements lie in the same class iff they are related: this is precisely the Representative Independence Lemma below.

Definition (Quotient Set)

The *quotient set* of A by R is the set of all equivalence classes:

$$A/R \;:=\; \{\, [a]_R \mid a \in A \,\}.$$

Definition 3.8.2 (Index of an Equivalence Relation). The *index* of R on A is the cardinality $|A/R|$, i.e. the number of equivalence classes.

Remark (Fully quantified form). $\mathrm{index}(R) = |A/R| = |\{[a]_R \mid a \in A\}|$.

Definition (Canonical Surjection)

The *canonical surjection* (quotient map) is the function

$$\pi : A \to A/R, \qquad \pi(a) := [a].$$

Remark 3.8.3 (Properties of π). π is surjective by construction. Elements $a, b \in A$ satisfy $\pi(a) = \pi(b)$ iff $(a, b) \in R$. The canonical surjection is the prototype for all quotient constructions: it reappears as the quotient homomorphism in algebra and the quotient map in topology.

Definition (Partition)

A *partition* of A is a collection $\mathcal{P}$ of subsets of A such that:

(i) every block is nonempty: $\forall P \in \mathcal{P},\ P \neq \varnothing$;

(ii) distinct blocks are disjoint: $\forall P, Q \in \mathcal{P},\ P \neq Q \to P \cap Q = \varnothing$;

(iii) the blocks cover A: $\bigcup_{P\in\mathcal{P}} P = A$.

The sets $P \in \mathcal{P}$ are called the *blocks* of the partition.

Remark 3.8.4 (Partition vs. cover). Every partition of A is a cover of A whose members are nonempty and pairwise disjoint. Partitions are precisely the covers satisfying the disjointness condition.

Lemma 3.8.5 (Representative Independence Lemma). *Let R be an equivalence relation on A. For any $a, b \in A$,*

$$[a] = [b] \iff (a, b) \in R.$$

Remark (Fully quantified form). $\forall a, b \in A : [a]_R = [b]_R \leftrightarrow (a, b) \in R$.

Remark 3.8.6 (Well-definedness on quotient sets). This lemma is the key fact underlying well-definedness of functions on quotient sets: a function $f : A \to B$ defined by $f([a]) = \cdots$ is well-defined iff the formula gives the same output for all representatives of $[a]$.

Theorem 3.8.7 (Equivalence Relations and Partitions). *Let A be a set.*

(i) If R is an equivalence relation on A, then A/R is a partition of A.

(ii) If $\mathcal{P}$ is a partition of A, then the relation $R_{\mathcal{P}}$ defined by

$$(a,b) \in R_{\mathcal{P}} \iff \exists P \in \mathcal{P} \text{ with } a \in P \text{ and } b \in P$$

is an equivalence relation on A.

(iii) These constructions are inverse: $R_{A/R} = R$ and $A/R_{\mathcal{P}} = \mathcal{P}$.

Remark (Fully quantified form). (i) $\forall a \in A,\ [a]_R \neq \varnothing$; $\forall a,b \in A,\ [a]_R \neq [b]_R \to [a]_R \cap [b]_R = \varnothing$; $\bigcup_{a\in A}[a]_R = A$.
(ii) $\forall a,b \in A:\ (a,b) \in R_{\mathcal{P}} \leftrightarrow \exists P \in \mathcal{P},\ a \in P \wedge b \in P$.

Remark 3.8.8 (Duality of perspectives). This theorem establishes a bijection between equivalence relations on A and partitions of A. The two perspectives--"same block" (partition) and "related" (equivalence relation)--are interchangeable and each is more natural in different contexts.

Example 3.8.9 (Extremal equivalence relations). On any set A:

1. The *equality relation* $(a,b) \in R \iff a = b$ gives singleton classes $[a] = \{a\}$ and the finest partition of A.
2. The *universal relation* $R = A\times A$ gives $[a] = A$ for all a, and the coarsest partition (one block).

All other equivalence relations on A lie strictly between these extremes.

Theorem 3.8.10 (Universal Property of the Quotient Map). *Let R be an equivalence relation on A, let $\pi : A \to A/R$ be the canonical surjection, and let $f : A \to B$ be any function. Then f is* constant on equivalence classes *meaning $(a,b) \in R$ implies $f(a) = f(b)$ if and only if there exists a unique function $\bar{f} : A/R \to B$ such that*

$$f = \bar{f} \circ \pi.$$

When it exists, $\bar{f}$ is defined by $\bar{f}([a]) = f(a)$.

Remark (Fully quantified form). $\big(\forall a,b \in A,\ (a,b) \in R \to f(a) = f(b)\big) \leftrightarrow \big(\exists!\, \bar{f} : A/R \to B,\ \forall a \in A,\ \bar{f}([a]) = f(a)\big)$.

Remark 3.8.11 (Why this is called the universal property). The condition $f = \bar{f}\circ\pi$ says the diagram commutes: going from A to B directly via f gives the same result as going first to A/R via π and then to B via $\bar{f}$. The word "universal" refers to the fact that $\pi : A \to A/R$ is the *unique* quotient with this property: any other map that "respects the equivalence" factors through π.

The uniqueness of $\bar{f}$ is forced by π being surjective: every element of A/R is $[a]$ for some a, so $\bar{f}([a]) = f(a)$ is the only possible definition.

Remark 3.8.12 (Well-definedness is the key verification). The heart of the theorem is well-definedness: the formula $\bar{f}([a]) = f(a)$ could be ambiguous if $[a] = [b]$ but $f(a) \neq f(b)$. The hypothesis that f is constant on equivalence classes is precisely what prevents this: $(a,b) \in R$ (i.e., $[a] = [b]$) implies $f(a) = f(b)$, so the output does not depend on which representative of the class is chosen.

This "well-definedness check" recurs throughout mathematics. Whenever a function is defined on equivalence classes by a formula involving representatives, the first obligation is to verify that the formula gives the same answer for all representatives. The universal property tells exactly what condition on f makes this work.

Remark 3.8.13 (Where this appears downstream). This theorem is not an abstract curiosity. It is the engine behind:

- *Integer construction*: the integers $\mathbb{Z}$ are defined as equivalence classes of pairs of natural numbers, and arithmetic operations are defined by formulas on representatives. The universal property verifies the operations are well-defined.
- *Rational construction*: $\mathbb{Q}$ is defined as equivalence classes of pairs (p, q) with $q \neq 0$, and addition/multiplication on representatives must be checked.
- *Quotient groups in algebra*: the quotient group G/N is defined by the same factorization, and the quotient homomorphism is π.
- *Quotient topology*: the quotient topological space is defined as the finest topology on A/R making π continuous a condition stated precisely via the universal property.

3.9 Functions

Functions Quick Reference

Concept	Meaning	Detail
Function	Left-total, right-unique relation	Def
Domain / Codomain	$\text{dom}(f) = A,\ \text{cod}(f) = B$ for $f : A \to B$	Def
Image of function	$\text{im}(f) = \{f(a) \mid a \in A\}$	Def
Image of set	$f(S) = \{f(a) \mid a \in S\}$ for $S \subseteq A$	Def
Preimage	$f^{-1}(T) = \{a \in A \mid f(a) \in T\}$ for $T \subseteq B$	Def
Fiber	$f^{-1}(\{b\})$: preimage of a singleton	Def
Graph	$\{(a, b) \in A \times B \mid b = f(a)\}$	Def
Injective	Distinct inputs $\Rightarrow$ distinct outputs	Def
Surjective	Every codomain element is achieved	Def
Bijective	Injective and surjective	Def
Identity	$\text{id}_A(a) = a$	Def
Inclusion map	$\iota : A \hookrightarrow B$, $\iota(a) = a$ for $A \subseteq B$	Def
Composition	$(g \circ f)(a) = g(f(a))$	Def
Inverse	Defined for bijections; $f^{-1} \circ f = \text{id}_A$	Def
Left / Right inverse	Section and retraction	Def
Restriction	$f\|_C : C \to B$	Def
Extension	$g : A' \to B$ with $g\|_A = f$	Def

Definition (Function)

Let A and B be sets. A *function* from A to B is a relation $f \subseteq A \times B$ such that:

(i) (*Existence / left-total*) for every $a \in A$, there exists $b \in B$ with $(a, b) \in f$;

(ii) (*Uniqueness / right-unique*) if $(a, b_1) \in f$ and $(a, b_2) \in f$ then $b_1 = b_2$.

If $(a, b) \in f$ we write $f(a) = b$.

Remark 3.9.1 (English reading). A function is a rule assigning to each input exactly one output. The two conditions formalize "defined everywhere" (existence) and "single-valued" (uniqueness).

Remark 3.9.2 (Function as relation). Every function is a relation, but not every relation is a function. The two conditions that distinguish functions are left-totality and right-uniqueness.

Definition (Domain and Codomain)

If f is a function from A to B, we write $f : A \to B$, where A is the *domain* $\mathrm{dom}(f)$ and B the *codomain* $\mathrm{cod}(f)$.

Definition (Image of a Function)

The *image* (or *range*) of $f : A \to B$ is:

$$\mathrm{im}(f) \;:=\; \{\, b \in B \mid \exists a \in A,\; f(a) = b \,\}.$$

The image may be a proper subset of the codomain.

Definition (Image of a Set)

For $S \subseteq A$, the *image of S under f* is:

$$f(S) \;:=\; \{\, f(a) \mid a \in S \,\}.$$

Note $f(A) = \mathrm{im}(f)$.

Definition (Preimage)

For $T \subseteq B$, the *preimage* (inverse image) of T under f is:

$$f^{-1}(T) \;:=\; \{\, a \in A \mid f(a) \in T \,\}.$$

Remark 3.9.3 (Preimage notation warning). $f^{-1}(T)$ does *not* denote an inverse function. It is defined for any function f, regardless of whether f is injective or bijective.

Definition (Fiber)

The *fiber* of f over $b \in B$ is the preimage of the singleton:

$$f^{-1}(\{b\}) \;=\; \{\, a \in A \mid f(a) = b \,\}.$$

Remark 3.9.4 (Fibers partition the domain). The collection $\{f^{-1}(\{b\}) \mid b \in \operatorname{im}(f)\}$ is a partition of A. Every function thus induces an equivalence relation $a_1 \sim_f a_2 \iff f(a_1) = f(a_2)$, whose equivalence classes are precisely the fibers.

Definition (Graph of a Function)

The *graph* of $f : A \to B$ is:

$$\operatorname{Graph}(f) \;:=\; \{\, (a, b) \in A \times B \mid b = f(a) \,\}.$$

Remark 3.9.5 (Function as graph). A relation $G \subseteq A \times B$ is the graph of a function $f : A \to B$ iff G is left-total and right-unique. Functions may therefore be identified with their graphs: a function *is* a special kind of relation.

Definition (Injective Function)

$f : A \to B$ is *injective* (one-to-one) if distinct elements of A have distinct images:

$$\forall a_1, a_2 \in A,\; f(a_1) = f(a_2) \implies a_1 = a_2.$$

Definition (Surjective Function)

$f : A \to B$ is *surjective* (onto) if every element of B is achieved:

$$\forall b \in B,\; \exists a \in A \text{ such that } f(a) = b.$$

Equivalently, $\operatorname{im}(f) = B$.

Definition (Bijective Function)

$f : A \to B$ is *bijective* if it is both injective and surjective.

Remark 3.9.6 (Fiber interpretation). Injectivity: each fiber has at most one element. Surjectivity: every fiber is nonempty. Bijectivity: every fiber has exactly one element.

Definition (Identity and Inclusion)

The *identity function* on A is $\mathrm{id}_A : A \to A$, $\mathrm{id}_A(a) = a$.
If $A \subseteq B$, the *inclusion map* is $\iota : A \hookrightarrow B$, $\iota(a) = a$.

Remark 3.9.7 (Identity vs. inclusion). Both send each element to itself, but the inclusion map has a strictly larger codomain when $A \subsetneq B$. The inclusion map is always injective.

Definition 3.9.8 (Constant Function). $f : A \to B$ is *constant* if there exists $b_0 \in B$ with $f(a) = b_0$ for all $a \in A$.

Remark (Fully quantified form). f is constant $\leftrightarrow$ $\exists b_0 \in B,\ \forall a \in A,\ f(a) = b_0$.

3.10 Composition, Inverses, and Set-Image Laws

Composition & Inverses Quick Reference

Result	Statement	Proof method	Detail
Associativity of $\circ$	$h \circ (g \circ f) = (h \circ g) \circ f$	element-wise	Thm
Identity and $\circ$	$f \circ \mathrm{id}_A = \mathrm{id}_B \circ f = f$	element-wise	Thm
Inj/Surj under $\circ$	Closed under composition; partial converses	element-wise	Thm
Inverse characterization	$f^{-1} \circ f = \mathrm{id}_A,\ f \circ f^{-1} = \mathrm{id}_B$	bijectivity	Thm
Inverse of composition	$(g \circ f)^{-1} = f^{-1} \circ g^{-1}$	verify both sides	Thm
One-sided inverses	Left inverse $\iff$ injective; right $\iff$ surjective		Thm
Preimage preserves ops	f^{-1} commutes with $\cup, \cap, \setminus, c$	element-wise	Thm
Image and set ops	Images preserve $\cup$; contain $\subseteq$ for $\cap$	counterexample	Thm

Definition (Composition)

Let $f : A \to B$ and $g : B \to C$. The *composition* $g \circ f : A \to C$ is:

$$(g \circ f)(a) := g\big(f(a)\big) \quad \text{for all } a \in A.$$

Remark 3.10.1 (Non-commutativity). Composition is generally not commutative: $g \circ f \neq f \circ g$ even when both are defined.

Theorem 3.10.2 (Associativity of Composition). *For $f : A \to B$, $g : B \to C$, $h : C \to D$:*

$$h \circ (g \circ f) = (h \circ g) \circ f.$$

Remark (Fully quantified form). $\forall a \in A :\ h(g(f(a))) = h(g(f(a)))$, i.e. $\forall f : A \to B,\ g : B \to C,\ h : C \to D,\ \forall a \in A :\ [h \circ (g \circ f)](a) = [(h \circ g) \circ f](a)$.

Theorem 3.10.3 (Identity and Composition). *For any $f : A \to B$:*

$$f \circ \mathrm{id}_A = f \quad \textit{and} \quad \mathrm{id}_B \circ f = f.$$

Remark (Fully quantified form). $\forall f : A \to B,\ \forall a \in A :\ (f \circ \mathrm{id}_A)(a) = f(a)$ and $(\mathrm{id}_B \circ f)(a) = f(a)$.

Theorem 3.10.4 (Injectivity and Surjectivity Under Composition). *Let $f : A \to B$ and $g : B \to C$.*

(i) If f and g are injective, then $g \circ f$ is injective.

(ii) If f and g are surjective, then $g \circ f$ is surjective.

(iii) If $g \circ f$ is injective, then f is injective.

(iv) If $g \circ f$ is surjective, then g is surjective.

Remark (Fully quantified form). (i) $(\forall a_1, a_2 \in A,\ f(a_1) = f(a_2) \to a_1 = a_2) \wedge (\forall b_1, b_2 \in B,\ g(b_1) = g(b_2) \to b_1 = b_2) \ \to\ \forall a_1, a_2 \in A,\ g(f(a_1)) = g(f(a_2)) \to a_1 = a_2$.
(iii) $(\forall a_1, a_2 \in A,\ g(f(a_1)) = g(f(a_2)) \to a_1 = a_2) \ \to\ \forall a_1, a_2 \in A,\ f(a_1) = f(a_2) \to a_1 = a_2$.

Remark 3.10.5 (Sharpness of the implications). In (iii), g need not be injective. In (iv), f need not be surjective. Bijectivity of $g \circ f$ does not imply bijectivity of either f or g individually.

Definition (Inverse Function)

Let $f : A \to B$ be bijective. The *inverse function* is $f^{-1} : B \to A$ defined by: $f^{-1}(b) = a$ iff $f(a) = b$.

Theorem 3.10.6 (Characterization of Inverse Functions). *Let $f : A \to B$ be bijective. Then*

$$f^{-1} \circ f = \mathrm{id}_A \quad \text{and} \quad f \circ f^{-1} = \mathrm{id}_B .$$

Conversely, a function admits an inverse iff it is bijective.

Remark (Fully quantified form). $\forall a \in A :\ f^{-1}(f(a)) = a$ and $\forall b \in B :\ f(f^{-1}(b)) = b$.
Converse: $(\exists g : B \to A,\ g \circ f = \mathrm{id}_A \wedge f \circ g = \mathrm{id}_B) \ \leftrightarrow\ f$ is bijective.

Theorem 3.10.7 (Inverse of a Composition). *Let $f : A \to B$ and $g : B \to C$ be bijective. Then $g \circ f$ is bijective and*

$$(g \circ f)^{-1} = f^{-1} \circ g^{-1}.$$

Remark (Fully quantified form). $\forall c \in C :\ (g \circ f)^{-1}(c) = f^{-1}(g^{-1}(c))$.

Definition (Left and Right Inverses)

Let $f : A \to B$.
A *left inverse* of f is a function $g : B \to A$ with $g \circ f = \mathrm{id}_A$.
A *right inverse* (or *section*) of f is a function $h : B \to A$ with $f \circ h = \mathrm{id}_B$.

Theorem 3.10.8 (One-Sided Inverses and Function Properties). *Let $f : A \to B$.*

(i) f has a left inverse $\iff$ f is injective (and $A \neq \varnothing$ or $A = B = \varnothing$).

(ii) f has a right inverse $\iff$ f is surjective.

(iii) If f has both a left inverse g and a right inverse h, then $g = h$ and f is bijective.

Remark (Fully quantified form). (i) $(\exists g : B \to A,\ \forall a \in A,\ g(f(a)) = a) \ \leftrightarrow\ \forall a_1, a_2 \in A,\ f(a_1) = f(a_2) \to a_1 = a_2$.
(ii) $(\exists h : B \to A,\ \forall b \in B,\ f(h(b)) = b) \ \leftrightarrow\ \forall b \in B,\ \exists a \in A,\ f(a) = b$.

Remark 3.10.9 (Axiom of Choice). The existence of a right inverse for every surjective function is equivalent to the Axiom of Choice.

Definition (Restriction and Extension)

Let $f : A \to B$ and $C \subseteq A$. The *restriction* of f to C is $f|_C : C \to B$, $f|_C(a) = f(a)$ for all $a \in C$.
Let $A \subseteq A'$, and let $f : A \to B$. A function $g : A' \to B$ is an *extension* of f if $g|_A = f$.

Remark 3.10.10 (Extensions not unique). An extension agrees with f on all of A but may be defined on a larger domain A'. Extensions are generally not unique; uniqueness requires additional constraints (such as continuity in analysis or linearity in algebra), leading to important results like the Tietze Extension Theorem and the Hahn-Banach Theorem.

Theorem 3.10.11 (Preimages Preserve Set Operations). *Let $f : A \to B$ and $S, T \subseteq B$. Then*

(i) $f^{-1}(S \cup T) = f^{-1}(S) \cup f^{-1}(T)$,

(ii) $f^{-1}(S \cap T) = f^{-1}(S) \cap f^{-1}(T)$,

(iii) $f^{-1}(S \setminus T) = f^{-1}(S) \setminus f^{-1}(T)$,

(iv) $f^{-1}(S^c) = (f^{-1}(S))^c$.

Remark (Fully quantified form). (i) $\forall a \in A : \ a \in f^{-1}(S \cup T) \leftrightarrow (f(a) \in S \vee f(a) \in T)$.
(ii) $\forall a \in A : \ a \in f^{-1}(S \cap T) \leftrightarrow (f(a) \in S \wedge f(a) \in T)$.
(iv) $\forall a \in A : \ a \in f^{-1}(S^c) \leftrightarrow f(a) \notin S$.

Theorem 3.10.12 (Images and Set Operations). *Let $f : A \to B$ and $S, T \subseteq A$. Then*

(i) $f(S \cup T) = f(S) \cup f(T)$,

(ii) $f(S \cap T) \subseteq f(S) \cap f(T)$,

(iii) $f(S \setminus T) \supseteq f(S) \setminus f(T)$.

Equality holds in (ii) and (iii) for all S, T iff f is injective.

Remark (Fully quantified form). (i) $\forall b \in B : \ b \in f(S \cup T) \leftrightarrow (\exists a \in S,\ f(a) = b) \vee (\exists a \in T,\ f(a) = b)$.
(ii) $\forall b \in B : \ (\exists a \in S \cap T,\ f(a) = b) \to (\exists a \in S,\ f(a) = b) \wedge (\exists a \in T,\ f(a) = b)$; equality iff f injective.

Remark 3.10.13 (Asymmetry between images and preimages). Forward images preserve unions but generally not intersections or complements. This asymmetry between images and preimages is fundamental and explains why preimages appear more naturally in topology and measure theory.

3.11 Ordered Sets

Ordered Sets Quick Reference

Concept	Meaning	Detail
Ordered set	$(A, \leq)$: set with a partial order	Def
Strict order	$a < b \iff a \leq b \wedge a \neq b$	Def
Comparable / incomparable	$a \leq b$ or $b \leq a$ / neither	Def
Total (linear) order	Partial order with all pairs comparable	Def
Upper / lower bound	$u \geq s$ / $\ell \leq s$ for all $s \in S$	Def
Minimal / maximal	No smaller/larger element in S	Def
Least / greatest	Smaller/larger than all elements of S	Def
Order-preserving	$a \leq b \Rightarrow f(a) \leq' f(b)$	Def
Order isomorphism	Bijective order-preserving map with order-preserving inverse	Def
Well-ordered set	Every nonempty subset has a least element	Def
Chain / antichain	All comparable / none comparable	Def
Initial segment	Downward-closed subset	Def

Definition (Ordered Set)

An *ordered set* (or *partially ordered set, poset*) is a pair $(A, \leq)$ where $\leq$ is a partial order on A: a relation that is reflexive, antisymmetric, and transitive.

Remark 3.11.1 (English reading). An ordered set is a set equipped with a specified way to compare elements, but without requiring all pairs to be comparable.

Definition (Strict Order)

Let $(A, \leq)$ be an ordered set. The *strict order* $<$ is defined by:

$$a < b \iff (a \leq b \wedge a \neq b).$$

Remark 3.11.2 (Strict and non-strict are equivalent). The relation $<$ is irreflexive and transitive. Conversely, given any strict partial order $<$, one recovers a non-strict order by setting $a \leq b \iff (a < b \vee a = b)$. The two formulations carry exactly the same information.

Definition 3.11.3 (Comparable and Incomparable Elements). In an ordered set $(A, \leq)$, elements $a, b \in A$ are *comparable* if $a \leq b$ or $b \leq a$; they are *incomparable* if neither holds.

Remark (Fully quantified form). a, b comparable $\leftrightarrow$ $a \leq b \vee b \leq a$. a, b incomparable $\leftrightarrow$ $\neg(a \leq b) \wedge \neg(b \leq a)$.

Remark 3.11.4 (Incomparable elements). Incomparable elements can only occur in partial orders. In a total order, every pair is comparable by definition.

Definition (Total / Linear Order)

An ordered set $(A, \leq)$ is a *total order* (or *linear order*) if every pair of elements is comparable:
$$\forall a, b \in A,\ a \leq b \lor b \leq a.$$

Remark 3.11.5 (Examples). $(\mathbb{R}, \leq)$ is a total order. $(\mathcal{P}(A), \subseteq)$ is a partial order that is generally not total.

Definition (Upper and Lower Bounds)

Let $(A, \leq)$ be an ordered set and $S \subseteq A$.
An element $u \in A$ is an *upper bound* of S if $\forall s \in S,\ s \leq u$.
An element $\ell \in A$ is a *lower bound* of S if $\forall s \in S,\ \ell \leq s$.

Remark 3.11.6 (Consequence). Bounds need not exist and need not be unique. They need not lie in S itself. Bounds are central to the definition of completeness for ordered fields.

Definition (Minimal, Maximal, Least, Greatest Elements)

Let $S \subseteq A$ in an ordered set $(A, \leq)$.
$m \in S$ is a *minimal element* of S if $\nexists s \in S$ with $s < m$.
$M \in S$ is a *maximal element* of S if $\nexists s \in S$ with $M < s$.
$\ell \in S$ is the *least element* if $\forall s \in S,\ \ell \leq s$.
$g \in S$ is the *greatest element* if $\forall s \in S,\ s \leq g$.

Remark 3.11.7 (Minimal vs. least). A *least* element is below every element of S; it must be unique if it exists. A *minimal* element merely has nothing below it within S; there may be many. Every least element is minimal, but not conversely.

Definition (Order-Preserving Map and Isomorphism)

Let $(M, \leq)$ and $(M', \leq')$ be partially ordered sets.
A function $f : M \to M'$ is *order-preserving* (or *monotone*) if
$$\forall a, b \in M, \quad a \leq b \implies f(a) \leq' f(b).$$
f is an *order isomorphism* if it is bijective and
$$\forall a, b \in M, \quad a \leq b \iff f(a) \leq' f(b).$$

Remark 3.11.8 (Order-isomorphic posets). An order isomorphism preserves and reflects the order structure exactly, including comparability, minimal and maximal elements, and bounds. Two posets are *order-isomorphic* if such a map exists; they are structurally identical from the order-theoretic viewpoint.

Definition (Well-Ordered Set)

An ordered set $(A, <)$ is *well-ordered* if every nonempty subset $S \subseteq A$ has a least element:

$$\forall S \subseteq A, \ \big(S \neq \varnothing \ \Rightarrow \ \exists m \in S \text{ s.t. } m \leq s \ \forall s \in S\big).$$

Remark 3.11.9 (Well-order implies total order). Every well-ordered set is totally ordered. The well-ordering condition is strictly stronger: it requires a least element in every nonempty subset, not just in A as a whole.

Example 3.11.10 (Well-order examples). $(\mathbb{N}, \leq)$ is well-ordered. $(\mathbb{Z}, \leq)$ is not: the set $\{\dots, -3, -2, -1\}$ has no least element. $(\mathbb{R}, \leq)$ is not: $(0, 1)$ has no least element.

Remark 3.11.11 (Connection to ordinal numbers). An *ordinal number* is defined as an equivalence class of well-ordered sets under order isomorphism: it measures the *order type* of a well-ordered set, not merely its cardinality.

Definition 3.11.12 (Chain and Antichain). A subset $C \subseteq A$ of a poset $(A, \leq$ $)$ is a *chain* if every pair of elements in C is comparable. It is an *antichain* if no two distinct elements of C are comparable.

Remark (Fully quantified form). C is a chain $\leftrightarrow$ $\forall a, b \in C, \ a \leq b \vee b \leq a$. C is an antichain $\leftrightarrow$ $\forall a, b \in C, \ a \neq b \to \neg(a \leq b) \wedge \neg(b \leq a)$.

Definition 3.11.13 (Initial Segment). A subset $I \subseteq A$ is an *initial segment* of $(A, \leq)$ if

$$a \in I \text{ and } b \leq a \ \Rightarrow \ b \in I.$$

Remark (Fully quantified form). I is an initial segment $\leftrightarrow$ $\forall a \in I, \ \forall b \in A, \ b \leq a \to b \in I$.

Remark 3.11.14 (Transition). Ordered sets provide the abstract framework for order structures on the real numbers, function spaces, and metric spaces. In later sections, order interacts with topology and analysis through intervals, monotone functions, and the completeness axiom for $\mathbb{R}$.

Definition (Directed Set)

A preorder $(I, \leq)$ is *directed* (or a *directed set*) if every finite subset of I has an upper bound in I: for all $i, j \in I$ there exists $k \in I$ with $i \leq k$ and $j \leq k$.

Remark 3.11.15 (What directed means). In a directed set, any two elements can always be "dominated" by a third. The condition does not require a maximum element to exist, only that no two elements are ever incomparable in a final way: there is always a common upper bound available. Every totally ordered set is directed (take $k = \max(i, j)$), but directed sets can be far more general.

Example 3.11.16 (Canonical directed sets). (i) $(\mathbb{N}, \leq)$ is directed: given $m, n \in \mathbb{N}$, take $k = \max(m, n)$.

(ii) The collection of all finite subsets of a set X, ordered by inclusion, is directed: given finite sets $F_1, F_2 \subseteq X$, take $k = F_1 \cup F_2$.

(iii) The set of all open neighborhoods of a point x in a topological space, ordered by reverse inclusion ($U \leq V$ iff $U \supseteq V$), is directed: given neighborhoods U and V of x, take $k = U \cap V$, which is also a neighborhood of x.

(iv) A partially ordered set is directed if and only if every finite subset has an upper bound. Partially ordered sets that are *not* directed have "incomparable maximal elements" elements with no common upper bound.

Definition (Net)

Let $(I, \leq)$ be a directed set and X a set. A *net* in X is a function $x : I \to X$. The value at $i \in I$ is written x_i, and the net is written $(x_i)_{i \in I}$.

Remark 3.11.17 (Nets generalize sequences). A sequence is a net with index set $(\mathbb{N}, \leq)$. Nets are the correct generalization of sequences to general topological spaces: in a metric space, sequences suffice to detect all topological properties (closure, continuity, compactness). In a general topological space, sequences can fail to detect these properties one needs nets. Specifically, a subset A of a topological space is closed if and only if every convergent net in A has its limit in A; and a function is continuous if and only if it preserves limits of nets.

The definition of net is placed here because it is purely order-theoretic: a directed index set and a function. The analytic content what it means for a net to converge belongs to topology and functional analysis. By establishing the set-theoretic definition now, those chapters can immediately state convergence conditions without set-theoretic digression.

3.12 Supremum and Infimum

Supremum, Infimum, and Bounds Quick Reference

Concept	Meaning	Detail
Upper bound	$u \geq s$ for all $s \in S$	Def
Lower bound	$\ell \leq s$ for all $s \in S$	Def
Supremum	Least upper bound of S	Def
Infimum	Greatest lower bound of S	Def
Uniqueness	Suprema and infima are unique when they exist	Prop
Max/min vs. sup/inf	Maximum $\Rightarrow$ supremum; not conversely	Rem
Duality	sup in $(A, \leq)$ = inf in $(A, \geq)$	Prop

Every student of analysis encounters the least upper bound property of $\mathbb{R}$ early and prominently. But this property is not a peculiarity of real numbers: it is an order-theoretic concept that can be formulated in any partially ordered set. Before we reach $\mathbb{R}$, it is essential to understand supremum and infimum in their natural habitat abstract ordered sets so that when the completeness axiom appears in Volume II, you will recognise exactly what it is asserting and why $\mathbb{Q}$ fails to satisfy it.

The definitions in this section are also the backbone of measure theory. The extended real line $\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$, equipped with the usual order, is the ambient poset in which measures take their values. Countable additivity, the monotone convergence theorem, and Fatou's lemma are all statements about suprema of sequences. Understanding supremum and infimum at the abstract level here means those later theorems will feel inevitable rather than arbitrary.

The definition of upper and lower bound was given in the Ordered Sets section. For convenience we recall: $u \in A$ is an *upper bound* of $S \subseteq A$ if $\forall s \in S,\ s \leq u$; a *lower bound* satisfies the reverse. We say S is *bounded above* (resp. *bounded below*, *bounded*) if it has at least one upper (resp. lower, both) bound.

Remark 3.12.1 (Bounds depend on the ambient poset). Whether S is bounded above depends on the poset $(A, \leq)$, not on S alone. The set $\{r \in \mathbb{Q} : r^2 < 2\}$ is bounded above in $(\mathbb{Q}, \leq)$ the rational number 2 is an upper bound but it has *no* supremum inside $\mathbb{Q}$. The same set in $(\mathbb{R}, \leq)$ has supremum $\sqrt{2} \in \mathbb{R}$. Being bounded above is necessary but not sufficient for a supremum to exist; the completeness axiom for $\mathbb{R}$ is precisely the assertion that sufficiency holds in $\mathbb{R}$.

Definition (Supremum and Infimum)

Let $(A, \leq)$ be a poset and $S \subseteq A$.
The *supremum* (or *least upper bound*) of S, written $\sup S$, is an element $u^* \in A$ satisfying:

(i) u^* is an upper bound of S: $\forall s \in S,\ s \leq u^*$;

(ii) u^* is the *least* upper bound: if u is *any* upper bound of S, then $u^* \leq u$.

The *infimum* (or *greatest lower bound*) of S, written $\inf S$, is an element $\ell^* \in A$ satisfying:

(i) ℓ^* is a lower bound of S: $\forall s \in S,\ \ell^* \leq s$;

(ii) ℓ^* is the *greatest* lower bound: if ℓ is *any* lower bound of S, then $\ell \leq \ell^*$.

Remark (Fully quantified form). $u^* = \sup S \ \leftrightarrow\ (\forall s \in S,\ s \leq u^*) \ \wedge\ (\forall u \in A,\ (\forall s \in S,\ s \leq u) \ \to\ u^* \leq u)$.
$\ell^* = \inf S \ \leftrightarrow\ (\forall s \in S,\ \ell^* \leq s) \ \wedge\ (\forall \ell \in A,\ (\forall s \in S,\ \ell \leq s) \ \to\ \ell \leq \ell^*)$.

Remark 3.12.2 (How to read these definitions). Condition (i) says u^* is a *candidate*: it stands above everything in S. Condition (ii) says u^* is the *best* candidate: it is at most as large as any other upper bound. Together they say: u^* is the unique element of A that is simultaneously an upper bound of S and no larger than any other upper bound.

A useful restatement of condition (ii) is its contrapositive: if $v < u^*$, then v is *not* an upper bound of S, meaning there exists some $s \in S$ with $v < s$. This is how the condition is typically used in proofs in analysis: to verify that $u^* = \sup S$, you show (i) u^* is an upper bound, and (ii) for every $\varepsilon > 0$, the value $u^* - \varepsilon$ is *not* an upper bound, i.e., there exists $s \in S$ with $s > u^* - \varepsilon$.

Proposition 3.12.3 (Uniqueness of Supremum and Infimum). *Let $(A, \leq)$ be a poset and $S \subseteq A$. If $\sup S$ exists, it is unique. If $\inf S$ exists, it is unique.*

Remark (Fully quantified form). $\forall u^*, v^* \in A$: if both u^* and v^* satisfy the two conditions in the definition of $\sup S$, then $u^* = v^*$.

Remark 3.12.4 (Proof idea). If u^* and v^* are both suprema, then u^* is an upper bound, so the minimality of v^* gives $v^* \leq u^*$; and symmetrically $u^* \leq v^*$. Antisymmetry of the partial order then forces $u^* = v^*$. This is a standard antisymmetry argument: when two elements are mutually $\leq$-related in a partial order, they must be equal.

Remark 3.12.5 (Supremum need not belong to S). The set $S = (0, 1) \subseteq \mathbb{R}$ is open: it contains neither endpoint. Yet $\sup S = 1$ and $\inf S = 0$. Neither 0 nor 1 belongs to S.

When the supremum does happen to lie in S, it is called the *maximum* of S, written $\max S$. Thus $\max S = \sup S$ when $\sup S \in S$, and $\max S$ is undefined when $\sup S \notin S$. Every maximum is a supremum, but the converse fails: a supremum may exist without being a maximum. The same relationship holds between infimum and minimum.

This distinction is critical in analysis. A continuous function on a closed bounded interval always achieves its maximum (the Extreme Value Theorem), but on an open interval it may approach a supremum without ever attaining it.

Remark 3.12.6 (Supremum need not exist). Being bounded above is necessary but not sufficient for a supremum to exist. Consider $S = \{r \in \mathbb{Q} : r^2 < 2\}$ in the poset $(\mathbb{Q}, \leq)$. Every element of S satisfies $r < 2$, so $2 \in \mathbb{Q}$ is an upper bound. But the candidate supremum would need to be $\sqrt{2}$, and $\sqrt{2} \notin \mathbb{Q}$. Any rational upper bound $q > \sqrt{2}$ is not the least upper bound, because $\frac{q+\sqrt{2}}{2}$ is a smaller upper bound (and is also irrational, which means we can always find a rational strictly between any rational upper bound and $\sqrt{2}$, and that rational is still an upper bound). In short: S is bounded above in $\mathbb{Q}$, but $\sup S$ does not exist in $\mathbb{Q}$.

This is the precise failure that the *completeness axiom* for $\mathbb{R}$ corrects: it asserts that every nonempty subset of $\mathbb{R}$ that is bounded above has a supremum in $\mathbb{R}$. The rational numbers satisfy every other ordered field axiom but fail this one.

Example 3.12.7 (Sup and inf in divisibility). Order the set $D = \{1, 2, 3, 4, 6, 12\}$ by divisibility: $a \leq b$ iff $a \mid b$. Take $S = \{4, 6\}$.

Upper bounds of S: an element $u \in D$ must satisfy $4 \mid u$ and $6 \mid u$, i.e., u must be a common multiple of 4 and 6 in D. The only such element is 12, so $\sup S = 12 = \operatorname{lcm}(4, 6)$.

Lower bounds of S: an element $\ell \in D$ must satisfy $\ell \mid 4$ and $\ell \mid 6$, i.e., ℓ must be a common divisor of 4 and 6 in D. The common divisors are $\{1, 2\}$; the greatest is 2, so $\inf S = 2 = \gcd(4, 6)$.

This example reveals a deep pattern: in any divisibility poset, $\sup$ computes the least common multiple and $\inf$ computes the greatest common divisor. Order theory and number theory are speaking the same language.

Example 3.12.8 (Sup and inf for function spaces). Let $B(X)$ be the set of bounded real-valued functions on a set X, ordered pointwise: $f \leq g$ iff $f(x) \leq g(x)$

for all $x \in X$. For a collection $\{f_\alpha\}$ of bounded functions, the pointwise supremum is $(\sup_\alpha f_\alpha)(x) = \sup_\alpha f_\alpha(x)$. In measure theory and integration theory, one constantly takes suprema and infima of sequences of functions; this example shows that such operations are instances of the abstract sup/inf defined here.

Proposition 3.12.9 (Characterisation of Supremum). *Let $(A, \leq)$ be a poset, $S \subseteq A$, and $u^* \in A$. Then $u^* = \sup S$ if and only if:*

(i) u^* is an upper bound of S; and

(ii) for every $v \in A$ with $v < u^*$, there exists $s \in S$ with $v < s$.

Remark (Fully quantified form). $u^* = \sup S \leftrightarrow (\forall s \in S,\ s \leq u^*) \wedge (\forall v \in A,\ v < u^* \rightarrow \exists s \in S,\ v < s)$.

Remark 3.12.10 (Why this characterisation is useful). Condition (ii) here is the contrapositive of the minimality condition in the definition. In analysis this is the working form: to check that u^* is the supremum, you verify that nothing strictly below u^* is an upper bound. In $\mathbb{R}$ this becomes: for every $\varepsilon > 0$, there exists $s \in S$ with $s > u^* - \varepsilon$. This is the exact form used in the proof that $\sup$ interacts correctly with limits, integrals, and measures.

Proposition 3.12.11 (Duality of Supremum and Infimum). *Let $(A, \leq)$ be a poset and $S \subseteq A$. Then $\inf_{(A,\leq)} S = \sup_{(A,\geq)} S$, where $(A, \geq)$ denotes the dual poset.*

In particular: if every nonempty bounded-above subset of $(A, \leq)$ has a supremum, then every nonempty bounded-below subset of $(A, \leq)$ has an infimum.

Remark (Fully quantified form). $\forall S \subseteq A$: ℓ^* is the infimum of S in $(A, \leq)$ $\leftrightarrow$ ℓ^* is the supremum of S in $(A, \geq)$.

Remark 3.12.12 (Consequence for $\mathbb{R}$). This proposition explains why the completeness axiom for $\mathbb{R}$ is stated only for suprema: the infimum statement comes for free by duality. The axiom says every nonempty bounded-above subset has a supremum; the proposition immediately gives that every nonempty bounded-below subset has an infimum. You will use this constantly without comment in Volume II.

Remark 3.12.13 (Sequences and sup/inf). A sequence $(a_n)_{n=1}^{\infty}$ in $\mathbb{R}$ is a function $\mathbb{N} \to \mathbb{R}$, so its range $\{a_n : n \in \mathbb{N}\}$ is a subset of $\mathbb{R}$. The quantities $\sup_n a_n$ and $\inf_n a_n$ are the supremum and infimum of this range set in the poset $(\mathbb{R}, \leq)$.

More subtle are the *limit superior* and *limit inferior*:

$$\limsup_{n\to\infty} a_n = \inf_{n\geq 1} \sup_{k\geq n} a_k, \qquad \liminf_{n\to\infty} a_n = \sup_{n\geq 1} \inf_{k\geq n} a_k.$$

These are iterated suprema and infima. The fact that they always exist in $\overline{\mathbb{R}}$ (the extended reals, where $\pm\infty$ are adjoined to supply missing bounds) is a direct application of the completeness of $\mathbb{R}$. Mastering the abstract definitions here makes those formulas transparent rather than mysterious.

Remark 3.12.14 (Transition). Supremum and infimum are defined for arbitrary subsets of a poset, but they need not always exist. When *every* two-element subset $\{a, b\}$ has both a supremum and an infimum, the poset has enough structure to define binary operations $a \vee b = \sup\{a, b\}$ and $a \wedge b = \inf\{a, b\}$. This additional structure is called a *lattice*, and it is developed in the next section. Lattices are the precise abstract setting for the σ-algebras and topologies encountered in measure theory and topology.

3.13 Lattices

Lattices Quick Reference

Concept	Meaning	Detail
Lattice	Poset with $\sup\{a,b\}$ and $\inf\{a,b\}$ for all a,b	Def
Join / Meet	$a \vee b = \sup\{a,b\}$; $a \wedge b = \inf\{a,b\}$	Def
Complete lattice	Every subset has sup and inf	Def
Distributive lattice	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	Def
Boolean lattice	Complemented distributive lattice	Def
$\mathcal{P}(X)$	Complete Boolean lattice under $\subseteq$	Ex
σ-algebras	Sub-lattices of $\mathcal{P}(X)$ closed under countable ops	Rem
Knaster–Tarski	Every monotone map on a complete lattice has a fixed point	Thm

The preceding section defined supremum and infimum for arbitrary subsets of a poset. In many posets that appear in practice, every pair of elements already has a supremum and an infimum. When this is the case, the binary operations $a \vee b = \sup\{a,b\}$ and $a \wedge b = \inf\{a,b\}$ are well-defined everywhere, giving the poset the algebraic structure of a *lattice*.

Lattices are ubiquitous:

- The power set $\mathcal{P}(X)$ ordered by $\subseteq$ is a complete lattice, with join = union and meet = intersection.
- Every σ-algebra is a sub-lattice of $\mathcal{P}(X)$ that is additionally closed under countable joins and meets and under complementation.
- Every topology is a sub-structure of $\mathcal{P}(X)$ that is closed under arbitrary joins and finite meets.
- In functional analysis, the lattice of closed subspaces of a Hilbert space is a complete lattice under inclusion.

Understanding lattices here means that when σ-algebras, topologies, and sub-σ-algebras are encountered downstream, their closure conditions will be recognisable instances of a single abstract pattern.

Definition (Lattice)

A *lattice* is a poset $(L, \leq)$ in which every two-element subset $\{a,b\} \subseteq L$ has both a supremum and an infimum in L.
The supremum $\sup\{a,b\}$ is called the *join* of a and b, written $a \vee b$. The infimum $\inf\{a,b\}$ is called the *meet* of a and b, written $a \wedge b$.

Remark (Fully quantified form). $(L, \leq)$ is a lattice $\leftrightarrow$ $\forall a,b \in L,\ \exists\, a \vee b \in L$ and $\exists\, a \wedge b \in L$, where $a \vee b$ and $a \wedge b$ satisfy the sup and inf conditions of Definition (Supremum and Infimum).

Remark 3.13.1 (Join and meet as operations). In a lattice, $\vee$ and $\wedge$ are binary operations $L \times L \to L$. They satisfy, for all $a,b,c \in L$:

(i) *Commutativity*: $a \vee b = b \vee a$ and $a \wedge b = b \wedge a$.

(ii) *Associativity*: $(a \vee b) \vee c = a \vee (b \vee c)$ and $(a \wedge b) \wedge c = a \wedge (b \wedge c)$.

(iii) *Idempotency*: $a \vee a = a$ and $a \wedge a = a$.

(iv) *Absorption*: $a \vee (a \wedge b) = a$ and $a \wedge (a \vee b) = a$.

Properties (i)-(iv) follow directly from the order-theoretic definitions of join and meet. Conversely, any set with two binary operations satisfying (i)-(iv) can be given a partial order making it a lattice: define $a \leq b$ iff $a \wedge b = a$. The two approaches order-theoretic and algebraic are equivalent.

Example 3.13.2 (Power set lattice). For any set X, the power set $(\mathcal{P}(X), \subseteq)$ is a lattice: $A \vee B = A \cup B$ and $A \wedge B = A \cap B$. The algebraic laws of the Set Algebra section (commutativity, associativity, distributivity, absorption) are exactly the lattice laws in this example. This is not a coincidence: Boolean algebra is a special kind of lattice.

Example 3.13.3 (Divisibility lattice). The divisibility order on $\mathbb{N}^+ = \{1, 2, 3, \ldots\}$ makes it a lattice: $a \vee b = \operatorname{lcm}(a, b)$ and $a \wedge b = \gcd(a, b)$, as we noted in Example 3.12.7.

Example 3.13.4 (Not every poset is a lattice). Consider the poset $(\{a, b, c, d\}, \leq)$ with $a < c$, $a < d$, $b < c$, $b < d$, and a, b incomparable. The subset $\{a, b\}$ has two minimal upper bounds, c and d, which are incomparable. Hence $\sup\{a, b\}$ does not exist, and this poset is not a lattice.

The power set $\mathcal{P}(\{1, 2, 3\})$ restricted to subsets of cardinality $\neq 1$ provides a cleaner example: $\{1, 2\}$ and $\{1, 3\}$ have no meet in this restricted collection.

Definition (Complete Lattice)

A lattice $(L, \leq)$ is a *complete lattice* if every subset $S \subseteq L$ (including the empty set and L itself) has both a supremum and an infimum in L:

$$\forall S \subseteq L, \quad \sup S \in L \ \text{ and } \ \inf S \in L.$$

Remark (Fully quantified form). $(L, \leq)$ is a complete lattice $\leftrightarrow$ $\forall S \subseteq L$, $\exists u^* \in L$, $\exists \ell^* \in L$ satisfying the sup and inf conditions of Definition (Supremum and Infimum).

Remark 3.13.5 (The empty set forces top and bottom). When $S = \varnothing$, the sup condition says: u^* is an upper bound of the empty set (which is vacuously satisfied by every element of L), and u^* is at most every other upper bound. This forces $u^* = \inf L$, the *bottom element* of L (often written $\perp$). Symmetrically, $\inf \varnothing = \sup L$, the *top element* (often written $\top$).

Concretely: in $(\mathcal{P}(X), \subseteq)$, we have $\sup \varnothing = \varnothing$ (the empty union) and $\inf \varnothing = X$ (the empty intersection equals the whole universe, by convention). Every complete lattice therefore has a top element $\top = \sup L$ and a bottom element $\perp = \inf L$.

Example 3.13.6 ($\mathcal{P}(X)$ is a complete lattice). For any set X, the power set $(\mathcal{P}(X), \subseteq)$ is a complete lattice: for any family $\{A_\alpha\}_{\alpha \in I} \subseteq \mathcal{P}(X)$,

$$\sup_{\alpha \in I} A_\alpha = \bigcup_{\alpha \in I} A_\alpha, \qquad \inf_{\alpha \in I} A_\alpha = \bigcap_{\alpha \in I} A_\alpha.$$

The bottom element is $\varnothing$ and the top element is X. This is the canonical example of a complete lattice, and every σ-algebra on X is a complete sub-lattice of $(\mathcal{P}(X), \subseteq)$.

Example 3.13.7 ($[0,\infty]$ is a complete lattice). The extended positive half-line $[0,\infty] = [0,\infty) \cup \{+\infty\}$ with the usual order is a complete lattice. Every nonempty subset has a supremum (which may be $+\infty$) and an infimum (which may be 0). This is the value space for measures: a measure assigns to each measurable set a value in $[0,\infty]$, and the key properties of measures (countable additivity, monotone convergence) are stated in terms of sups and infs in this complete lattice.

Definition (Distributive Lattice)

A lattice $(L, \leq)$ is *distributive* if for all $a, b, c \in L$:

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c).$$

Remark 3.13.8 (One law implies the other). By the duality principle for lattices, distributivity of $\wedge$ over $\vee$ is equivalent to distributivity of $\vee$ over $\wedge$: $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$. Proving one gives the other for free.

Remark 3.13.9 (Why distributivity matters). The distributive law is what makes a lattice behave like set algebra. In the power set $\mathcal{P}(X)$, the distributive laws are exactly the set identities $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ from Theorem 3.22. Distributivity is the condition that separates lattices that "look like sets" from those that do not (e.g., the lattice of closed subspaces of a Hilbert space is not distributive).

Definition (Complement and Boolean Lattice)

Let $(L, \leq)$ be a bounded distributive lattice with top element $\top$ and bottom element $\bot$. An element $a' \in L$ is a *complement* of $a \in L$ if

$$a \vee a' = \top \quad \text{and} \quad a \wedge a' = \bot.$$

A *Boolean lattice* (or *Boolean algebra*) is a bounded distributive lattice in which every element has a complement.

Remark 3.13.10 (The power set is the canonical Boolean lattice). In $\mathcal{P}(X)$, the complement of a set A is $A^c = X \setminus A$, since $A \cup A^c = X = \top$ and $A \cap A^c = \varnothing = \bot$. The power set is therefore a complete Boolean lattice. Every σ-algebra on X is a Boolean sub-algebra of $\mathcal{P}(X)$: it is closed under complements (by definition) and under finite meets and joins (since every σ-algebra is an algebra).

Remark 3.13.11 (σ-algebras as sub-lattices). A σ-algebra $\mathcal{M}$ on X is, from the lattice-theoretic viewpoint, a sub-structure of $(\mathcal{P}(X), \subseteq)$ that is:

- a complete sub-lattice (closed under all countable joins and meets);
- a Boolean sub-algebra (closed under complementation).

The condition "closed under countable joins" is exactly the σ-algebra axiom that countable unions of measurable sets are measurable. The condition "closed under meets" for countable families follows from De Morgan together with closure under countable unions and complements.

This perspective makes the definition of σ-algebra feel natural: it is exactly a complete Boolean sub-algebra of the power set lattice, where "complete" is interpreted countably (countable joins and meets), not set-theoretically.

Remark 3.13.12 (Topologies as join-sub-structures). A topology τ on X is *not* a sub-lattice of $\mathcal{P}(X)$ in general, because it is not closed under arbitrary meets (i.e., arbitrary intersections of open sets need not be open). However, τ is closed under arbitrary joins (arbitrary unions of open sets are open) and finite meets (finite intersections of open sets are open). This makes τ an *inf-semilattice* for finite meets and a complete *join-semilattice*. The failure of closure under infinite meets is fundamental: it is precisely what allows open sets to "separate" points, and it is the reason topology is structurally different from measure theory.

Theorem (Knaster–Tarski Fixed-Point Theorem)

Let $(L, \leq)$ be a complete lattice and $f : L \to L$ a monotone function (i.e., $a \leq b \Rightarrow f(a) \leq f(b)$). Then f has a least fixed point and a greatest fixed point in L. Specifically:

$$\operatorname{lfp}(f) = \inf\{\, a \in L : f(a) \leq a \,\}, \qquad \operatorname{gfp}(f) = \sup\{\, a \in L : a \leq f(a) \,\}.$$

Remark (Fully quantified form). $\forall$ complete lattices $(L, \leq)$, $\forall$ monotone $f : L \to L$: $\exists p, q \in L$ such that $f(p) = p$, $f(q) = q$, and $\forall r \in L$ with $f(r) = r$: $p \leq r \leq q$.

Remark 3.13.13 (Why this theorem appears here). The Knaster-Tarski theorem is not needed for most of Volume I, but it is placed here because its hypotheses complete lattice, monotone function are now in place and the statement is short. Its significance reaches further:

- In set theory, it is used to prove the Schröder-Bernstein theorem without the Axiom of Choice (the monotone map sends $A \mapsto X \setminus g(Y \setminus f(A))$ on $\mathcal{P}(X)$).
- In measure theory, it is used to construct the smallest σ-algebra containing a given class of sets (the generated σ-algebra is a fixed point of the "closure under countable operations" operator).
- In theoretical computer science and domain theory, it is the foundation for denotational semantics.

For now, note that $(\mathcal{P}(X), \subseteq)$ is a complete lattice and any set operation that is monotone (such as $A \mapsto \overline{A}$, closure in a topological space, or $A \mapsto \sigma(A)$, generated σ-algebra) has a fixed point by this theorem.

Remark 3.13.14 (Transition). With the notion of lattice in hand, the Hasse diagram becomes a more powerful visualisation tool: the Hasse diagram of a lattice has a unique bottom element ($\inf L$) and a unique top element ($\sup L$), and the join and meet of any two elements can be read off as the unique path structure in the diagram. The next section introduces Hasse diagrams, the duality principle, and the connection between duality and the passage from sup to inf.

3.14 Hasse Diagrams and the Duality Principle

Hasse Diagrams and Duality Quick Reference

Concept	Meaning	Detail
Cover relation	$a \lessdot b$: $a < b$ with nothing strictly between	Def
Hasse diagram	Graph encoding a poset; edges only for cover relations	Def
Dual poset	$(A, \geq)$ obtained by reversing the order on $(A, \leq)$	Def
Duality principle	Every theorem about posets yields a dual theorem for free	Prop

A partial order can be an enormous set of pairs, impossible to visualise directly. The *Hasse diagram* is a compression: it retains only the pairs that are "immediate" in the order, discarding those implied by transitivity and reflexivity. From the compressed diagram the full order can be recovered by taking transitive closure. Hasse diagrams are the standard visual language for finite posets; knowing how to read and draw them is essential for understanding the order examples in topology and lattice theory.

Definition (Cover Relation)

Let $(A, \leq)$ be a poset and $a, b \in A$. We say b *covers* a, written $a \lessdot b$, if $a < b$ and there is no $c \in A$ with $a < c < b$.

Remark (Fully quantified form). $a \lessdot b \;\leftrightarrow\; (a < b) \;\wedge\; \neg(\exists c \in A,\; a < c \;\wedge\; c < b)$.

Remark 3.14.1 (Intuition). The cover relation strips the partial order down to its "essential edges." If $a \lessdot b$, then b is the *immediate successor* of a: you cannot insert anything between them. In a finite poset, every strict relation $a < b$ factors as a finite chain of cover steps: $a = a_0 \lessdot a_1 \lessdot \cdots \lessdot a_k = b$. In an infinite poset, this factorisation may fail (e.g., in $(\mathbb{R}, \leq)$, no two distinct real numbers are in a cover relation because there is always a rational strictly between them).

Definition (Hasse Diagram)

The *Hasse diagram* of a finite poset $(A, \leq)$ is the directed graph whose vertices are the elements of A, with an upward edge from a to b whenever $a \lessdot b$. Three graphical conventions apply:

(i) Reflexive loops ($a \leq a$) are suppressed.

(ii) Edges implied by transitivity are suppressed: if $a \lessdot b$ and $b \lessdot c$, no direct edge from a to c is drawn.

(iii) Direction is encoded by height: $a \lessdot b$ is drawn with b strictly above a, so arrowheads are unnecessary.

Remark 3.14.2 (How to recover the full order). The full order can be read back from the Hasse diagram: $a \leq b$ if and only if either $a = b$ or there is an upward path in the diagram from a to b (a sequence of cover edges going strictly upward). The diagram is therefore a lossless encoding of the partial order, compressed by removing the redundant reflexive and transitive edges.

Example 3.14.3 (Divisibility on $\{1, 2, 3, 4, 6, 12\}$). Order the set $D = \{1, 2, 3, 4, 6, 12\}$ by divisibility ($a \leq b$ iff $a \mid b$). The covers are:

$$1 \lessdot 2, \quad 1 \lessdot 3, \quad 2 \lessdot 4, \quad 2 \lessdot 6, \quad 3 \lessdot 6, \quad 4 \lessdot 12, \quad 6 \lessdot 12.$$

The Hasse diagram places 1 at the bottom and 12 at the top. The relation $1 \leq 4$ holds in the poset but the edge $1 \to 4$ is *not* drawn because $1 \lessdot 2 \lessdot 4$ provides an upward path. Drawing the direct edge would be redundant.

Notice also: 4 and 6 are incomparable, so there is no edge between them they sit at the same level with no path connecting them. The join $4 \vee 6 = 12 = \operatorname{lcm}(4, 6)$ and the meet $4 \wedge 6 = 2 = \gcd(4, 6)$ are readable from the diagram as the lowest common ancestor and the highest common descendant, respectively.

Example 3.14.4 (Power set $\mathcal{P}(\{a, b, c\})$ under inclusion). The Hasse diagram of $(\mathcal{P}(\{a, b, c\}), \subseteq)$ has four levels: $\varnothing$ at the bottom; the three singletons $\{a\}, \{b\}, \{c\}$ one level up; the three two-element sets $\{a, b\}, \{a, c\}, \{b, c\}$ above those; and $\{a, b, c\}$ at the top. Each singleton is covered by the two two-element sets containing it. This diagram is the Boolean lattice B_3: it has a bottom ($\varnothing$), a top ($\{a, b, c\}$), and every element has a complement ($A' = \{a, b, c\} \setminus A$).

Remark 3.14.5 (Limitation to finite posets). For infinite posets such as $(\mathbb{N}, \leq)$, the cover relation is well-defined ($n \lessdot n+1$) but the full diagram cannot be drawn. In $(\mathbb{R}, \leq)$ or $(\mathbb{Q}, \leq)$, the cover relation is empty: no two elements are in a cover relation. Hasse diagrams are a practical visualisation tool for finite or schematic examples and should not be used as a proof device in infinite settings.

Definition (Dual Poset)

Let $(A, \leq)$ be a poset. The *dual poset* is $(A, \geq)$, where $a \geq b$ is defined to mean $b \leq a$. The dual is also written $(A, \leq^{\mathrm{op}})$.

Remark 3.14.6 (Intuition). The dual poset is obtained by flipping the Hasse diagram upside down. Every structural feature is preserved but reflected: elements at the top are now at the bottom, minimal elements become maximal, upper bounds become lower bounds, and suprema become infima and vice versa (as established in Proposition 3.12.11).

Proposition 3.14.7 (Duality Principle for Posets). *Let Φ be any first-order statement about a poset $(A, \leq)$ expressed using only the relation $\leq$. Let Φ^* be the statement obtained from Φ by replacing every occurrence of $\leq$ with $\geq$ (equivalently, working in the dual poset). Then:*

$$(A, \leq) \models \Phi \iff (A, \geq) \models \Phi.$$

In particular, if Φ is a theorem about all posets, then so is Φ^*.

Remark (Fully quantified form). $\forall$ posets $(A, \leq)$, $\forall$ first-order sentences Φ in the language of $\leq$: $(A, \leq) \models \Phi \leftrightarrow (A, \geq) \models \Phi^*$, where Φ^* substitutes $\geq$ for every $\leq$.

Remark 3.14.8 (Theorems come in pairs at no extra cost). The duality principle means every proved theorem about posets automatically yields a second theorem its dual without any additional work. You will encounter this repeatedly in analysis. Here are the most important instances:

- *Supremum $\leftrightarrow$ Infimum.* Every theorem about sup yields a theorem about inf by duality. In particular, the completeness axiom for $\mathbb{R}$ (every nonempty bounded-above subset has a supremum) immediately implies that every nonempty bounded-below subset has an infimum not a separate axiom but a consequence by duality.

- *Minimal $\leftrightarrow$ Maximal.* A statement about minimal elements dualises to a statement about maximal elements. Zorn's Lemma (every poset in which every chain is bounded above has a maximal element) dualises to: every poset in which every chain is bounded below has a minimal element.

- *Well-ordering is not self-dual.* $(\mathbb{N}, \leq)$ is well-ordered (every nonempty subset has a least element); its dual $(\mathbb{N}, \geq)$ is not well-ordered because the whole set has no greatest element. Duality preserves the form of a definition but not whether a particular structure satisfies it.

- *Join $\leftrightarrow$ Meet in lattices.* Every lattice identity involving $\vee$ and $\wedge$ dualises by swapping the two operations. The distributive law $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ dualises to $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$: proving one gives the other for free.

Remark 3.14.9 (Transition). The order vocabulary developed across the order sections partial orders, total orders, well-orders, bounds, suprema, infima, lattices, directed sets, and duality is the complete toolkit for order-theoretic reasoning in analysis. In Volume II, the construction of $\mathbb{R}$ requires only the completeness axiom (existence of suprema) and the Dedekind cut or Cauchy sequence constructions, both of which use only the vocabulary established here. In Volume III, metric space arguments, the definition of compactness, and the construction of Lebesgue measure all draw on this vocabulary repeatedly.

3.15 Order Extensions

Order Extensions Quick Reference

Concept	Meaning	Detail
Preorder	Reflexive, transitive (not necessarily antisymmetric)	Def
Loset	Linear order = complete partial order	Def
Symmetric part	$xIy \iff xRy \wedge yRx$	Def
Asymmetric part	$xPy \iff xRy \wedge \neg(yRx)$	Def
Transitive closure	Smallest transitive relation containing R	Def
Extension	$\succsim'$ agrees with $\succsim$ on strict and weak ranks	Def
OWC	$xT(\succsim)y \Rightarrow \neg(y \succ x)$	Def
Szpilrajn's Thm	Every partial order extends to a linear order	Thm

Definition (Symmetric and Asymmetric Parts)

For any binary relation R on X, define:

- the *symmetric part*: $xIy \iff xRy$ and yRx
- the *asymmetric part*: $xPy \iff xRy$ but not yRx

Note that $R = P \cup I$.

Remark 3.15.1 (Preference-theoretic reading). In decision theory, R encodes a preference relation: xRy means "x is at least as good as y." Then I is the *indifference* relation (equally good) and P is the *strict preference* relation (strictly better). This decomposition is fundamental to utility representation theory.

Definition (Preorder and Loset)

A binary relation R on X is a *preorder* if it is reflexive and transitive.

A *loset* (linearly ordered set) is a pair (X, R) where R is a *linear order*: a complete partial order (R is reflexive, transitive, antisymmetric, and complete).

Remark 3.15.2 (Preorder vs. partial order). A preorder need not be antisymmetric: two distinct elements may satisfy xRy and yRx simultaneously (they are "indifferent" but not equal). Adding antisymmetry promotes a preorder to a partial order. Every partial order is a preorder, but not conversely.

Definition (Maximal Elements and Upper Bounds)

Let $\succsim$ be a binary relation on X.

- The set of *maximal elements* of X is
$$\text{Max}(X, \succsim) = \{x \in X \mid y \succsim x \text{ for no } y \in X \text{ with } y \neq x\}.$$
- The set of *upper bounds* of X is
$$M(X, \succsim) = \{x \in X \mid x \succsim y \text{ for all } y \in X\}.$$

Remark 3.15.3 (Maximal vs. greatest). A maximal element has nothing strictly above it; an upper bound (greatest element) is above everything. In a partial order, the greatest element is unique if it exists and is always maximal, but maximal elements need not be greatest. In a linear order, maximal and greatest coincide.

Definition (Transitive Closure)

The *transitive closure* $T(R)$ of a binary relation R on X is the smallest transitive relation containing R: that is, $T(R)$ is transitive, $xRy \Rightarrow xT(R)y$, and no strictly smaller relation is both transitive and contains R.

Remark 3.15.4 (Constructive description). Define $R^0 = R$ and xR^my if there exist $z_1, \ldots, z_m \in X$ such that $xRz_1R \cdots Rz_mRy$. Then

$$T(R) = R \cup \bigcup_{m \in \mathbb{N}} R^m.$$

Existence follows from the fact that the intersection of any collection of transitive relations is transitive, so the intersection of all transitive supersets of R is the smallest such relation.

Definition (Extension of a Preorder)

Let $\succsim$ be a preorder on X. A preorder $\succsim'$ is an *extension* of $\succsim$ if

$$x \succsim y \;\Rightarrow\; x \succsim' y, \qquad x \succ y \;\Rightarrow\; x \succ' y.$$

A *complete extension* is an extension that is also complete.

Remark 3.15.5 (What extensions do). An extension of $\succsim$ "fills in" the comparisons left undecided by $\succsim$ without reversing any existing strict comparison. The goal is to promote a partial ranking to a total ranking while respecting the original judgements.

Theorem (Szpilrajn, 1930)

For any nonempty set X and partial order $\succsim$ on X, there exists a linear order that is an extension of $\succsim$.

Remark 3.15.6 (Proof sketch via Hausdorff Maximum Principle). Let T_X be the set of all partial orders on X extending $\succsim$, ordered by inclusion. By the Hausdorff Maximum Principle (equivalent to AC), T_X has a maximal chain; its union $\succsim^*$ is a partial order extending $\succsim$. If $\succsim^*$ were not complete, one could enlarge it via transitive closure, contradicting maximality. Hence $\succsim^*$ is a linear order extending $\succsim$.

Dependence on AC. The theorem is equivalent to the Axiom of Choice for infinite sets; no proof avoiding AC is known.

Corollary 3.15.7 (Complete preorder extension). *For any nonempty set X and preorder $\succsim$ on X, there exists a complete preorder that is an extension of $\succsim$.*

Remark (Fully quantified form). $\forall X \neq \varnothing$, $\forall \succsim$ preorder on X: $\exists \succsim'$ complete preorder on X s.t. $(x \succsim y \to x \succsim' y) \wedge (x \succ y \to x \succ' y)$.

Remark 3.15.8 (Proof). Pass to the quotient $X/\sim$ (where $\sim$ is the symmetric part), apply Szpilrajn's Theorem to obtain a linear order on $X/\sim$, then pull back to a complete preorder on X.

Definition and Proposition (Only Weak Cycles)

Let $\succsim$ be a binary relation on X with asymmetric part $\succ$ and transitive closure $T(\succsim)$. We say $\succsim$ satisfies *only weak cycles* (OWC) if

$$xT(\succsim)y \;\Rightarrow\; \neg(y \succ x).$$

Proposition. A binary relation $\succsim$ on a nonempty set X can be extended to a complete preorder if and only if it satisfies OWC.

Remark 3.15.9 (Intuition for OWC). OWC prohibits the following: there is a "chain" of weak preferences $x_1 \succsim x_2 \succsim \cdots \succsim x_n$ but a strict reversal $x_n \succ x_1$. Such a configuration cannot be extended to a complete preorder because the chain forces $x_1 \succsim' x_n$ in any extension, but strict preference $x_n \succ x_1$ in the extension would mean $x_n \succsim' x_1$ but not $x_1 \succsim' x_n$ contradicting $x_1 \succsim' x_n$.

Remark 3.15.10 (Transition to AC). Szpilrajn's Theorem relies on Zorn's Lemma / Hausdorff Maximum Principle, which are equivalent to the Axiom of Choice. The next section develops these equivalences directly.

3.16 Induced Orders and Order Embeddings

Induced Orders and Order Embeddings — Quick Reference

Concept	Meaning	Detail
Induced preorder	$x \leq_f y \;\Leftrightarrow\; f(x) \leq' f(y)$; always a preorder	Def
Induced partial order	Induced order is a partial order iff f is injective	Prop
f-indistinguishable	$x \sim_f y$: both $x \leq_f y$ and $y \leq_f x$	Def
Order embedding	Injective, order-preserving, and order-reflecting	Def
Embedding vs. isomorphism	Embedding is iso onto image; isomorphism is surjective too	Prop
Suborder	Restriction of $\leq'$ to a subset $S \subseteq B$	Def
Key results:		
$\leq_f$ preorder: always. Antisymmetry: iff f injective. f order embedding $\Rightarrow (A, \leq_f) \cong (f(A), \leq')$.		

The simplest way to put an order on a set A is to compare elements of A *indirectly* through a function into an already-ordered set. This construction, called the *induced order* or *pullback order*, is ubiquitous: it explains how the usual order on $\mathbb{Z}$ restricts to $\mathbb{N}$, how functions on a common domain acquire a pointwise order, and how subsets inherit the order of the ambient space.

Definition (Induced Order)

Let $(B, \leq')$ be a partially ordered set and let $f : A \to B$ be a function. The *order induced by* f (or *pullback order along* f) is the relation $\leq_f$ on A defined by:

$$x \;\leq_f\; y \quad\Longleftrightarrow\quad f(x) \;\leq'\; f(y).$$

Remark 3.16.1 (Intuition). Two elements of A are compared by sending them through f and comparing their images in $(B, \leq')$. The order on A is entirely inherited from B via f: we never compare elements of A directly, only their f-values.

Remark 3.16.2 (Fully quantified form). $\forall x, y \in A, \quad x \leq_f y \iff f(x) \leq' f(y)$.

Proposition 3.16.3 ($\leq_f$ is always a preorder). *For any function $f : A \to B$ and partial order $(B, \leq')$, the relation $\leq_f$ is a preorder on A: it is reflexive and transitive.*

Remark (Fully quantified form). $\forall x \in A :\; f(x) \leq' f(x)$ (reflexivity); $\forall x, y, z \in A :\; f(x) \leq' f(y) \wedge f(y) \leq' f(z) \to f(x) \leq' f(z)$ (transitivity).

Remark 3.16.4 (Proof strategy). Both properties are inherited directly from $(B, \leq')$ by pulling back through f: reflexivity at x uses reflexivity at $f(x)$, and transitivity along x, y, z uses transitivity along $f(x), f(y), f(z)$.

Definition (f-Indistinguishable Elements)

Let $f : A \to B$ and $\leq_f$ be the induced order. Two elements $x, y \in A$ are f-*indistinguishable*, written $x \sim_f y$, if both $x \leq_f y$ and $y \leq_f x$, i.e. if

$$f(x) \leq' f(y) \quad \text{and} \quad f(y) \leq' f(x).$$

Remark 3.16.5 (Intuition). Two elements are f-indistinguishable when they map to the same position in the $\leq'$-order not necessarily to the same *point*, but to mutually comparable points. When $\leq'$ is a partial order (antisymmetric), $x \sim_f y$ forces $f(x) = f(y)$, making f non-injective the only obstruction to antisymmetry.

Proposition 3.16.6 ($\leq_f$ is a partial order iff f is injective). *Let $(B, \leq')$ be a partially ordered set and $f : A \to B$. The induced order $\leq_f$ is a partial order on A if and only if f is injective.*

Remark (Fully quantified form). $\leq_f$ is a partial order $\leftrightarrow \forall x, y \in A,\ f(x) \leq' f(y) \wedge f(y) \leq' f(x) \to x = y \ \leftrightarrow\ \forall x, y \in A,\ f(x) = f(y) \to x = y$ (injectivity of f, since $\leq'$ is antisymmetric).

Remark 3.16.7 (Common error). It is tempting to think the induced order is automatically a partial order "because $\leq'$ is one." It is not. The issue is antisymmetry: if f collapses two distinct points $x \neq y$ to the same or comparable images, we get $x \leq_f y$ and $y \leq_f x$ with $x \neq y$, violating antisymmetry. Reflexivity and transitivity lift freely; antisymmetry does not.

Example 3.16.8 (Non-injective f destroys antisymmetry). Let $B = (\mathbb{N}, \leq)$ and let $f : \{a, b\} \to \mathbb{N}$ be the constant function $f(a) = f(b) = 0$. Then $a \leq_f b$ (since $0 \leq 0$) and $b \leq_f a$ (since $0 \leq 0$), but $a \neq b$. So $\leq_f$ is not antisymmetric, confirming that non-injectivity forces a failure.

Example 3.16.9 (Injective f gives a genuine partial order). Let $B = (\mathcal{P}(\{1, 2, 3\}), \subseteq)$ and let $A = \{x, y, z\}$ with $f(x) = \{1\}$, $f(y) = \{2\}$, $f(z) = \{1, 2\}$. Then f is injective. The induced order has $x \leq_f z$ (since $\{1\} \subseteq \{1, 2\}$) and $y \leq_f z$ (since $\{2\} \subseteq \{1, 2\}$), but x and y are incomparable. This is a genuine partial order on A.

Definition (Suborder / Restriction)

Let $(B, \leq')$ be a partially ordered set and $S \subseteq B$. The *suborder* on S (or *induced order on* S) is the relation $\leq'_S$ defined by:

$$x \ \leq'_S \ y \iff x \leq' y, \quad \text{for } x, y \in S.$$

Remark 3.16.10 (Connection to induced orders). The suborder on $S \subseteq B$ is exactly the order induced by the inclusion map $\iota : S \hookrightarrow B$, $\iota(x) = x$. Since ι is injective, the suborder is always a partial order whenever $(B, \leq')$ is. This justifies speaking of "S with the inherited order" without further verification.

Definition (Order Embedding)

Let $(A, \leq)$ and $(B, \leq')$ be partially ordered sets. A function $f : A \to B$ is an *order embedding* if for all $x, y \in A$:

$$x \leq y \iff f(x) \leq' f(y).$$

Remark 3.16.11 (Two directions, one condition). The $(\Rightarrow)$ direction says f is order-preserving (see Definition (Order-Preserving Map)). The $(\Leftarrow)$ direction -- $f(x) \leq' f(y) \Rightarrow x \leq y$ -- is called *order-reflection*. An order embedding both preserves and reflects the order, so f is a perfect local copy of $(A, \leq)$ inside $(B, \leq')$.

Remark 3.16.12 (Relation to the induced order). If $f : A \to B$ is an order embedding from $(A, \leq)$ to $(B, \leq')$, then $\leq$ coincides with the induced order $\leq_f$:

$$x \leq y \iff f(x) \leq' f(y) \iff x \leq_f y.$$

In other words, an order embedding is exactly an injective function whose induced order *agrees with the existing order on* A.

Proposition 3.16.13 (Order embeddings are injective). *Every order embedding* $f : (A, \leq) \to (B, \leq')$ *is injective.*

Remark (Fully quantified form). $\forall x, y \in A : f(x) = f(y) \to x = y$.

Remark 3.16.14 (Proof strategy). Injectivity falls out of the reflection direction: if two elements have the same image, reflection forces them to be mutually $\leq$-related, and antisymmetry collapses them to equality.

Proposition 3.16.15 (Order embedding is isomorphism onto image). *If* $f : (A, \leq) \to (B, \leq')$ *is an order embedding, then* f *is an order isomorphism from* $(A, \leq)$ *to the suborder* $(f(A), \leq'_{f(A)})$.

Remark (Fully quantified form). $\forall x, y \in A : x \leq y \leftrightarrow f(x) \leq'_{f(A)} f(y)$, i.e. f is an order isomorphism $(A, \leq) \cong (f(A), \leq')$.

Remark 3.16.16 (Consequence). This proposition is the order-theoretic analogue of the first isomorphism theorem: an order embedding always realises $(A, \leq)$ as an isomorphic copy of a sub-poset of $(B, \leq')$. When f is also surjective, the embedding becomes a full order isomorphism (see Definition (Order Isomorphism)).

Example 3.16.17 (Standard embeddings). Each of the following is an order embedding:

(i) The inclusion $\mathbb{N} \hookrightarrow \mathbb{Z}$ with standard $\leq$ on both: $m \leq n \iff m \leq n$ trivially. The natural numbers sit inside the integers as an isomorphic copy of $(\mathbb{N}, \leq)$.

(ii) The function $f : \mathbb{N} \to \mathbb{N}$, $f(n) = 2n$, embeds $(\mathbb{N}, \leq)$ into itself: $m \leq n \iff 2m \leq 2n$. The image is the even numbers with the inherited order.

(iii) The map $A \mapsto A$ from $(\mathcal{P}(X), \subseteq)$ to itself is an order isomorphism (the identity), but any injective $g : \mathcal{P}(X) \to \mathcal{P}(X)$ that preserves and reflects $\subseteq$ is an order embedding.

Remark 3.16.18 (Transition). Induced orders and embeddings are the order-theoretic counterpart of substructures in algebra. Just as a subgroup is a subset closed under the group operations, a sub-poset is a subset with the inherited order -- and an order embedding is the morphism that witnesses this. In analysis, these ideas appear when restricting an order from $\mathbb{R}$ to subsets (intervals, sequences, function spaces), and when the completeness property is transferred between ordered structures.

3.17 Zorn's Lemma and the Axiom of Choice

AC Equivalences Quick Reference

Statement	Role
Axiom of Choice (AC)	Choice function from any collection of nonempty sets
Product form of AC	Cartesian product of nonempty sets is nonempty
Zorn's Lemma	Poset with every chain bounded $\Rightarrow$ maximal element
Hausdorff Maximum Principle	Every poset has a $\subseteq$-maximal chain
Well-Ordering Theorem	Every set admits a well-ordering

All five statements are equivalent (over ZF set theory).

Axiom of Choice

Let $\mathcal{A}$ be any collection of nonempty sets. Then there exists a function

$$f : \mathcal{A} \to \bigcup \mathcal{A}$$

such that $f(A) \in A$ for all $A \in \mathcal{A}$.

Equivalent form. The Cartesian product of an arbitrary collection of nonempty sets is nonempty.

Remark 3.17.1 (Why AC is non-constructive). For finite collections, the axiom is provable without extra assumptions (see SRF-STO-C08-S8.1). For infinite collections, no construction can produce the choice function: AC asserts its existence without exhibiting it. This non-constructive character is both AC's power and the source of the foundational controversy surrounding it.

Zorn's Lemma

Let $(P, \leq)$ be a poset in which every chain (loset) has an upper bound in P. Then P has at least one maximal element.

Remark 3.17.2 (Reading Zorn's Lemma). The hypothesis "every chain has an upper bound" does *not* require the bound to lie in the chain itself. The conclusion gives a maximal element: an element above which nothing in P sits. In a partial order, there may be many maximal elements; Zorn guarantees at least one.

Remark 3.17.3 (Typical proof pattern using Zorn's Lemma). To show an object of type X exists with a maximal property:

1. Partially order candidate objects by inclusion (or extension).
2. Verify every chain of candidates has an upper bound (usually its union).
3. Conclude by Zorn that a maximal candidate exists.
4. Show the maximal candidate has the desired property (often: if it lacked it, it could be enlarged, contradicting maximality).

This pattern recurs throughout algebra and analysis: maximal ideals, algebraic bases (Hamel bases), maximal consistent sets, and Szpilrajn's Theorem.

Hausdorff Maximum Principle

In every poset $(P, \leq)$ there exists a $\subseteq$-maximal chain; that is, a chain $C \subseteq P$ such that no chain $C' \subseteq P$ strictly contains C.

Remark 3.17.4 (Relation to Zorn). The Hausdorff Maximum Principle is an immediate corollary of Zorn's Lemma: take the poset of chains ordered by inclusion; every chain of chains is bounded by its union; Zorn gives a maximal chain. Conversel Hausdorff implies Zorn. Both are equivalent to AC over ZF.

Remark 3.17.5 (Application: Szpilrajn via Hausdorff). In the proof of Szpilrajn's Theorem (Thm), let T_X be the set of all partial orders on X extending $\succsim$, ordered by $\subseteq$. By the Hausdorff Maximum Principle, T_X has a maximal chain A; its union $\succsim^*= \bigcup A$ is a partial order extending $\succsim$. If $\succsim^*$ were not total, there would exist incomparable x, y and one could extend $\succsim^*$ by adding (x, y) (closing under transitivity), contradicting maximality of A. Hence $\succsim^*$ is a linear order. $\square$

Remark 3.17.6 (Transition). Zorn's Lemma is the workhorse for existence proofs throughout abstract algebra and analysis. It first appears in the project context here and at the proof stubs in §11 of the Chapter VIII exercises.

3.18 Cardinality and Infinite Sets

Cardinality Quick Reference

Concept	Meaning	Detail
Same cardinality	Bijection $f : A \to B$ exists; written $A \sim B$	Def
Finite set	$A \sim \{1, \ldots, n\}$ for some $n \in \mathbb{N}$, or $A = \varnothing$	Def
Countably infinite	$A \sim \mathbb{N}$	Def
Countable	Finite or countably infinite	Def
Uncountable	Infinite but not countable	Def
$\lvert A\rvert \le \lvert B\rvert$	Injection $A \hookrightarrow B$ exists	Def
$\mathbb{Q}$ countable	Diagonal enumeration	Thm
$\mathbb{R}$ uncountable	Cantor diagonal argument	Thm
Schröder–Bernstein	$\lvert A\rvert \le \lvert B\rvert$ and $\lvert B\rvert \le \lvert A\rvert \Rightarrow A \sim B$	Thm
Cantor's theorem	$\lvert A\rvert < \lvert\mathcal{P}(A)\rvert$ for every set A	Thm
Countable union	Countable union of countable sets is countable	Thm

The intuitive idea of "size" works perfectly for finite sets: $\{a, b, c\}$ has three elements, $\{1, 2, 3, 4\}$ has four, and any two finite sets can be compared by counting. For infinite sets, counting breaks down, and a different foundation is needed. The key insight, due to Cantor, is that two sets have the "same size" if and only if their elements can be matched perfectly one-to-one and onto without any element left over on either side. This single idea, made precise through the notion of a bijection, gives a theory of size that applies uniformly to both finite and infinite sets, and produces genuinely surprising results: the integers and the rationals turn out to have the same size as the natural numbers, while the real numbers are provably larger.

Definition (Same Cardinality)

Two sets A and B have the *same cardinality*, written $A \sim B$, if there exists a bijection $f : A \to B$.

Remark (Fully quantified form). $A \sim B \leftrightarrow \exists f : A \to B,\ (\forall a_1, a_2 \in A,\ f(a_1) = f(a_2) \to a_1 = a_2) \wedge (\forall b \in B,\ \exists a \in A,\ f(a) = b)$.

Remark 3.18.1 (Same cardinality is an equivalence relation). The relation $\sim$ is an equivalence relation on the class of all sets:

(i) *Reflexive*: $A \sim A$ via id_A.

(ii) *Symmetric*: if $f : A \to B$ is a bijection then $f^{-1} : B \to A$ is a bijection, so $A \sim B$ implies $B \sim A$.

(iii) *Transitive*: if $f : A \to B$ and $g : B \to C$ are bijections then $g \circ f : A \to C$ is a bijection, so $A \sim B$ and $B \sim C$ imply $A \sim C$.

This is a genuine equivalence relation, so all the theory of equivalence classes developed in the previous section applies. The equivalence class of A under $\sim$ is called the *cardinality* of A, written $|A|$.

Example 3.18.2 (Infinite sets can be equinumerous with proper subsets). Let $E = \{2, 4, 6, \dots\}$ be the set of positive even integers. The function $f : \mathbb{N} \to E$ defined by $f(n) = 2n$ is a bijection, so $\mathbb{N} \sim E$. This seems paradoxical: E is a proper subset of $\mathbb{N}$, yet the two sets have the same cardinality. This is not a contradiction but rather one of the defining characteristics of infinite sets a phenomenon that simply cannot occur for finite sets.

Remark 3.18.3 (Dedekind's characterization of infinite sets). The phenomenon in Example 3.18.2 is not an accident. An infinite set can always be matched bijectively with some proper subset of itself. This is sometimes taken as the *definition* of an infinite set (Dedekind's definition): a set is infinite if and only if it is equinumerous with a proper subset of itself.

Definition (Finite and Infinite Sets)

A set A is *finite* if $A = \varnothing$ or $A \sim \{1, 2, \dots, n\}$ for some $n \in \mathbb{N}$. A set is *infinite* if it is not finite.

Remark (Fully quantified form). A is finite $\leftrightarrow$ $A = \varnothing \vee \exists n \in \mathbb{N},\ \exists f : A \to \{1, \dots, n\}$, f bijective. A is infinite $\leftrightarrow$ $\neg(A$ is finite).

Definition (Countable and Uncountable Sets)

A set A is *countably infinite* if $A \sim \mathbb{N}$. A set is *countable* if it is finite or countably infinite. A set is *uncountable* if it is infinite but not countable.

Remark (Fully quantified form). A is countably infinite $\leftrightarrow$ $\exists f : A \to \mathbb{N}$, f bijective. A is countable $\leftrightarrow$ A finite $\vee$ $A \sim \mathbb{N}$. A is uncountable $\leftrightarrow$ A infinite $\wedge \neg(A \sim \mathbb{N})$.

Remark 3.18.4 (Countable means listable). A set A is countably infinite if and only if its elements can be arranged into an infinite list $a_1, a_2, a_3, \dots$ with no repetitions and every element of A appearing somewhere in the list. The bijection $f : \mathbb{N} \to A$ is precisely such a listing: $a_n = f(n)$. This is why countability is often informally described as "can be listed."

Definition (Comparing Cardinalities)

Write $|A| \leq |B|$ if there exists an injection $f : A \hookrightarrow B$. Write $|A| < |B|$ if $|A| \leq |B|$ but $A \not\sim B$.

Remark (Fully quantified form). $|A| \leq |B|$ $\leftrightarrow$ $\exists f : A \to B,\ \forall a_1, a_2 \in A,\ f(a_1) = f(a_2) \to a_1 = a_2$ (injection exists).

Remark 3.18.5 (Why injections, not surjections). An injection $A \hookrightarrow B$ places a "copy" of A inside B without overlap, which formalizes "A is no larger than B." The direction matters: an injection from A to B is not the same as an injection from B to A.

Theorem 3.18.6 ($\mathbb{Q}$ is countable). *The set $\mathbb{Q}$ of rational numbers is countable.*

Remark 3.18.7 (Proof strategy diagonal enumeration). The argument arranges all positive rationals into an infinite two-dimensional array: place p/q (in lowest terms, $p, q > 0$) in row p, column q. Then traverse the array along successive diagonals the anti-diagonal $p+q=k$ for $k=2,3,4,\dots$ skipping any fraction already seen. This produces an explicit list of all positive rationals, and one then interleaves the negative rationals and zero. The precise details are left as a guided proof exercise. The core idea is: a doubly infinite list can be converted into a singly infinite list by reading diagonals.

Remark 3.18.8 (What this means). $\mathbb{Q}$ is dense in $\mathbb{R}$ between any two real numbers there is a rational yet $\mathbb{Q}$ is countable. This means the rationals are in some precise sense "thin" even though they are everywhere. The uncountability of $\mathbb{R}$ that follows shows the reals are genuinely larger.

Theorem 3.18.9 (Countable union of countable sets is countable). *Let $\{A_n\}_{n\in\mathbb{N}}$ be a countable family of countable sets. Then $\bigcup_{n=1}^{\infty} A_n$ is countable.*

Remark 3.18.10 (Proof strategy). The argument again uses a diagonal enumeration. Arrange the elements of each A_n as a row in an infinite array (padding with repetitions if A_n is finite), then read the array along anti-diagonals. This produces a surjection from $\mathbb{N}$ onto $\bigcup A_n$, which is enough to establish countability. The formal proof is a standard exercise.

Remark 3.18.11 (Why this theorem matters downstream). This result is used constantly. In topology: a countable union of nowhere-dense sets is called a *meager* set, and Baire's theorem says a complete metric space is not meager the proof requires knowing that countable unions behave predictably. In measure theory: a countable union of sets of measure zero has measure zero, by countable additivity. Whenever an argument says "take the union over all $n\in\mathbb{N}$" and needs the result to remain manageable in size, this theorem is the justification.

Theorem 3.18.12 ($\mathbb{R}$ is uncountable). *The set $\mathbb{R}$ of real numbers is not countable.*

Remark 3.18.13 (Proof strategy Cantor's diagonal argument). This is one of the most important proof techniques in all of mathematics. Suppose, for contradictio that $\mathbb{R}$ were countable, so its elements could be listed: $r_1, r_2, r_3, \dots$. Write each r_n in decimal expansion. Construct a new real number x by choosing the n-th decimal digit of x to differ from the n-th decimal digit of r_n (and avoiding 9 to prevent the $0.999\cdots = 1$ ambiguity). Then $x \neq r_n$ for every n, because x and r_n differ in the n-th decimal place. So x is not on the list contradicting the assumption that the list contains every real number.

The key move is the diagonal construction: to defeat every row r_n simultaneously, make x disagree with row n at position n. This "diagonal sweep" is a technique that reappears in logic (Gödel's incompleteness theorem), computability theory (the halting problem), and the proof of Cantor's theorem below.

Remark 3.18.14 (Consequence). Since $\mathbb{Q}$ is countable and $\mathbb{R}$ is uncountable, the irrational numbers $\mathbb{R}\backslash\mathbb{Q}$ must be uncountable: if they were countable, then $\mathbb{R} = \mathbb{Q}\cup(\mathbb{R}\backslash\mathbb{Q})$ would be a countable union of countable sets, hence countable contradiction.

Theorem 3.18.15 (Cantor's Theorem). *For any set A, there is no surjection $A \to \mathcal{P}(A)$. In particular, $|A| < |\mathcal{P}(A)|$.*

Remark (Fully quantified form). $\forall A:\ \neg\exists f : A \to \mathcal{P}(A),\ \forall S \in \mathcal{P}(A),\ \exists a \in A,\ f(a) = S$. Equivalently: $\forall f : A \to \mathcal{P}(A),\ \exists D \in \mathcal{P}(A),\ \forall a \in A,\ f(a) \neq D$.

Remark 3.18.16 (Proof strategy the same diagonal idea). Suppose $f : A \to \mathcal{P}(A)$ is any function. Construct the set

$$D = \{ a \in A \mid a \notin f(a) \}.$$

D is a well-defined subset of A, so $D \in \mathcal{P}(A)$. Claim: D is not in the image of f. For suppose $D = f(a_0)$ for some $a_0 \in A$. Then:

- if $a_0 \in D$, then by definition $a_0 \notin f(a_0) = D$ contradiction.
- if $a_0 \notin D$, then by definition $a_0 \in f(a_0) = D$ contradiction.

Either case is impossible, so $D \neq f(a_0)$ for every $a_0 \in A$, and f is not surjective. Since f was arbitrary, no surjection $A \to \mathcal{P}(A)$ exists.

Notice the structure: D is defined by "$a \in D$ iff $a \notin f(a)$," which is a diagonal construction the membership of a in D is decided by looking at position (a, a) in the "membership table" of f, and then disagreeing. This is the abstract form of the same move used in the proof that $\mathbb{R}$ is uncountable.

Remark 3.18.17 (A hierarchy of infinities). Cantor's theorem implies there is no largest cardinality: for any set A, the power set $\mathcal{P}(A)$ is strictly larger. Starting from $\mathbb{N}$, the sets

$$\mathbb{N}, \quad \mathcal{P}(\mathbb{N}), \quad \mathcal{P}(\mathcal{P}(\mathbb{N})), \quad \ldots$$

form a strictly increasing sequence of cardinalities. One can show $|\mathcal{P}(\mathbb{N})| = |\mathbb{R}|$ via the characteristic function bijection (described below), so the second level is already the cardinality of the continuum.

Definition (Function Space B^A)

For sets A and B, the *function space* B^A denotes the set of all functions from A to B:

$$B^A := \{ f : A \to B \}.$$

Remark 3.18.18 (Why the exponential notation). For finite sets with $|A| = m$ and $|B| = n$, the number of functions $A \to B$ is n^m (there are n choices for each of the m inputs). The notation B^A reflects this count. In particular, $|\{0,1\}^A| = 2^{|A|}$ for finite A, which matches $|\mathcal{P}(A)| = 2^{|A|}$.

Remark 3.18.19 (The characteristic function bijection). For any set A, the power set $\mathcal{P}(A)$ and the function space $\{0,1\}^A$ have the same cardinality via the *characteristic function* bijection:

$$\mathcal{P}(A) \to \{0,1\}^A, \qquad S \mapsto \mathbf{1}_S,$$

where $\mathbf{1}_S(a) = 1$ if $a \in S$ and $\mathbf{1}_S(a) = 0$ if $a \notin S$. This bijection is why Cantor's theorem is sometimes stated as: there is no surjection $A \to \{0,1\}^A$. The notation 2^A is used interchangeably with $\{0,1\}^A$, and $|\mathcal{P}(A)| = 2^{|A|}$ is the general statement.

In functional analysis, spaces of the form B^A functions from A to B are the central objects of study. Knowing they are sets (with a definite cardinality when A and B are concrete) is the set-theoretic foundation for everything that follows.

Theorem 3.18.20 (Schröder-Bernstein Theorem). *If $|A| \leq |B|$ and $|B| \leq |A|$, then $A \sim B$.*

Equivalently: if there exist injections $f : A \hookrightarrow B$ and $g : B \hookrightarrow A$, then there exists a bijection $h : A \to B$.

Remark (Fully quantified form). $\big(\exists f : A \hookrightarrow B\big) \wedge \big(\exists g : B \hookrightarrow A\big) \ \to\ \exists h : A \to B,\ h$ bijective.

Remark 3.18.21 (Why this theorem is indispensable). In practice, proving $A \sim B$ by exhibiting an explicit bijection can be very difficult. Schröder-Bernstein gives a two-injection strategy: find any injection each way, and the theorem guarantees a bijection exists (even if you cannot describe it explicitly). This is a powerful tool for cardinality computations. For example, to show $|(0,1)| = |\mathbb{R}|$, find an injection $(0,1) \hookrightarrow \mathbb{R}$ (inclusion) and an injection $\mathbb{R} \hookrightarrow (0,1)$ (e.g., $x \mapsto \frac{1}{\pi}\arctan(x) + \frac{1}{2}$); the theorem does the rest.

Remark 3.18.22 (Proof idea). The proof constructs h by partitioning A into two pieces: those elements whose "ancestry" under repeated application of $g \circ f$ is finite (they go through f), and those whose ancestry is infinite (they go through g^{-1}). The formal argument is a standard exercise; the key insight is that the two injections together cover A in a structured enough way to stitch together a bijection.

Remark 3.18.23 (Sequences as functions). A *sequence* in a set X is a function $f : \mathbb{N} \to X$. The element $f(n)$ is written x_n, and the sequence is written $(x_n)_{n \in \mathbb{N}}$ or simply (x_n). This is not a new object: it is an element of the function space $X^{\mathbb{N}}$. When metric spaces and topology discuss convergence of sequences $x_1, x_2, x_3, \ldots$ in an arbitrary space X, the formal object is always a function $\mathbb{N} \to X$, and all the function machinery injectivity, composition, preimages applies to it directly.

Remark 3.18.24 (Transition). Cardinality distinguishes between the countable (manageable, listable) and the uncountable (genuinely larger). This distinction organizes much of what follows. In metric spaces: separable spaces have countable dense subsets, and separability is a strong structural property. In measure theory: σ-algebras are closed under *countable* unions not arbitrary ones precisely because countable operations preserve the kind of control that makes measure theory work. In functional analysis: L^2 spaces have countable orthonormal bases (Hilbert spaces are separable), while non-separable spaces behave very differently. The line between countable and uncountable is not a technicality; it is one of the organizing principles of analysis.

3.19 Systems of Sets

Systems of Sets Quick Reference

Structure	**Closed under**	**Detail**
Semiring	$\cap$; $A \setminus B = \bigsqcup_{k=1}^{n} C_k$ (finite disjoint)	Def
Ring of sets	$\cup$, $\setminus$ (hence $\cap$, $\triangle$)	Def
Algebra of sets	Finite $\cup$, $\cap$; complement	Def
σ-algebra	Countable $\cup$, $\cap$; complement	Def
Topology	Arbitrary $\cup$; finite $\cap$; $\varnothing, X$	Def
Generated σ-alg.	$\sigma(\mathcal{F})$: smallest σ-alg. containing $\mathcal{F}$	Def
Borel σ-algebra	$\mathcal{B}(X)$: generated by open sets of X	Def

Comparison of Set Systems

Property	**Semiring**	**Ring**	**Algebra**	**σ-algebra**	**Topology**
Contains $\varnothing$	✓	✓	✓	✓	✓
Contains X	—	—	✓	✓	✓
Closed under $\cap$ (finite)	✓	✓	✓	✓	✓
Closed under $\cup$ (finite)	—	✓	✓	✓	✓
Closed under $\cup$ (countable)	—	—	—	✓	—
Closed under $\cup$ (arbitrary)	—	—	—	—	✓
Closed under complement	—	—	✓	✓	—
Difference $A \setminus B$	finite disj. union	✓	✓	✓	—

The definitions and theorems of the previous sections have been about individual sets and their operations. A second level of structure emerges when one asks: which *collections* of sets are closed under given operations? This question is not abstract generality for its own sake. The three subjects that depend most heavily on this chapter topology, measure theory, and functional analysis each begin by specifying a particular type of collection on a given set X. A topology specifies which subsets are "open." A σ-algebra specifies which subsets are "measurable." In both cases, the definition is precisely a list of closure conditions: which set operations applied to members of the collection must produce another member.

Mastering these four structures here, as purely set-theoretic objects, means that when topology and measure theory are encountered, all the foundational work is already done. The task will be to understand the *geometric* or *analytic* meaning of the axioms, not to struggle with the set-theoretic mechanics.

Definition (Semiring of Sets)

A collection $\mathcal{S}$ of subsets of a set X is a *semiring* if:

(i) $\varnothing \in \mathcal{S}$;

(ii) $A, B \in \mathcal{S}$ implies $A \cap B \in \mathcal{S}$;

(iii) for any $A, B \in \mathcal{S}$, the difference $A \setminus B$ is a *finite disjoint union* of members of $\mathcal{S}$: there exist pairwise disjoint $C_1, \ldots, C_n \in \mathcal{S}$ with $A \setminus B = C_1 \sqcup \cdots \sqcup C_n$.

Example 3.19.1 (Half-open intervals form a semiring). Let $X = \mathbb{R}$ and let $\mathcal{S}$ be the collection of all half-open intervals $[a,b) = \{x \in \mathbb{R} : a \leq x < b\}$, together with $\varnothing$. Then $\mathcal{S}$ is a semiring:

- $[a,b) \cap [c,d) = [\max(a,c), \min(b,d))$, which is again a half-open interval (or $\varnothing$);
- $[a,b) \setminus [c,d)$ is either $\varnothing$, $[a,c)$, $[d,b)$, or $[a,c) \sqcup [d,b)$ in every case, a finite disjoint union of half-open intervals.

This semiring is the natural starting point for constructing Lebesgue measure: one first defines length on $\mathcal{S}$ (as $b-a$ for $[a,b)$), then extends to the generated ring and σ-algebra.

Remark 3.19.2 (Why semirings appear in measure theory). Lebesgue measure is most naturally defined first on a semiring, because semirings have enough structure to state the countable additivity condition for a *premeasure*, yet are simple enough that explicit formulas for sets like $[a,b)$ are easy to write down. The general extension theorem (Carathéodory's theorem) then extends the premeasure to the full σ-algebra. This is why Kolmogorov & Fomin introduce semirings before algebras: they are the base case of the extension hierarchy.

Definition (Ring of Sets)

A collection $\mathcal{R}$ of subsets of X is a *ring of sets* if:

(i) $\varnothing \in \mathcal{R}$;

(ii) $A, B \in \mathcal{R}$ implies $A \cup B \in \mathcal{R}$;

(iii) $A, B \in \mathcal{R}$ implies $A \setminus B \in \mathcal{R}$.

Remark 3.19.3 (Derived closure properties of a ring). A ring is closed under finitely many of each operation. Specifically, if $\mathcal{R}$ is a ring then:

- $A \cap B = A \setminus (A \setminus B) \in \mathcal{R}$ (intersection follows from set difference);
- $A \triangle B = (A \setminus B) \cup (B \setminus A) \in \mathcal{R}$ (symmetric difference follows from union and difference).

A ring is closed under all finite Boolean combinations of its elements.

Remark 3.19.4 (Why "ring"). The name comes from algebra. Under the operations of symmetric difference $\triangle$ (playing the role of addition) and intersection

$\cap$ (multiplication), a ring of sets is a commutative ring in the algebraic sense: $\triangle$ is commutative and associative with identity $\varnothing$, every element is its own inverse ($A \triangle A = \varnothing$), and $\cap$ distributes over $\triangle$.

Definition (Algebra of Sets)

A collection $\mathcal{A}$ of subsets of X is an *algebra of sets* (or *Boolean algebra of sets*, or *field of sets*) if:

(i) $X \in \mathcal{A}$;

(ii) $A \in \mathcal{A}$ implies $A^c = X \setminus A \in \mathcal{A}$;

(iii) $A, B \in \mathcal{A}$ implies $A \cup B \in \mathcal{A}$.

Remark 3.19.5 (Derived properties of an algebra). An algebra is a ring that also contains X. Since $\varnothing = X^c$ and $X \in \mathcal{A}$, every algebra contains $\varnothing$. Closure under finite intersection follows: $A \cap B = (A^c \cup B^c)^c \in \mathcal{A}$ by De Morgan. An algebra is therefore closed under all finite Boolean operations: finite unions, finite intersections, complements, and differences.

Definition (σ-Algebra)

A collection $\mathcal{M}$ of subsets of X is a *σ-algebra* (or *σ-field*) on X if:

(i) $X \in \mathcal{M}$;

(ii) $A \in \mathcal{M}$ implies $A^c \in \mathcal{M}$;

(iii) if $\{A_n\}_{n \in \mathbb{N}}$ is a countable family in $\mathcal{M}$, then $\bigcup_{n=1}^{\infty} A_n \in \mathcal{M}$.

The pair $(X, \mathcal{M})$ is called a *measurable space*.

Remark 3.19.6 (Derived closure properties of a σ-algebra). Every σ-algebra is an algebra (take the union of a finite family by padding with $\varnothing$). But a σ-algebra is additionally closed under countable intersections: by De Morgan,

$$\bigcap_{n=1}^{\infty} A_n = \left(\bigcup_{n=1}^{\infty} A_n^c \right)^c \in \mathcal{M}.$$

The σ in "σ-algebra" is notation for "countable" (from the German *Summe*), signalling the step up from finite to countable closure.

Remark 3.19.7 (Why countable and not arbitrary). A σ-algebra is closed under countable unions but not arbitrary ones. This is not a deficiency it is the right condition for measure theory. Measure is defined to be countably additive: if $\{A_n\}$ is a countable disjoint family, then $\mu\left(\bigcup A_n\right) = \sum \mu(A_n)$. Extending to uncountable unions would force every set to have either measure zero or infinite measure (a useless theory). The countability restriction is what makes the theory nontrivial.

By contrast, a topology (defined below) is closed under *arbitrary* unions but only *finite* intersections. This is a different trade-off: open sets are defined by local conditions, and any union of open sets is open because being open is a pointwise property. Countable intersections of open sets are *not* required to be open; those are a separate class called G_δ sets.

Definition (Topology)

A *topology* on a set X is a collection τ of subsets of X satisfying:

(i) $\varnothing \in \tau$ and $X \in \tau$;

(ii) if $\{U_\alpha\}_{\alpha\in I}$ is any family of sets in τ (indexed by any set I), then $\bigcup_{\alpha\in I} U_\alpha \in \tau$;

(iii) if $U_1, \dots, U_n \in \tau$ (finitely many), then $U_1 \cap \cdots \cap U_n \in \tau$.

The pair (X, τ) is a *topological space*. Members of τ are called *open sets*.

Remark 3.19.8 (Why arbitrary unions but only finite intersections). A topology formalizes the idea of "nearness" without a distance function. The condition that arbitrary unions of open sets are open reflects the following: if a point x is near some set, and that set is a union of smaller sets, then x is near one of those smaller sets. This is a purely local condition and works for any union.

Finite intersections must be open because: if x lies in the interior of U_1 and also in the interior of U_2, it lies in the interior of $U_1 \cap U_2$. This argument works for finitely many sets (take the smallest neighborhood). For a countably infinite intersection, the "smallest neighborhood" might shrink to nothing consider $\bigcap_{n=1}^{\infty}(-1/n, 1/n) = \{0\}$ in $\mathbb{R}$, which is not open. So infinite intersections of open sets are not required to be open.

Theorem 3.19.9 (Intersection of σ-algebras is a σ-algebra). *Let $\{\mathcal{M}_\alpha\}_{\alpha\in I}$ be any family of σ-algebras on X (indexed by any set I). Then*

$$\mathcal{M} := \bigcap_{\alpha\in I} \mathcal{M}_\alpha$$

is a σ-algebra on X.

Remark (Fully quantified form). $\big(\forall \alpha \in I,\ \mathcal{M}_\alpha$ is a σ-algebra on $X\big) \to \bigcap_{\alpha\in I} \mathcal{M}_\alpha$ is a σ-
Explicitly: $X \in \bigcap \mathcal{M}_\alpha$; $A \in \bigcap \mathcal{M}_\alpha \to A^c \in \bigcap \mathcal{M}_\alpha$; $\forall n,\ A_n \in \bigcap \mathcal{M}_\alpha \to \bigcup_n A_n \in \bigcap \mathcal{M}_\alpha$.

Remark 3.19.10 (Proof). Check each axiom: (i) $X \in \mathcal{M}_\alpha$ for all α, so $X \in \bigcap_\alpha \mathcal{M}_\alpha$. (ii) If $A \in \mathcal{M}$ then $A \in \mathcal{M}_\alpha$ for all α, so $A^c \in \mathcal{M}_\alpha$ for all α, so $A^c \in \mathcal{M}$. (iii) If $\{A_n\} \subseteq \mathcal{M}$ then $\{A_n\} \subseteq \mathcal{M}_\alpha$ for all α, so $\bigcup A_n \in \mathcal{M}_\alpha$ for all α, so $\bigcup A_n \in \mathcal{M}$. All three conditions are verified. The same argument works for algebras, rings, and topologies (with appropriate closure conditions).

Definition (Generated σ-Algebra)

Let $\mathcal{F}$ be any collection of subsets of X. The *σ-algebra generated by $\mathcal{F}$*, written $\sigma(\mathcal{F})$, is the smallest σ-algebra on X that contains $\mathcal{F}$:

$$\sigma(\mathcal{F}) := \bigcap \big\{ \mathcal{M} \ \big|\ \mathcal{M} \text{ is a } \sigma\text{-algebra on } X \text{ and } \mathcal{F} \subseteq \mathcal{M} \big\}.$$

The collection $\mathcal{F}$ is called a *generating family* for $\sigma(\mathcal{F})$.

Remark 3.19.11 (Why the definition is valid). The set of σ-algebras on X containing $\mathcal{F}$ is nonempty: $\mathcal{P}(X)$ is a σ-algebra containing every subset of X. Theorem 3.19.9 guarantees their intersection is again a σ-algebra. So $\sigma(\mathcal{F})$ is always well-defined. The same construction works for generated algebras, generated rings, and (with a small modification for topologies) generated topologies.

Remark 3.19.12 (How to think about generated structures). $\sigma(\mathcal{F})$ contains $\mathcal{F}$, and then contains everything that must be present in any σ-algebra that contains $\mathcal{F}$: complements of elements of $\mathcal{F}$, countable unions of elements of $\mathcal{F}$, countable unions of complements of elements of $\mathcal{F}$, and so on iterating indefinitely. The generated σ-algebra is the "closure" of $\mathcal{F}$ under all σ-algebraic operations. In practice, one specifies $\mathcal{F}$ as a convenient collection (e.g., intervals) and then works with the full σ-algebra by knowing which operations are allowed.

Definition (Borel σ-Algebra)

Let (X, τ) be a topological space. The *Borel σ-algebra* on X, written $\mathcal{B}(X)$, is the σ-algebra generated by the open sets:

$$\mathcal{B}(X) \ := \ \sigma(\tau).$$

Members of $\mathcal{B}(X)$ are called *Borel sets.*

Example 3.19.13 (F_σ and G_δ sets). In a topological space X:

- A G_δ *set* is a countable intersection of open sets: $A = \bigcap_{n=1}^{\infty} U_n$ with each U_n open.
- An F_σ *set* is a countable union of closed sets: $A = \bigcup_{n=1}^{\infty} F_n$ with each F_n closed.

These names come from German: G for *Gebiet* (open set), δ for *Durchschnitt* (intersection); F for *Ferme* (closed), σ for *Summe* (union). Both G_δ and F_σ sets are Borel sets. They are dual: A is G_δ if and only if A^c is F_σ.

These classes appear in analysis and measure theory as the natural "next level" above open and closed sets. In $\mathbb{R}$: the set of continuity points of a monotone function is a G_δ set; the rational numbers $\mathbb{Q}$ are an F_σ set; the irrational numbers $\mathbb{R} \setminus \mathbb{Q}$ are a G_δ set.

Remark 3.19.14 (Transition). The four structures semiring, ring, algebra, σ-algebra and the topology form a hierarchy of increasing closure strength. Measure theory builds from the bottom (semiring, ring) upward; topology is its own branch with a different closure pattern. The Borel σ-algebra bridges the two: it is generated by the topology, so it captures all sets describable by topological operations, and it is the standard "default" σ-algebra in any analytic context. Whenever a later chapter says "let f be a measurable function" without specifying the σ-algebra, it almost certainly means measurability with respect to the Borel σ-algebra.

Proofs

Principle of Set Duality

Remark (Return). ← Back to Corollary (Principle of Set Duality) in Notes

Proof. **Claim.** Any set-theoretic identity involving $\cup$, $\cap$, $\varnothing$, and U remains valid when each operation and constant is replaced by its dual: $\cup \leftrightarrow \cap$ and $\varnothing \leftrightarrow U$.
Given. De Morgan's laws (Theorem (De Morgan's Laws)): $(A \cup B)^c = A^c \cap B^c$ and $(A \cap B)^c = A^c \cup B^c$. The involution law: $(A^c)^c = A$ for all $A \subseteq U$. The boundary cases: $\varnothing^c = U$ and $U^c = \varnothing$.
Goal. To show that if an identity $E = F$ holds for all subsets of U, then its dual identity $E^* = F^*$ also holds for all subsets of U, where the dual is obtained by swapping $\cup \leftrightarrow \cap$ and $\varnothing \leftrightarrow U$.
Strategy. We show that if we take complements of both sides of $E = F$ and apply De Morgan, we obtain $E^* = F^*$.

Given the identity $E = F$, take complements of both sides: $E^c = F^c$.
Now apply De Morgan's laws to every occurrence of $\cup$ and $\cap$ in E^c and F^c. The key substitutions are:

- $(A \cup B)^c = A^c \cap B^c$ complement turns $\cup$ into $\cap$.
- $(A \cap B)^c = A^c \cup B^c$ complement turns $\cap$ into $\cup$.
- $\varnothing^c = U$ and $U^c = \varnothing$ the boundary constants swap.
- $(A^c)^c = A$ double complements cancel, restoring the original variables.

After applying these substitutions throughout E^c and F^c, each variable A appears without complement (the two complementations cancel), and every $\cup$ has become $\cap$ and vice versa. The constants $\varnothing$ and U have swapped. The result is precisely $E^* = F^*$.
Since the identity $E = F$ holds for all subsets of U, so does $E^c = F^c$, and therefore $E^* = F^*$ holds for all subsets of U. **as required.** □

Remark (Proof shape). The argument is a *structural transformation*: complement both sides of the known identity, then use De Morgan to rewrite the complements of compound expressions. The double-complement law cleans up the variables, and the boundary swaps handle the constants. The result is the dual identity.

Remark (This is a metatheorem). The Principle of Set Duality is a statement about *all* set-theoretic identities, not about any particular one. It says we never need to prove both of a dual pair: one proof suffices and the other is free. For example, proving $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ immediately gives $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ by duality.

Remark (Dependencies). The proof depends on: De Morgan's Laws, the involution of complement ($(A^c)^c = A$), and the boundary facts $\varnothing^c = U$ and $U^c = \varnothing$.

$\leq_f$ is Always a Preorder

Remark (Return). ← Back to Proposition ($\leq_f$ is always a preorder) in Notes

Proof. **Claim.** For any function $f : A \to B$ and partial order $(B, \leq')$, the induced relation $\leq_f$ defined by $x \leq_f y \Leftrightarrow f(x) \leq' f(y)$ is a preorder on A.
Given. The induced order (Definition (Induced Order)): $x \leq_f y \Leftrightarrow f(x) \leq' f(y)$. The partial order $(B, \leq')$: $\leq'$ is reflexive, transitive, and antisymmetric.
Goal. To verify reflexivity and transitivity of $\leq_f$ on A.

Reflexivity. Let $x \in A$. Since $\leq'$ is reflexive, $f(x) \leq' f(x)$. By definition of $\leq_f$, $x \leq_f x$.

Transitivity. Suppose $x \leq_f y$ and $y \leq_f z$. By definition, $f(x) \leq' f(y)$ and $f(y) \leq' f(z)$. Since $\leq'$ is transitive, $f(x) \leq' f(z)$. By definition, $x \leq_f z$.
as required. □

Remark (Proof shape). Both properties are one-line pullbacks. Reflexivity at $x \in A$ pulls back reflexivity at $f(x) \in B$; transitivity along (x, y, z) in A pulls back transitivity along $(f(x), f(y), f(z))$ in B. The structure of the proof mirrors the structure of the definition: $\leq_f$ is defined by comparing f-images, so properties of $\leq'$ transfer immediately to $\leq_f$.

Remark (Antisymmetry does not lift freely). Antisymmetry of $\leq'$ does not automatically give antisymmetry of $\leq_f$. If f is not injective, there may exist $x \neq y$ in A with $f(x) = f(y)$, giving $x \leq_f y$ and $y \leq_f x$ with $x \neq y$. This is why $\leq_f$ is only guaranteed to be a preorder, not a partial order. See $\leq_f$ is a partial order iff f is injective.

Remark (Dependencies). The proof depends on: the Definition of induced order (Definition (Induced Order)) and reflexivity and transitivity of $(B, \leq')$.

$\leq_f$ is a Partial Order iff f is Injective

Remark (Return). ← Back to Proposition ($\leq_f$ is a partial order iff f is injective) in Notes

Proof. **Claim.** Let $(B, \leq')$ be a partially ordered set and $f : A \to B$. The induced order $\leq_f$ is a partial order on A if and only if f is injective.
Given. $\leq_f$ is always a preorder ($\leq_f$ is always a preorder). The induced order definition: $x \leq_f y \Leftrightarrow f(x) \leq' f(y)$. Antisymmetry of $\leq'$: $f(x) \leq' f(y)$ and $f(y) \leq' f(x)$ imply $f(x) = f(y)$.
Goal. To show $\leq_f$ is antisymmetric if and only if f is injective. (Since $\leq_f$ is already a preorder, the only missing property for a partial order is antisymmetry.)

(⇒) If $\leq_f$ is antisymmetric, then f is injective.
Suppose $f(x) = f(y)$. Then $f(x) \leq' f(y)$ (by reflexivity of $\leq'$) and $f(y) \leq' f(x)$. By definition, $x \leq_f y$ and $y \leq_f x$. By antisymmetry of $\leq_f$, $x = y$. So $f(x) = f(y) \Rightarrow x = y$, i.e. f is injective.

(⇐) If f is injective, then $\leq_f$ is antisymmetric.
Suppose $x \leq_f y$ and $y \leq_f x$. By definition, $f(x) \leq' f(y)$ and $f(y) \leq' f(x)$. By antisymmetry of $\leq'$, $f(x) = f(y)$. By injectivity of f, $x = y$.

Therefore $\leq_f$ is antisymmetric, and since it is also a preorder, it is a partial order. **as required.** $\square$

Remark (Proof shape). This is a clean biconditional proof. Each direction is one short chain: $(\Rightarrow)$ shows that antisymmetry of $\leq_f$ forces injectivity by constructing the right test case ($f(x) = f(y)$); $(\Leftarrow)$ shows injectivity gives antisymmetry by tracing the chain $x \leq_f y \leq_f x \Rightarrow f(x) = f(y) \Rightarrow x = y$.

Remark (The exact obstruction). The proposition identifies injectivity as the *exact* condition for antisymmetry to lift. Any non-injective f (with $f(x) = f(y)$ for some $x \neq y$) produces a violation: $x \leq_f y$ and $y \leq_f x$ but $x \neq y$. Injectivity rules out all such pairs simultaneously.

Remark (Dependencies). The proof depends on: $\leq_f$ is always a preorder, the Definition of induced order (Definition (Induced Order)), antisymmetry of $(B, \leq')$, and the definition of injectivity.

Complete Preorder Extension

Remark (Return). ← Back to Corollary (Complete preorder extension) in Notes

Proof. **Claim.** For any nonempty set X and preorder $\succsim$ on X, there exists a complete preorder that is an extension of $\succsim$.
Given. Szpilrajn's Theorem (Theorem (Szpilrajn, 1930)): every partial order on a nonempty set extends to a linear order. The definition of the symmetric part of $\succsim$: $x \sim y$ iff $x \succsim y$ and $y \succsim x$. The quotient construction: $\sim$ is an equivalence relation, and the strict part $\succ$ of $\succsim$ descends to a well-defined strict partial order on the quotient $X/{\sim}$.
Goal. To construct a complete preorder on X extending $\succsim$.
Strategy. Pass to the quotient, apply Szpilrajn's Theorem, then pull back.

Step 1: The quotient. Define $x \sim y$ iff $x \succsim y$ and $y \succsim x$ (indifference). Since $\succsim$ is a preorder, $\sim$ is an equivalence relation. Let $[x]$ denote the equivalence class of x and $X/{\sim}$ the quotient set.

Step 2: The induced order on the quotient. Define $[x] \leq^* [y]$ iff $x \succsim y$ on $X/{\sim}$. This is well-defined (the relation $\succsim$ is constant on equivalence classes) and antisymmetric (if $[x] \leq^* [y]$ and $[y] \leq^* [x]$, then $x \succsim y$ and $y \succsim x$, so $x \sim y$, i.e. $[x] = [y]$). Since $\succsim$ is also reflexive and transitive, $\leq^*$ is a partial order on $X/{\sim}$.

Step 3: Apply Szpilrajn. By Szpilrajn's Theorem, there exists a linear order $\leq^{**}$ on $X/{\sim}$ extending $\leq^*$.

Step 4: Pull back to X. Define $x \succsim' y$ iff $[x] \leq^{**} [y]$. We verify:
Complete preorder: $\succsim'$ is reflexive (since $[x] \leq^{**} [x]$), transitive (since $\leq^{**}$ is), and complete (since $\leq^{**}$ is a linear order, any two classes are comparable, hence any two elements of X are comparable under $\succsim'$).
Extension: If $x \succsim y$, then $[x] \leq^* [y]$, so $[x] \leq^{**} [y]$ (since $\leq^{**}$ extends $\leq^*$), so $x \succsim' y$. If $x \succ y$ (i.e. $x \succsim y$ but not $y \succsim x$), then $[x] <^* [y]$ strictly, so $[x] <^{**} [y]$ strictly (since $\leq^{**}$ extends the strict part), so $x \succ' y$.
Hence $\succsim'$ is a complete preorder extending $\succsim$. **as required.** $\square$

Remark (Proof shape). The proof is a *quotient-then-extend-then-pull-back* construction. The difficulty with applying Szpilrajn directly to a preorder is that preorders are not antisymmetric (indifferent elements are not forced to be equal). The quotient removes this obstruction, turning the preorder into a genuine partial order. Szpilrajn then provides a linear extension. Pulling back along the quotient map reintroduces the indifference structure as a complete preorder.

Remark (Dependence on the Axiom of Choice). Szpilrajn's Theorem relies on Zorn's Lemma (or equivalently, the Axiom of Choice). This corollary inherits that dependence. For finite sets, the Axiom of Choice is not needed: one can construct a linear extension algorithmically.

Remark (Dependencies). The proof depends on: Szpilrajn's Theorem, the definition of quotient by an equivalence relation, the definition of a complete preorder (Definition (Preorder and Loset)), and the definition of an extension of a preorder (Definition (Extension of a Preorder)).

Capstone

3.20 Capstone Assessment: Sets, Relations, and Functions

Purpose. This capstone assesses mastery of elementary set theory, relations, and functions as used in analysis and abstract mathematics. All proofs must be written using precise definitions and logical reasoning.

Instructions. Each problem requires a complete proof. You must explicitly invoke definitions (e.g. function, equivalence relation) when they are used. No appeal to diagrams or intuition is permitted.

Problem 1 Equivalence Relations

Let $\sim$ be a relation on a set A. Prove that $\sim$ is an equivalence relation if and only if its equivalence classes form a partition of A.

Your proof must establish both directions.

Problem 2 Images and Preimages

Let $f : A \to B$ be a function and let $S \subseteq A$. Prove that

$$S \subseteq f^{-1}(f(S)).$$

Give an example where equality does not hold.

Problem 3 Injectivity and Left Inverses

Prove that a function $f : A \to B$ is injective if and only if there exists a function $g : B \to A$ such that

$$g \circ f = \mathrm{id}_A.$$

Your proof must clearly indicate where injectivity is used.

Problem 4 Composition of Relations

Let $R \subseteq A \times B$ and $S \subseteq B \times C$ be relations. Prove that if both R and S are transitive relations (on their respective domains), then their composition need not be transitive.

Your proof must include a concrete counterexample.

Problem 5 Order Relations

Let $(A, \leq)$ be a partially ordered set. Prove that $\leq$ is antisymmetric if and only if

$$(x \leq y \ \wedge \ y \leq x) \ \to \ x = y$$

holds for all $x, y \in A$.

Your proof must explicitly use the definition of antisymmetry.

Completion Criterion. You have mastered sets, relations, and functions if all five proofs:

- correctly invoke definitions,
- handle element-wise reasoning rigorously,
- distinguish relations from functions,
- and avoid implicit assumptions.

Successful completion certifies readiness to proceed to foundations of the real line and completeness.

Chapter 4

Axiom Systems

Chapter Status

This chapter is in governance rebuild status. The previous active chapter has been archived, and the active files are placeholders for a clean rebuild.

Rebuild Roadmap

The planned order is governed by the chapter notes, proofs, and exercises routers. Active mathematical content has not yet been restored here.

Axiom Systems Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

4.0.1 Signatures

Signatures Toolkit — Quick Reference

Concept	Role	Detail
Signature	non-logical vocabulary	Definition
Language	signature together with logical syntax	Definition
Constant symbol	names a distinguished object syntactically	Definition
Function symbol	names an operation syntactically	Definition
Relation symbol	names a relation syntactically	Definition
Arity	records number of input places	Definition

Signature Presentation: A Simple Arithmetic Language

Let

$$\Sigma_{\mathrm{arith}} = \left(\mathcal{C}_{\mathrm{arith}}, \mathcal{F}_{\mathrm{arith}}, \mathcal{R}_{\mathrm{arith}}, \mathrm{ar}\right)$$

where

$$\mathcal{C}_{\text{arith}} = \{0, 1\}, \qquad \mathcal{F}_{\text{arith}} = \{+, \cdot, S\}, \qquad \mathcal{R}_{\text{arith}} = \{<\}.$$

The arity function is specified by

$$\text{ar}(0) = \text{ar}(1) = 0, \qquad \text{ar}(S) = 1, \qquad \text{ar}(+) = \text{ar}(\cdot) = 2, \qquad \text{ar}(<) = 2.$$

Symbol	Kind	Arity	Intended Reading
$0, 1$	constants	0	distinguished constant names
S	function symbol	1	successor operation symbol
$+, \cdot$	function symbols	2	binary operation symbols
$<$	relation symbol	2	binary order relation symbol

The signature records only the available non-logical symbols and their arities. It does not assert any axioms about those symbols.

Definition 4.0.1 (Signature). A ***signature*** is a collection of non-logical symbols divided into constant symbols, function symbols, and relation symbols, together with an arity assignment for each symbol. Constant symbols are recorded as arity-0 symbols.

Remark (Interpretation). A signature is the formal vocabulary available before any axioms are chosen. It says which symbols may occur and how many input places each symbol has.

Definition 4.0.2 (Arity). The ***arity*** of a symbol is the number of input places assigned to that symbol by a signature. Constants have arity 0; unary symbols have arity 1; binary symbols have arity 2.

Remark (Interpretation). Arity is bookkeeping for grammatical well-formedness. It determines whether a symbol has been supplied with the correct number of arguments.

Remark (Dependencies).

- Signature

Definition 4.0.3 (Language). A ***language*** determined by a signature Σ consists of the non-logical symbols of Σ together with the logical symbols, variables, punctuation, and formation rules used to build terms and formulas.

Remark (Interpretation). The signature supplies the subject-specific vocabulary. The language adds the logical grammar needed to form expressions from that vocabulary.

Remark (Dependencies).

- Signature

Definition 4.0.4 (Constant Symbol). A *constant symbol* is a non-logical symbol of arity 0 in a signature. It may occur as a term without taking any input terms.

Remark (Interpretation). A constant symbol is a name-forming symbol. In the arithmetic signature above, 0 and 1 are constant symbols.

Remark (Dependencies).

- Signature
- Arity

Definition 4.0.5 (Function Symbol). A *function symbol* is a non-logical symbol in a signature assigned an arity $n \geq 1$. If f has arity n and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term.

Remark (Interpretation). A function symbol is an operation-forming symbol. In the arithmetic signature above, S is unary, while $+$ and $\cdot$ are binary.

Remark (Dependencies).

- Signature
- Arity

Definition 4.0.6 (Relation Symbol). A *relation symbol* is a non-logical symbol in a signature assigned an arity $n \geq 1$. If R has arity n and $t_1, \ldots, t_n$ are terms, then $R(t_1, \ldots, t_n)$ is an atomic formula.

Remark (Interpretation). A relation symbol is a statement-forming symbol once it is applied to the right number of terms. In the arithmetic signature above, $<$ is a binary relation symbol.

Remark (Dependencies).

- Signature
- Arity

Remark (Structural remarks). The same signature may support different choices of axioms later in the chapter. For example, a single arithmetic vocabulary can be used with stronger or weaker axiom sets.

The same signature may also be read over different mathematical universes once interpretations are introduced. At this stage, the signature itself records only the common vocabulary; it does not choose any particular interpretation.

4.0.2 Axioms

Axioms Toolkit — Quick Reference

Concept	Role	Detail
Axiom	formal assertion adopted without proof	Definition
Axiom system	collection of assertions in one language	Definition
Assumption	temporary assertion for reasoning	Definition
Consistency	absence of contradiction	forward reference
Theory	deductive closure of axioms	forward reference

Axiom Presentation: A Simple Arithmetic Axiom System

Reuse the arithmetic signature

$$\Sigma_{\mathrm{arith}} = \big(\{0,1\}, \{+, \cdot, S\}, \{<\}, \mathrm{ar}\big)$$

from the Signatures section. The signature supplies the symbols. An axiom system supplies assertions formed with those symbols.
For example, the following are familiar arithmetic-style axioms:

$$\forall x\ (x + 0 = x),$$

$$\forall x\ (S(x) \neq 0),$$
$$\forall x \forall y\ (S(x) = S(y) \to x = y).$$

These examples are illustrative. The signature says that 0, $+$, and S are available symbols. The axioms assert formal relationships involving those symbols.

Remark (Structural remarks). **Vocabulary vs assertion.** A signature is vocabulary: it specifies what may be said. An axiom is an assertion: it specifies what is adopted as true within that vocabulary. The signature supplies symbols; the axioms supply assertions involving those symbols.

Same signature, different axiom systems. A single signature may support multiple collections of axioms. For example, geometric languages can be used for Euclidean or non-Euclidean axiom systems. The usual group signature can be used for group theory, while adding the commutativity axiom gives the stronger axiom system for abelian groups. The vocabulary can remain fixed while the assertions change.

Forward conceptual roadmap. The chapter proceeds from collections of adopted assertions to their deductive and semantic roles:

$$\begin{array}{c}\texttt{Axiom System}\\ \Downarrow\\ \texttt{Theory}\\ \Downarrow\\ \texttt{Models}\end{array}$$

The present section introduces the assertion layer. Later sections formalize the theory generated by axioms and the models in which those assertions hold.

Definition 4.0.7 (Axiom). An *axiom* is a formal statement adopted without proof within a specified language.

Remark (Interpretation). An axiom is not part of the vocabulary itself. It is an assertion made using the vocabulary of a language. The signature determines which symbols may appear; the axiom states something with those symbols.

Remark (Dependencies).

- Language

Definition 4.0.8 (Axiom System). An ***axiom system*** is a collection of axioms expressed in a common language.

Remark (Interpretation). An axiom system records the assertions chosen for a mathematical setting. Two axiom systems may use the same language while adopting different assertions. The shared language fixes the vocabulary; the axiom system fixes the adopted formal claims.

Remark (Dependencies).

- Axiom
- Language

Definition 4.0.9 (Assumption). An ***assumption*** is a statement temporarily accepted for purposes of reasoning.

Remark (Interpretation). An axiom is adopted as part of the formal background of a system. An assumption is local to an argument: it may be introduced to prove a conditional, to reason within a case, or to derive a contradiction. Axioms belong to the system; assumptions belong to the proof context.

Remark (Dependencies).

- Axiom

4.0.3 Theories

Theories Toolkit — Quick Reference

Concept	**Role**	**Detail**
Theory	all consequences generated by axioms	Definition
Theorem	statement derivable from axioms	Definition
Consequence	statement following from a theory	Definition
Derivation	chain of justified steps	forward reference
Deductive closure	all derivable statements	Definition

Theory Presentation: Arithmetic Consequences

Start with the arithmetic-style axiom system from the previous section. It uses the arithmetic signature and includes assertions such as

$$\forall x\ (x + 0 = x),$$

$$\forall x\ (S(x) \neq 0),$$

$$\forall x \forall y\ (S(x) = S(y) \rightarrow x = y).$$

Those axioms are the starting assertions. From them, together with other arithmetic axioms and ordinary logical reasoning, additional statements may be established. For instance, one may later prove new equations, cancellation facts, uniqueness facts, and order facts that were not listed as axioms.

A theory contains the original axioms. A theory also contains the statements derivable from those axioms. Thus the axioms are the generators, while the theory is the full collection of what those generators imply.

Remark (Structural remarks). **Axioms vs theory.** Axioms are starting assertions. A theory includes all logical consequences of those assertions. The distinction is the next step after the vocabulary and assertion distinction: a signature supplies vocabulary, axioms supply adopted assertions, and a theory contains everything derivable from those assertions.

Finite axioms, infinite theory. A small axiom system can generate an enormous collection of consequences. The listed axioms may be finite, but the statements derivable from them need not be. This is why a theory is usually larger than the axiom system that presents it.

Forward roadmap. The next conceptual step is semantic:

$$\begin{array}{c}\text{Theory}\\ \Downarrow\\ \text{Models}\end{array}$$

A model will later be introduced as a mathematical structure in which the statements of a theory hold.

Definition 4.0.10 (Theory). A *theory* is a collection of statements in a language that contains the axioms and all statements derivable from them.

Remark (Interpretation). A theory is not merely the list of axioms. It is the closure of those axioms under derivability: every statement that follows from the axioms belongs to the theory. The axioms start the system; the theory records everything the system proves.

Remark (Dependencies).

- Language
- Axiom System

Definition 4.0.11 (Theorem). A *theorem* is a statement derivable from the axioms of a theory.

Remark (Interpretation). Within an axiom system, a theorem is not an additional starting assumption. It is a statement reached from the adopted axioms by reasoning. Axioms are given; theorems are obtained.

Remark (Dependencies).

- Theory

Definition 4.0.12 (Consequence). A statement is a *consequence* of a theory if it follows from the statements of that theory.

Remark (Interpretation). Consequence is the relation that connects a theory to what it implies. If the axioms generate a theory, then consequences are the statements carried along by those axioms and by the statements already obtained from them.

Remark (Dependencies).

- Theory

Definition 4.0.13 (Deductive Closure). The *deductive closure* of an axiom system is the collection of all statements derivable from that axiom system.

Remark (Interpretation). Deductive closure is the operation that turns an axiom system into the full theory it generates. Starting with the axioms, close under derivability; the result is the theory determined by those starting assertions.

Remark (Dependencies).

- Axiom System

4.0.4 Models

Models Toolkit — Quick Reference

Concept	Role	Detail
Interpretation	assigns meanings to symbols	Definition
Structure	domain with interpreted symbols	Definition
Model	structure satisfying the theory	Definition
Satisfaction	truth of a statement in a structure	Definition
Countermodel	model where a statement fails	Definition

Model Presentation: Arithmetic as a Model

Reuse the arithmetic signature and arithmetic-style axioms from the previous sections. A model begins by interpreting the symbols of that signature in an actual mathematical structure.

$$\begin{aligned} \text{Domain} &= \text{the natural numbers}, \\ 0 &\mapsto \text{the number zero}, \\ S &\mapsto \text{the successor operation}, \\ + &\mapsto \text{addition}. \end{aligned}$$

In this structure, the statement

$$\forall x\ (x + 0 = x)$$

holds because adding zero to any natural number gives back that same natural number.

The formal symbol 0 is only a symbol until it is interpreted as the number zero. The formal symbol + is only a symbol until it is interpreted as addition. The symbols acquire meaning only after interpretation.

Remark (Structural remarks). **Syntax vs meaning.** Signatures, axioms, and theories are formal objects. They belong to syntax: they specify vocabulary, adopted assertions, and consequences. Models provide semantic meaning by assigning mathematical content to the symbols and by determining which statements hold.

One theory, many models. The same theory may have multiple models. Euclidean geometry may be realized in different coordinate systems. The group axioms have many models, including different concrete groups. The theory fixes the assertions; a model is one mathematical realization in which those assertions hold.

Countermodels. A statement fails to follow from a theory if there is a model of the theory in which that statement is false. Countermodels are therefore evidence that a claim is not forced by the axioms.

Forward roadmap. The next sections use models to explain consistency and independence:

$$\begin{array}{c} \texttt{Models} \\ \Downarrow \\ \texttt{Consistency} \\ \Downarrow \\ \texttt{Independence} \end{array}$$

Definition 4.0.14 (Interpretation). An *interpretation* is an assignment of meanings to the non-logical symbols of a signature.

Remark (Interpretation). An interpretation turns formal vocabulary into mathematical content. It tells what each constant, function symbol, and relation symbol means in a particular setting.

Remark (Dependencies).

- Signature

Definition 4.0.15 (Structure). A *structure* is a domain together with interpretations of the symbols in a signature.

Remark (Interpretation). A structure is the environment in which formal symbols receive meaning. The domain supplies the objects, and the interpretation explains how the symbols refer to objects, operations, and relations on that domain.

Remark (Dependencies).

- Signature
- Interpretation

Definition 4.0.16 (Model). A *model* is a structure in which all axioms, equivalently all statements of the theory, are satisfied.

Remark (Interpretation). A model is a structure that makes the theory true. It is not just an interpretation of the vocabulary; it is an interpretation in which the adopted assertions and their consequences hold.

Remark (Dependencies).

- Structure
- Theory

Definition 4.0.17 (Satisfaction). A statement is *satisfied* in a structure if it evaluates as true under the given interpretation.

Remark (Interpretation). Satisfaction is the semantic test for a statement. Once the symbols have been interpreted in a structure, the statement can be checked as true or false in that setting.

Remark (Dependencies).

- Structure

Definition 4.0.18 (Countermodel). A *countermodel* is a model of a theory in which a particular statement fails.

Remark (Interpretation). A countermodel shows that the theory does not force the statement. It keeps the axioms true while making the proposed statement false.

Remark (Dependencies).

- Model
- Satisfaction

4.0.5 Consistency

Consistency Toolkit — Quick Reference

Concept	Role	Detail
Contradiction	assertion that cannot be true in an interpretation	Definition
Consistency	absence of contradiction	Definition
Inconsistency	contradiction in or from axioms	Definition
Model existence	evidence that axioms fit together	Definition
Relative consistency	consistency transferred from another theory	Definition

Consistency Presentation: A Simple Example

Consider two informal axiom systems.

Example A.

- All objects are red.
- Some object is not red.

These assertions cannot simultaneously hold. The first assertion says every object has the property; the second says at least one object lacks it.

Example B.

- All objects are red.
- Object a is red.

These assertions can hold simultaneously. The second assertion agrees with what the first assertion already requires.

These examples are only for intuition. A consistent axiom system is one whose assertions fit together without contradiction. An inconsistent axiom system is one whose assertions cannot be held together coherently.

Remark (Structural remarks). **Why contradictions matter.** Contradictory systems are unusable because their assertions do not fit together. If a system both requires and forbids the same situation, then it no longer gives a coherent basis for reasoning.

Models as evidence. If a model exists, the axioms hold together coherently in at least one interpretation. A model is therefore evidence that the axioms are mutually compatible.

Absolute vs relative consistency. Most consistency results in mathematics are relative consistency results. They show that one theory is consistent provided some other background theory is consistent. This section records the idea only; the technical development belongs elsewhere.

Forward roadmap. The next section uses models and consistency to introduce independence:

$$\begin{array}{c}\texttt{Consistency}\\ \Downarrow \\ \texttt{Independence}\end{array}$$

Definition 4.0.19 (Contradiction)**.** A *contradiction* is a statement that cannot be true under a given interpretation.

Remark (Interpretation)*.* A contradiction marks a breakdown in compatibility. It identifies an assertion that cannot be made true in the interpretation under consideration.

Remark (Dependencies)*.*

- Satisfaction

Definition 4.0.20 (Consistent Axiom System)**.** A *consistent axiom system* is an axiom system that does not lead to contradiction.

Remark (Interpretation)*.* Consistency says that the adopted assertions fit together. A consistent axiom system avoids forcing an impossible situation.

Remark (Dependencies)*.*

- Axiom System
- Contradiction

Definition 4.0.21 (Inconsistent Axiom System)**.** An *inconsistent axiom system* is an axiom system containing or producing contradictory assertions.

Remark (Interpretation)*.* Inconsistency means that the adopted assertions cannot all be maintained coherently. The system asks for incompatible requirements at once.

Remark (Dependencies)*.*

- Axiom System
- Contradiction

Definition 4.0.22 (Model Existence Criterion (Informal))**.** Informally, if an axiom system has a model, then the axioms are mutually compatible.

Remark (Interpretation)*.* This is an informal criterion in this section, not a formal theorem. The idea is that a model provides a setting in which all the axioms hold together, so the axioms have at least one coherent realization.

Remark (Dependencies)*.*

- Model

Definition 4.0.23 (Relative Consistency)**.** A *relative consistency* statement says that one theory is consistent provided another theory is consistent.

Remark (Interpretation)*.* Relative consistency does not establish consistency from nothing. It compares systems: if the background theory avoids contradiction, then the target theory does as well.

Remark (Dependencies)*.*

- Consistent Axiom System

Independence Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

Comparing Theories Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

Examples And Previews Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

Summary Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

Proofs

Proofs

Proof entries for this chapter are planned. No proof files have been regenerated.

Signatures Proofs - Planned

Proofs for this section are planned.
No proof files have been regenerated.

Axioms Proofs - Planned

Proofs for this section are planned.
No proof files have been regenerated.

Theories Proofs - Planned

Proofs for this section are planned.
No proof files have been regenerated.

Models Proofs - Planned

Proofs for this section are planned.
No proof files have been regenerated.

Consistency Proofs - Planned

Proofs for this section are planned.
No proof files have been regenerated.

Independence Proofs - Planned

Proofs for this section are planned.
No proof files have been regenerated.

Comparing Theories Proofs - Planned

Proofs for this section are planned.
No proof files have been regenerated.

Examples And Previews Proofs - Planned

Proofs for this section are planned.
No proof files have been regenerated.

Summary Proofs - Planned

Proofs for this section are planned.

No proof files have been regenerated.

Exercises

Exercises

Exercises for this chapter are planned. No exercise content has been restored here yet.

Chapter 5

Proof Techniques

Where You Are in the Journey

Propositional Logic → Predicate Calculus → Sets & Functions → **Proof Techniques** → Real Analysis → Algebraic Structures → Linear Algebra → ⋯

How we got here. You have spent the first part of this curriculum learning to reason precisely. Propositional logic gave you the algebra of truth and the laws of valid inference. Predicate logic extended that to statements about objects: what it means to say "for all x" or "there exists y." Sets and functions gave you the universe of objects to reason about. Every tool you will ever use in a mathematical proof is ultimately grounded in those foundations.

What this chapter builds. This chapter answers a different question: *how do you actually write a proof?* Knowing the laws of logic is not the same as knowing how to sit down in front of a blank page, read a mathematical statement, and construct a valid argument. This chapter makes that process explicit and systematic. You will learn to read a statement and immediately identify its proof strategy; to execute that strategy step by step without losing track of what you have established; to use the three fundamental proof structures in the situations where each is appropriate; to deploy induction correctly and confidently; and to recognize the handful of algebraic tactics that appear in virtually every proof in abstract mathematics.

Where this leads. Every proof you write from this point forward uses the machinery developed here. In real analysis, convergence and continuity are proved using direct proof and contradiction. In abstract algebra, uniqueness results use the assume-two-and-compare pattern and equalities use satisfy-and-cite. Induction threads through number theory, algebra, and combinatorics. The patterns are not numerous, but they are universal: master them here and nothing in the later curriculum will feel structurally unfamiliar.

Structural Roadmap

This chapter is organized in six sections of increasing specificity.

Architecture $\longrightarrow$ **Construction** $\longrightarrow$ **Structures** $\longrightarrow$ **Induction** $\longrightarrow$ **Tactics** $\longrightarrow$

Reference $\longrightarrow$ Analysis Preview

Sections 1–4 should be read in order on your first pass. They form a complete, coherent account of how proofs are constructed from start to finish. Section 1 answers the question of *what kind of argument* a given statement calls for. Section 2 shows you how to execute that argument *line by line*. Section 3 goes deep on the five structural patterns that cover almost every proof you will write. Section 4 gives induction the full treatment it deserves: not just the mechanical template, but the reasoning behind it, the different variants, and the mistakes that derail students who never understood why it works.

Sections 5–6 are reference material. Read Section 5 (Tactics) once, carefully, when you encounter an algebraic proof that stalls at a specific step. After that, treat it as a lookup table. Section 6 is a pure reference: a map from "stuck situations" to strategies.

Section 7 is a forward-looking preparation for Volume III. It does not teach real analysis; it equips you with a framework so that when you arrive at sequences and metric spaces, the proof patterns will not feel alien. It introduces the six structural types that every analysis exercise falls into, the twelve canonical proof moves used throughout metric space theory, and the four-layer hierarchy that organizes those moves. Read it after completing Sections 1–4 and before beginning Volume III.

Remark 5.0.1 (Four distinct questions that students conflate). The most common source of proof-writing paralysis is conflating four questions that have different answers and require different tools. **Architecture** asks: what kind of proof is this, and what skeleton should it have? **Construction** asks: once the skeleton is chosen, how do I execute it line by line? **Structures** asks: what do the standard proof patterns look like in practice? **Tactics** asks: once the proof is in progress, which specific algebraic moves are available? A student who asks "how do I start?" may be asking any of the first three questions simultaneously Separating them is the first act of a practised proof writer.

5.1 Proof Architecture

The Two Questions Before You Write Anything. Before writing a single symbol, ask two questions in order.

> **Question 1.** *What is the logical form of the statement?*
> Read the claim and identify its outermost logical structure: universal, conditional, biconditional, existential, uniqueness, equality, or structural assertion.
>
> **Question 2.** *What proof strategy does this form demand?*
> The form determines the skeleton of the proof almost mechanically, before any mathematics is done.

Remark 5.1.1 (Why order matters -- and why most students get it backwards). Question 2 cannot be answered before Question 1, but almost every beginner tries to answer it first. The default instinct is to reach for a familiar technique -- proof by contradiction is the usual reflex -- and start writing. This is the wrong order.

Reading the statement *is* the first mathematical act of the proof. The logical form of the statement is not preliminary bookkeeping; it is the information that determines the entire structure of the argument. A proof whose structure mismatches the statement's form is not just inelegant -- it is often invalid or at best needlessly circular.

Until you are fluent enough to read the form automatically, slow down deliberately before every proof. Write the statement out. Identify its outermost connective. Then and only then choose a strategy.

Example 5.1.2 (Applying the two questions). **Statement.** *The identity element of a group is unique.*

Q1. Logical form. Strip the natural-language phrasing down to its logical skeleton. "The identity is unique" means: for every group G and for any two elements $e, e' \in G$ that both satisfy the identity axiom, we have $e = e'$. The outermost structure is a universal statement with a uniqueness conclusion: $\forall G,\ \forall e, e' \in G$, [both satisfy identity axiom] $\Rightarrow e = e'$.

Q2. Strategy. A universally quantified uniqueness claim demands the *assume-two-and-compare* pattern: introduce an arbitrary group G, assume two objects e and e' both satisfying the definition, and derive $e = e'$.

The strategy is forced entirely by the form. No creative insight is required to select it. The selection is a consequence of reading.

Remark 5.1.3 (What "logical form" means in practice). You do not need to write out the full first-order formula every time. You need to identify one thing: the *outermost* structure.

Is the statement "for all x, ..."? Then introduce an arbitrary x. Is it "P implies Q"? Then assume P. Is it "there exists x"? Then construct a witness. Is it "there exists a unique x"? Then split into existence and uniqueness. Is it "$A = B$ as sets"? Then prove double inclusion. Is it "$a = b$ as elements in an algebraic structure"? Then either do algebra or use satisfy-and-cite.

The statement map in the next section makes this correspondence explicit.

Statement Forms and Their Proof Strategies. The following table maps every common statement form to the proof strategy it demands. This is not a list of suggestions — it is a mechanical lookup. Once you have identified the outermost logical form of the statement you are trying to prove, the table tells you how to open your proof.

Statement Form	Proof Strategy	Opening Move
$\forall x,\ P(x)$	Introduce arbitrary element	"Let x be arbitrary."
$P \Rightarrow Q$	Direct proof	"Assume P."
$P \Rightarrow Q$	Contrapositive	"Assume $\neg Q$."
$P \Rightarrow Q$ (when negation more useful)	Contradiction	"Assume P and $\neg Q$."
$P \Leftrightarrow Q$	Two separate directions	Prove $P \Rightarrow Q$, then $Q \Rightarrow P$.
$\exists x,\ P(x)$	Construct a witness	"Define $x := \dots$ and verify $P(x)$."
$\exists!\, x,\ P(x)$	Existence then uniqueness	Construct witness; then assume-two-and-compare.
$A = B$ (sets)	Double inclusion	Prove $A \subseteq B$ and $B \subseteq A$.
$a = b$ (algebraic elements)	Satisfy-and-cite or algebra	Show a satisfies the defining property of b.
$P(n)$ for all $n \in \mathbb{N}$	Induction	Base case; inductive step.
"X is unique"	Assume-two-and-compare	"Let x, y both satisfy the definition."

Remark 5.1.4 (Reading the table: defaults and alternatives). Some statement forms admit more than one strategy, and the table reflects this with multiple rows for the same form.

For $P \Rightarrow Q$, the default is direct proof: assume P and derive Q by forward reasoning. Contrapositive is reached for when $\neg Q$ is more useful than P as an assumption -- this happens when the negation of the conclusion is a strong, concrete condition (for example, "n is odd" is concrete; "n is not even" is the same thing but sounds weaker). Contradiction is reached for when neither P alone nor $\neg Q$ alone gets you far, but combining them produces a quick absurdity.

The hierarchy is: try direct proof first. If the hypothesis is hard to use forward, try contrapositive. If neither works, try contradiction. Do not reach for contradiction as a default just because the proof feels difficult.

Remark 5.1.5 (Set equality versus element equality are genuinely different). The two equality rows in the table look similar but demand completely different techniques, and conflating them is a common error.

$A = B$ as *sets* is proved by double inclusion because sets are defined extensionally two sets are equal if and only if they have the same elements. Proving $A = B$ by any other means -- saying they are "clearly the same" or "follow from the definition" -- is not a proof. The double inclusion must be made explicit.

$a = b$ as *elements in an algebraic structure* is different. Here b is typically a uniquely-determined object: an inverse, an identity, a limit, a fixed point. The strategy is to show a satisfies the same defining property as b, and then invoke the previously proved uniqueness theorem to conclude $a = b$. This is the satisfy-and-cite pattern (Section 5.1).

The mistake is trying to use double inclusion on algebraic elements, or trying to use algebraic manipulation on sets. The form of the objects determines which technique applies.

Remark 5.1.6 (The biconditional is always two separate proofs). A biconditional $P \Leftrightarrow Q$ is *never* proved in a single argument that runs from P to Q and "loops back." It is always proved in two distinct proofs:

($\Rightarrow$) Assume P. Prove Q.
($\Leftarrow$) Assume Q. Prove P.

Label the two directions explicitly. The two directions often require completely different techniques and different amounts of work. In characterisation theorems in analysis, one direction is typically a two-line unraveling of a definition while the other requires a nontrivial inequality argument. Treating them as one argument is how gaps get hidden.

The Five Proof Archetypes. Every mathematical proof, at the highest level of compression, is an instance of one of five archetypes. The statement map tells you the specific opening move. The archetypes tell you the *global shape* of the argument — what kind of reasoning is being done, and why it constitutes a valid proof of the claim.

The Five Proof Archetypes

1. **Construct something.** Exhibit a concrete object and verify it has every required property. This is the proof structure for all existence claims.

2. **Assume two and compare.** Assume two objects both satisfy a definition; derive that they are equal. This is the proof structure for all uniqueness claims.

3. **Take a minimal element.** Apply the well-ordering principle to a nonempty set of counterexamples; exploit the minimality of the least element to derive a contradiction. Used in number theory, descent arguments, and equivalence with induction.

4. **Induct.** Verify a base case; show the property propagates to successors. Used whenever the claim concerns all natural numbers or any recursively defined structure.

5. **Chase an arbitrary element.** Take an arbitrary element satisfying one condition; track it through definitions and established facts until it satisfies the target condition. Used for set inclusions, function properties, and direct proofs where the logic flows in a straight line from hypothesis to conclusion.

Remark 5.1.7 (Why five archetypes, not more?). The five archetypes are not a classification chosen for convenience. They correspond to the five fundamental proof obligations that mathematical statements impose:

To prove something *exists*, you must produce it. To prove something is *unique*, you must show that any two instances are equal. To prove a statement holds *for all natural numbers*, the inductive structure of $\mathbb{N}$ provides the only available tool. To prove a *conditional* claim $P \Rightarrow Q$, you must trace an arbitrary element or object satisfying P to a situation where Q holds. The minimal element archetype is the well-ordering counterpart to induction, available whenever you need to reason about the smallest counterexample.

Everything else in proof writing is refinement of these five. The specific proof structures in Section 5.3 (direct proof, contrapositive, contradiction, cases, existence-uniqueness) are instantiations of these archetypes adapted to specific logical forms. The tactics in Section 5.5 are the move-level tools used to execute whichever archetype applies.

Remark 5.1.8 (Archetypes can nest, and this is a feature). A single proof often uses more than one archetype. The most common nested pattern in abstract algebra is: construct a witness (archetype 1), then invoke a uniqueness theorem (archetype 2) to conclude that the constructed object equals a previously named one.

This is the satisfy-and-cite tactic (Section 5.1). It appears in almost every proof of the form "$a = b$" where b is an algebraically defined object. The nesting is not a complication -- it is the standard pattern for proving equalities in algebraic structures, and once recognized, it becomes mechanical.

Another common nesting: induct (archetype 4), and inside the inductive step, use a direct proof argument (archetype 5) to move from $P(n)$ to $P(n+1)$. The outer structure is induction; the inner argument is element-chasing.

Remark 5.1.9 (The archetype identifies the proof's centre of gravity). When you read a finished proof in a textbook, the archetype is usually visible in the first line or two. "Suppose x, y both satisfy the definition" -- that is archetype 2. "Define $f : X \to Y$ by $f(x) = \dots$" -- that is archetype 1. "Let $n \in \mathbb{N}$ be arbitrary" followed by induction markup -- that is archetype 4. "Let $x \in A$" followed by a chain of implications -- that is archetype 5.

Reading proofs actively means identifying the archetype at the start and then watching how the author executes it. This is far more efficient than trying to absorb a proof line by line without a sense of its overall shape.

The Satisfy-and-Cite Pattern. There is a proof pattern that does not appear in any of the standard proof archetypes by name, yet appears constantly — in every chapter of abstract algebra and analysis. Most undergraduates have never been taught it explicitly, which means they either reinvent it poorly each time or miss it entirely and write longer, less clear proofs.

The pattern is called **satisfy-and-cite**. Here is its logic.

The Satisfy-and-Cite Pattern

Setup. You want to prove $a = b$, where b is a *uniquely determined* object: the unique identity, the unique inverse, the unique limit, the unique fixed point, the unique element satisfying some algebraic condition.

Step 1. Show that a satisfies the defining property of b.
Step 2. Cite the previously proved uniqueness theorem: there is exactly one object satisfying that property.
Step 3. Conclude that $a = b$, since b is the unique such object and a is one too.

Remark 5.1.10 (Why this is not the same as assume-two-and-compare). The confusion between these two patterns is extremely common, so it is worth being precise about what each one does.

The **assume-two-and-compare** pattern *proves* a uniqueness theorem. You use it when the goal is to establish that some object is unique. You say: let x and y both satisfy the definition; then show $x = y$.

The **satisfy-and-cite** pattern *uses* an already proved uniqueness theorem. You use it when you already know some object b is unique and you want to identify a with b. You show a satisfies b's defining property, then invoke uniqueness to collapse a to b.

Assume-two-and-compare appears in the proof that the identity is unique. Satisfy-and-cite appears in every proof that uses that fact afterward. The first creates the tool; the second uses it. Conflating them means writing a new uniqueness proof every time you want to identify two equal objects, which is circular at best and wrong at worst.

Example 5.1.11 (Socks-shoes property: $(ab)^{-1} = b^{-1}a^{-1}$). We want to show $(ab)^{-1} = b^{-1}a^{-1}$ in a group.

The inverse of ab is, by definition, the unique element x satisfying $(ab)x = e$ and $x(ab) = e$.

We compute directly, using only associativity and the inverse axiom:

$$(ab)(b^{-1}a^{-1}) = a(bb^{-1})a^{-1} = aea^{-1} = aa^{-1} = e.$$

Similarly $(b^{-1}a^{-1})(ab) = e$.

So $b^{-1}a^{-1}$ *satisfies the defining property of* $(ab)^{-1}$. By the uniqueness of inverses (which we have already proved), we conclude $(ab)^{-1} = b^{-1}a^{-1}$. □

Notice what did not happen: we never assumed two inverses of ab existed and compared them. We showed one specific element satisfies the inverse condition, then cited uniqueness to identify it with the named inverse.

Example 5.1.12 (Ring theorem: $a \cdot (-b) = -(ab)$). We want to show $a(-b) = -(ab)$ in a ring.

The additive inverse $-(ab)$ is, by definition, the unique element x satisfying $ab + x = 0$.

We compute using distributivity:

$$ab + a(-b) = a(b + (-b)) = a \cdot 0 = 0.$$

So $a(-b)$ *satisfies the defining property of* $-(ab)$. By the uniqueness of additive inverses (previously proved), $a(-b) = -(ab)$. □

Same pattern: compute, satisfy, cite.

Remark 5.1.13 (The pattern across mathematics). Once you see satisfy-and-cite, you will see it everywhere. It appears wherever uniqueness theorems have been proved:

In **algebra**: every proof of the form "this element equals the inverse/identity/zero" uses it. The four examples above are $(ab)^{-1} = b^{-1}a^{-1}$, $a\cdot(-b) = -(ab)$, $-(-a) = a$, and $a \cdot 0 = 0$.

In **analysis**: the uniqueness of limits means that to show $\lim_{n\to\infty} x_n = L$, you verify x_n converges to L by the definition; uniqueness then implies L equals any other expression you have shown to be the limit of the same sequence.

In **linear algebra**: the zero vector is unique; the additive inverse is unique; a linear transformation is uniquely determined by its values on a basis. Each uniqueness theorem creates a new application of satisfy-and-cite.

Each time a uniqueness theorem is proved, it creates a new proof tool that can be activated by satisfy-and-cite for every subsequent argument in that structure. The theorem is not just a fact -- it is a lever.

5.2 The Proof Construction Algorithm

Remark 5.2.1 (What this section is for). Architecture (Section 5.1) tells you what kind of proof to write. This section tells you how to write it, line by line, once the architecture is chosen.

The procedure below is intentionally explicit and more detailed than anything you will find in a standard textbook. That is on purpose. With practice, these steps will become internalized and automatic. But until they are -- until you have written enough proofs that the process feels natural -- working through them consciously will prevent the most common errors: undefined objects, unjustified steps, and forgotten stop conditions.

A longer proof with every step explicit is always better than a shorter proof with implicit gaps.

Step 0: Classify the Statement. Before anything else, identify the logical form of the claim. Common forms:

- Universal: $\forall x\, P(x)$
- Conditional: $P \to Q$
- Biconditional: $P \leftrightarrow Q$
- Existential: $\exists x\, P(x)$
- Existence-and-uniqueness: $\exists!\, x\, P(x)$
- Set equality: $A = B$
- Algebraic equality: $a = b$
- Structural assertion: "f is injective," "R is an equivalence relation"

The logical form determines the skeleton of the proof. Consulting the statement map (Section 5.1) is the direct link from this classification to an opening strategy.

Step 1: Restate the Givens. Write out explicitly what is given. Introduce every set, relation, function, and algebraic structure, and record any properties assumed about them.

> Let G be a group. Let $a \in G$.

This is not a formality. By recording the givens, you license their use in the proof — you can now invoke associativity, the existence of inverses, and the identity element without reintroducing them each time. What is not in the givens cannot be used.

Step 2: Identify Objects and Their Types. Before reasoning begins, verify the *type* of each object you will work with: is it an element, a set, a function, or a relation? What is its ambient universe? What operations apply to it?

$$x \in A, \quad f : A \to B, \quad R \subseteq A \times A.$$

This step prevents the most common logical errors in undergraduate proofs: writing $x \in f$ when you mean $x \in \mathrm{dom}(f)$; writing $A = x$ when you mean $x \in A$; confusing membership ($\in$) with inclusion ($\subseteq$). Every object used in a proof must have a declared type. If you cannot say what type something is, you do not yet understand what you are working with.

Step 3: Introduce Arbitrary Elements. If the claim is universal ($\forall x\ P(x)$), immediately introduce an arbitrary element:

$$\text{Let } x \in A \text{ be arbitrary.}$$

The word "arbitrary" is not decorative. It means the element is not given any properties beyond those of the set A. Whatever you prove about this x will hold for all $x \in A$, precisely because you have assumed nothing about it beyond membership. The moment you specialize — "let $x = 0$" or "let x be such that …" — you have lost the ability to conclude the universal statement.

Step 4: Expand the Goal. Rewrite the conclusion using definitions rather than named concepts.

- To prove f is injective: expand injectivity as "$f(a) = f(b) \Rightarrow a = b$" and take that as the new goal.
- To prove $A = B$ (sets): take as new goal "$A \subseteq B$ and $B \subseteq A$."
- To prove $x \in A \cup B$: rewrite the goal as "$x \in A$ or $x \in B$."
- To prove $\{x_n\}$ converges: expand to "$\forall \varepsilon > 0\ \exists N$ s.t. $n \geq N \Rightarrow d(x_n, L) < \varepsilon$."

This step is the one most students skip, and skipping it is the single most reliable predictor of a stuck proof. You cannot make progress toward a goal you have not named in operative terms. "Prove f is injective" is not an operative goal. "Prove that $f(a) = f(b)$ implies $a = b$" is.

Step 5: Introduce Helper Objects. Introduce any auxiliary objects needed for the argument: witnesses for existential claims, intermediate elements for indirect arguments, bounds for inequality proofs, function values.

$$\text{Let } N_1 \in \mathbb{N} \text{ s.t. } n \geq N_1 \Rightarrow d(x_n, L) < \varepsilon/2.$$

No object should appear in a proof without being explicitly introduced. If you write "let c be the cancellation element" without first establishing that such an element exists, your proof has an undefined object. Every name carries an obligation: the obligation to have said what the name refers to.

Step 6: Apply One Rule at a Time. Each line of a proof follows from exactly one of four sources:

1. A definition (unpacking what a term means),
2. A hypothesis (something assumed in the current proof),
3. A previously proved theorem (cited by name and reference),
4. A basic logical rule (modus ponens, universal instantiation, etc.).

Lines that are justified by "clearly" or "obviously" or no reason at all are gaps. At the learning stage, every step should have an explicit justification. When a proof stalls, the right question is always: which of the four sources has not been consulted? Almost always, the answer is that some definition has not been unpacked.

Step 7: Use Forward and Backward Reasoning. Proofs alternate between two modes of reasoning, and recognizing which mode is productive at each stage is a significant skill.

Forward reasoning: deduce consequences from what you have established so far. If you know $a^2 = e$ in a group, derive that $a(ae) = a \cdot a \cdot e = a^2 \cdot e = e$. You are moving away from the hypotheses, toward the goal.

Backward reasoning: examine the goal and ask what would suffice to establish it. If the goal is "$a = a^{-1}$," ask: by what definition would that follow? It follows if $aa = e$, since that is the condition for a to be its own inverse. Now your new goal is to prove $aa = e$, which may be more directly accessible.

In practice, most proofs proceed in both directions simultaneously: you derive forward from the hypotheses while simplifying the goal backward until the two sides meet. When you are stuck, try the direction you have not been working in.

Step 8: Handle Cases Explicitly. If a statement splits into mutually exclusive and exhaustive cases, enumerate all of them.

$$\text{Either } n \text{ is even, or } n \text{ is odd.}$$

Handle each case separately. Close each case before opening the next. Verify, explicitly, that the cases together cover every possibility. A proof with unlabeled cases — where it is unclear which case is being argued or whether all cases have been covered — is not a proof.

Step 9: Close the Argument. Once the desired conclusion has been established, state it explicitly:

- "Thus $x \in B$, and so $A \subseteq B$."
- "Hence $f(a) = f(b)$ implies $a = b$, so f is injective."
- "In both cases $P(n + 1)$ holds."

The explicit closing step serves two functions. First, it tells the reader (and you) that the proof goal has been achieved. Second, it forces you to check that what you have proved actually *is* the goal. Students who reach the end of a proof and write □without stating a conclusion often discover, on re-reading, that they proved something adjacent to the goal but not the goal itself.

Step 10: Signal Completion. Conclude with $\square$, $\blacksquare$, or "This completes the proof."

Legal Moves and Stop Conditions

What you may never do:

- Choose an existential witness before proving it exists.
- Introduce an "arbitrary" element and then give it special properties.
- Assume the conclusion at any point in the proof.
- Apply a definition partially (all conditions must be checked).

A proof is complete when:

- A universal claim has been proved for a genuinely arbitrary element.
- An existential claim has produced a verified witness.
- A set equality has established both inclusions explicitly.
- Each direction of a biconditional has been proved separately.
- An existence-uniqueness claim has done both parts.

Line-by-Line Discipline. Before writing each line, ask three silent questions:

(i) Has every object mentioned in this line been defined or introduced earlier?

(ii) Which definition, hypothesis, or theorem justifies this step?

(iii) Does this step move the argument toward the stated goal?

If any answer is no, stop and resolve it before continuing.

5.3 Proof Structures

Direct Proof. A direct proof of $P \Rightarrow Q$ assumes P and derives Q through a chain of valid inferences.

Template: Direct Proof

Given. P.
Goal. Q.

Assume P.
$\vdots$ [expand definitions; apply lemmas; derive consequences]
Therefore Q. $\square$

Remark 5.3.1 (Direct proof is the default -- always try it first). Direct proof is not one option among equals. It is the default. Every time you sit down to prove $P \Rightarrow Q$, the first thing you write is "Assume P" and the first thing you do is unpack what P says. You use what P gives you, apply definitions and previously proved results, and derive Q.

Reach for contrapositive or contradiction only when direct proof has genuinely stalled -- when you have unpacked the hypothesis and the goal and there is no visible path between them. Do not reach for contradiction just because the statement feels hard. Most statements that feel hard are hard only because a definition has not been unpacked.

Remark 5.3.2 (The anatomy of a direct proof). Every direct proof has three phases that correspond to the first four steps of the construction algorithm:

Phase 1 -- Unpack the hypothesis. P is a named concept or a logical statement. Expand it into its operative form. If P says "G is a group," this licenses associativity, the existence of an identity, and the existence of inverses. Write those down as available tools.

Phase 2 -- Expand the goal. Q is also a named concept. Expand it into what you need to establish. If Q says "$a = a^{-1}$," the operative target is showing a satisfies the defining property of an inverse: $aa = e$.

Phase 3 -- Bridge the gap. Apply definitions, lemmas, and axioms to move from the expanded hypothesis to the expanded conclusion.

When Phase 3 stalls, the cause is almost always that Phase 1 or Phase 2 was not completed. The most common error is having a vague goal ("show $a = a^{-1}$") without having written out what that means in terms of the group axioms.

Example 5.3.3 (Direct proof in a group). **Claim.** In a group G, if $a^2 = e$ then $a = a^{-1}$.

Proof. Assume $a^2 = e$, meaning $aa = e$.

We want to show $a = a^{-1}$, which by definition of inverse means we need $aa = e$ and $ea = a$ (or equivalently, a is its own inverse).

Multiply both sides of $aa = e$ on the right by a^{-1}:

$$(aa)a^{-1} = e \cdot a^{-1}.$$

By associativity: $a(aa^{-1}) = a^{-1}$. By the inverse axiom: $ae = a^{-1}$. By the identity axiom: $a = a^{-1}$.

Hence $a^2 = e$ implies $a = a^{-1}$. $\square$

Each step is justified by exactly one of: the hypothesis ($aa = e$), the inverse axiom ($aa^{-1} = e$), the identity axiom ($ae = a$), or associativity. No step is unjustified.

Proof by Contrapositive. The contrapositive of $P \Rightarrow Q$ is $\neg Q \Rightarrow \neg P$. These two statements are logically equivalent: one is true if and only if the other is. A proof by contrapositive establishes $\neg Q \Rightarrow \neg P$ by a direct argument, and since this is equivalent to $P \Rightarrow Q$, the original claim follows.

Template: Proof by Contrapositive

Goal. $P \Rightarrow Q$.
Equivalent goal. $\neg Q \Rightarrow \neg P$.

Assume $\neg Q$.

$\vdots$ [derive $\neg P$ from $\neg Q$]
Therefore $\neg P$.
Hence $P \Rightarrow Q$. $\square$

Remark 5.3.4 (When contrapositive is the right tool). Contrapositive is useful when the negation of the conclusion is more concrete and easier to work with than the hypothesis.

The canonical example is injectivity. To prove $P \Rightarrow Q$ where P says "$f(a) = f(b)$" and Q says "$a = b$": the contrapositive is $a \neq b \Rightarrow f(a) \neq f(b)$, which may be directly accessible if the function is order-preserving or explicitly defined.

Another signal: when Q is the statement that some object exists or some set is non-empty. The negation $\neg Q$ says no such object exists, which may be a very restrictive and workable condition.

The general diagnostic: after writing $\neg Q$, look at what it says. If it gives you a strong, concrete assumption -- something you can actually manipulate -- contrapositive is likely the right tool. If $\neg Q$ is vague or complicated, try a different approach.

Remark 5.3.5 (Contrapositive versus contradiction: the essential difference). Students routinely conflate contrapositive and contradiction. They are not the same, and knowing the difference matters for clarity.

In a **contrapositive** proof, you assume *only* $\neg Q$ and derive $\neg P$. The hypothesis P is never mentioned. The logical engine is: you proved $\neg Q \Rightarrow \neg P$, which by equivalence gives $P \Rightarrow Q$.

In a **contradiction** proof, you assume *both* P and $\neg Q$ simultaneously and derive *any* absurdity -- not necessarily $\neg P$. The logical engine is: the assumption $P \wedge \neg Q$ leads to a false statement, so $P \wedge \neg Q$ must itself be false, so $P \Rightarrow Q$.

When contrapositive works, it is cleaner: you need one assumption, not two, and the conclusion you derive ($\neg P$) is specific rather than "any contradiction." Always try contrapositive before reaching for contradiction.

Example 5.3.6 (Contrapositive for divisibility). **Claim.** If n^2 is even then n is even.

Proof (contrapositive). We prove the contrapositive: if n is odd then n^2 is odd.

Assume n is odd. Then $n = 2k + 1$ for some $k \in \mathbb{Z}$.

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which has the form $2m + 1$ and is therefore odd.

Hence n odd $\Rightarrow$ n^2 odd, which is the contrapositive of n^2 even $\Rightarrow$ n even. $\square$

Notice: the hypothesis "n^2 is even" was never used. We proved the contrapositive directly, and the original implication follows by logical equivalence. There is no need to loop back and mention the original statement; the contrapositive suffices.

Proof by Contradiction. To prove P, assume $\neg P$ and derive a statement that contradicts a known truth: a hypothesis, an axiom, or a previously proved theorem. Since the assumption $\neg P$ led to a false statement, $\neg P$ is false, and therefore P is true.

Template: Proof by Contradiction

Goal. P.

Suppose for contradiction that $\neg P$.

$\vdots$ [derive consequences; identify the absurdity]
This contradicts [name the violated fact].
Therefore P. $\square$

Remark 5.3.7 (The logic: why it works). Contradiction relies on the law of the excluded middle: every statement is either true or false. To prove P, it suffices to show $\neg P$ is false. You show $\neg P$ is false by deriving from it a statement that is known to be false -- a contradiction.

The contradiction can be anything: $0 = 1$, a set that is both empty and non-empty, a number that is simultaneously even and odd, an inequality $x < x$. The only requirement is that the contradicted fact was genuinely established before the current proof began.

This is the move that students most often get wrong: they think they have derived a contradiction when they have merely derived something surprising. A statement being counterintuitive is not a contradiction. The contradicted fact must be something you can point to: a hypothesis, an axiom, a definition, or a theorem with a name.

Remark 5.3.8 (When contradiction is the right tool). Contradiction is best reserved for claims whose negation is a strong, productive assumption. The archetypes where contradiction excels are:

Irrationality proofs. Assuming $\sqrt{2} \in \mathbb{Q}$ gives you a fraction in lowest terms; arithmetic then forces the numerator and denominator to be simultaneously even, contradicting the lowest-terms assumption.
Infinitude arguments. Assuming there are finitely many primes lets you list them; constructing a number not divisible by any of them contradicts the assumption that the list was complete.
Nonexistence results. If you want to show no object satisfies some property, assume one exists and derive an absurdity.
Uniqueness (via assume-two-and-compare). Assuming two distinct objects both satisfy a uniqueness condition produces a contradiction from the definitions.

Avoid contradiction as a default. When a direct proof or contrapositive works, prefer it: the argument is shorter and the logical structure is more transparent. Contradiction is a sign that the natural direction of inference does not work, and that you need to reason from an impossible assumption instead.

Remark 5.3.9 (Always name the contradiction explicitly). The closing line of a contradiction proof must name what has been contradicted. "This is a contradiction" with no further specification is a red flag: it suggests the student either does not know what was contradicted, or is hoping the reader will not check.

Write: "This contradicts the assumption that $\gcd(p, q) = 1$." Write: "This contradicts Theorem X, which asserts ..." Write: "This contradicts $m \in S$ being the least element."

Naming the violated fact performs two functions: it confirms that the contradiction is genuine, and it locates the logical break in the argument so that anyone reading the proof can verify it.

Example 5.3.10 ($\sqrt{2}$ is irrational). *Proof.* Suppose for contradiction that $\sqrt{2} \in \mathbb{Q}$. Write $\sqrt{2} = p/q$ where $p, q \in \mathbb{Z}$, $q \neq 0$, and $\gcd(p, q) = 1$ (in lowest terms; this is possible by the definition of $\mathbb{Q}$).

Squaring: $2q^2 = p^2$. So p^2 is even, which forces p to be even (by the contrapositive example above). Write $p = 2k$.

Then $2q^2 = (2k)^2 = 4k^2$, so $q^2 = 2k^2$. So q^2 is even, which forces q to be even.

But then both p and q are even, so $\gcd(p, q) \geq 2$. This contradicts $\gcd(p, q) = 1$.

Therefore $\sqrt{2} \notin \mathbb{Q}$. $\square$

The structure is: assume $\neg P$ (i.e., $\sqrt{2} \in \mathbb{Q}$); extract information from this assumption (lowest-terms representation); derive a consequence (both p and q even); identify the contradiction (conflicts with lowest terms); conclude P.

Proof by Cases. If the hypotheses or conclusion naturally divide into a finite number of mutually exclusive and collectively exhaustive alternatives, handle each case separately.

Template: Proof by Cases

Goal: Prove Q, given that exactly one of $P_1, P_2, \ldots, P_k$ holds (and one of them always holds).

Verify exhaustiveness: Every possibility falls under some P_i.
Verify mutual exclusivity (when needed).

Case 1: Assume P_1. [Argue.] Hence Q.

Case 2: Assume P_2. [Argue.] Hence Q.

$\vdots$

Case k: Assume P_k. [Argue.] Hence Q.

Since $P_1, \ldots, P_k$ are exhaustive and Q holds in each case, Q holds unconditionally. $\square$

Remark 5.3.11 (The exhaustiveness obligation is non-negotiable). A case proof is only valid when the cases together cover every possibility. Always verify exhaustiveness explicitly, even when it feels obvious.

The most common exhaustive splits:

- n is even, or n is odd.
- $a > 0$, $a = 0$, or $a < 0$.
- $a \mid b$, or $a \nmid b$.
- $x \in A$, or $x \notin A$.
- $x = y$, or $x \neq y$.

Writing "we consider two cases" without naming what they are, or naming cases that do not cover all possibilities, is a structural error. The proof is not valid until exhaustiveness is established.

Remark 5.3.12 (Each case is a self-contained argument). Inside each case, you have an additional assumption: P_i is true. That assumption is local to the case. You may use it within Case i, but you may not carry it into Case j.

When you close a case -- "hence Q, completing Case 1" -- you are signalling that the local assumption is no longer in scope. Case 2 starts fresh, with only the original given plus the assumption P_2.

This discipline prevents the most common case-analysis error: drifting from one case into another by accidentally retaining a local assumption beyond its scope.

Remark 5.3.13 (Symmetric cases). When two cases are exactly symmetric -- when the argument for Case 2 is obtained from the argument for Case 1 by renaming variables, with no other change -- it is acceptable to write "Case 2 is symmetric to Case 1 by swapping a and b." But be careful: only write this when the symmetry is complete. A partial symmetry that requires a different step at one point must be written out in full.

Example 5.3.14 (Parity of $n(n+1)$). **Claim.** For every $n \in \mathbb{Z}$, $n(n+1)$ is even.

Proof. Every integer is either even or odd (these cases are exhaustive and mutually exclusive).

Case 1: n is even. Write $n = 2k$ for some $k \in \mathbb{Z}$. Then $n(n+1) = 2k(n+1)$. Since this is 2 times an integer, $n(n+1)$ is even.

Case 2: n is odd. Write $n = 2k+1$ for some $k \in \mathbb{Z}$. Then $n+1 = 2k+2 = 2(k+1)$, so $n(n+1) = (2k+1)\cdot 2(k+1)$. This is 2 times an integer, so $n(n+1)$ is even.

In both cases $n(n+1)$ is even. Since every integer falls into one of these cases, the proof is complete. $\square$

Notice that the cases were named before the argument, the exhaustiveness was verified ("every integer is even or odd"), and each case was closed explicitly before moving to the next.

Example 5.3.15 (Case split induced by a definition). **Claim.** The discrete metric satisfies the triangle inequality.

Proof. Let $x, y, z \in X$. We must show $d(x,z) \leq d(x,y) + d(y,z)$.

Case 1: $x = z$. Then $d(x,z) = 0$. Since $d(x,y) \geq 0$ and $d(y,z) \geq 0$, we have $0 \leq d(x,y) + d(y,z)$.

Case 2: $x \neq z$. Then $d(x,z) = 1$. We cannot have both $x = y$ and $y = z$, for that would force $x = z$. So at least one of $d(x,y), d(y,z)$ equals 1, hence $d(x,y) + d(y,z) \geq 1 = d(x,z)$.

The cases $x = z$ and $x \neq z$ are exhaustive. The triangle inequality holds in both cases. □

The case split here is induced by the two-value structure of the discrete metric: distances are 0 or 1. The right case split to use is the one naturally suggested by the definition you are working with.

Existence and Uniqueness. The statement $\exists!\, x\ P(x)$ asserts two things simultaneously: that some object satisfying P exists, and that it is the *only* such object. These two obligations are proved separately, in order.

Template: Existence and Uniqueness ($\exists!\, x\ P(x)$)

Step 1 — Existence.
Define $x :=$ [explicit construction].
Verify that $P(x)$ holds. (Check every condition in P.)

Step 2 — Uniqueness (assume-two-and-compare).
Let x and y be arbitrary objects both satisfying P.
Apply the definition of x with y as input; apply the definition of y with x as input. The two applications collapse.
Conclude $x = y$.

Step 3 — Conclude.
By Steps 1 and 2, there exists exactly one object satisfying P. □

Remark 5.3.16 (Existence must be proved before uniqueness). The uniqueness argument has the form: "let x and y both satisfy P." This assumption is only meaningful if we already know that at least one object satisfying P exists. Without Step 1, the uniqueness argument is vacuous: we are asserting something about objects that might not exist.

This is not pedantry. There are statements where the existence fails and the uniqueness argument would appear to go through -- but the whole proof would then be establishing uniqueness for an empty set of objects, which is trivially true and useless. Existence first is not just a convention; it is logically necessary.

Remark 5.3.17 (The key move in assume-two-and-compare). The assume-two-and-compare argument has a characteristic structure that appears across mathematics. The critical move is this: if x and y both satisfy the defining property P, apply x's property with y as the argument, and simultaneously apply y's property with x as the argument. The two applications collapse to $x = y$.

Here is the pattern in the group identity example: if e satisfies the identity axiom, then $e \cdot e' = e'$ (using e as identity with argument e'). If e' satisfies the identity axiom, then $e \cdot e' = e$ (using e' as identity with argument e). Combining: $e' = e$.

The same move works for: uniqueness of inverses, uniqueness of the zero vector, uniqueness of limits, uniqueness of the supremum. In every case, the two assumed objects are "played against each other": each one's defining property is applied with the other as input.

Example 5.3.18 (Uniqueness of the group identity). *Proof.* Let G be a group. Suppose e and e' both satisfy the identity axiom:

$$e \cdot g = g \cdot e = g \quad \text{for all } g \in G,$$

$$e' \cdot g = g \cdot e' = g \quad \text{for all } g \in G.$$

Apply e''s identity property with $g = e$:

$$e' \cdot e = e.$$

Apply e's identity property with $g = e'$:

$$e \cdot e' = e'.$$

But $e' \cdot e = e \cdot e'$ (both equal the product $e \cdot e'$ in G by commutativity -- actually, both equal $e \cdot e'$ directly). Actually, more directly: $e = e' \cdot e = e'$ where the first step uses e' as identity and the second uses e as identity. Hence $e = e'$. □

The single line is: $e = e \cdot e' = e'$. First equality: e acts as identity on e'. Second equality: e' acts as identity on e. Both identities applied to the same product; both sides collapse to the same expression; $e = e'$.

Remark 5.3.19 (Why proving uniqueness creates a proof tool). Every uniqueness theorem, once proved, creates a new application of the satisfy-and-cite tactic. This is the connection that most students miss.

Proving uniqueness says: there is exactly one object satisfying P. This means: if you can ever show that some element a satisfies P, you automatically get $a = b$, where b is the canonical name for the unique object satisfying P.

The group inverse provides the clearest example. Once you have proved that inverses are unique, the pattern for proving $(ab)^{-1} = b^{-1}a^{-1}$ is immediate: show $b^{-1}a^{-1}$ satisfies the defining property of the inverse of ab, then cite uniqueness. You never need to set up another assume-two-and-compare argument. The uniqueness theorem absorbs all future comparisons.

This is why uniqueness results are not just facts -- they are *tools*. Prove them early; use them everywhere afterward.

5.4 Mathematical Induction

Why Induction Works. Mathematical induction is often presented as a technique: base case, inductive step, done. That presentation is useful but incomplete. Induction *feels* like a procedure, but it is actually a theorem, and it is worth understanding why it is true before using it.

Remark 5.4.1 (The Peano axiom behind induction). The natural numbers are characterize by the Peano axioms. The critical one for induction is Axiom P5:

If $A \subseteq \mathbb{N}$ *satisfies (i)* $0 \in A$ *and (ii)* $n \in A \Rightarrow S(n) \in A$*, then* $A = \mathbb{N}$.

This axiom says that $\mathbb{N}$ is the *smallest* set containing 0 and closed under the successor function S. There is no proper subset of $\mathbb{N}$ with these two properties.

The induction principle is a direct translation. Let $P(n)$ be any statement. Define $A = \{n \in \mathbb{N} : P(n)\}$. If $P(0)$ holds, then $0 \in A$. If $P(n) \Rightarrow P(S(n))$ for every n, then A is closed under S. By P5, $A = \mathbb{N}$. Therefore $P(n)$ holds for every $n \in \mathbb{N}$.

The full development of this from the Peano axioms is in the Axiom Systems chapter. Here, the takeaway is: induction is not a trick. It is a theorem derivable from the definition of $\mathbb{N}$.

Remark 5.4.2 (Why the base case is not optional). The inductive step alone says: *if* $P(n)$ holds for some n, then $P(n+1)$ holds. This is a conditional, not an assertion. Without a base case, there is no n for which $P(n)$ is known to hold. The chain of conditionals has no starting point, and the proof produces nothing.

The most famous illustration is the false theorem "all horses are the same colour." A flawed argument runs: base case trivial ($n = 1$), inductive step [fatally flawed reasoning]. The lesson is not just that the step is wrong -- it is that even a correct base case does not rescue a bad step, and even a correct step does not rescue a missing base case. Each part is independent; both are necessary.

In terms of P5: the base case establishes $0 \in A$. The inductive step establishes closure under S. Both conditions of P5 are required for the conclusion $A = \mathbb{N}$.

Remark 5.4.3 (The domino picture and its limitations). The common picture of induction as an infinite row of falling dominoes is a useful mnemonic but a misleading explanation. It suggests that induction requires each domino to knock over the next, which captures the inductive step. But it does not explain why the argument is valid -- the validity comes from P5, not from a physical metaphor.

More importantly, the domino picture suggests that the steps happen *in time*: first domino 0 falls, then 1, then 2, and so on. But induction proves a statement about *all* natural numbers simultaneously, not by a sequential process. The proof is instantaneous: once you have established the base case and the inductive step, the conclusion $P(n)$ for all n follows at once from P5.

Use the domino picture as memory aid. For understanding, use P5.

Remark 5.4.4 (Induction and well-ordering are equivalent). The induction principle and the well-ordering principle for $\mathbb{N}$ (every nonempty subset has a least element) are logically equivalent. This means every induction argument can be rephrased as a minimal-counterexample argument, and vice versa. Both are valid proof tools. The well-ordering approach is sometimes more natural when you want to reason about the *first* failure rather than climbing from a base case. The equivalence is proved in Section 5.4.

Theorem: Weak (Standard) Induction

Let $P(n)$ be a statement depending on $n \in \mathbb{N}$. Suppose:

1. **Base case.** $P(n_0)$ is true for some starting value n_0.
2. **Inductive step.** For all $n \geq n_0$, $P(n) \Rightarrow P(n+1)$.

Then $P(n)$ is true for all $n \geq n_0$.

Template: Weak Induction Proof

Let $P(n)$ denote the statement: [*write it out completely as a sentence*].

Base case ($n = [n_0]$). [*Verify $P(n_0)$ directly.*] Hence $P(n_0)$ holds.

Inductive step. Let $n \geq n_0$ and assume $P(n)$ holds. [*State explicitly what $P(n)$ says.*] We show $P(n+1)$ holds.
[*Rewrite $P(n+1)$ to expose $P(n)$ inside it. Substitute the inductive hypothesis. Simplify.*]
Hence $P(n+1)$ holds.

By induction, $P(n)$ holds for all $n \geq n_0$. $\square$

Remark 5.4.5 (The most important discipline: write $P(n)$ as a sentence). The single most impactful habit in induction proofs is writing the inductive hypothesis as a *complete mathematical sentence* before the inductive step begins.

"Assume $P(n)$ holds" is not sufficient if you have not first written out what $P(n)$ says. Write: "Assume $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$." That sentence is your inductive hypothesis. It is the precise tool you have available in the inductive step.

Without this sentence, students typically make one of two errors: they try to use $P(n)$ without knowing what it says, guessing at it from context; or they prove the inductive step using information beyond what $P(n)$ supplies, making the proof circular or invalid. Write the sentence. It takes five seconds and eliminates the most common induction error.

Remark 5.4.6 (The core skill of the inductive step). Every inductive step involves the same core action: *rewrite $P(n+1)$ to expose $P(n)$ inside it, then substitute the inductive hypothesis.*

For sum formulas, this means: split off the last term.

$$\sum_{k=1}^{n+1} k = \left(\sum_{k=1}^{n} k\right) + (n+1).$$

Now substitute $P(n)$: replace $\sum_{k=1}^{n} k$ with $\frac{n(n+1)}{2}$. Simplify to get the $P(n+1)$ form.

For divisibility, this means: write $4^{n+1} - 1 = 4(4^n - 1) + 3$ to expose $4^n - 1$ inside $4^{n+1} - 1$.

In both cases, the step is: factor out the $P(n)$ term, substitute, and verify the arithmetic produces $P(n+1)$.

When the rewriting step is not obvious, it usually means $P(n)$ needs to be strengthened (see Section 5.4).

Example 5.4.7 (Sum formula). **Proposition.** For all $n \geq 1$: $\displaystyle\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

Proof. Let $P(n)$ denote: $\displaystyle\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

Base case ($n = 1$). $\sum_{k=1}^{1} k = 1 = \frac{1\cdot 2}{2}$. Hence $P(1)$ holds.

Inductive step. Let $n \geq 1$ and assume $\displaystyle\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$. *(Inductive hypothesis.)*

We show $\displaystyle\sum_{k=1}^{n+1} k = \frac{(n+1)(n+2)}{2}$.

Split off the last term:

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \left(\sum_{k=1}^{n} k\right) + (n+1) \\ &= \frac{n(n+1)}{2} + (n+1) && \text{(inductive hypothesis)} \\ &= (n+1)\left(\frac{n}{2} + 1\right) = \frac{(n+1)(n+2)}{2}.\end{aligned}$$

Hence $P(n+1)$ holds. By induction, $P(n)$ holds for all $n \geq 1$. □

Example 5.4.8 (Divisibility). **Proposition.** For all $n \geq 0$, $3 \mid (4^n - 1)$.

Proof. Let $P(n)$ denote: $3 \mid (4^n - 1)$.

Base case ($n = 0$). $4^0 - 1 = 0 = 3 \cdot 0$. Hence $3 \mid 0$ and $P(0)$ holds.

Inductive step. Assume $3 \mid (4^n - 1)$, i.e., $4^n - 1 = 3m$ for some $m \in \mathbb{Z}$.

Expose $4^n - 1$ inside $4^{n+1} - 1$:

$$4^{n+1} - 1 = 4 \cdot 4^n - 1 = 4(4^n - 1) + 4 - 1 = 4(4^n - 1) + 3 = 4 \cdot 3m + 3 = 3(4m + 1).$$

Hence $3 \mid (4^{n+1} - 1)$ and $P(n+1)$ holds.

By induction, $P(n)$ holds for all $n \geq 0$. □

The rewriting step $4^{n+1} - 1 = 4(4^n - 1) + 3$ is the key move: it exposes the inductive hypothesis $3 \mid (4^n - 1)$ inside the goal $3 \mid (4^{n+1} - 1)$.

Choosing $P(n)$: The Hard Part. In textbook problems, the statement $P(n)$ is usually handed to you: the claim is explicitly "prove that $\sum_{k=1}^{n} k = n(n+1)/2$", and $P(n)$ is that equality. In original proofs, you must construct $P(n)$ from the statement of the goal. And sometimes even when $P(n)$ is given, the straightforward choice fails and must be strengthened.

Remark 5.4.9 (Rule 1: $P(n)$ mirrors the logical form of the goal). Every claim to be proved by induction has the form $\forall n, \Phi(n)$ for some predicate Φ. Strip the universal quantifier: what remains is $P(n)$.

Goal form	**Shape of $P(n)$**
$\forall n,\ f(n) = g(n)$	Equality: $P(n) :\equiv f(n) = g(n)$
$\forall n,\ A(n) \Rightarrow B(n)$	Conditional: $P(n) :\equiv A(n) \Rightarrow B(n)$
$\forall n,\ A(n) \Leftrightarrow B(n)$	Biconditional: $P(n) :\equiv A(n) \Leftrightarrow B(n)$

Do not simplify. Do not add conditions. Transcribe the logical form of $\Phi(n)$ exactly into $P(n)$.

Remark 5.4.10 (Rule 2: induct on the recursive variable). When the goal involves a recursively defined operation, identify which variable the definition recurses on. Induct on that variable and hold all others fixed as arbitrary.

To find the recursive variable: locate the definition of the operation and identify which variable is reduced in the recursive clause.

Operation	**Recursive clause**	**Induct on**
Addition (Peano)	$(n{+}{+}) + m := (n+m){+}{+}$	n
Multiplication (Peano)	$(n{+}{+}) \times m := (n \times m) + m$	n
Exponentiation	$m^{n{+}{+}} := m^n \times m$	n

Remark 5.4.11 (The strengthening principle: when $P(n)$ is too weak). Sometimes the natural $P(n)$ fails not because the statement is false but because the inductive hypothesis is insufficient to prove the step. The fix is counterintuitive: ***add more to*** $P(n)$, making the claim stronger.

This works because a stronger $P(n)$ gives you more in the inductive hypothesis, which is what you actually need to prove $P(n{+}1)$. The stronger $P(n)$ is harder to prove overall but easier to propagate.

The signal that strengthening is needed: you write the inductive step, you need to use some fact about the sequence that $P(n)$ alone does not provide, and you find yourself thinking "if only I also knew $P(n{-}1)$." That thought is the signal to switch to strong induction (which implicitly strengthens $P(n)$ to "$P(k)$ for all $k \leq n$").

Example 5.4.12 (Strengthening: Fibonacci bound requires two prior values). **Goal.** Show that the Fibonacci sequence satisfies $F_n < 2^n$ for all $n \geq 1$.

Naive attempt. Let $P(n)$: "$F_n < 2^n$."

Inductive step: assume $F_n < 2^n$; show $F_{n+1} < 2^{n+1}$.

We have $F_{n+1} = F_n + F_{n-1}$. We know $F_n < 2^n$ by the inductive hypothesis. But what do we know about F_{n-1}? Nothing: $P(n)$ says nothing about F_{n-1}. The step is stuck.

Fix: strong induction gives two terms. Use the strong inductive hypothesis: $F_k < 2^k$ for all $k \leq n$. Then:

$$F_{n+1} = F_n + F_{n-1} < 2^n + 2^{n-1} < 2^n + 2^n = 2^{n+1}.$$

The step completes because strong induction provides $F_{n-1} < 2^{n-1}$. This is the canonical signal that strong induction is needed: the recursive step requires knowing $P(k)$ for some $k < n$ other than $k = n-1$.

Remark 5.4.13 (Vacuous base cases in conditional statements). When $P(n)$ is a conditional and its hypothesis is false at the base value, the base case is vacuously true. This is expected and requires no further argument.

For example, if the claim is "for all $n \geq 1$, if $n > 0$ and $m > 0$ then $nm > 0$" and you induct starting at $n = 0$, then $P(0)$: "if $0 > 0$ and $m > 0$ then $0{\times}m > 0$" has a false hypothesis ($0 > 0$ is false), so $P(0)$ is vacuously true.

This signals that the claim only has content for values of n where the hypothesis holds. The induction then establishes the claim for all n where the hypothesis is satisfied.

Remark 5.4.14 (Diagnostic questions when stuck choosing $P(n)$). 1. What exactly do I need to prove for $n = 0, 1, 2, 3$? Write it out. The pattern of $P(n)$ will appear.

2. Does the inductive step require knowing $P(n-1)$ as well as $P(n)$? If yes, switch to strong induction.

3. Does the inductive step produce a bound or equation slightly weaker than $P(n+1)$? If yes, strengthen $P(n)$.

4. After writing $P(n{+}{+})$, can I apply the recursive definition to expose $P(n)$? If not, the wrong variable may have been chosen.

Theorem: Strong Induction

Let $P(n)$ be a statement depending on $n \in \mathbb{N}$. Suppose that for every $n \in \mathbb{N}$:

$$\Big(\forall k < n,\ P(k)\Big) \Rightarrow P(n).$$

Then $P(n)$ holds for all $n \in \mathbb{N}$.

Remark 5.4.15 (Strong and weak induction prove the same statements). Strong induction is sometimes described as "more powerful" than weak induction, but this is imprecise. Both principles are logically equivalent (they both follow from P5 and each implies the other). Strong induction is not more powerful in the sense of proving statements that weak induction cannot -- every statement provable by one is provable by the other.

What strong induction offers is convenience: when the proof of $P(n+1)$ genuinely requires knowing $P(k)$ for some $k < n$ that is not $n-1$, strong induction supplies that information directly. Rephrasing the proof as weak induction would require artificially strengthening $P(n)$ to "$P(k)$ for all $k \leq n$", which is precisely the hypothesis that strong induction builds in automatically.

Use strong induction when the step needs information from more than one earlier case. Use weak induction when $P(n)$ alone suffices.

Remark 5.4.16 (The base case in strong induction). When $n = 0$, the hypothesis $\forall k < 0,\ P(k)$ is vacuously true: there are no natural numbers less than 0. So the step for $n = 0$ requires proving $P(0)$ with no inductive hypothesis at all -- this is the base case, and it must be proved directly.

In practice, always write the base case explicitly, even though it is implicit in the universal statement. Students who skip the base case in strong induction often write "by the inductive hypothesis applied to $n-1$" when $n = 0$, which is an error: $n-1 = -1$ is not a natural number and has no inductive hypothesis.

Remark 5.4.17 (When to use strong induction: the diagnostic). The diagnostic for strong induction is simple. Write out the inductive step: $P(n+1) = f(P(?), P(?), \ldots)$, and ask what earlier values of P you need. If the answer includes anything other than exactly $P(n)$, use strong induction.

Canonical situations where strong induction is needed:

- Recursive sequences with two prior terms: $a_{n+1} = a_n + a_{n-1}$ requires both $P(n)$ and $P(n-1)$.
- Prime factorization: writing $n = ab$ with $a, b < n$ requires $P(a)$ and $P(b)$ where a and b are arbitrary.
- Any argument that splits $n+1$ into parts: you need P at the part values, which are not predetermined.

Template: Strong Induction Proof

Let $P(n)$ denote: [*write it out*].

Base case ($n =$ [start]). [*Verify directly, with no inductive hypothesis.*]

Inductive step. Let $n >$ [start] and assume $P(k)$ holds for all [start] $\leq k < n$. *(Strong inductive hypothesis.)* We show $P(n)$ holds.
[*Use $P(k)$ for some specific $k < n$.*]
Hence $P(n)$ holds.
By strong induction, $P(n)$ holds for all $n \geq$ [start]. □

Example 5.4.18 (Every integer $n \geq 2$ has a prime factor). *Proof.* Let $P(n)$: "n has a prime factor."

Base case ($n = 2$). 2 is prime, so 2 is a prime factor of itself. $P(2)$ holds.

Inductive step. Let $n > 2$ and assume $P(k)$ holds for all $2 \leq k < n$.
Case 1: n is prime. Then n is its own prime factor, and $P(n)$ holds.
Case 2: n is composite. Write $n = ab$ with $2 \leq a, b < n$. By the strong inductive hypothesis applied to a, a has a prime factor p. Then $p \mid a$ and $a \mid n$, so $p \mid n$. Hence $P(n)$ holds.
In both cases $P(n)$ holds. By strong induction, $P(n)$ holds for all $n \geq 2$. □

Why weak induction cannot work directly: in Case 2, the value a satisfies $2 \leq a < n$ but is otherwise arbitrary. It is not necessarily $n - 1$. Weak induction supplies $P(n-1)$, which is useless when $a \neq n-1$. Strong induction supplies $P(k)$ for all $k < n$, covering whatever value a takes.

Well-Ordering Principle

Every nonempty subset of $\mathbb{N}$ has a least element.

Remark 5.4.19 (The minimal counterexample argument). Well-ordering drives a proof strategy that is the structural dual of induction: the **minimal counterexample argument**.

Instead of climbing up from a base case, you assume the claim fails somewhere, take the smallest failure, and derive a contradiction from its minimality. The logic:

1. Suppose $P(n)$ fails for some n. Let $S = \{n \in \mathbb{N} : \neg P(n)\}$. By assumption $S \neq \emptyset$.
2. By well-ordering, S has a least element m.

3. Derive a contradiction: either show $P(m)$ holds (contradicting $m \in S$), or find some $k < m$ with $\neg P(k)$ (contradicting the minimality of m).

The key tool is minimality: the fact that m is the *smallest* failure means all smaller values are successes, which is a powerful assumption. It is equivalent to the strong inductive hypothesis.

Proposition 5.4.20 (Induction and well-ordering are equivalent). *Over the axioms of $\mathbb{N}$ without P5, the following three principles are logically equivalent:*

1. *The (weak) induction principle.*
2. *The well-ordering principle.*
3. *The strong induction principle.*

Remark 5.4.21 (Proof sketch of the equivalence). **Induction $\Rightarrow$ Well-ordering.** Suppose $S \subseteq \mathbb{N}$ has no least element. Let $P(n)$: "$n \notin S$." Then $P(0)$ holds (otherwise $0 \in S$ would be the least element). If $P(k)$ holds for all $k \leq n$, then $n+1 \notin S$ (otherwise $n+1$ would be the least element of S). By induction, $P(n)$ holds for all n, so $S = \emptyset$.

Well-ordering $\Rightarrow$ Induction. Suppose the base case $P(n_0)$ holds and the inductive step holds. Let $S = \{n \geq n_0 : \neg P(n)\}$. If $S \neq \emptyset$, let m be its least element. Since $P(n_0)$ holds, $m \neq n_0$, so $m > n_0$ and $m - 1 \geq n_0$. By minimality of m, $P(m-1)$ holds. But by the inductive step, $P(m-1) \Rightarrow P(m)$, contradicting $m \in S$. Hence $S = \emptyset$.

Template: Minimal Counterexample

Suppose for contradiction that $P(n)$ fails for some $n \in \mathbb{N}$.
Let $S = \{n \in \mathbb{N} : \neg P(n)\}$. By assumption $S \neq \emptyset$. By well-ordering, S has a least element m.
[*Derive a contradiction using minimality of m: either show $P(m)$ holds, or construct a smaller counterexample.*]
This contradicts [minimality of m / the base case]. Therefore $P(n)$ holds for all $n \in \mathbb{N}$. □

Example 5.4.22 (Minimal counterexample: exponential bound). **Claim.** For all $n \geq 1$, $2^n > n$.

Proof (minimal counterexample). Suppose the claim fails. Let $S = \{n \geq 1 : 2^n \leq n\}$. By assumption $S \neq \emptyset$; let m be its least element.

Since $2^1 = 2 > 1$, we have $m \neq 1$, so $m \geq 2$. Then $m-1 \geq 1$ and $m-1 < m$, so $m - 1 \notin S$ (by minimality of m). Thus $2^{m-1} > m - 1$.

Then:

$$2^m = 2 \cdot 2^{m-1} > 2(m-1) = 2m - 2 \geq m$$

(using $m \geq 2$ for the last inequality).

This contradicts $m \in S$ (which requires $2^m \leq m$). Therefore $S = \emptyset$ and the claim holds for all $n \geq 1$. □

The minimal counterexample argument proceeds by taking the smallest failure and showing it cannot be the smallest failure. The contradiction is always of the same form: either the "failure" was not a failure, or there is a smaller one.

Structural Induction (Preview). Induction on $\mathbb{N}$ is a special case of a general principle that applies to any recursively defined structure. You do not need $\mathbb{N}$ to do induction; you need a recursive definition.

Remark 5.4.23 (The general principle). Let X be a set defined by:

1. A collection of *base cases* (atomic elements not built from other elements), and
2. A collection of *constructor rules* (ways of building larger elements from smaller ones).

Structural induction on X: to prove $P(x)$ for all $x \in X$, prove P for all base cases, then prove that if P holds for the parts, it holds for the whole.

This is exactly standard induction where the "less than" ordering on $\mathbb{N}$ is replaced by the "proper sub-element" ordering on X.

Example 5.4.24 (Induction on algebraic terms). Define an *algebraic term* over a ring R by:

- Base case: any element $r \in R$ is a term.
- Constructor: if s, t are terms, so are $s + t$ and $s \cdot t$.

To prove a property P holds for all terms:

1. Prove $P(r)$ for all $r \in R$ (base cases).
2. Prove: if $P(s)$ and $P(t)$ hold, then $P(s + t)$ holds.
3. Prove: if $P(s)$ and $P(t)$ hold, then $P(s \cdot t)$ holds.

The recursive structure of terms determines the structure of the proof.

Remark 5.4.25 (Where structural induction appears in this project). Induction on $\mathbb{N}$ is structural induction on the Peano structure $(\mathbb{N}, 0, S)$: the base case is $n = 0$, the constructor is $n \mapsto n{+}{+}$.

In the Abstract Algebra chapter, polynomial properties are proved by induction on degree, which is structural induction on the recursive definition of polynomials over a ring.

In computer science, correctness of recursive algorithms is proved by structural induction on the input data structure (lists, trees, expressions). The full treatment of structural induction appears in the Abstract Algebra chapter.

Common Induction Mistakes.

Remark 5.4.26 (Mistake 1: Missing base case). A proof that establishes the inductive step but not the base case proves nothing. The step says: *if* $P(n)$ holds for some n, *then* $P(n{+}1)$ holds. Without a base case, there is no n for which $P(n)$ is established, so the chain never starts.

The classic illustration: the false theorem "all horses are the same colour." The base case for $n = 1$ is trivially true (one horse has one colour). The inductive step has a subtle flaw in the overlap argument. The lesson is that both parts are essential and independent. Always verify the base case, even when it seems trivial. A trivial base case takes one line; write the one line.

Remark 5.4.27 (Mistake 2: Circular inductive step). The inductive step proves $P(n+1)$ using only $P(n)$ and previously established facts. It is a fatal error to assume $P(n+1)$ at any point in the proof of $P(n+1)$.

This error most often appears when a student "derives" both sides of an equation and meets in the middle, inadvertently assuming what they are trying to prove. The symptom: the final few lines of the step are not deductions from $P(n)$ but rather manipulations of $P(n+1)$ itself.

The cure: always work in one direction. Start with the left side of $P(n+1)$ (or whatever the goal is), apply the inductive hypothesis once, and algebraically arrive at the right side. Never write the target as an intermediate line and work toward it from both sides.

Remark 5.4.28 (Mistake 3: Unstated inductive hypothesis). The most common error in practice is writing "assume $P(n)$" without saying what $P(n)$ actually says. This is a sign that $P(n)$ has not been written out as a full sentence.

The effect is that the inductive step becomes vague at the crucial moment: the student needs to use the inductive hypothesis but cannot cite it precisely, so they either paraphrase it loosely (which may introduce errors) or implicitly use a stronger version than what was stated.

Write $P(n)$ as a complete sentence at the top of the proof. Then write "assume $P(n)$: [full sentence]." This forces precision and prevents the most common errors in the body of the proof.

Remark 5.4.29 (Mistake 4: Wrong base case). Induction proves $P(n)$ for all $n \geq n_0$, where n_0 is whatever value you use as the base case. If the claim is only true for $n \geq 2$, the base case $n = 0$ will fail, and the proof is invalid.

Always check: what is the smallest n for which the claim is true? That is your base case. Do not default to $n = 0$ or $n = 1$ without verifying that the claim holds there. For divisibility statements, $n = 0$ often gives $0 = 0 \cdot k$, which works. For growth bounds, $n = 1$ or $n = 2$ may be needed. For Fibonacci identities, specific small values may require individual checking.

Remark 5.4.30 (Mistake 5: Using strong induction where weak suffices, and vice versa). Strong induction is not automatically better than weak. A proof that invokes the full strong hypothesis "$P(k)$ for all $k < n$" when only $P(n-1)$ is actually used is unnecessarily complicated and obscures the proof's logic.

Conversely, a proof that tries to use weak induction on a recursion requiring two prior terms will stall at the step.

The diagnostic: write the step and check explicitly which prior values you use. If only $P(n)$, use weak. If $P(k)$ for some $k < n$ that is not $n-1$, use strong.

Remark 5.4.31 (Mistake 6: Forgetting vacuity in strong induction). In strong induction, the case $n = 0$ has a vacuously true hypothesis $\forall k < 0,\ P(k)$, since no natural number is less than 0. This means $P(0)$ must be proved *directly*, not from the inductive hypothesis. Many students write "by the inductive hypothesis applied to $n-1$" for $n = 0$, which refers to $n - 1 = -1$, a non-existent value.

The base case in strong induction is always proved from scratch. The strong hypothesis is only available for $n \geq 1$.

5.5 Algebraic Tactics

The DU/TA/AM Framework. Every line of an algebraic proof is justified by exactly one of three sources. This framework makes that obligation explicit and provides the tagging system used throughout these notes.

The Three Line Tags

- DU **(Definition / Unpack).** The step applies a definition or unpacks a named concept into its explicit conditions. *Example*: expanding "e is an identity" into "$ea = ae = a$ for all $a \in G$".
- TA **(Theorem / Axiom).** The step cites a previously proved proposition or a structural axiom (group axiom, ring axiom, field axiom, Peano axiom). *Example*: citing G1 (associativity) to regroup $(ab)c = a(bc)$.
- TA **(Algebraic Manipulation, abbreviated AM).** The step performs a computation whose justification is itself a combination of DU and TA moves, compressed for readability. *Example*: simplifying $a \cdot e = a$ in one step after the identity axiom has already been cited.

Remark 5.5.1 (Why the framework matters: finding where proofs break). The DU/TA/AM system serves a diagnostic purpose. Every unjustified step in a proof is either a DU step that has not been taken (some definition has not been unpacked) or a TA step that has not been cited (some axiom or theorem is being used silently).

When an algebraic proof stalls, the right question is: which definition have I not yet unpacked? Which axiom am I using but not citing? The tags make these omissions visible.

In practice, stalling almost always traces to a definition that has been left in its named form ("G is a group") when it needed to be expanded into operative conditions ("associativity holds; an identity exists; every element has an inverse"). Expand the definition and the stuck proof typically becomes unstuck.

Remark 5.5.2 (The hierarchy: DU and TA are atomic; AM is shorthand). DU and TA are the atomic justifications. There is nothing more basic. AM is not a separate category of reasoning -- it is a compressed sequence of DU and TA steps that have been verified and labeled as a single unit for concision.

A fully formal proof contains only DU and TA steps, explicitly tagged. The three-column proof format used in this project -- tag in the left column, statement in the middle, commentary in the right -- enforces explicit justification of every line and is the recommended format while you are learning.

As fluency develops, AM steps can absorb more and more of the lower-level manipulations. But the underlying DU/TA structure is always present, even when not written. The ability to decompose any AM step into its DU and TA components is the hallmark of genuine understanding.

Remark 5.5.3 (A proof is a chain of citations). The deepest way to understand the DU/TA/AM system is this: a mathematical proof is a sequence of statements, each of which is justified by citing something that is already known (a definition, an axiom, or a previously proved theorem). The proof is valid if and only if every step has a valid citation.

When you read a proof and feel uncertain whether it is correct, the right move is to label every step. If you can assign a DU or TA justification to every line, the proof is valid. If any line has no justification, there is a gap.

This is not just a learning technique. Working mathematicians who find a suspicious step in a proof do exactly this: they ask "what justifies this?" and try to answer. If no answer is available, the step is an error.

Catalog of Algebraic Tactics. The following catalog lists the specific algebraic moves available once a proof is in progress and the architecture has been chosen. Each entry states the tactic, its precondition (what must already be established or assumed), and its effect (what it achieves in the proof).

Tactic	Precondition	Effect
Multiply by inverse	$a \neq 0$ in a field; or $a \in G$ in a group	Multiply both sides by a^{-1}; use associativity to collapse $a^{-1}a = e$; isolates the target variable.
Cancellation	$ac = bc$ (or $ca = cb$) in a group or integral domain	Conclude $a = b$. In a group: multiply by c^{-1}. In an integral domain: cite no-zero-divisors.
Distributivity bridge	Multiplication by 0, -1, or a negative element	Rewrite the product as a sum using distributivity, then use additive group structure to collapse. Crosses from $\times$ to $+$.
Add zero	Need to introduce a term without changing the expression	Write $a = a+0 = a+(b+(-b))$; regroup to expose the needed structure.
Satisfy-and-cite	A uniqueness theorem for some named object b is in scope	Show a satisfies b's defining property; cite uniqueness; conclude $a = b$. Avoids assume-two-and-compare entirely.
Double negation	$-(-a)$ appears or is the goal	$-a$ is the unique element x with $a + x = 0$; $-(-a)$ satisfies this; by uniqueness, $-(-a) = a$.
Absorb into axiom	A subexpression matches the LHS of an axiom	Rewrite using the axiom. Tag as TA.

Remark 5.5.4 (The distributivity bridge in detail). The distributivity bridge is the central tactic for ring and field proofs involving zero and negatives. Every instance of "$a \cdot 0 = 0$" and "$a(-b) = -(ab)$" uses it. Once you have seen the pattern, it is immediately recognizable.

The pattern always has four steps:

1. Introduce $a \cdot 0$ or $a \cdot (-b)$ into the argument.
2. Expand: write $0 = x + (-x)$ or $-b = b + (-b) + (-b)$ using the additive inverse definition.
3. Apply distributivity: $a(x + y) = ax + ay$.
4. Use uniqueness of the additive inverse to identify the result with $-(ab)$ or 0.

The reason this requires distributivity as a bridge is that the additive structure (with its identity 0 and its inverses $-a$) and the multiplicative structure (with its products ab) are separate. The only connection between them is distributivity. To get from the multiplicative side ($a \cdot 0$) to the additive side ($0 = 0 + 0$), you must cross the bridge.

Remark 5.5.5 (Cancellation: two theorems that must not be conflated). "Cancellation" names two distinct results with different preconditions.

Group cancellation. In any group, $ac = bc \Rightarrow a = b$. Proved by multiplying both sides on the right by c^{-1}, which always exists in a group. The hypothesis $c \neq 0$ is not needed because inverses always exist in a group.

Domain cancellation (or integral domain cancellation). In an integral domain, $ac = bc$ and $c \neq 0$ imply $a = b$. Proved by rewriting as $(a-b)c = 0$ and invoking the no-zero-divisors property. The hypothesis $c \neq 0$ is essential: without it, cancellation can fail (e.g., in $\mathbb{Z}_6$, $2 \cdot 3 = 4 \cdot 3$ but $2 \neq 4$).

The error to avoid: using group cancellation in an integral domain argument (it works but uses stronger machinery than needed and may not be available), or trying to use domain cancellation without verifying $c \neq 0$.

Remark 5.5.6 (Add zero: the simplest but most overlooked tactic). The add-zero tactic sounds trivial but appears in almost every algebraic proof where an intermediate term needs to be introduced without changing the value of an expression. The template is:

$$a = a + 0 = a + (b + (-b)) = (a + b) + (-b).$$

This exposes $a+b$ as a subexpression of a, which can then be worked with. It is the algebraic equivalent of adding and subtracting the same term in a real analysis inequality argument.

When you need a term to appear in an expression and cannot see how to introduce it, ask: can I write some zero as $x + (-x)$ and distribute?

5.6 Proof Strategies Reference

Lookup Table: Stuck Situations and Strategies.

Remark 5.6.1 (How to use this table). When progress on a proof stalls, locate the row that describes your situation. The middle column names the strategy. The right column gives the specific first move and a cross-reference to the section where that strategy is developed in detail.

This table is not a substitute for understanding the strategies; it is a navigation aid for when you know what you are trying to do but have forgotten the precise machinery. The full explanations and worked examples are in the sections cited.

Stuck because...	**Strategy**	**First move / See**
I need to prove P but don't know where to start	Two-question check	Write the logical form of P. Consult the statement map. §5.1
I need to show two things are equal and one is uniquely defined	Satisfy-and-cite	Show the first satisfies the definition of the second. Cite the uniqueness theorem. §5.1
I need to show something is the only object of its kind	Assume-two-and-compare	"Let x, y both satisfy the definition. Show $x = y$." §5.3
I have a conditional $P \Rightarrow Q$ and P is hard to use forward	Contrapositive	"Assume $\neg Q$." Derive $\neg P$. §5.3
I have a conditional and the conclusion seems impossible to reach directly	Contradiction	"Assume P and $\neg Q$." Derive any contradiction. §5.3
The claim splits naturally into a finite number of cases	Case analysis	List all cases explicitly. Verify exhaustiveness first. §5.3
The claim is about all $n \in \mathbb{N}$	Weak induction	Write $P(n)$ as a full sentence. Prove the base case directly. §5.4
The inductive step needs $P(k)$ for some $k < n$ (not just $n - 1$)	Strong induction	Replace "assume $P(n)$" with "assume $P(k)$ for all $k < n$." §5.4
The claim is about all $n \geq n_0$ and feels easier to negate	Minimal counterexample	"Let $S = \{n : \neg P(n)\} \neq \emptyset$. Let $m = \min S$." §5.4
I need to prove $a \cdot 0 = 0$ or $a(-b) = -(ab)$ in a ring	Distributivity bridge	Write $0 = x + (-x)$; apply distributivity; use uniqueness of additive inverse. §5.5
I have $ac = bc$ and need $a = b$	Cancellation	Group: multiply by c^{-1}. Domain: write $(a - b)c = 0$; cite no zero divisors. §5.5
I need to prove two sets are equal	Double inclusion	Prove $A \subseteq B$: let $x \in A$ arbitrary, derive $x \in B$. Then prove $B \subseteq A$. §5.1
A definition has not been unpacked	DU move	Identify the named concept and write out its definition. Tag the line DU. §5.5
A step uses an axiom or theorem by name	TA move	Cite the axiom or theorem explicitly by label or full name. Tag the line TA. §5.5

Remark 5.6.2 (Cross-references to primary worked examples). The following table maps each strategy to a worked example in these notes where it is the primary tool.

Strategy	Primary worked examples
Assume-two-and-compare	Uniqueness of group identity (this section)
Satisfy-and-cite	$(ab)^{-1} = b^{-1}a^{-1}$; $a(-b) = -(ab)$ (this section)
Distributivity bridge	$a \cdot 0 = 0$; $a(-b) = -(ab)$ (Tactics section)
Cancellation (group)	Group left-cancellation (Tactics section)
Cancellation (domain)	Integral domain cancellation (Tactics section)
Weak induction	Sum formula $\sum k = n(n+1)/2$; $3 \mid 4^n - 1$
Strong induction	Fibonacci bound; prime factor theorem
Minimal counterexample	$2^n > n$
Contrapositive	n^2 even $\Rightarrow$ n even
Contradiction	$\sqrt{2} \notin \mathbb{Q}$

5.7 Analysis: What Changes and Why

Why Analysis Proofs Feel Different. Everything in the preceding sections applies to mathematics broadly: direct proof, induction, contradiction, and the rest are the universal grammar of mathematical argument. But real analysis — sequences, metric spaces, limits, continuity — introduces a particular *flavour* of proof work that surprises many students when they first encounter it.

This section prepares you for that surprise before you arrive at Volume III. The goal is not to teach you analysis; it is to give you the framework so that when you get there, the proofs do not feel alien.

Remark 5.7.1 (The shift from computation to verification). In calculus, most tasks are *computational*: differentiate this function, evaluate this integral, find this limit by l'Hôpital's rule. You apply known procedures and the answer is a number or a function.

In analysis, most tasks are *verificational*: prove that this sequence converges, show that this space is complete, establish that this function is continuous. The answer is a logical argument. There is no procedure to apply; there is a structure to find.

This shift is not just psychological. It changes what "progress" means. In a computation, you know you are making progress when the expressions simplify. In a proof, you know you are making progress when the definitions are unpacked, the hypotheses are applied, and the conclusion comes into reach. The habits of mind required are different.

Remark 5.7.2 (Definitions are not background; they are the machinery). The single most important habit in analysis is **returning to definitions**. Every concept -- convergence, Cauchy, compactness, continuity -- has a precise definition in terms of quantifiers and inequalities. That definition is not background knowledge. It is the working machinery of every proof involving that concept.

Here is the canonical example. To prove that $1/n \to 0$, a calculus student says: "obviously it approaches zero." An analysis student unpacks the definition:

For every $\varepsilon > 0$*, there exists* $N \in \mathbb{N}$ *such that* $n \geq N$ *implies* $|1/n - 0| < \varepsilon$*.*

Then they find N: choose $N > 1/\varepsilon$ (possible by the Archimedean property). For $n \geq N$: $1/n \leq 1/N < \varepsilon$.

The proof is three lines, and every line is either the definition (unpacked), the Archimedean property (a theorem), or arithmetic. There is no mystery. The entire proof is the definition plus two lines of calculation. This is the prototype of an analysis proof.

The lesson: when a proof stalls, the first question to ask is always: *which definition have I not yet unpacked?*

Remark 5.7.3 (Analysis proofs are structured, not creative). Here is perhaps the most liberating thing to understand about analysis: **proofs are far less creative than they look.**

Experienced analysts do not invent proofs from scratch. They recognize the type of exercise, recall the standard move for that type, and apply it. The vocabulary of standard moves is small -- twelve moves cover virtually all of the proofs in a first course in metric spaces. The creativity, if any, is in recognizing which move to apply.

This is the same observation that governs the Architecture section of this chapter (Section 5.1), scaled down to the specific landscape of analysis. The statement map tells you the global structure. In analysis, a more fine-grained map is available: there are specific named moves for specific recurring situations. Learning to name them is the same skill as learning to read a statement and immediately identify its proof type.

The rest of this section makes that vocabulary explicit.

The Six Exercise Types. Every exercise in sequences and metric spaces belongs to one of six structural types. Identifying the type is the first step before writing anything. The type determines the shape of the solution — not the details, but the overall structure.

Quick Reference: The Six Types

Type	Signal words	Shape	First move
Proof	*Prove, show, establish*	Hypotheses to conclusion	Unpack all definitions
Verification	*Verify that ... is a metric, show ... satisfies*	Check each condition	List every condition
Prove/Counterexample	*True? Prove or find a counterexample*	Decide, then justify	Decide first
Construction	*Find, construct, exhibit, give an example of*	Name the object; verify it	Name the object
Computation	*Compute, calculate, find the diameter of*	Evaluate from a definition	Write the definition
Characterization	*If and only if, characterize all ... that*	Two separate directions	Split two directions

Remark 5.7.4 (Type 1 (Proof): you are told the statement is true). In a proof exercise, the statement is true and your job is to trace the logical path from the hypotheses to the conclusion. There is no choice to make about truth value.

The opening move is always the same: unpack every definition in the hypotheses and in the conclusion. The definitions turn abstract concepts into concrete quantifier-and-inequality statements that can be manipulated. Without this step, you have nothing to work with.

Example. *Prove that every convergent sequence is Cauchy.* Unpack "convergent": $\forall \varepsilon > 0\ \exists N$ s.t. $n \geq N \Rightarrow d(x_n, p) < \varepsilon$. Unpack "Cauchy": $\forall \varepsilon > 0\ \exists M$ s.t. $m, n \geq M \Rightarrow d(x_m, x_n) < \varepsilon$. Now the proof goal is visible: use the convergence condition (which controls $d(x_n, p)$) to establish the Cauchy condition (which asks for control over $d(x_m, x_n)$). The triangle inequality bridges them.

Remark 5.7.5 (Type 2 (Verification): check every condition). In a verification exercise, a specific named object is given and you must confirm it satisfies every condition of a definition. The definition has a finite list of conditions. Your job is to work through every one, explicitly and separately.

The error students make: checking four out of five conditions and writing QED. A verification is complete only when every condition is checked. When you begin a verification, write the complete list of conditions first. Then work through them in order.

Example. *Verify that the discrete metric is a metric.* A metric has four conditions: non-negativity, identity of indiscernibles, symmetry, triangle inequality. List all four. Check each one. The triangle inequality requires a case split (on whether $x = z$ or $x \neq z$), which is itself a standard move in analysis.

Remark 5.7.6 (Type 3 (Prove or counterexample): decide before you argue). This is the most dangerous type for students who rush. The exercise gives you a statement and asks you to determine its truth value, then justify. If you skip the determination stage and try to prove a false statement, you will waste effort and grow confused.

Stage 1 is always: decide. Test the statement on simple cases -- $\mathbb{R}$ with the standard metric first, then discrete spaces, then other special cases. One counterexample falsifies; failure to find a counterexample after testing several cases suggests the statement is true.

Stage 2: justify. If true, write a proof. If false, exhibit a specific counterexample and verify it fails.

Example. *Is the open ball always equal to the closed ball?* Test in $\mathbb{R}$: $B(0,1)=(-1,1)$ and $C(0,1)=[-1,1]$. The point -1 is in $C(0,1)$ but not $B(0,1)$. False. Exhibit this specific counterexample in the solution, with the witness (-1) and the calculation ($d(-1,0)=1$, so $-1\in C(0,1)$ and $-1\notin B(0,1)$).

Remark 5.7.7 (Type 4 (Construction): name first, verify second). A construction exercise asks you to produce an object with given properties. The solution has two parts: stating the object, and verifying it has every required property. Both parts are required.

State the object completely and unambiguously before any verification. If you cannot state it clearly, you do not yet have a solution. Then verify every required property from scratch -- do not assume properties that were not established.

Example. *Find a metric on $\mathbb{N}$ such that $\mathbb{N}$ is bounded.* The idea: embed $\mathbb{N}$ into $(0,1]\subset\mathbb{R}$ via $n\mapsto 1/n$ and pull back the metric. Define $d(m,n)=|1/m-1/n|$. Then verify: (i) it is a metric (four conditions), and (ii) it is bounded ($d(m,n)\leq 1$ for all m,n).

Remark 5.7.8 (Type 5 (Computation): the definition is the answer). In a computation exercise, a specific numerical or set-theoretic value is asked for. The answer always comes from writing out the relevant definition as a supremum, infimum, or set, and then evaluating it.

Do not skip straight to the answer. Write the definition first. This is not just for clarity -- it prevents errors that arise from misremembering what is being computed.

Example. *Compute* $\operatorname{diam}([a,b])$. Write: $\operatorname{diam}([a,b])=\sup\{|x-y| : x,y\in[a,b]\}$. Upper bound: $|x-y|\leq b-a$ for all $x,y\in[a,b]$. Attained: $|b-a|=b-a$. Therefore $\operatorname{diam}([a,b])=b-a$.

Remark 5.7.9 (Type 6 (Characterization): two proofs, not one). A characterization exercise asks for an equivalence or an if-and-only-if. The solution is always two separate proofs, labeled $(\Rightarrow)$ and $(\Leftarrow)$. Label them before writing a single line.

The two directions often require completely different techniques and different amounts of work. The $(\Rightarrow)$ direction may be three lines while the $(\Leftarrow)$ direction requires a sequence argument. Writing them separately prevents the error of running arguments in both directions simultaneously and losing track of which assumptions are in play.

Example. *Show that a sequence converges if and only if it is eventually constant (in the discrete metric).* $(\Leftarrow)$: Suppose eventually constant; then convergence holds trivially. $(\Rightarrow)$: Suppose it converges; apply the definition with $\varepsilon=1/2$ and use the fact that distances in the discrete metric are either 0 or 1.

The Twelve Canonical Proof Moves. A *proof move* is a recurring logical or computational technique that appears across many different proofs in analysis. Once you learn to name these moves, reading and writing proofs becomes a matter of recognizing which move applies — not inventing an argument from scratch.

The moves below are not tricks. Each one encodes a genuine mathematical idea. But once the idea is understood, the move can be deployed almost mechanically in the right situations.

The Twelve Moves at a Glance

#	Move	Use when ...	Level
1	Unpack a definition	starting any proof	L1
2	Triangle inequality	need to bound $d(x, z)$ via y	L2
3	ε-splitting	triangle ineq. produces a sum	L2
4	Tail argument	combining two N's	L3
5	Case splitting	discrete or binary structure	L2
6	Choose N explicitly	ε-N arguments	L3
7	Contradiction	uniqueness or non-existence	L4
8	Subsequence extraction	sequence misbehaves globally	L3
9	Squeeze (sandwich)	bound $d(x_n, p)$ from above	L2
10	Inherited metric / embedding	constructing a new metric	L4
11	Open cover / finite subcover	compactness arguments	L4
12	Diagonal argument	simultaneous subsequence conditions	L3/4

Remark 5.7.10 (Move 1: Unpack a definition). This is always the first move. Every analysis concept -- convergence, Cauchy, compactness, closure, diameter -- is a compressed form of a precise statement with quantifiers and inequalities. Unpacking the definition turns the abstract concept into something you can work with.

Without this step, you have named objects but no machinery. With it, you have the raw material for every subsequent move.

Template. Write out the definition of every term in the hypothesis and in the goal. The hypotheses become your assumptions; the goal becomes your target inequality or quantifier statement.

Remark 5.7.11 (Moves 2 and 3: Triangle inequality and ε-splitting). These two moves almost always appear together. The triangle inequality lets you insert an intermediate point y and split $d(x,z)$ into $d(x,y)+d(y,z)$. ε-splitting means choosing the bounds on each piece so they add up to ε.

The intermediate point y is not arbitrary: it is chosen to be something you already have control over (a limit, a fixed center, a sequence element). The fractions $\varepsilon/2$, $\varepsilon/3$ are chosen *before* picking N, so that the final bound comes out to ε exactly.

Template. When you need $d(x_m, x_n) < \varepsilon$ and you know each term is close to a limit p: insert p via the triangle inequality, budget $\varepsilon/2$ for each piece, and choose N so both pieces are bounded.

Remark 5.7.12 (Moves 4 and 6: Tail argument and choosing N). The ε-N definition of convergence requires you to *produce* an N. This is Move 6: work backwards from the target inequality to determine what N must satisfy, then assert its existence (typically by the Archimedean property: for any $\varepsilon > 0$ there is an integer greater than $1/\varepsilon$).

Move 4 (the tail argument) handles the situation where two separate conditions each give their own N_i. The resolution: take $N = \max(N_1, N_2, \ldots)$. For $n \geq N$, all conditions hold simultaneously.

These two moves are the engine of every ε-N argument. They appear in virtually every proof about convergent sequences.

Remark 5.7.13 (Move 5: Case splitting in analysis). Case splitting is the same structural technique described in Section 5.3, now applied to the specific context of metric spaces. The most common case split in metric space proofs is: $x = y$ (distance is 0) versus $x \neq y$ (distance is 1 in the discrete metric, or some positive value in other spaces).

Case splitting is particularly important in verification exercises: the triangle inequality almost always requires splitting on whether the two outer points coincide.

Remark 5.7.14 (Move 7: Contradiction in analysis). In analysis, contradiction is used most characteristically for *uniqueness* results: suppose two limits exist; assume they are different; derive a contradiction from the triangle inequality and the convergence hypothesis.

The template: suppose the goal fails (e.g., two limits exist with $d(x, y) > 0$). Choose $\varepsilon = d(x, y)/2$. Then for large enough n, both $d(x_n, x) < \varepsilon$ and $d(x_n, y) < \varepsilon$. The triangle inequality gives $d(x, y) \leq 2\varepsilon = d(x, y)$, a contradiction.

Notice: contradiction here uses the triangle inequality (Move 2) and the tail argument (Move 4) internally. Proof moves nest and combine.

Remark 5.7.15 (Moves 8 and 12: Subsequence extraction and diagonal argument). Subsequence extraction is the operation of selecting indices $n_1 < n_2 < \cdots$ so that the restricted sequence $\{x_{n_k}\}$ has some desired property (being monotone, being close to a given point, etc.). It is a Level 3 operation: you are selecting from an infinite sequence, which requires genuine sequence machinery.

The diagonal argument applies subsequence extraction infinitely many times and diagonalizes the result: the k-th term of the diagonal sequence is $x_k^{(k)}$, where $\{x_n^{(k)}\}$ is the k-th extracted subsequence. The diagonal sequence satisfies all conditions simultaneously. This is the technique behind Arzelà-Ascoli and related compactness results.

Remark 5.7.16 (Move 9: The squeeze (sandwich)). If you can show $0 \leq d(x_n, p) \leq c_n$ where $c_n \to 0$, then $x_n \to p$ follows immediately: any ε that works for c_n also works for $d(x_n, p)$.

The squeeze is particularly useful when $d(x_n, p)$ is hard to bound directly but can be dominated by a simpler sequence whose convergence is known.

Remark 5.7.17 (Move 10: Inherited metric / embedding). To construct a metric on a new space X, it is often easiest to find an injective function $f : X \to Y$ into a space Y whose metric is known, and define $d_X(x, z) := d_Y(f(x), f(z))$. All four metric axioms are then inherited from d_Y via f. The only things to verify are injectivity of f (to ensure the identity axiom) and whatever additional properties are required.

This is a Level 4 move because it operates on the global structure of the space: you are choosing an ambient space and an embedding, not just manipulating inequalities.

Remark 5.7.18 (Move 11: Open cover / finite subcover). Compactness says: every open cover of a compact set K has a finite subcover. The power of this is that it converts infinite problems into finite ones. If you need a single bound δ that works everywhere on K, you find a δ_x at each x (infinitely many), then extract a finite subcover and take the minimum of finitely many δ_x. A finite minimum exists; a minimum over infinitely many values might not.

This move appears in all compactness arguments: uniform continuity on compact sets, compactness implies boundedness, compactness implies completeness in complete metric spaces.

The Four-Layer Hierarchy. The twelve moves are not all at the same level. They form a four-layer hierarchy that reflects the logical structure of metric space proofs: each layer depends on the one below it, and together the four layers describe a progression from the most basic (unpacking definitions) to the most structural (global topological reasoning).

Understanding the hierarchy is not an organizational convenience. It tells you something deep: *why* metric spaces are defined the way they are, and why nearly every proof in the subject follows the same basic upward shape. Once you see this, metric space theory stops looking like a collection of unrelated tricks and starts looking like a small, elegant machine built from a few core mechanisms.

The Four Levels

L	Name	Moves	What it does
L1	Definition Expansion	Move 1	Turns abstract concepts into inequalities
L2	Inequality Mechanics	Moves 2, 3, 5, 9	Combines and bounds distances
L3	Sequence Control	Moves 4, 6, 8, 12	Controls asymptotic and tail behavior
L4	Structural / Topological	Moves 7, 10, 11	Reasons about the space as a whole

Remark 5.7.19 (Level 1: Definition Expansion -- the foundation of everything). Every other move operates on inequalities. Those inequalities do not appear until you unpack a definition. Before Level 1, you have a statement about abstract objects. After Level 1, you have quantifiers and specific inequality conditions that can be manipulated.

This is why Level 1 is always the first step in any analysis proof, regardless of how complex the argument becomes. The inequality $d(x_n, p) < \varepsilon$ does not exist in the proof until you write it down. You write it down by unpacking the definition of convergence.

What Level 1 produces:

- "$x_n \to p$" $\mapsto$ $\forall \varepsilon > 0\, \exists N$ s.t. $n \geq N \Rightarrow d(x_n, p) < \varepsilon$
- "$\{x_n\}$ is Cauchy" $\mapsto$ $\forall \varepsilon > 0\, \exists M$ s.t. $m, n \geq M \Rightarrow d(x_m, x_n) < \varepsilon$
- "K is compact" $\mapsto$ every open cover of K has a finite subcover
- "E is closed" $\mapsto$ every convergent sequence in E converges to a point of E

The abstract word is a compressed form of the inequality. Unpacking it is not simplification; it is *activation*.

Remark 5.7.20 (Level 2: Inequality Mechanics -- the algebra of metric spaces). Once Level 1 has produced inequalities, Level 2 manipulates them. This is the algebra of metric spaces: you combine, bound, split, and chain inequalities to move from what you know toward what you need.

The triangle inequality is a *metric axiom* for a reason: it was included in the definition of a metric precisely because it is the tool needed to chain distance bounds. Without it, you cannot get from control over $d(x, y)$ and $d(y, z)$ to control over $d(x, z)$. The axiom is there because it enables Level 2.

Notice that Level 2 does not involve sequences, indices, or tails. It is purely about bounding distances between specific points. The sequence machinery comes at the next level.

Remark 5.7.21 (Level 3: Sequence Control -- the engine of limit arguments). Analysis is fundamentally about sequences approaching limits, distances becoming arbitrarily small, behavior stabilizing as an index grows large. Level 3 controls this asymptotic behavior.

The key idea: sequence control is about managing the word "eventually." The definition of convergence says that for large enough n, the distance $d(x_n, p)$ is small. The tail argument combines multiple "large enough n" conditions by taking the maximum index. Choosing N explicitly produces the specific threshold. Subsequence extraction selects which terms of the sequence to keep.

These moves require genuine sequence machinery -- you are reasoning about indexed infinite processes, not just about individual distances between points. That is what places them at Level 3 rather than Level 2.

Remark 5.7.22 (Level 4: Structural / Topological -- global reasoning). Level 4 moves operate on the global structure of the metric space, not on individual sequences or inequalities. Contradiction reasons about the logical structure of the whole proof. Embedding reasons about how the space sits inside a larger ambient space. Compactness reasons about the global covering properties of the space.

Compactness is the flagship Level 4 property. It converts infinite problems into finite ones: an infinite family of open sets reduces to a finite subcover, and a finite collection has a minimum, a maximum, a single controlling constant. This is the most powerful tool in metric space theory.

Level 4 and Level 3 interact bidirectionally. Sequential compactness (every sequence has a convergent subsequence) is a Level 3 characterization of a Level 4 property. Many deep results in analysis are precisely about these interactions.

Remark 5.7.23 (Reading a proof using the hierarchy). When you read a proof in the textbook, label each step by level. This is not a mechanical exercise; it reveals the logical architecture and tells you why each step appears where it does.

Here is the proof that convergent implies Cauchy, labeled:

Let $\varepsilon > 0$. *[L1: introduce ε from the Cauchy definition]*

Since $x_n \to p$, there exists N such that $n \geq N \Rightarrow d(x_n, p) < \varepsilon/2$. *[L3: choose N from the convergence hypothesis]*

For $m, n \geq N$: *[L3: tail condition]*

$d(x_m, x_n) \leq d(x_m, p) + d(p, x_n)$ *[L2: triangle inequality]*

$< \varepsilon/2 + \varepsilon/2 = \varepsilon$. *[L2: ε-splitting]*

Hence $\{x_n\}$ is Cauchy. *[L1: conclusion matches Cauchy definition]*

This proof uses Levels 1, 2, and 3, but not Level 4. It is a local argument about sequences, not a global structural argument. To reach Level 4, you would need compactness or an embedding argument.

The levels a proof reaches tell you its depth and what tools it uses. A verification exercise stays at Levels 1-2. A limit argument reaches Level 3. A compactness argument reaches Level 4.

Remark 5.7.24 (Why the metric axioms are the axioms they are). The metric axioms -- non-negativity, identity of indiscernibles, symmetry, triangle inequality -- are not arbitrary. They are exactly the conditions needed to make Level 2 possible.

Without the triangle inequality, you cannot chain distance bounds. Without identity of indiscernibles, the notion of a limit point degenerates. Without symmetry, the definitions of balls and convergence become asymmetric and unworkable.

The axioms are the *minimal conditions on a distance function that support the full four-level proof hierarchy.* When you study a new property of metric spaces, ask: which level does it live at? The answer tells you what tools to expect and what kind of proofs will be required.

How to Read an Analysis Exercise Before Writing Anything. The preceding sections have given you the vocabulary: six exercise types, twelve proof moves, four levels. This section shows how to put them together in the five minutes before you write your first symbol.

Pre-Writing Checklist

1. **Identify the type.** Which of the six types is this: Proof, Verification, Prove-or-Counterexample, Construction, Computation, Characterization? Write the type at the top of your scratch paper before anything else.

2. **State the hypotheses and the goal.** In plain English, what are you assuming? What are you trying to show? Do not proceed until both are stated explicitly.

3. **Unpack every definition in the goal.** If the goal involves a named concept — convergence, compactness, closed, bounded — write out its definition in full. The unpacked goal is your actual target.

4. **Identify the key move.** Which of the twelve moves is most likely to bridge the hypothesis to the unpacked goal? Which level does that move belong to? Knowing the level tells you roughly how deep the proof will go and what other moves will likely appear alongside it.

5. **Sketch the shape in words.** Write in plain language: "I will choose $\varepsilon > 0$. I will use convergence of $\{x_n\}$ to find N_1. I will use convergence of $\{y_n\}$ to find N_2. I will take $N = \max(N_1, N_2)$. Then I will apply the triangle inequality." Only after this sketch do you fill in the symbols.

Remark 5.7.25 (The shape tells you what you need before you need it). Step 5 is the most important and the most skipped. Students who jump directly to symbols frequently paint themselves into corners: they choose N for one condition and then need it to satisfy a second condition that their choice does not support.

If you write the shape of the proof in words first, you see *how many conditions N must satisfy* before you commit to a specific value. Then you choose N to satisfy all of them simultaneously. The tail argument tells you how: take the maximum of however many separate N_i's the conditions require.

The shape also tells you how many terms the triangle inequality will produce and therefore what the right ε-splitting fractions are. These are not things you discover mid-proof; they are visible from the sketch.

Remark 5.7.26 (Common proofs and their shapes). The following table summarizes the shape of the most common proof patterns in a first analysis course, listed by the move sequence and the levels they traverse.

Goal	Move sequence	Levels
Convergent $\Rightarrow$ Cauchy	Unpack both definitions; triangle inequality; ε-splitting; tail argument ($\max$ of two N's).	L1-L3
Uniqueness of limits	Contradiction (assume two limits); triangle inequality; choose $\varepsilon = d(x,y)/2$; tail argument.	L2-L4
Convergent $\Rightarrow$ bounded	Unpack convergence; choose $\varepsilon = 1$; tail argument (bound $n \geq N$); take $\max$ of first N terms.	L1-L3
Verify d is a metric	List four conditions; check non-negativity, identity, symmetry directly; case split for triangle inequality.	L1-L2
Compute diameter of $[a,b]$	Write diameter as supremum; give upper bound; exhibit achieving pair.	L1-L2

Remark 5.7.27 (A note on templates). For each of the six exercise types, there is a standard template structure. These templates live in Appendix A of the companion *Primer for Real Analysis Exercises*, which you will receive at the start of Volume III. The templates are starting-point structures, not scripts to be memorized. Their purpose is to ensure that every solution has all required parts and that no structural element is accidentally omitted.

What matters at this stage is the vocabulary: six types, twelve moves, four levels. You now have the map. Volume III will fill in the terrain.

Proofs

Capstone

Bibliography